SYSTEMS LABORATORY

Evaluarea Performanțelor

EP

performance crunch

SYSTEMS LABORATORY

Performance Evaluation

EP

performance crunch

SYSTEMS LABORATORY

Team

EP performance crunch

Emilian Rădoi
emilian.radoi@upb.ro

Radu Mantu
andru.mantu@gmail.com

Adrian Cosma
cosma.i.adrian@gmail.com

Vlad Ștefănescu
vlad.a.stefanescu@gmail.com

Alex Marin
alexandru.marin.alex00@gmail.com

Cosmin Petre
Petregcosmin@gmail.com

Cezar Crăciunoiu
Cezar.craciunoiu@gmail.com

Alexandru Mircea
Alex.mircea98@gmail.com

Andrei Mirciu
Constantinandrei.mirciu@gmail.com

SYSTEMS LABORATORY

Course objectives

EP performance crunch

1. Concepts and techniques for conducting relevant performance analysis.
    - Measurement
    - Statistical modeling
    - Experimental design
    - Simulation
    - Queuing theory
2. Offer a suite of tools for monitoring the performance of the different subsystems of an OS (Linux and Windows).
3. Present methods for clear data presentation.

SYSTEMS LABORATORY

The course will cover

EP performance crunch

- Specifying performance requirements
- Evaluating design alternatives
- Comparing systems
- System tuning (determining optimal value of a parameter)
- Bottleneck identification
- Workload characterisation
- Capacity planning
- Predicting the performance at future workloads

SYSTEMS LABORATORY

Lectures structure

EP performance crunch

1. Introduction
2. Common mistakes
3. Metrics - Selection of techniques and metrics
4. Metrics - Selection of techniques and metrics
5. Workloads
6. Monitors, Capacity planning and Benchmarking
7. Data presentation and Ratio games
8. Summarizing measured data and Comparing systems using sample data
9. Introduction to experimental design and Simulation
10. Performance evaluation on Windows (BitDefender)

SYSTEMS LABORATORY

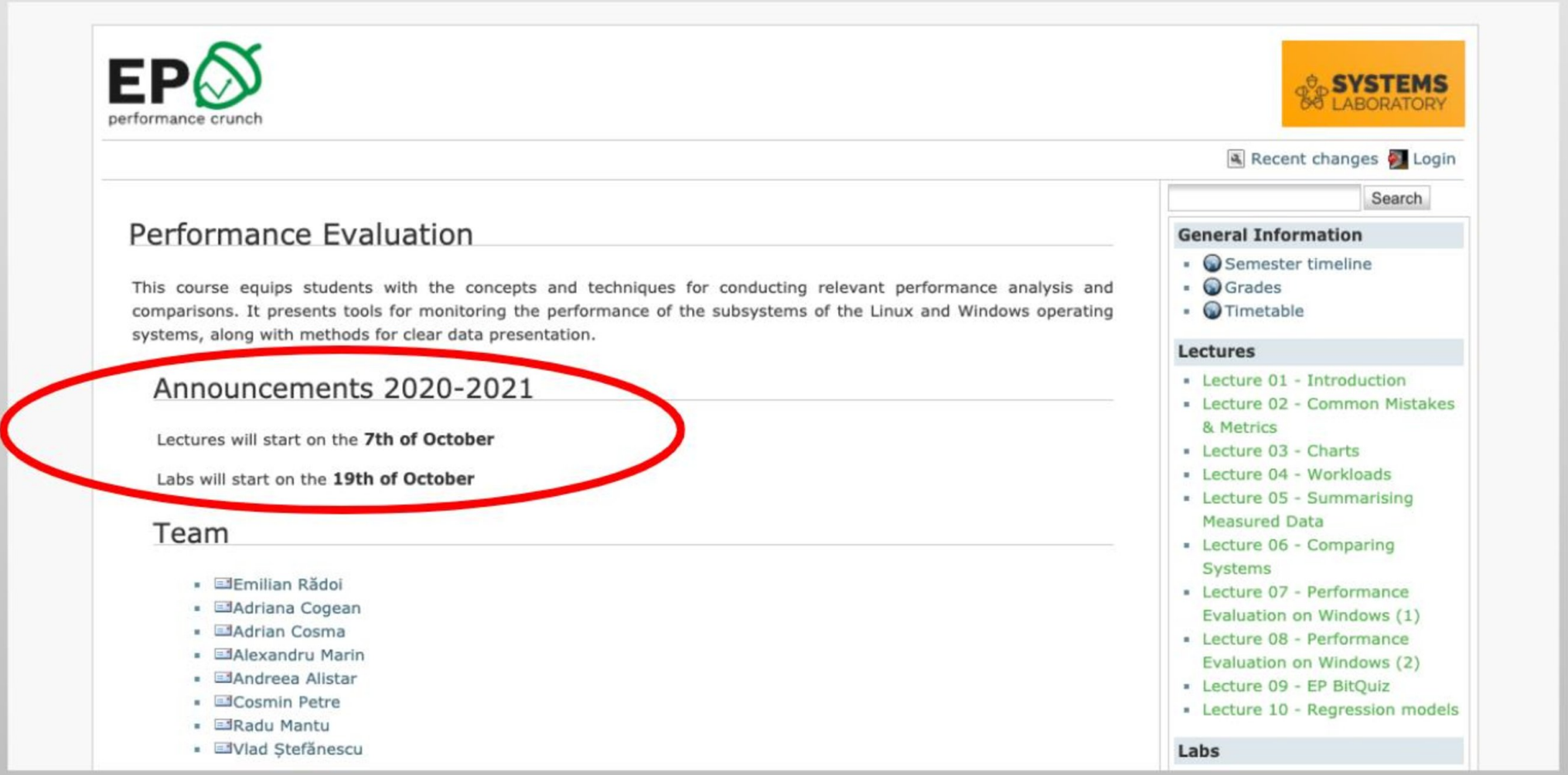
Labs structure

EP performance crunch

1. CPU (Linux)
2. Memory (Linux)
3. I/O (Linux)
4. Network (Linux)
5. Plotting
6. More plotting
7. CPU and Memory (Windows)
8. I/O and Network (Windows)
9. -
10. Machine learning
11. Student presentations
12. Student presentations

Course materials

https://ocw.cs.pub.ro/courses/ep



SYSTEMS LABORATORY

Timetable

EP performance crunch

Teaching schedule					
	Monday	**Tuesday**	**Wednesday**	**Thursday**	**Friday**
8 - 10	-	**341C5-sg2** Adrian **344C5-sg1** Cosmin **Opt-sg1** Adrian / Cosmin	**342C5-sg1** Alex **343C5-sg1** Andrei **Opt-sg2** Alex / Andrei	-	-
10 - 12	-	**341C5-sg1** Radu **344C5-sg2** Cezar	**342C5-sg2** Vlad **343C5-sg2** Alexandru	-	-
12 - 14	-	-	-	-	-
14 - 16	-	-	-	-	-
16 - 18	-	-	-	-	-
18 - 21	-	-	**Lecture** Emilian	-	-

SYSTEMS LABORATORY

Grading

EP performance crunch

- **1.5p** - Lab
- **0.5p** - Lecture attendance (random)
- **1p** - Lecture tests
- **1.5p** - Portfolio
- **1.5p** - Assignment 1
- **4p** - Exam

Total = 10p

SYSTEMS LABORATORY

Grading

EP performance crunch

- **1.5p** - Lab
- **0.5p** - Lecture attendance (random)
- **1p** - Lecture tests
- **1.5p** - Portfolio
- **1.5p** - Assignment 1
- **4p** - Exam

Total = 10p

- **1p** - Dataset

Total = 11p

SYSTEMS LABORATORY

Assignment 1

EP performance crunch

Main components:

1. App implementation (TBD)
2. Monitor behaviour and resource utilization
3. Plot results
4. Performance analysis (analyze/interpret results)

SYSTEMS LABORATORY

Portfolio

EP performance crunch

1. 5 top conferences
2. Choose a paper
3. Rewrite “Related work” section
4. Presentation in week 13
5. **Hard deadline** for document and slide submission: 15.01.2021

SYSTEMS LABORATORY

Portfolio

EP performance crunch

1. [2p/10p] Present the solution offered by the paper
2. [3p/10p] Present related papers
   a. +0.5p - for papers not referenced in your paper
   b. +0.5p - for each presented paper (at least 5 for maximum points)
3. [2p/10p] Presentation slides structure / outline
4. [3p/10p] Presentation

SYSTEMS LABORATORY

Săptămâna	Luna	Luni	Marți	Miercuri	Joi	Vineri	Sâmbătă	Duminică	Laborator	Teme
						1	2	3		
1		4	5	6	7	8	9	10		
2	Octombrie	11	12	13	14	15	16	17	L1	Curs 1
3		18	19	20	21	22	23	24	L2	
4		25	26	27	28	29	30	31	L3	Publicare Portofoliu
5		1	2	3	4	5	6	7	L4	Publicare T1
6	Noiembrie	8	9	10	11	12	13	14	L5	
7		15	16	17	18	19	20	21	L6	
8		22	23	24	25	26	27	28	L7	Deadline T1
Vacanță		29	30	1	2	3	4	5		
9		6	7	8	9	10	11	12	L8	
10	Decembrie	13	14	15	16	17	18	19	L9	EP BitQuiz
Vacanță		20	21	22	23	24	25	26		
Vacanță		27	28	29	30	31	1	2		
Vacanță		3	4	5	6	7	8	9		
11	Ianuarie	10	11	12	13	14	15	16	No Lab	Deadline Portofoliu
12		17	18	19	20	21	22	23	L10	
13		24	25	26	27	28	29	30	L11	
Sesiune		31	1	2	3	4	5	6		
Sesiune		7	8	9	10	11	12	13		
Sesiune	Februarie	14	15	16	17	18	19	20		
		21	22	23	24	25	26	27		
		28	29	30						
Restanțe	Septembrie									TBD
		Laborator	Laborator	Examene	Vacanță	Zile libere				

EP performance crunch

SYSTEMS LABORATORY

EP BitQuiz

EP performance crunch

- Prizes
- 15th of December
- Teams - ?# students (formed on the day)
- Attendance is equivalent to 1 lecture test
- More details to come

Dataset

EP performance crunch

SYSTEMS LABORATORY

Recent changes Login

Search

Performance Evaluation

This course equips students with the concepts and techniques for conducting relevant performance analysis and comparisons. It presents tools for monitoring the performance of the subsystems of the Linux and Windows operating systems, along with methods for clear data presentation.

Announcements 2021-2022

Lectures will start on the **13th of October**

Labs will start on the **12th of October**

Team

- Emilian Rădoi
- Radu Mantu
- Adrian Cosma
- Vlad Ștefănescu
- Alex Marin
- Cosmin Petre
- Alexandru Mircea
- Andrei Mirciu
- Cezar Crăciunoiu

Timetable

Teaching schedule					
	Monday	**Tuesday**	**Wednesday**	**Thursday**	**Friday**
8 - 10	-	**341C5-sg2** Adrian **344C5-sg1** Cosmin **Opt-sg1** Adrian / Cosmin	**342C5-sg1** Alex **343C5-sg1** Andrei **Opt-sg2** Alex / Andrei	-	-
10 - 12	-	**341C5-sg1** Radu **344C5-sg2** Cezar	**342C5-sg2** Vlad **343C5-sg2** Alexandru	-	-
12 - 14	-	-	-	-	-
14 - 16	-	-	-	-	-
16 - 18	-	-	-	-	-
18 - 21	-	-	**Lecture** Emilian	-	-

General Information

- Semester timeline
- Grades

Lectures

- Lecture 01 - Introduction
- Lecture 02 - Common Mistakes & Metrics
- Lecture 03 - Workloads
- Lecture 04 - Charts
- Lecture 05 - Summarising Measured Data
- Lecture 06 - Comparing Systems
- Lecture 07 - Performance Evaluation on Windows (1)
- Lecture 08 - Performance Evaluation on Windows (2)
- Lecture 09 - EP BitQuiz
- Lecture 10 - Regression models

Labs

- **Lab 01 - CPU Monitoring (Linux)**
- **Lab 02 - Memory Monitoring (Linux)**
- **Lab 03 - I/O Monitoring (Linux)**
- **Lab 04 - Networking Monitoring (Linux)**
- **Lab 05 - Plotting (Numpy & Matplotlib)**
- **Lab 06 - Advanced plotting (seaborn & pandas)**
- **Lab 07 - Test**
- **Lab 08 - I/O Monitoring (Windows)**
- **Lab 09 - Memory, CPU and Network Monitoring (Windows)**
- **Lab 10 - Machine Learning**
- **Lab 11 - Portfolio presentations**
- **Lab 12 - Portfolio presentations**

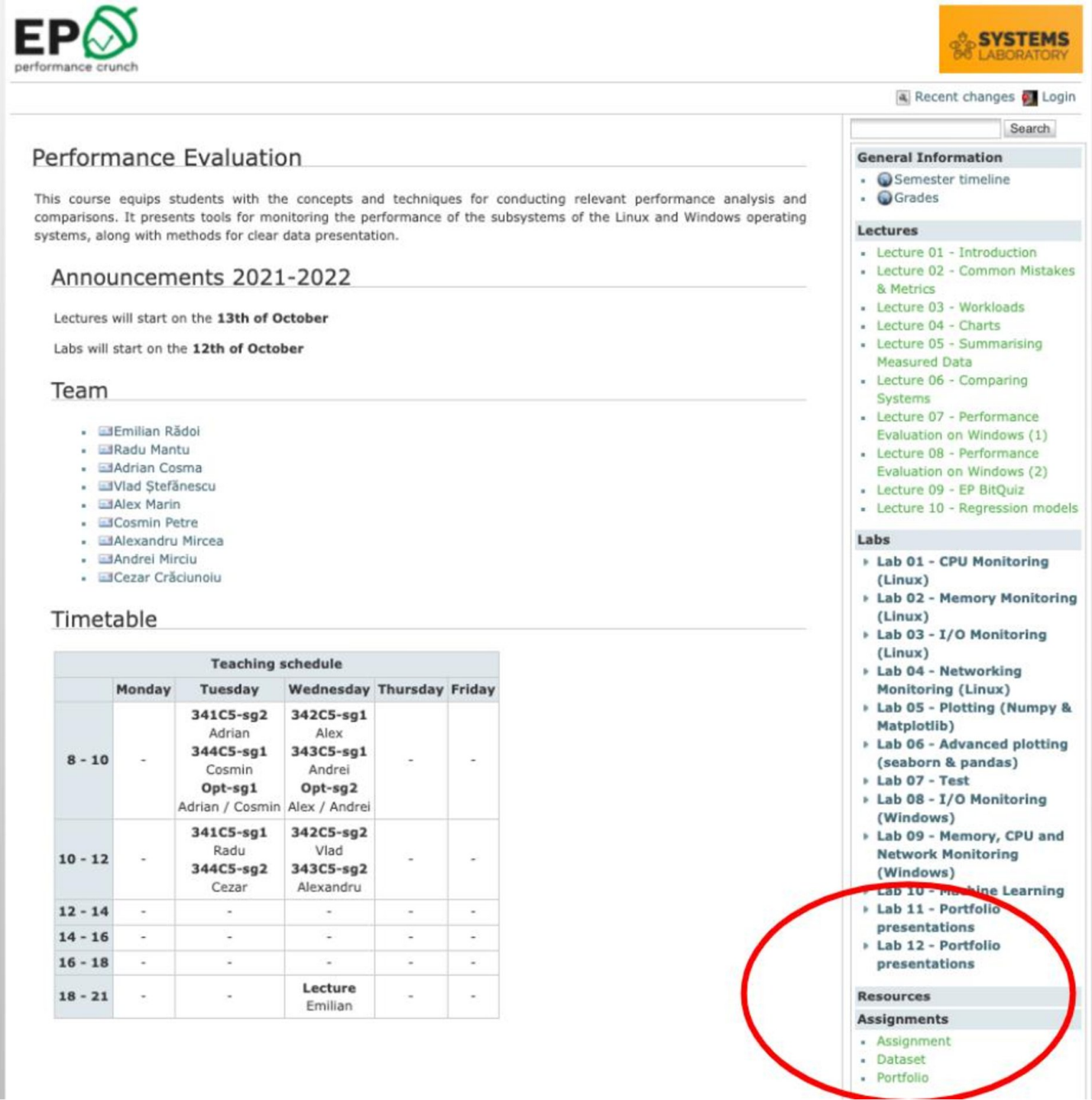
Resources

Assignments

- Assignment
- Dataset
- Portfolio

EP performance crunch

Dataset



Recent changes Login

Search

Performance Evaluation

This course equips students with the concepts and techniques for conducting relevant performance analysis and comparisons. It presents tools for monitoring the performance of the subsystems of the Linux and Windows operating systems, along with methods for clear data presentation.

Announcements 2021-2022

Lectures will start on the **13th of October**

Labs will start on the **12th of October**

Team

- Emilian Rădoi
- Radu Mantu
- Adrian Cosma
- Vlad Ștefănescu
- Alex Marin
- Cosmin Petre
- Alexandru Mircea
- Andrei Mirciu
- Cezar Crăciunoiu

Timetable

Teaching schedule					
	Monday	**Tuesday**	**Wednesday**	**Thursday**	**Friday**
8 - 10	-	**341C5-sg2** Adrian **344C5-sg1** Cosmin **Opt-sg1** Adrian / Cosmin	**342C5-sg1** Alex **343C5-sg1** Andrei **Opt-sg2** Alex / Andrei	-	-
10 - 12	-	**341C5-sg1** Radu **344C5-sg2** Cezar	**342C5-sg2** Vlad **343C5-sg2** Alexandru	-	-
12 - 14	-	-	-	-	-
14 - 16	-	-	-	-	-
16 - 18	-	-	-	-	-
18 - 21	-	-	**Lecture** Emilian	-	-

General Information
- Semester timeline
- Grades

Lectures
- Lecture 01 - Introduction
- Lecture 02 - Common Mistakes & Metrics
- Lecture 03 - Workloads
- Lecture 04 - Charts
- Lecture 05 - Summarising Measured Data
- Lecture 06 - Comparing Systems
- Lecture 07 - Performance Evaluation on Windows (1)
- Lecture 08 - Performance Evaluation on Windows (2)
- Lecture 09 - EP BitQuiz
- Lecture 10 - Regression models

Labs
- **Lab 01 - CPU Monitoring (Linux)**
- **Lab 02 - Memory Monitoring (Linux)**
- **Lab 03 - I/O Monitoring (Linux)**
- **Lab 04 - Networking Monitoring (Linux)**
- **Lab 05 - Plotting (Numpy & Matplotlib)**
- **Lab 06 - Advanced plotting (seaborn & pandas)**
- **Lab 07 - Test**
- **Lab 08 - I/O Monitoring (Windows)**
- **Lab 09 - Memory, CPU and Network Monitoring (Windows)**
- **Lab 10 - Machine Learning**
- **Lab 11 - Portfolio presentations**
- **Lab 12 - Portfolio presentations**

Resources

Assignments
- Assignment
- Dataset
- Portfolio

Dataset

Dataset

Optional (1p)

This assignment represents an opportunity for you to contribute to the research performed in our Computer Science department of the University Politehnica of Bucharest, by helping us collect a large dataset which will enable an interdisciplinary study on personality traits and movement. This requires you filling in approximately 5 personality questionnaires and walking multiple times in front of three cameras.

Above is an example of a run. There are three cameras at different angles (0, 45 and 90 degrees) that capture walking patterns. It is required to perform the same walk, but with a few variations:

1. Normal walking (Baseline) - 4 runs
2. Different clothing - 2 runs
3. Carry bag - 2 runs
4. Slower walking speed - 2 runs
5. Faster walking speed - 2 runs
6. Walking while texting - 2 runs
7. Walking while talking on the phone - 2 runs

The walking pattern will be extracted using pose estimation methods.

Tasks

1. Fill in questionnaire

<Questionnaire form will be available soon>

This Google Form contains 5 questionnaires for self-reported psychometrics of various personality traits (Big Five, Self-Esteem, Assertiveness, Aggression, Anxiety etc).

- Duration: approx. 30 minutes

2. Book time slot

<Doodle form will be available soon>

After filling in the questionnaires use this Doodle to book a time slot to capture your walking patterns.

- Duration: approx. 1 minute

3. Walk in a straight line

The camera setup is located in the **FitBit Lab (EG 302)**.

General Information
- Semester timeline
- Grades

Lectures
- Lecture 01 - Introduction
- Lecture 02 - Common Mistakes & Metrics
- Lecture 03 - Workloads
- Lecture 04 - Charts
- Lecture 05 - Summarising Measured Data
- Lecture 06 - Comparing Systems
- Lecture 07 - Performance Evaluation on Windows (1)
- Lecture 08 - Performance Evaluation on Windows (2)
- Lecture 09 - EP BitQuiz
- Lecture 10 - Regression models

Labs
- **Lab 01 - CPU Monitoring (Linux)**
- **Lab 02 - Memory Monitoring (Linux)**
- **Lab 03 - I/O Monitoring (Linux)**
- **Lab 04 - Networking Monitoring (Linux)**
- **Lab 05 - Plotting (Numpy & Matplotlib)**
- **Lab 06 - Advanced plotting (seaborn & pandas)**
- **Lab 07 - Test**
- **Lab 08 - I/O Monitoring (Windows)**
- **Lab 09 - Memory, CPU and Network Monitoring (Windows)**
- **Lab 10 - Machine Learning**
- **Lab 11 - Portfolio presentations**
- **Lab 12 - Portfolio presentations**

Resources

Assignments
- Assignment
- Dataset
- Portfolio

SYSTEMS LABORATORY

Performance evaluation

EP

performance crunch

SYSTEMS LABORATORY

Whose ?

A **system** can be any collection of hardware, software, and firmware components.

Other keywords:

Metrics - criteria used to evaluate the performance of a system

Workloads - the requests made by the users of a system

SYSTEMS LABORATORY

Why do we need this?

EP performance crunch

- Determine a system's capabilities
- Determine feasibility of future expansion
- Systems comparison
- Identify performance problems / bottlenecks
- System tuning

Process

1. Select appropriate evaluation techniques, performance metrics, system workload
2. Conduct performance measurements
3. Use statistical techniques for comparing alternatives
4. Design simulation/measurement experiments to get maximum results with least effort
5. Perform simulations correctly
6. Mathematical techniques

SYSTEMS LABORATORY

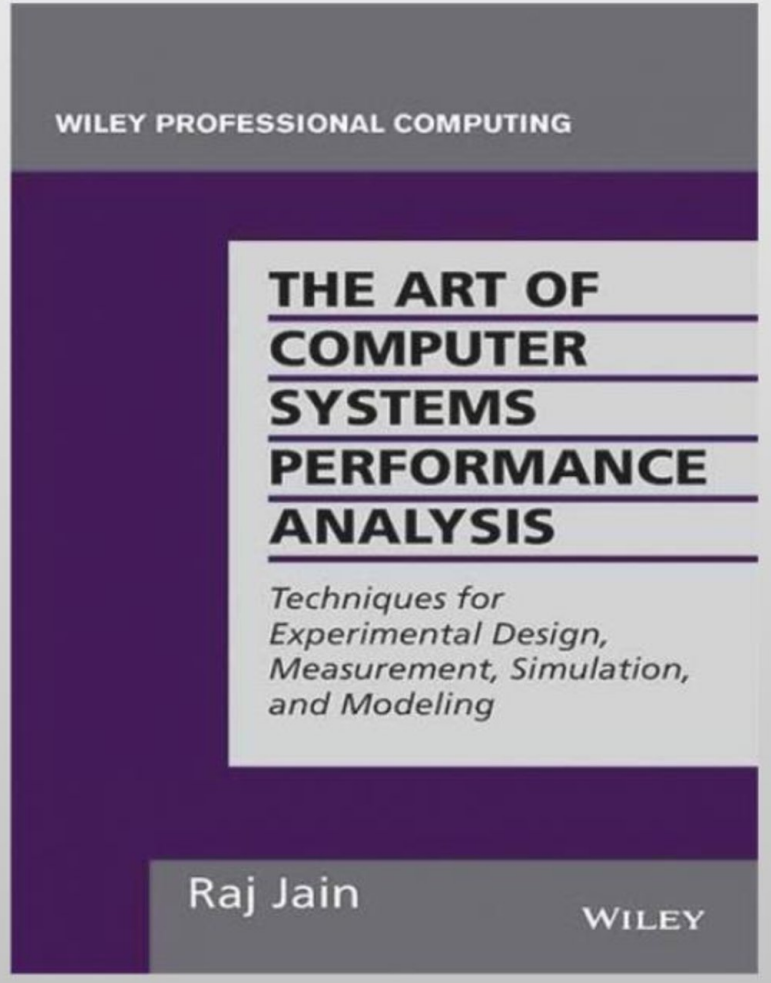
EP performance crunch

“Contrary to common belief, performance evaluation is an art. Like a work of art, successful evaluation cannot be produced mechanically. Every evaluation requires an intimate knowledge of the system being modeled and a careful selection of the methodology, workload, and tools.”

Raj K. Jain

References

Raj K. Jain, "*Art of Computer Systems Performance Analysis: Techniques for Experimental Design Measurements, Simulation and Modeling*", John Wiley, 2015.



SYSTEMS LABORATORY

Lecture 2
Common Mistakes & Metrics

EP
performance crunch

SYSTEMS LABORATORY

Recap

EP performance crunch

System - any collection of hardware, software, and firmware components.

Metrics - criteria used to evaluate the performance of a system

Workloads - the requests made by the users of a system

SYSTEMS LABORATORY

Common mistakes

EP performance crunch

1. No goals
2. Biased goals
3. Unsystematic approach
4. Analysis without understanding the problem
5. Incorrect performance metrics
6. Unrepresentative workload
7. Wrong evaluation technique
8. Overlook important parameters
9. Ignore significant factors
10. Inappropriate experimental design
11. Inappropriate level of detail

SYSTEMS LABORATORY

Common mistakes

EP performance crunch

12. No analysis
13. Erroneous analysis
14. No sensitivity analysis
15. Ignoring errors in input
16. Improper treatment of outliers
17. Assuming no change in the future
18. Ignoring variability
19. Too complex analysis
20. Improper presentation of results
21. Ignoring social aspects
22. Omitting assumptions and limitation

SYSTEMS LABORATORY

Checklist for avoiding common mistakes

EP performance crunch

Look it up in the book.

SYSTEMS LABORATORY

A systematic approach to perf. evaluation

EP performance crunch

1. State goals and define the system
2. List services and outcomes
3. Select metrics
4. List parameters
5. Select factors to study
6. Select evaluation technique
7. Select workload
8. Design experiments
9. Analyze and interpret data
10. Present results

Repeat

Example

1. System definition

   The network connecting two hosts.

2. Services

   Instant messaging application (including file transfers).

SYSTEMS LABORATORY

Example (cont)

EP performance crunch

3. Metrics
   - No errors or failures.
   - Rate (throughput), time (latency, rtt), resource per service.

   Leads to these metrics:

   i. Time per message or file
   ii. Maximum transfer rate per unit of time
   iii. Local and remote CPU time per transmission

4. System parameters
   - Speed of local CPU, remote CPU, network
   - Operating system
   - Network reliability

Example (cont)

5. Workload parameters
   - Time between successive message/file transmissions
   - Sizes of messages/files
   - Type of channel
   - Other loads on the local and remote CPUs and the network
6. Factors
   - Type of channel
   - Size of network
   - Size of message/file transmitted
   - Keep fixed: type of CPU, OS, no other loads on hosts and network

Example (cont)

7. Evaluation technique
   - Implement prototype => measurements
   - Use analytical modeling for validation
8. Workload
   - Synthetic program generating the specified types of channel requests
   - Null channel requests => resources used in monitoring and logging
9. Experimental design
   - A full factorial experiment design e.g. 2*3*3
10. Data Analysis
11. Data Presentation

SYSTEMS LABORATORY

Summary

EP performance crunch

- Analysis techniques, metrics, workload - depend on the goal
- Metrics are based on the services provided by the system
- System and workload parameters determine the right set of experiments

SYSTEMS LABORATORY

Metrics

EP

performance crunch

SYSTEMS LABORATORY

EP performance crunch

Selecting evaluation technique

Selecting metrics

SYSTEMS LABORATORY

Techniques

EP performance crunch

- Mathematical Modeling
- Simulation
- Measurement

SYSTEMS LABORATORY

Mathematical Modeling

EP performance crunch

- Skill set required
  - Math
- Abstraction of system
  - High-level
- Assumptions
  - Many
- Results
  - Asymptotic results, low accuracy
- Outliers
  - Not captured

SYSTEMS LABORATORY

Simulation

EP performance crunch

- Represents an almost “real-world” view of the system
- System can be modeled as a set of interacting subsystems and objects
- Various levels of abstraction possible
- Fine-grained performance metrics can be measured
- Coding errors
- Does not capture real-world environment

SYSTEMS LABORATORY

Measurement

EP performance crunch

- Testbed
- Experimental results

Selecting an evaluation technique

Criterion	Analytical Model	Simulation	Measuring
1. Stage	Any	Any	Prototype
2. Time required	Small	Medium	Varies
3. Tools	Analysts	Computer languages	Instrumentation
4. Accuracy	Low	Moderate	Varies
5. Trade-off evaluation	Easy	Moderate	Difficult
6. Cost	Low	Medium	High
7. Saleability	Low	Medium	High

SYSTEMS LABORATORY

Rules of validation

EP performance crunch

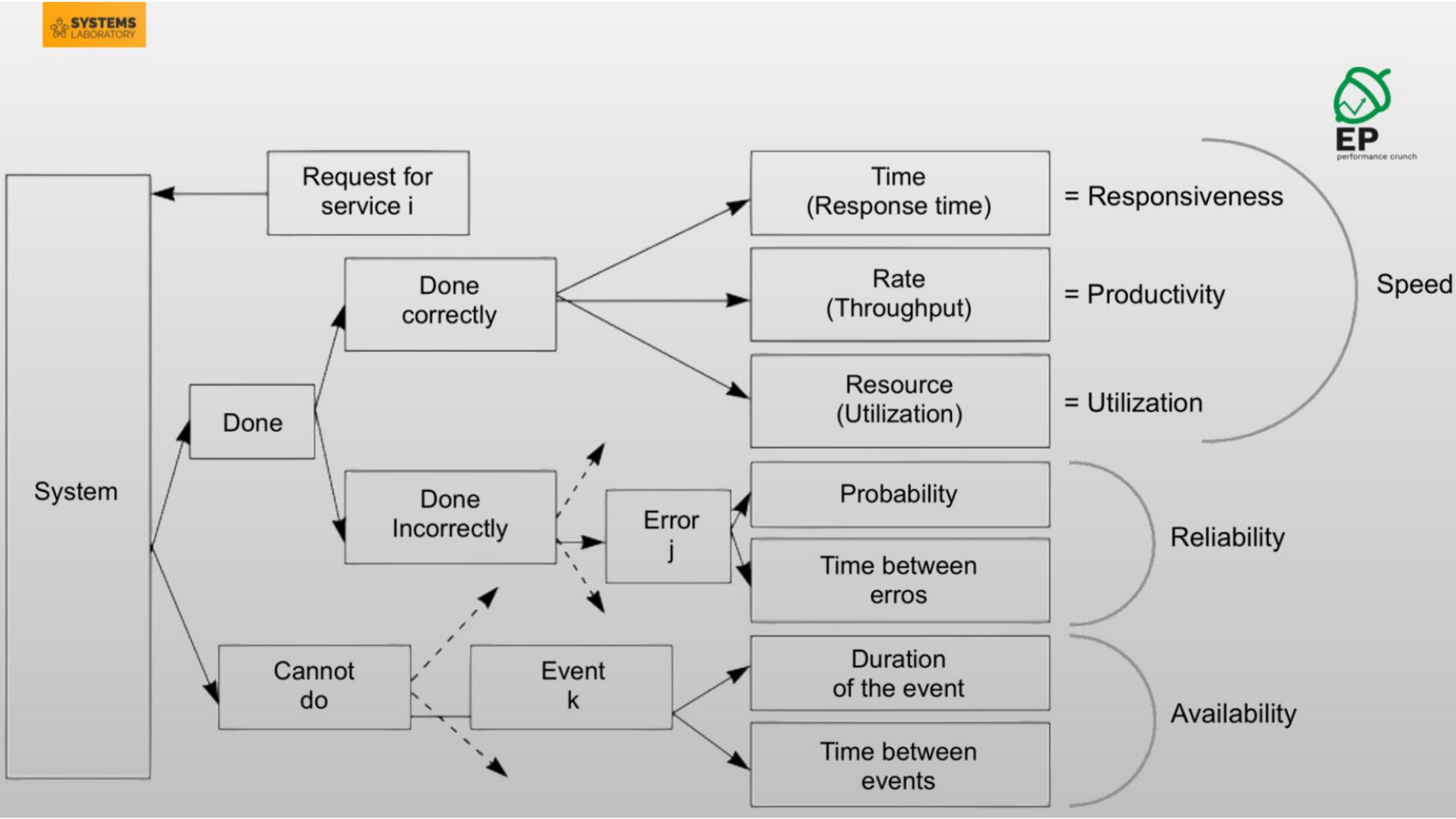
- Do not trust the results of a **simulation model** until validated with analytical modeling or measurements
- Do not trust the results of an **analytical model** until validated with a simulation model or measurements
- Do not trust the results of a **measurement** until validated with simulation or analytical modeling

SYSTEMS LABORATORY

Performance metrics

EP performance crunch

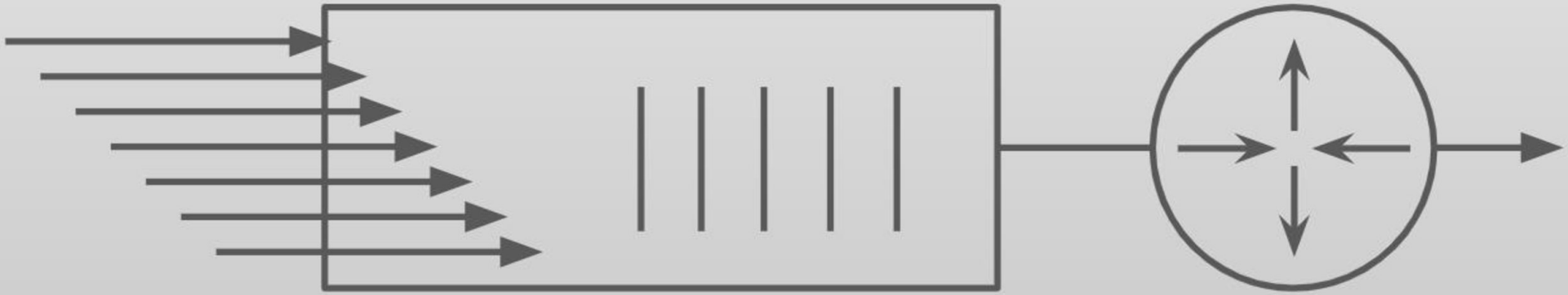
- List services and outcomes


SYSTEMS LABORATORY
EP performance crunch
System
Request for service i
Done
Done correctly
Done Incorrectly
Cannot do
Error j
Event k
Time (Response time)
Rate (Throughput)
Resource (Utilization)
Probability
Time between erros
Duration of the event
Time between events
= Responsiveness
= Productivity
= Utilization
Speed
Reliability
Availability

SYSTEMS LABORATORY

Global vs individual

EP performance crunch



SYSTEMS LABORATORY

Common metrics

EP performance crunch

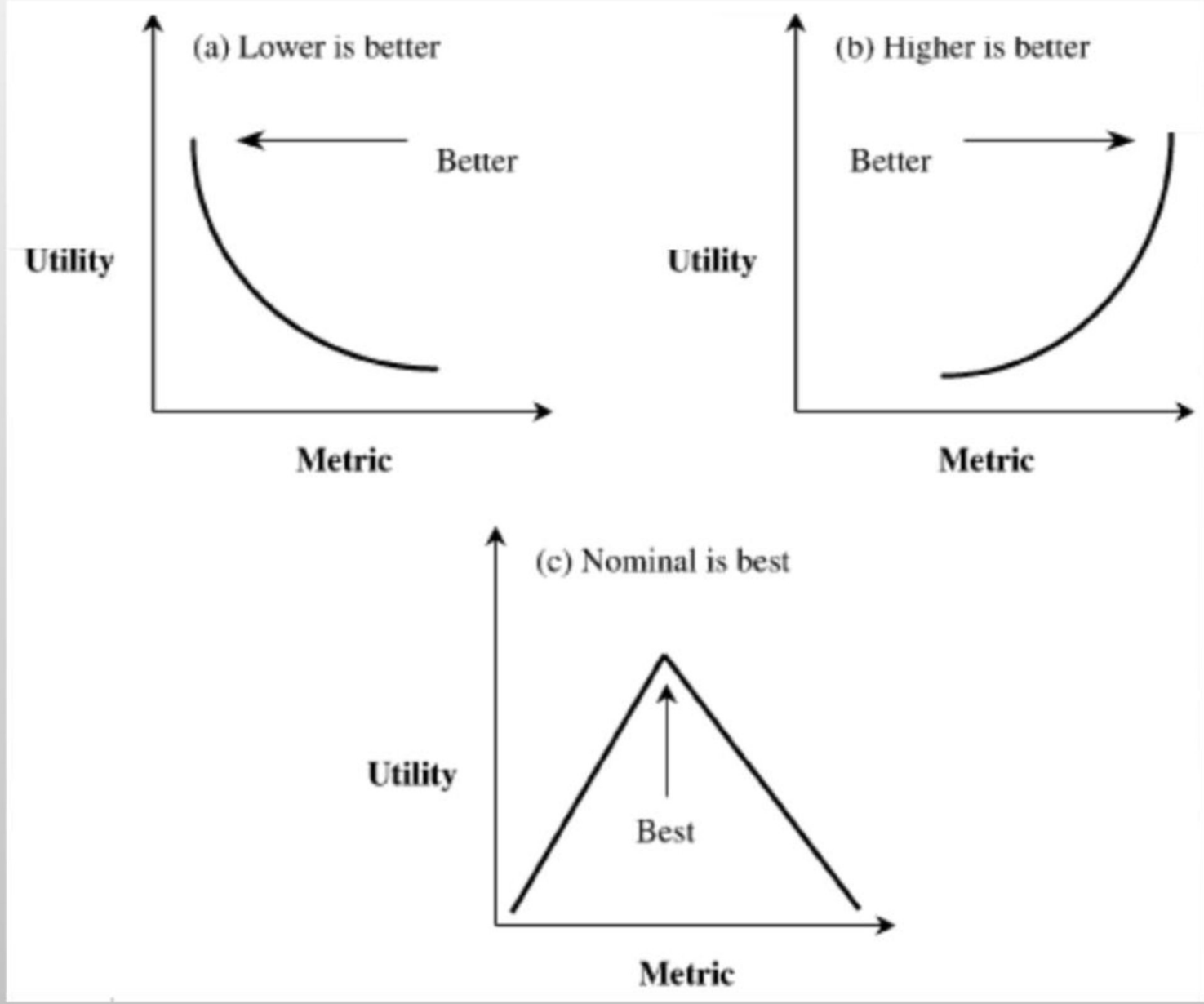
- Response time
  - Interval between a user's request and the system response
- Turnaround time
  - For batch stream
  - Time between the submission of a batch job and the completion of its output

Common metrics (cont)

- Throughput
  - The rate (requests per unit of time) at which the requests can be serviced by the system
- Nominal capacity
  - The maximum achievable throughput under ideal workload conditions
  - Bandwidth
- Usable capacity
  - The maximum throughput achievable without exceeding a prespecified response time limit (Often the response time at maximum throughput is too high to be acceptable.)
- Efficiency
  - Usable capacity / nominal capacity

Common metrics (cont)

- Utilization
  - The fraction of time the resource is busy servicing requests
- Reliability
  - The probability of errors or the mean time between errors
- Availability
  - The fraction of the time the system is available to service users' requests
  - Mean Time To Failure (MTTF) - mean uptime
- Cost/performance ratio
  - Metric for comparing systems
  - E.g. two transaction processing systems may be compared in terms of dollars per TPS


(a) Lower is better
Better
Utility
Metric
(b) Higher is better
Better
Utility
Metric
(c) Nominal is best
Utility
Best
Metric

SYSTEMS LABORATORY

Requirements

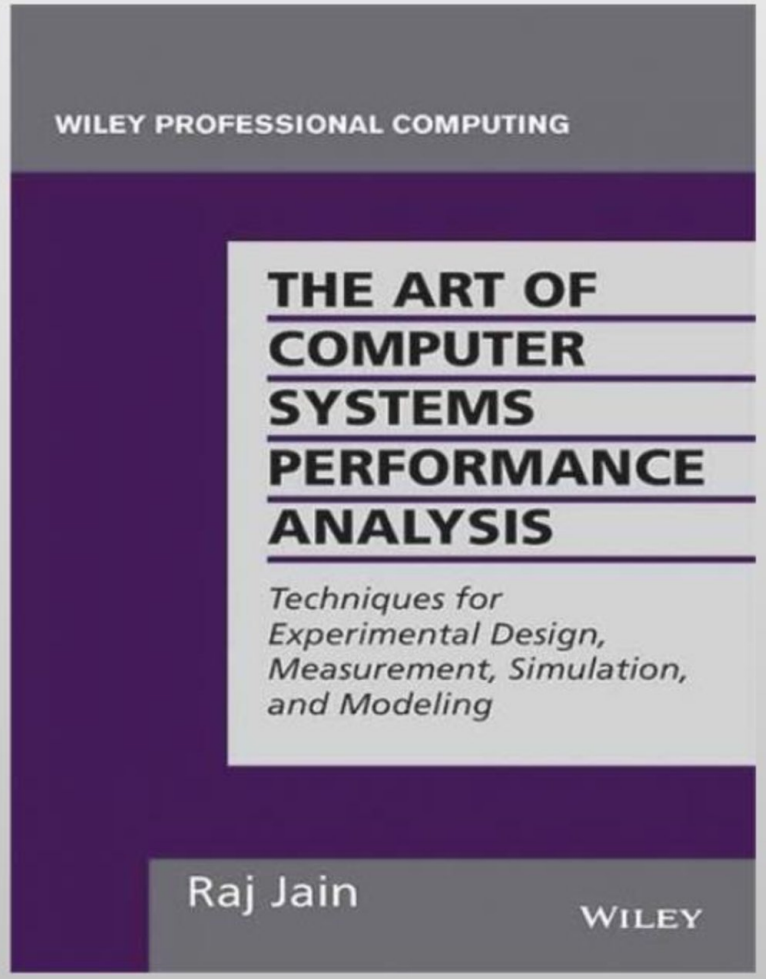
EP performance crunch

- “The system should be both processing and memory efficient. It should not create excessive overhead.”
- “There should be an extremely low probability that the network will duplicate a packet, deliver a packet to the wrong destination, or change the data in a packet.”

- Unacceptable
  - Non-Specific
  - Non-Measurable
  - Non-Acceptable
  - Non-Realizable
  - Non-Thorough

References

Raj K. Jain, "*Art of Computer Systems Performance Analysis: Techniques for Experimental Design Measurements, Simulation and Modeling*", John Wiley, 2015.



SYSTEMS LABORATORY

Lecture 3
Workloads

EP

performance crunch

SYSTEMS LABORATORY

EP performance crunch

- Lecture Test 1
- Portfolio goes live tonight

SYSTEMS LABORATORY

Test workload types

EP performance crunch

- Real workload
  - System being used for normal operations (by actual people)
- Synthetic workload
  - Similar to real workload
  - Can be applied repeatedly in a controlled manner
  - No large real-world data files
  - No sensitive data
  - Easily modified without affecting operations
  - Easily ported to different systems due to its small size
  - May have built-in measurement capabilities

SYSTEMS LABORATORY

Desired characteristics of a benchmark

EP performance crunch

- Representative
  - of an application area
- Portable
  - run on many different architectures and implementations
- Unbiased
  - not designed to favour a particular system
- Scalable
  - run on both small and large systems
- Measurable
  - easy to measure => broad acceptance
- Repeatable
  - minimum variance
- Explainable
  - single number easy to understand

SYSTEMS LABORATORY

Workloads for computer systems

EP performance crunch

1. Add instructions
2. Instruction mixes - usage frequency
3. Kernels - matches order - micro-benchmark
4. Synthetic programs - bigger programs
5. Application benchmarks - matches I/O

Synthetic programs

- Synthetic programs
  - Whetstone (1972) - floating point intensive numerical code
  - Linpack (1976) - linear algebra package - still used - top5000.org to rank supercomputers
  - Dhrystone (1984) - integer only, system code

- Vendor specific benchmarks
  - WinSAT (Windows System Assessment Tool)
    - Control Panel → Performance Information and Tools
    - Reports Windows Experience Index (WEI)
    - Five sub-scores: processor, memory, 2D graphics, 3D graphics, disk
    - Overall score = **min** of five sub-scores
  - SAP, Lotus/IBM, Oracle, etc
  - Use binary / machine code

SYSTEMS LABORATORY

Synthetic programs

EP performance crunch

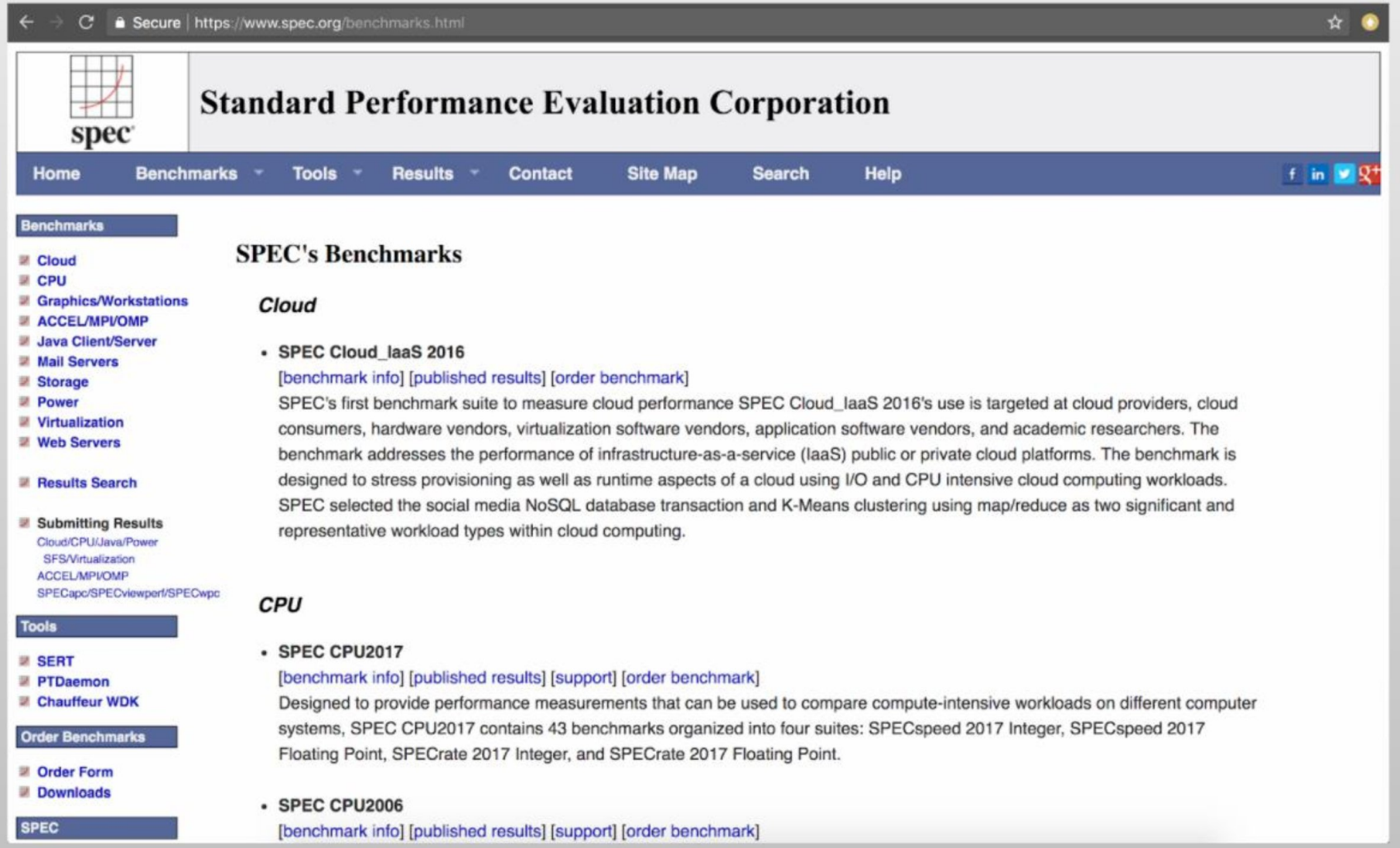
- Industry benchmarks associations
  - SPEC (Systems Performance Evaluation Cooperative)
  - BAPCo (Business Applications Performance Corporation)
  - EEMBC (The Embedded Microprocessor Benchmark Consortium)
  - TPC (Transaction Performance Council) - database systems
  - SPC (Storage Performance Council) - storage systems

https://en.wikipedia.org/wiki/Benchmark_(computing)

SYSTEMS LABORATORY

SPEC Benchmarks

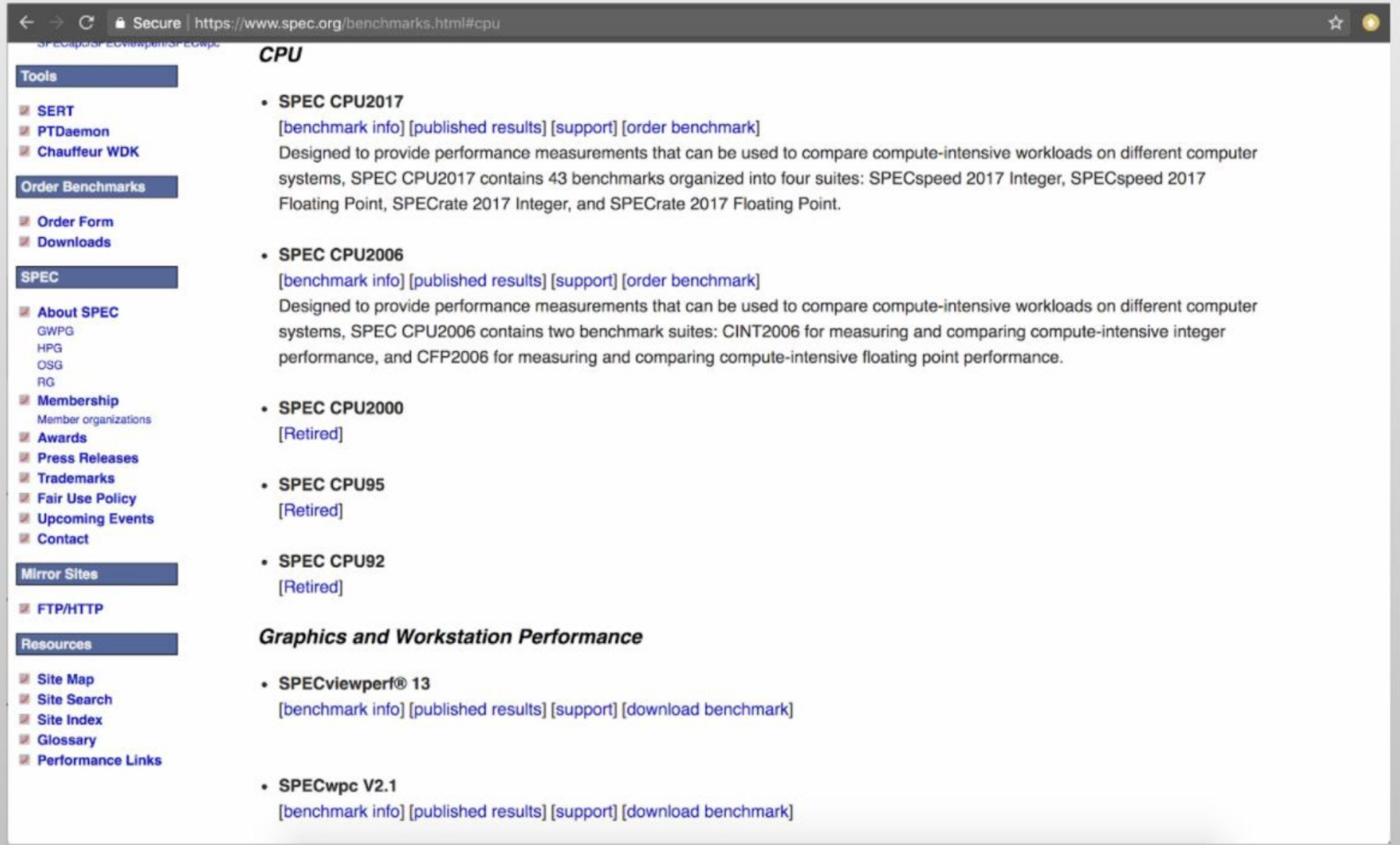
EP performance crunch



SYSTEMS LABORATORY

SPEC Benchmarks

EP performance crunch



SYSTEMS LABORATORY

SPEC CPU benchmark

EP performance crunch

- Consists of many programs
- Ported to various platforms to remove dependency on I/O or OSs
- Compilation and execution is automated
  - Teste supplies a configuration file with location of C compiler, compilation flags, etc
- Speed
  - Execution time on the system normalised by a well known system
- Geometric mean of ratios is reported as SPECint or SPECfp
- Rate - throughput - jobs/s. Geometric mean = SPECrate

SYSTEMS LABORATORY

Benchmark challenges / issues

EP performance crunch

Main problem - optimising/tuning to score higher on benchmarks

- Optimised compilations not representative of real-world
  - Baseline and peak numbers (vendors publicise only peak numbers)
- n-CPU systems run n copies in parallel
  - Not representative since there is no queue of waiting jobs
- Source codes of commonly used programs are not available for use in benchmarks
  - Mostly open source
- Long run times allow jobs to run from cache
  - Not representative
- High performance on high priced systems
  - Price/performance ratio

Summary

- Benchmarks have moved from instructions (MIPS), kernels, synthetic benchmarks to industry standard benchmarks.
- Benchmarks should be representative, portable, unbiased, scalable, measurable, repeatable and explainable.
- Difficult to ensure representativeness of benchmark (compiler flags, run time, configurations - can be manipulated)
- Price and energy consumption are important along with performance

SYSTEMS LABORATORY

EP performance crunch

Workload Selection

SYSTEMS LABORATORY

Workload selection

EP performance crunch

- Services exercised
- Level of detail
- Loading level
- Timeliness

SYSTEMS LABORATORY

Services exercised

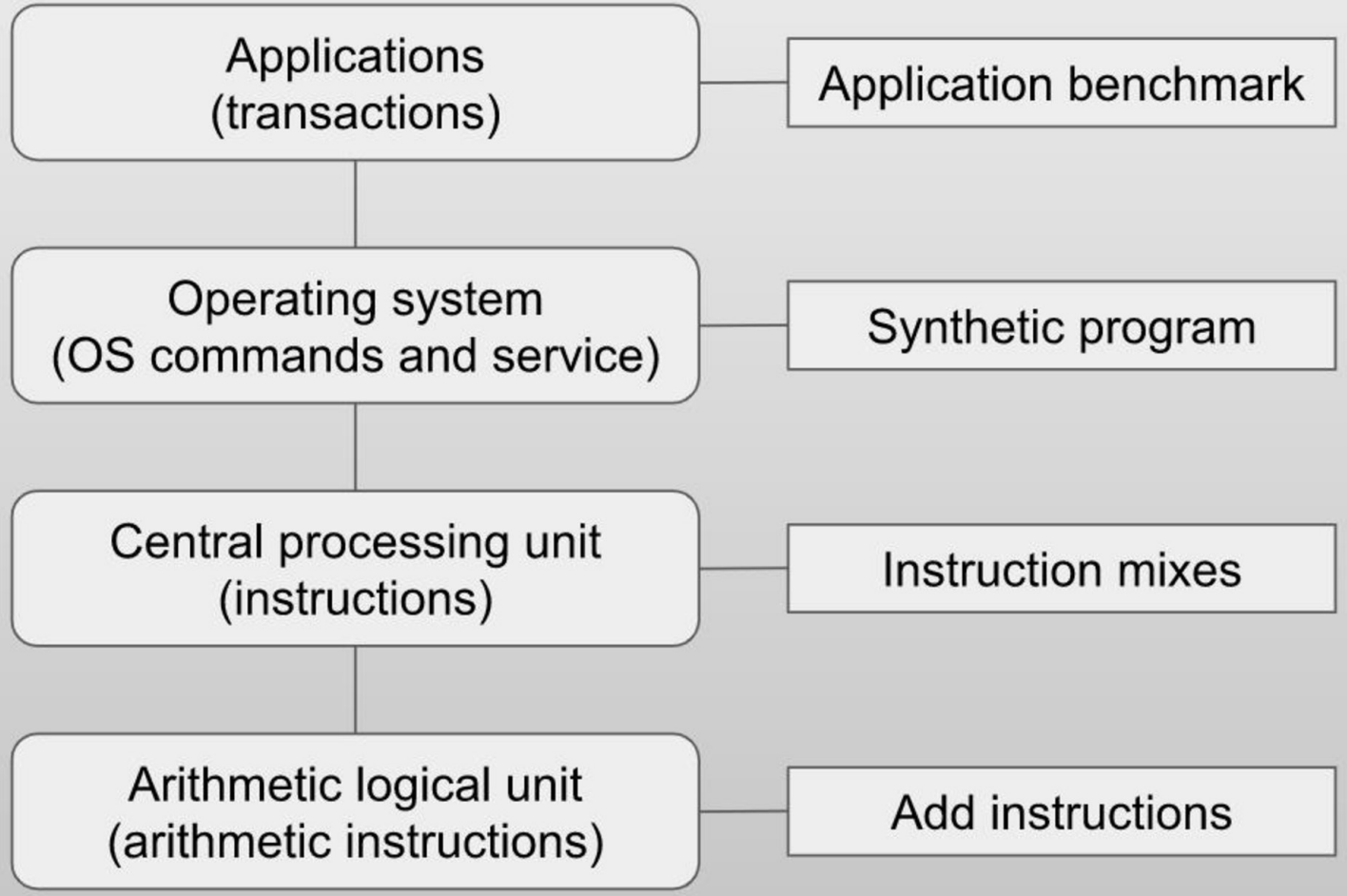
EP performance crunch

System services determine the workload and metrics

System <- Services <- Metrics and Workload

- System Under Test (SUT) vs Component Under Study (CUT)
- Metrics depend upon SUT
  - MIPS is ok for 2 CPUs, but not for 2 time-sharing systems
- Workload depends upon SUT
- Examples
  - CPU - instructions
  - System - transactions
  - Multiple services - exercise as complete a set of services as possible

Example


Applications
(transactions)
Application benchmark
Operating system
(OS commands and service)
Synthetic program
Central processing unit
(instructions)
Instruction mixes
Arithmetic logical unit
(arithmetic instructions)
Add instructions

Level of detail

- Most frequent operations
  - Only if one service is significantly more frequent than others
  - Example: Add instructions
- Frequency of request types
  - Example: Instruction mixes
- Time-stamped sequence of requests
  - Probably too detailed
  - May require exact reproduction of component behaviour
- Average resource demand
- Distribution of resource demands

SYSTEMS LABORATORY

Representativeness

EP performance crunch

- The test workload and the real workload should have the same:
  - Elapsed time
  - Resource demands
  - Resource usage profile - sequence and amounts in which the different resources are used

SYSTEMS LABORATORY

Timeliness

EP performance crunch

It has to represent today’s usage!

Other considerations

- Loading level - a workload may exercise a system to its:
  - Full capacity (best case)
  - Beyond its capacity (worst case)
  - At a load level observed in real world (typical case)
- Impact of external components
  - Do not use a workload that makes external component a bottleneck => all alternatives to this system give equally good performance.
- Repeatability

SYSTEMS LABORATORY

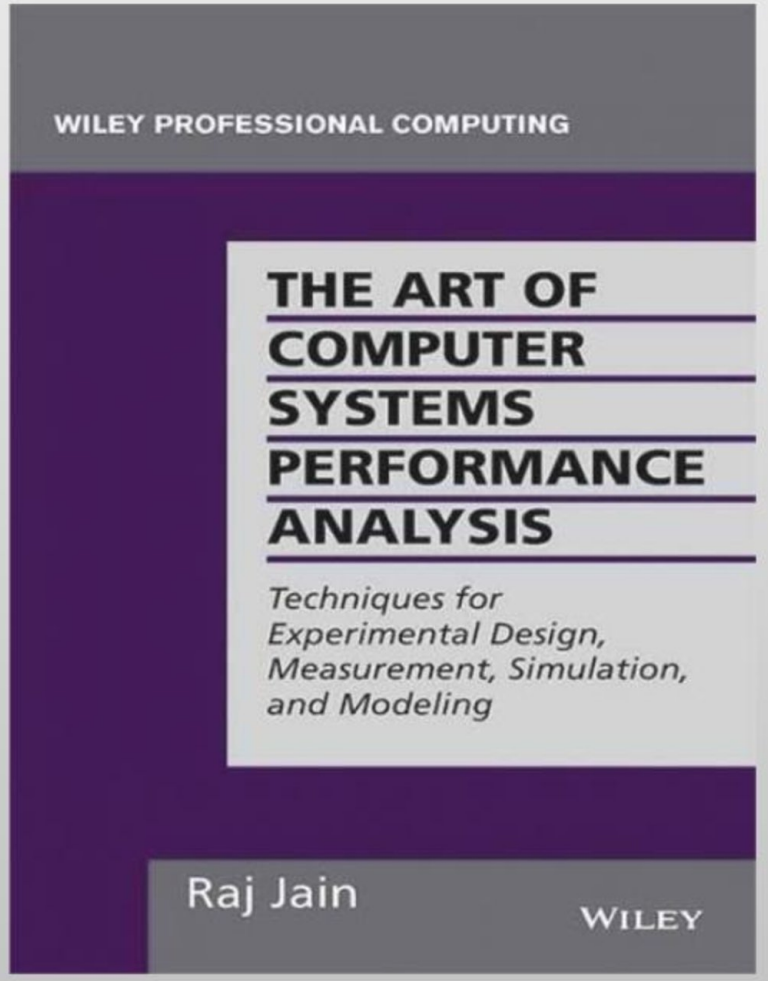
Summary

EP performance crunch

- Services determine the workload (and metrics)
- Level of detail of the workload should match that of the model being used
- Workload should be representative of the real systems' usage in recent past

References

1. Raj K. Jain, "*Art of Computer Systems Performance Analysis: Techniques for Experimental Design Measurements, Simulation and Modeling*", John Wiley, 2015.
2. Raj Jain, CSE 567M Computer Systems Analysis, Washington University in St. Louis, 2013.



SYSTEMS LABORATORY

Lecture 4
Charts

performance crunch

The aim of performance analysis

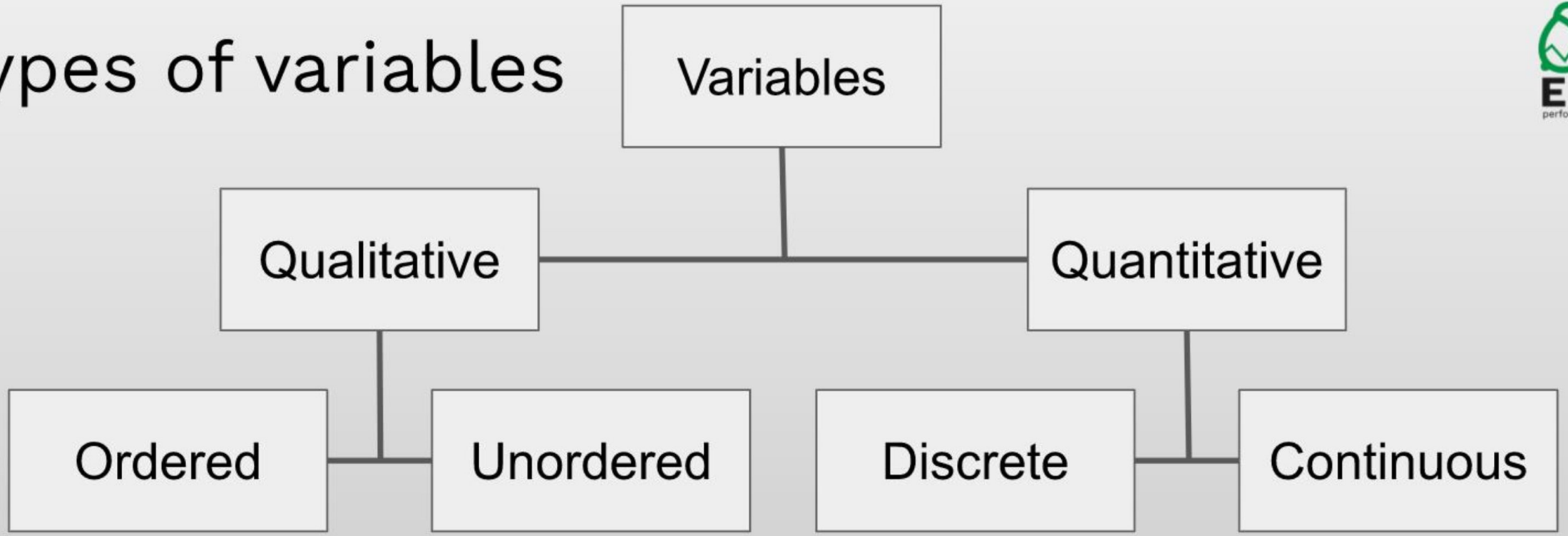
1. What is the aim of every performance analysis?
   Help in making a decision

2. **Presentation of final results** - one of the important steps in every performance evaluation study

Why use graphs

1. Save readers' time
   a. Present the same information more concisely
2. Interest the reader
   a. Easier to look at the figures to quickly grasp the main points of the study and read the text only for details
3. Emphasize or clarify a point
4. Reinforce a conclusion
5. Summarize the results of a study

Types of variables



Ordered - Computer type: supercomputers, minicomputers, microcomputers
Unordered - Workload type: scientific, engineering, educational
Discrete (Integer) - Number of processors in a multiprocessor system. Size of disks in blocks.
Continuous (Real) - System response time for a job.

The type of chart to be used depends on the type of variable

1. A **line** chart is used to show the ***relationship*** between two **continuous** variables.

2. A **column** chart or bar chart is used when the ***independent*** variable (also called x-variable) is a **discrete** or a **qualitative** variable.

SYSTEMS LABORATORY

Common chart types

EP performance crunch

- Line chart
  - Usually shows change over time
  - Compare things over time
- Bar chart
  - Compare similar things in a single category
- Double bar chart
  - Compare data
- Histogram
  - Type of bar chart
  - Compares ranges of data in a single category
- Stemplots
  - Like histograms, except they show each data point

Common chart types

- Pie chart
  - Shows parts of a whole
  - Often shows percentages
- Scatter plot
  - Dots of information that are not connected
  - Often show a correlation
- Timeplot
  - Shows how a variable changes over time
  - Convention
    - time on OX axis
    - values of the variable on the OY axis
- Box and whisker
  - Show variation among your data

SYSTEMS LABORATORY

Guidelines for good graphs

EP performance crunch

1. Require minimum effort from the reader
   a. Legends vs direct labeling
2. Maximise information - self contained
   a. CPU usage vs Daily CPU usage
3. Minimise the amount printed
   a. Grid lines - shown only if necessary to read accurate values
4. Use commonly accepted practices
   a. What people expect: origin in (0,0), linear scales, increase left to right, etc.
5. Avoid ambiguity
   a. Show coordinate axes, scale divisions, and origin.

Checklist for good graphics -1-

Checklist proposed by Raj Jain in [1]

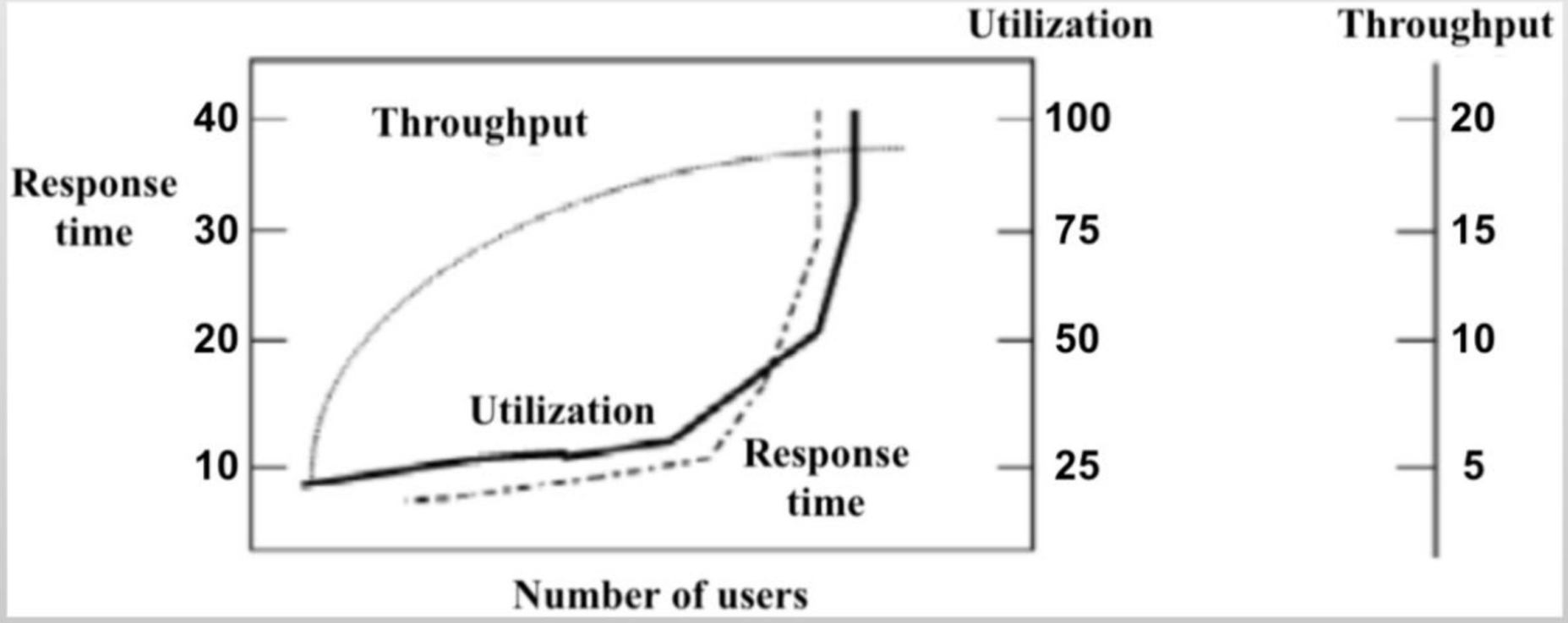
1. Are both coordinate axes shown and labeled?
2. Are the axes labels self-explanatory and concise?
3. Are the scales and divisions shown on both axes?
4. Are the minimum and maximum of the ranges shown on the axes appropriate to present the maximum information?
5. Is the number of curves reasonably small? (A line chart should generally have no more than six curves).
6. Do all graphs use the same scale? (Multiple scales on the same chart are confusing.)
7. Is there no curve that can be removed without reducing the information?
8. Are the curves on a line chart individually labeled?
9. Are the cells in a bar chart individually labeled?
10. Are all symbols on the graph accompanied by appropriate textual explanations?
11. If the curves cross, are the line patterns different to avoid confusion?
12. Are the units of measurement indicated?
13. Is the horizontal scale increasing from left to right?

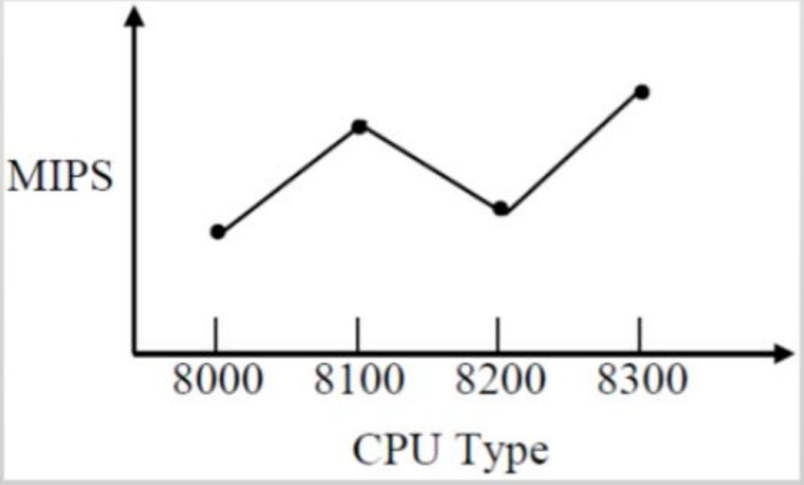
Checklist for good graphics -2-

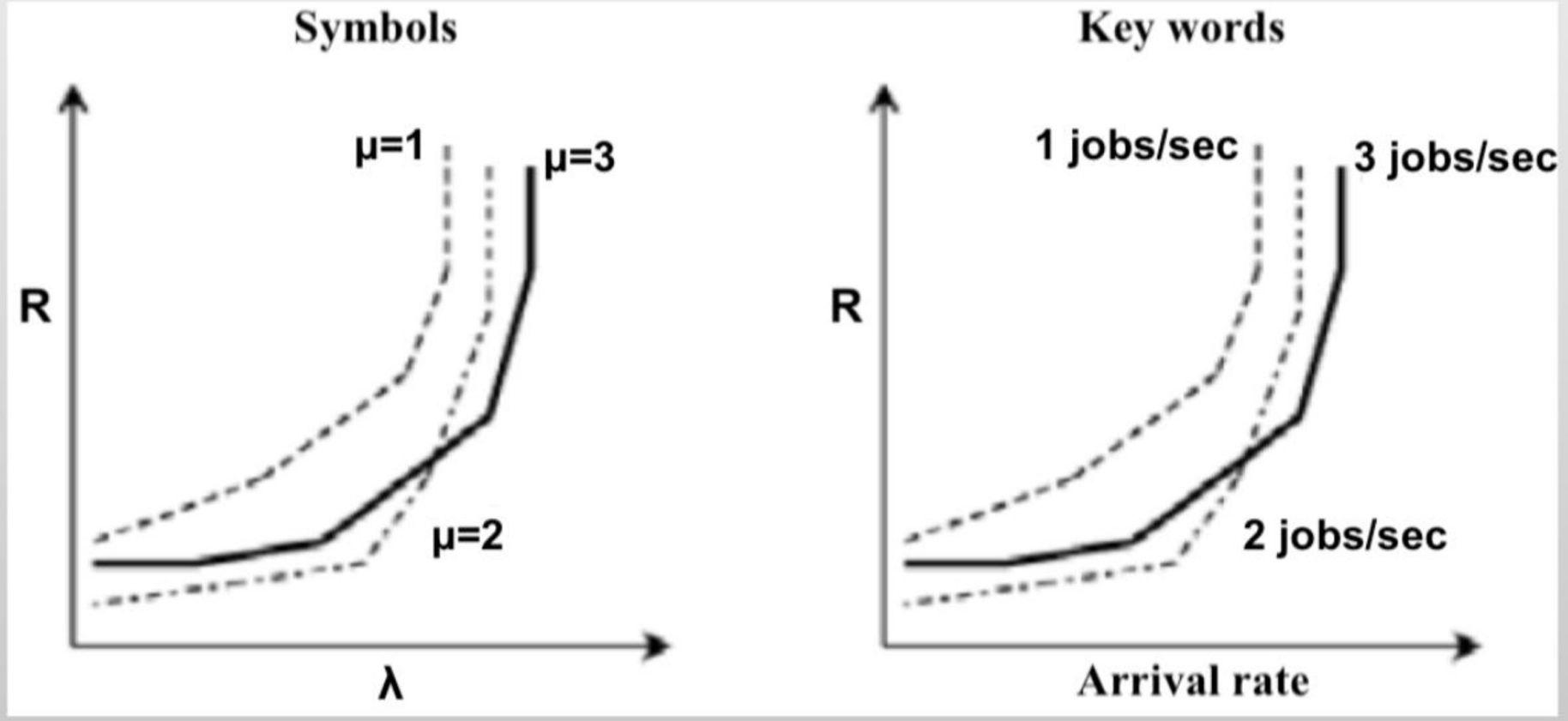
14. Is the vertical scale increasing from bottom to top?
15. Are the grid lines aiding in reading the curves? (If not, the grid lines should not be shown.)
16. Does this whole chart add to information available to the reader?
17. Are the scales contiguous? (Breaks in the scale should be avoided or clearly shown.)
18. Is the order of bars in a bar chart systematic? (Alphabetic, temporal, or best-to-worst ordering is to be preferred over random placement.)
19. If the vertical axis represents a random quantity, are confidence intervals shown?
19. If the vertical axis represents a random quantity, are confidence intervals shown?
20. Are there no curves, symbols, or texts on the graph that can be removed without affecting the information?
21. Is there a title for the whole chart?
22. Is the chart title self-explanatory and concise?
23. Do the variables plotted on this chart give more information than other alternatives?
24. Does the chart clearly bring out the intended message?
25. Is the figure referenced and discussed in the text of the report?

Common mistakes

1. Too much data on a graph
    a. Rule of thumb
        i. Max: **6 lines**/curves on a line chart, **8 bars** in a bar/column chart, **8 slices** in a pie chart
        ii. Max **10 values** on an axis
2. Using symbols in place of text
3. Using line chart instead of column chart


Utilization
Throughput
Response time
40
30
20
10
Throughput
Utilization
Response time
100
75
50
25
20
15
10
5
Number of users


MIPS
8000
8100
8200
8300
CPU Type


Symbols
μ=1
μ=3
μ=2
R
λ
Key words
1 jobs/sec
3 jobs/sec
2 jobs/sec
R
Arrival rate

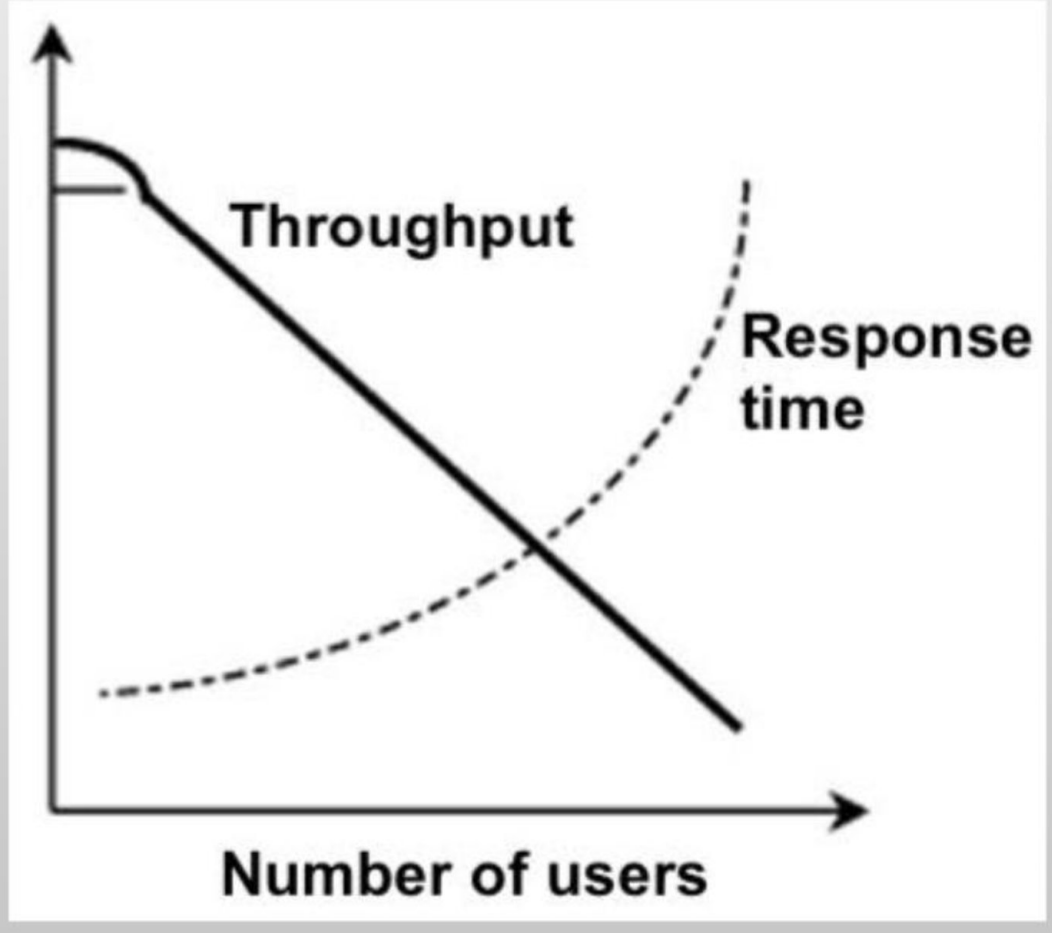
SYSTEMS LABORATORY

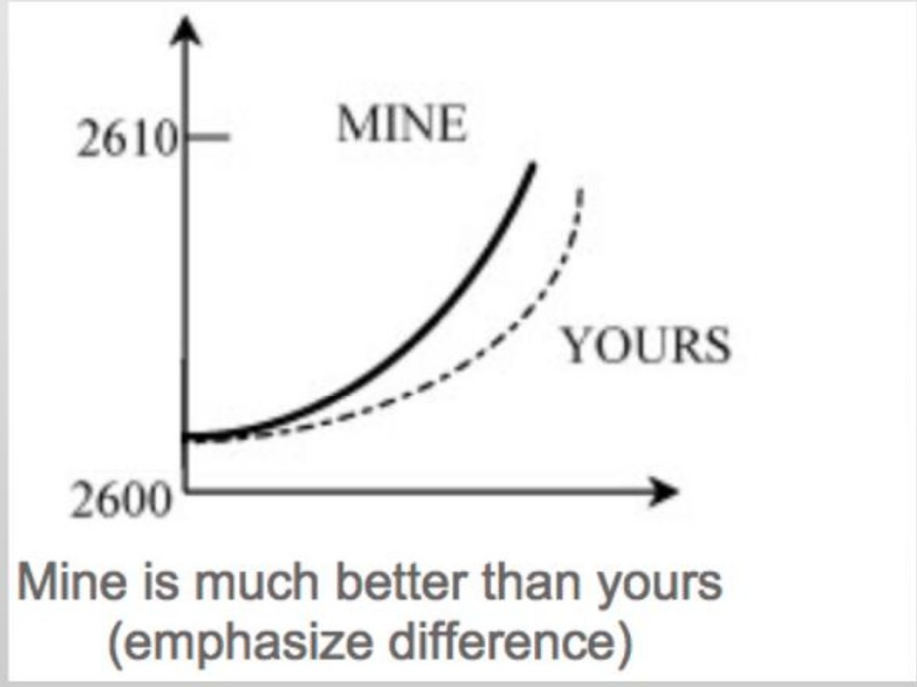
Misleading graphs

EP performance crunch

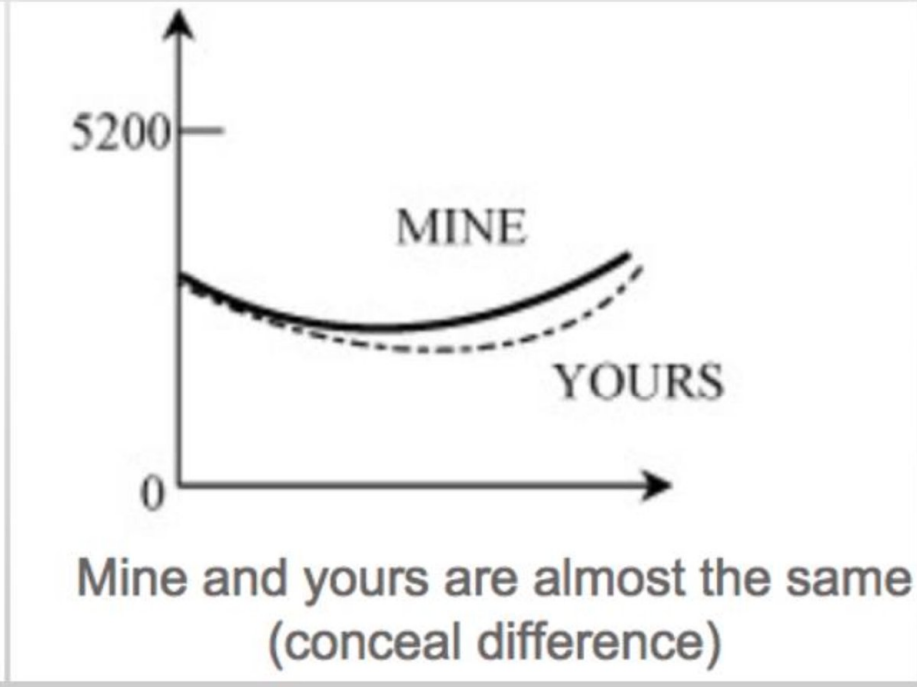
1. Using non-zero origins to emphasize the difference
2. Improper scaling
3. Plotting related metrics for ‘dramatisation’
4. Omitting data
5. Improper extraction

6. Labels, numbers, scale, context

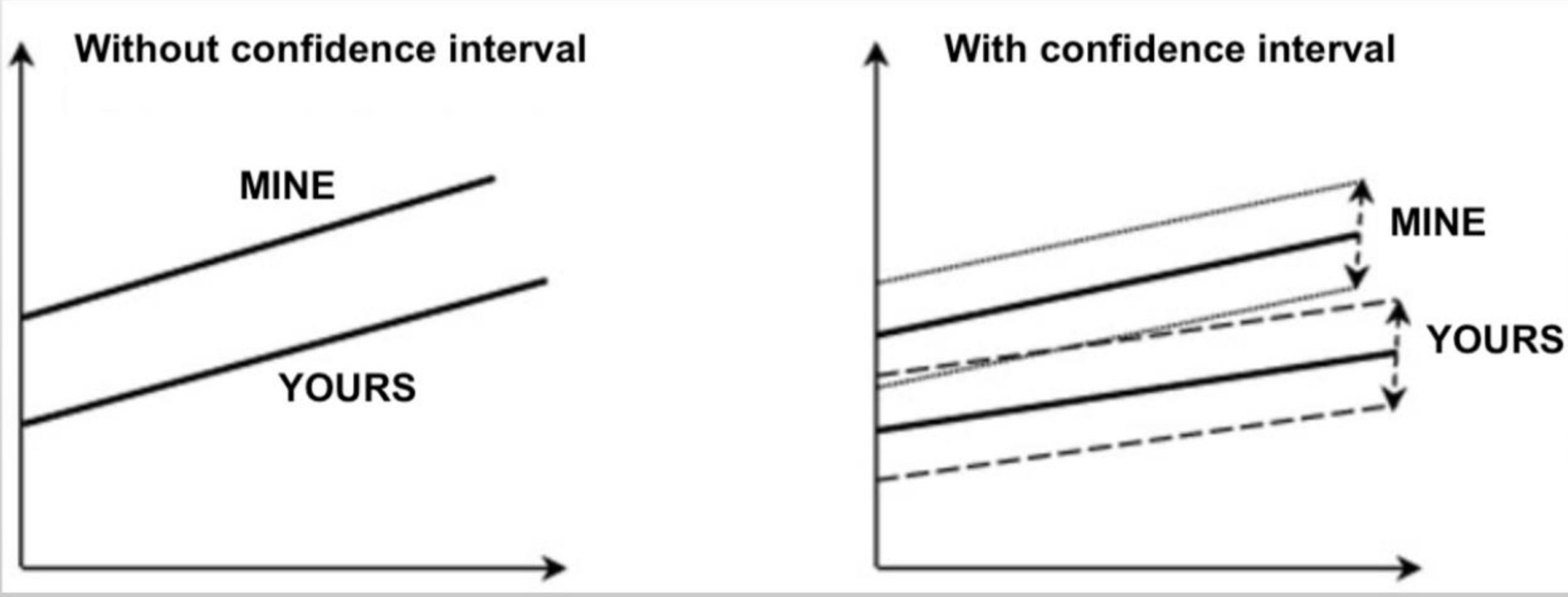

Throughput
Response time
Number of users



Mine is much better than yours
(emphasize difference)

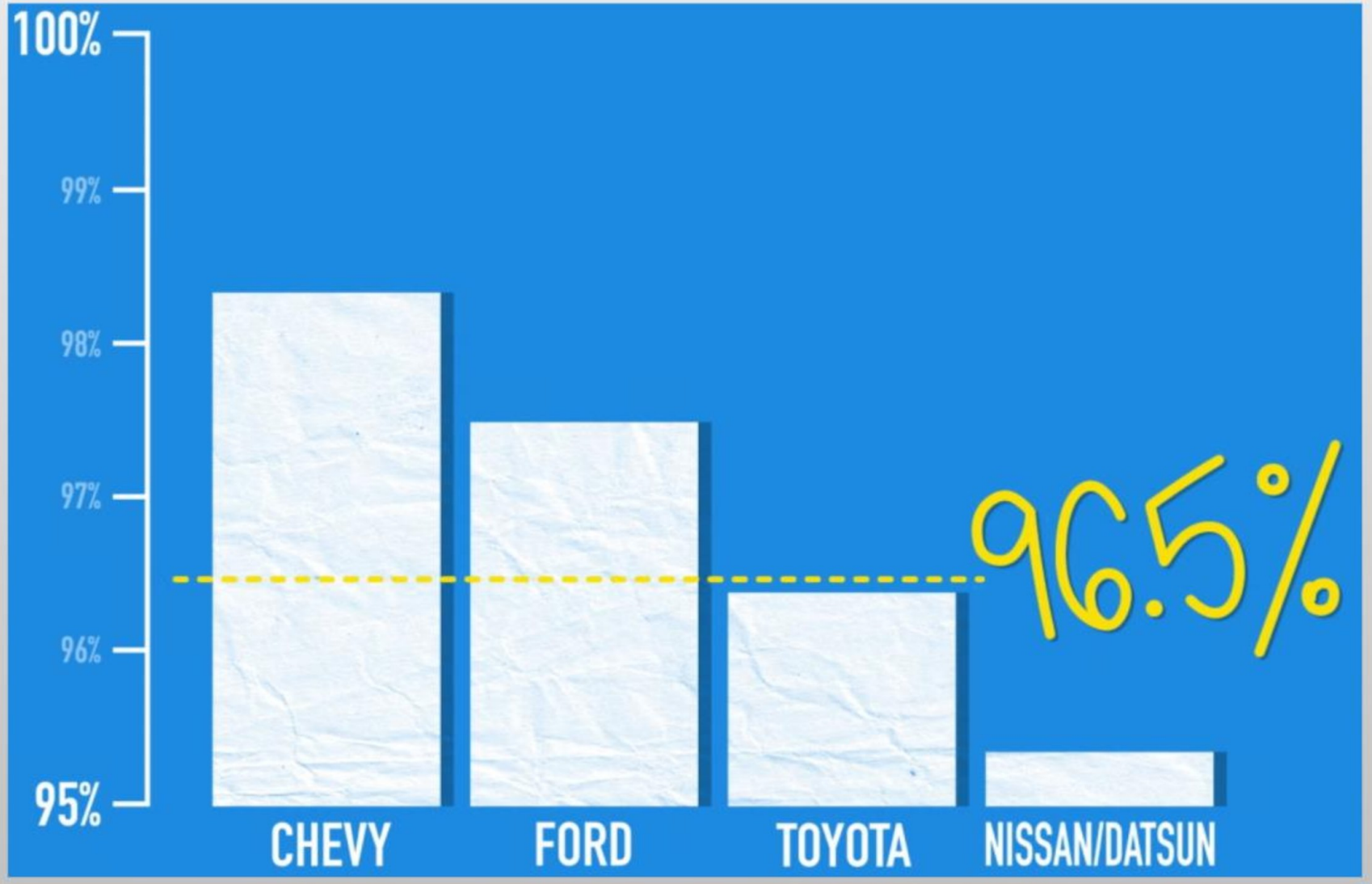


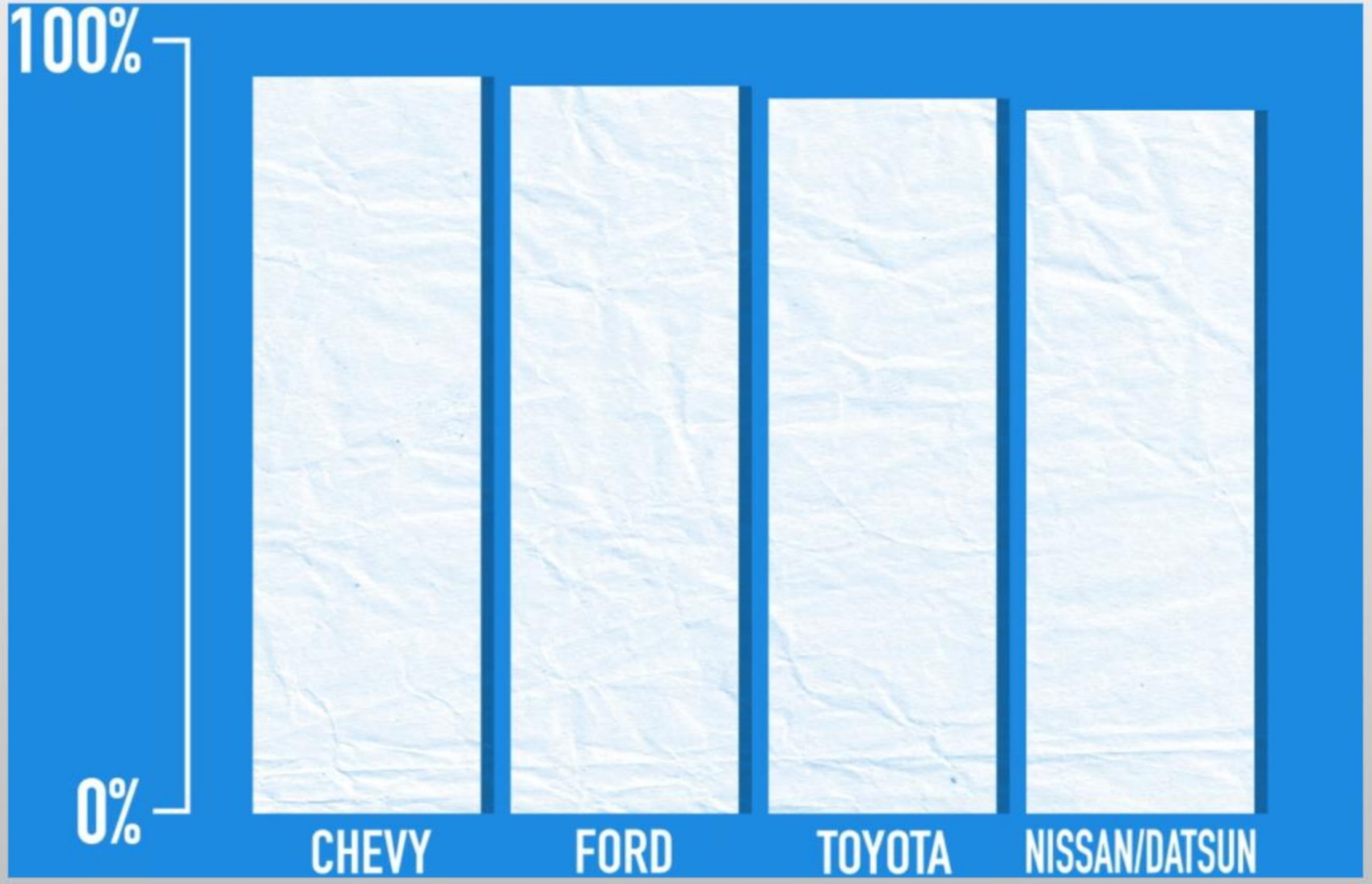
Mine and yours are almost the same
(conceal difference)

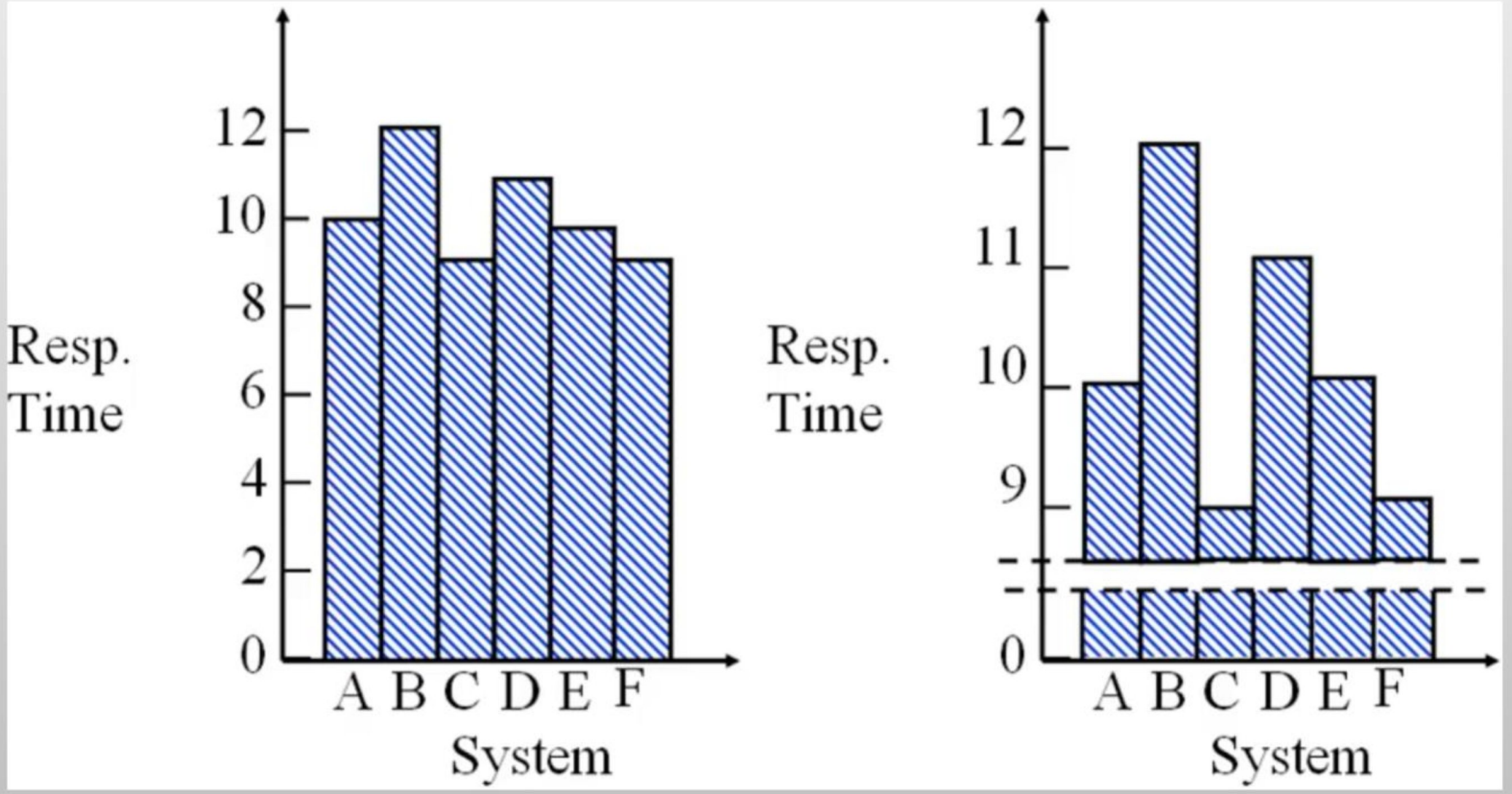

Without confidence interval
MINE
YOURS
With confidence interval
MINE
YOURS

SYSTEMS LABORATORY

EP performance crunch



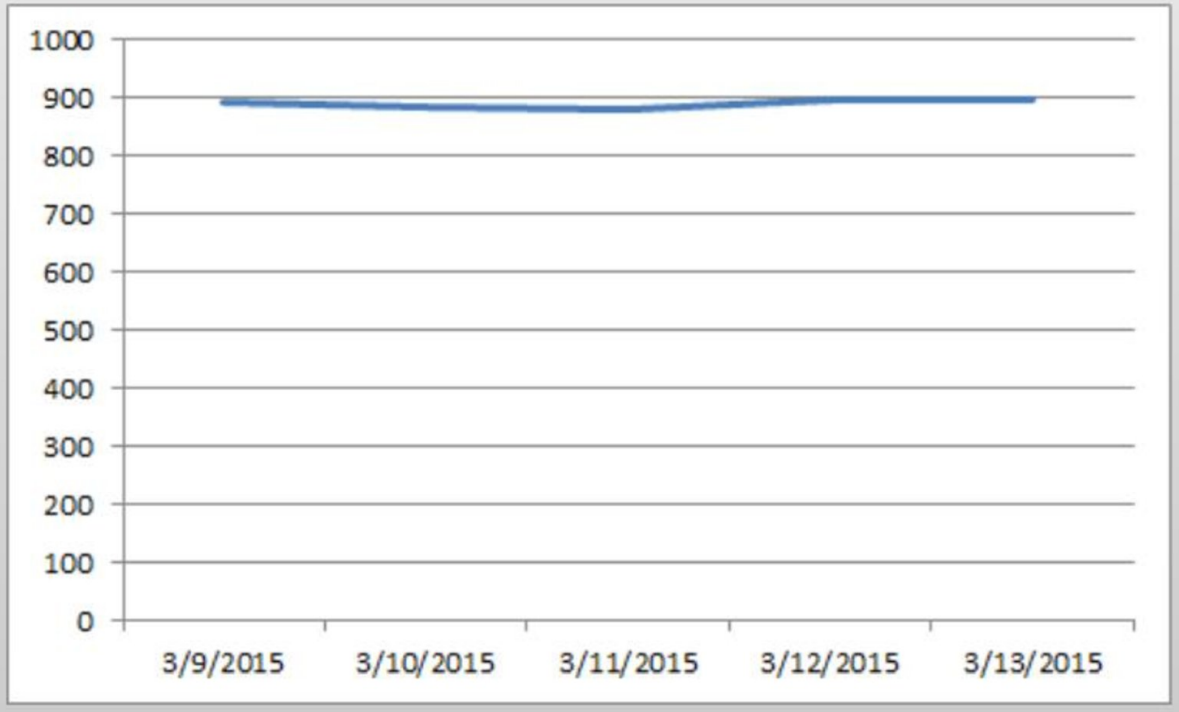

100%
0%
CHEVY
FORD
TOYOTA
NISSAN/DATSUN


Resp.
Time
12
10
8
6
4
2
0
A B C D E F
System
Resp.
Time
12
11
10
9
0
A B C D E F
System

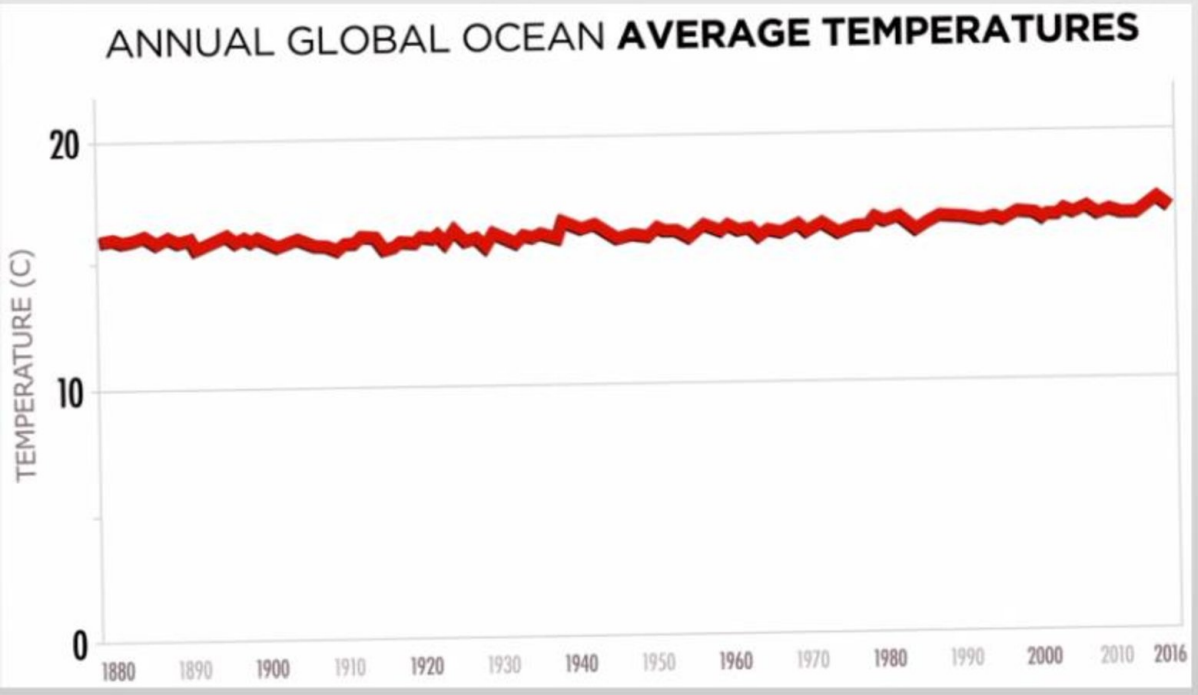
JOB LOSS BY QUARTER
7 MIL
9 MIL
13.5 MIL
15 MIL
DEC '07
SEPT '08
MAR '09
JUN '10
6
15

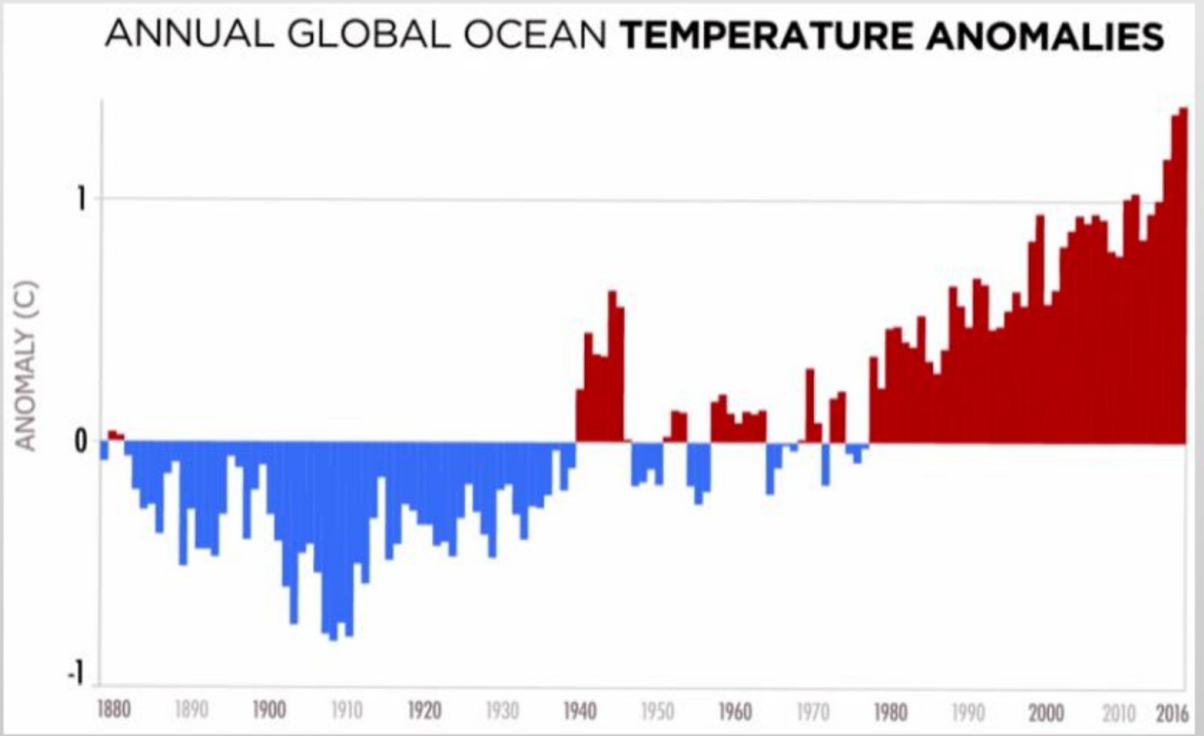
JOB LOSS BY QUARTER
7 MIL
15 MIL
DEC '07
SEPT '08
MAR '09
JUN '10

EP performance crunch



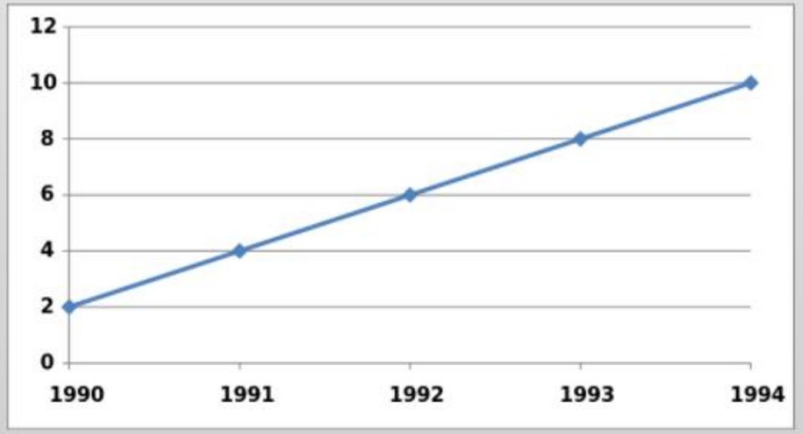
EP performance crunch

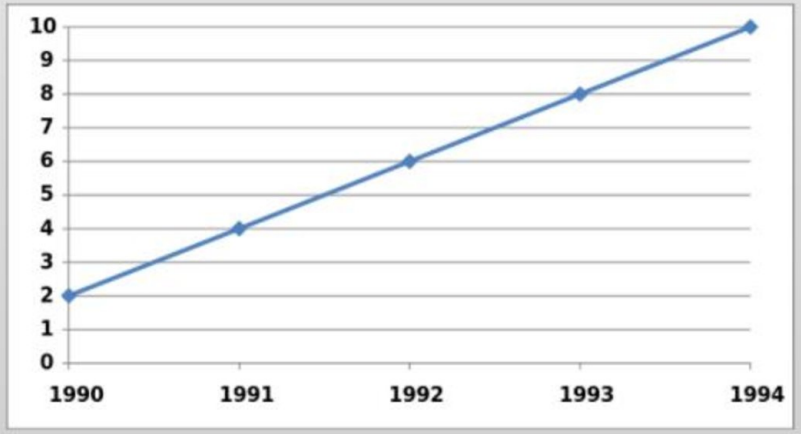


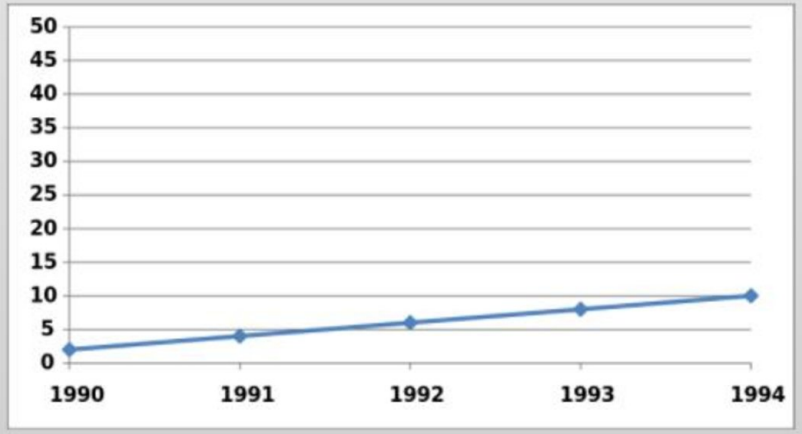


SYSTEMS LABORATORY

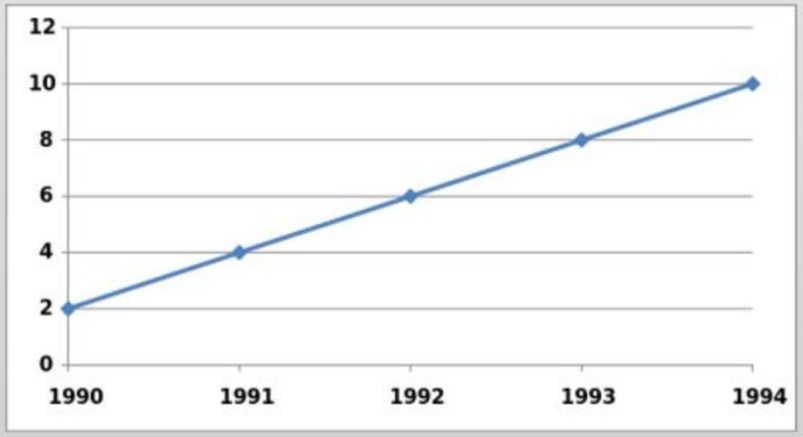
EP performance crunch

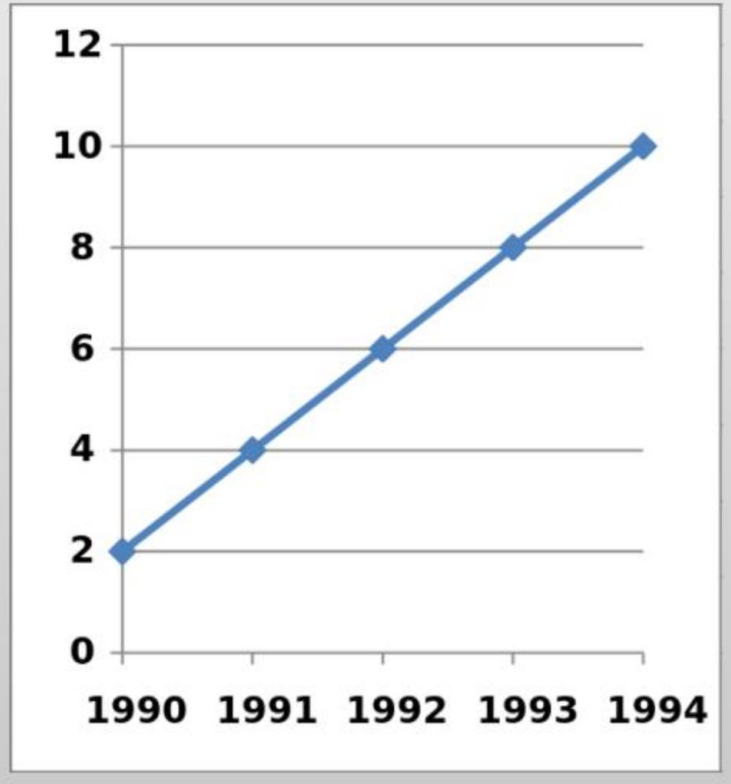


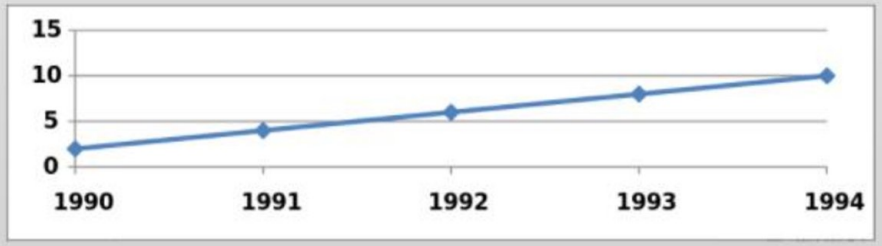




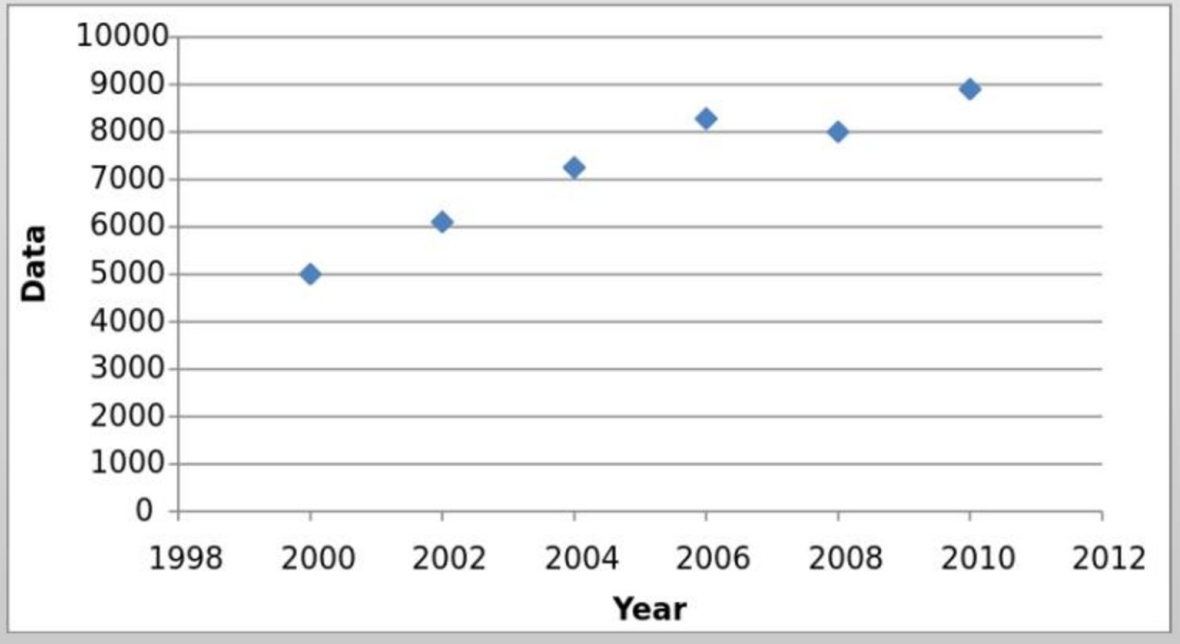
EP
performance crunch

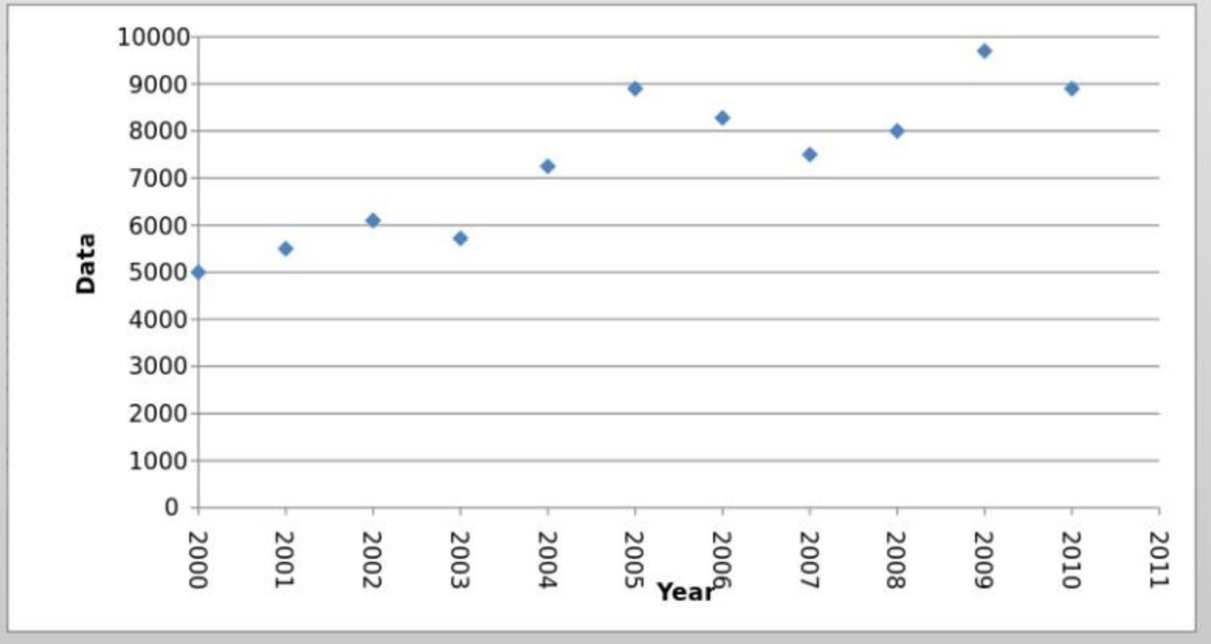




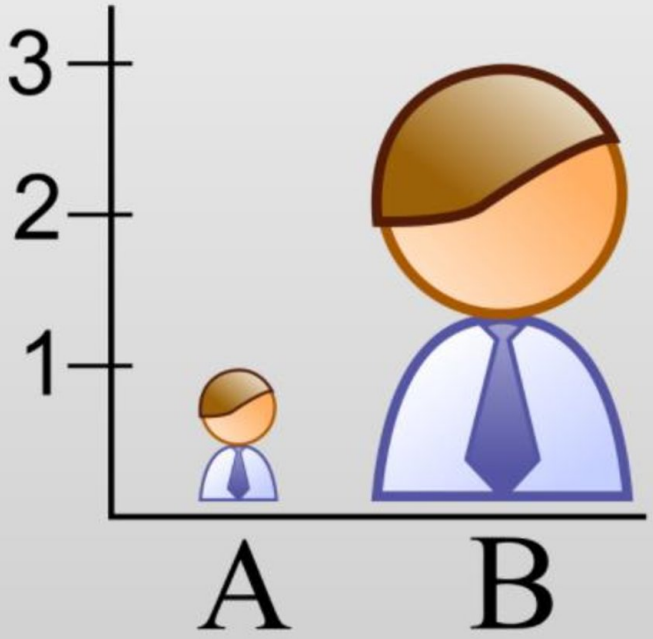


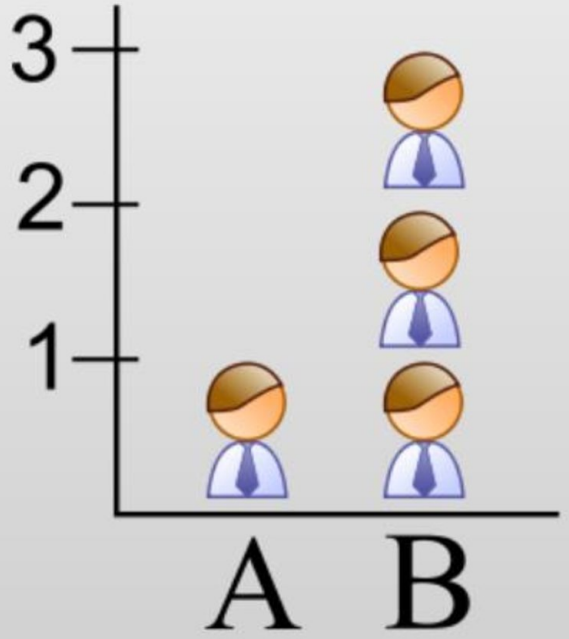
EP performance crunch

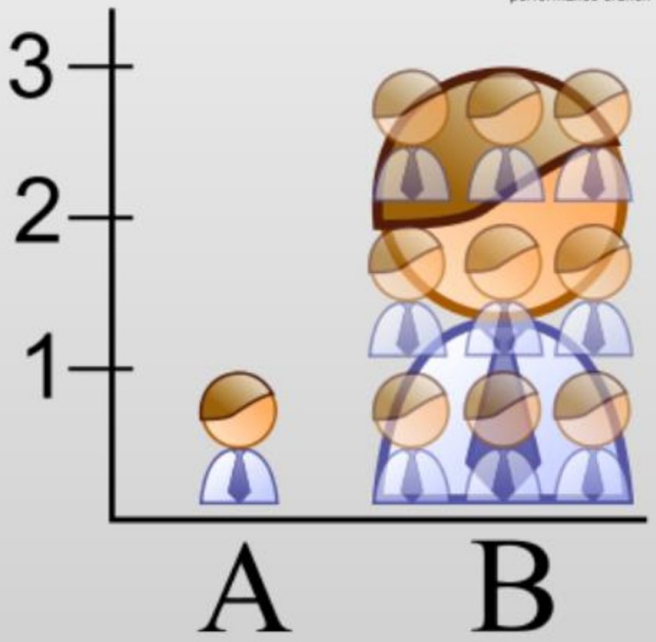


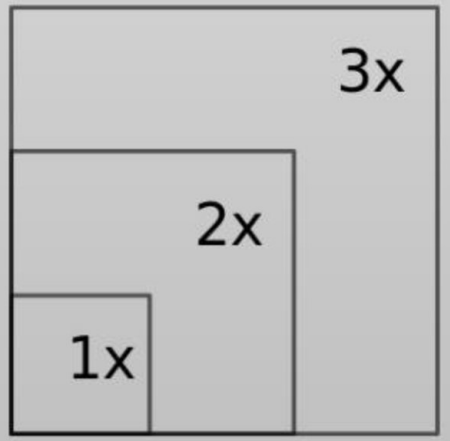


EP
performance crunch









SYSTEMS LABORATORY

Overview

EP performance crunch

- Importance
- Guidelines / best practices

- Matplotlib
  - Why
  - Intro
  - More details

SYSTEMS LABORATORY

Plotting tools

EP performance crunch

What is a plotting tool?

A plotting **tool** is computer software which helps to **analyze** and **visualize** data, often of a scientific nature.
Using this type of software, users can generate plots of **functions**, **data** and **data fits**.
Software of this nature typically includes additional functionality, such as **data analysis functions** including curve fitting.

SYSTEMS LABORATORY

Plotting tools

EP performance crunch

Many

1. Online / Offline
2. Operating systems (including mobile OSs)
3. Open source / Closed source
4. Free / Paid

Over 100 alternatives here -

https://en.wikipedia.org/wiki/List_of_information_graphics_software

SYSTEMS LABORATORY

Plotting tools

EP performance crunch

Free, linux-based:

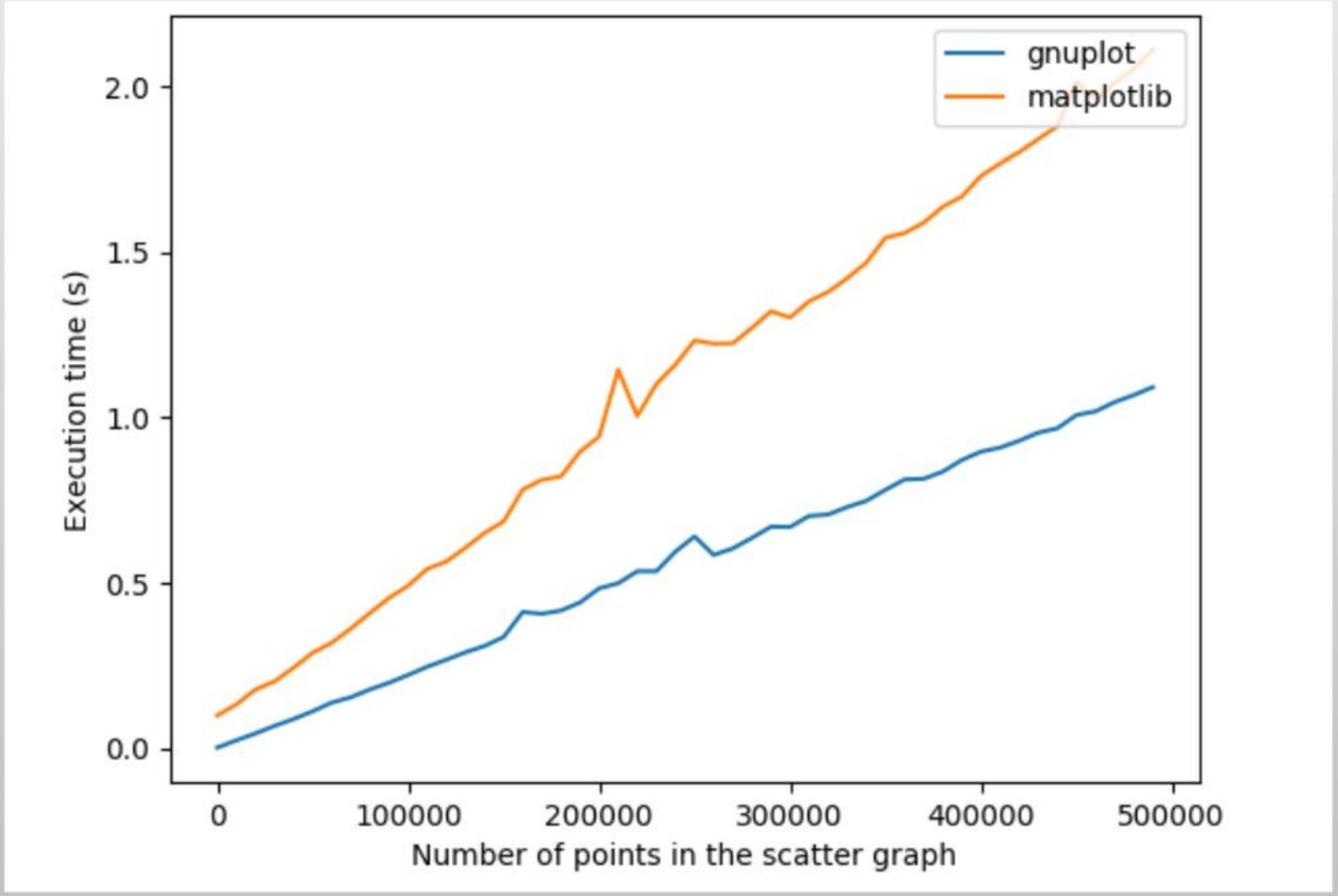
- **gnuplot** - Command-line driven interactive plotting program
- **Ctioga** - Command-line plotting utility
- **LabPlot** - Data plotting and function analysis tool for KDE
- **matplotlib** - Python based plotting system in a style similar to Matlab
- **QtiPlot** - Fully fledged plotting software
- **ROOT** - Solves the data analysis challenges of high-energy physics
- **SCaVis** - Produces professional graphics and has many libraries for data manipulation
- **GLE** - Graphics Layout Engine
- **PLplot** - Scientific plotting library
- **RLPlot** - Generate publication quality graphs
- **Genius** - Educational tool in mathematics and for research

* http://www.linuxlinks.com

SYSTEMS LABORATORY

Gnuplot vs Matplotlib

EP performance crunch



https://stackoverflow.com/questions/911655/gnuplot-vs-matplotlib

SYSTEMS LABORATORY

Matplotlib

EP performance crunch

Price	- Free to download
Language	- Python
License	- Open source
Developer	- NumFOCUS - nonprofit organisation in the US
Website	- https://matplotlib.org
Support	- Documentation

Gnuplot

- A comprehensive library for creating static, animated, and interactive visualizations in Python

- Large user base

  > great documentation

  > actively maintained

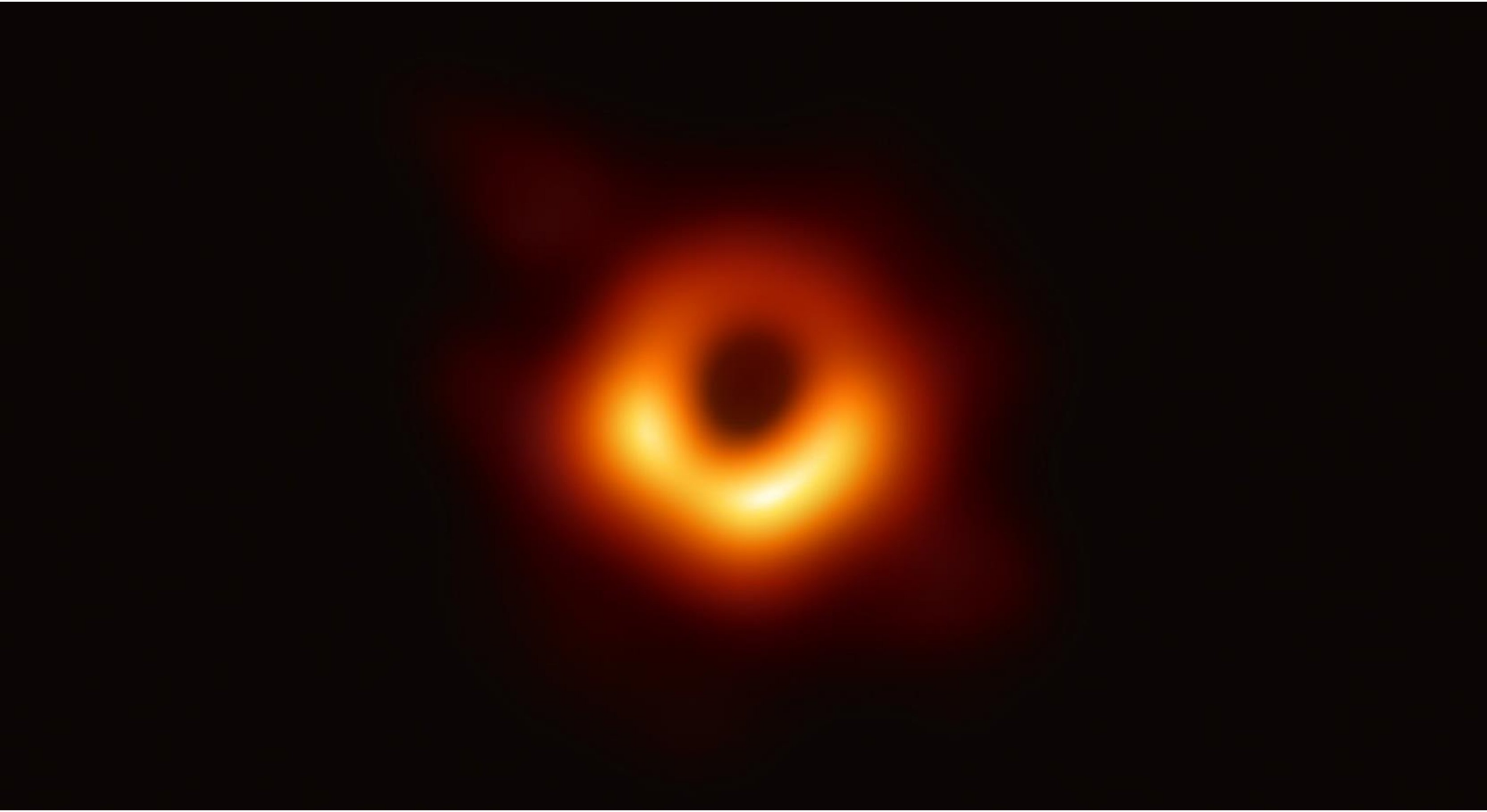
  > gained popularity in recent years

SYSTEMS LABORATORY

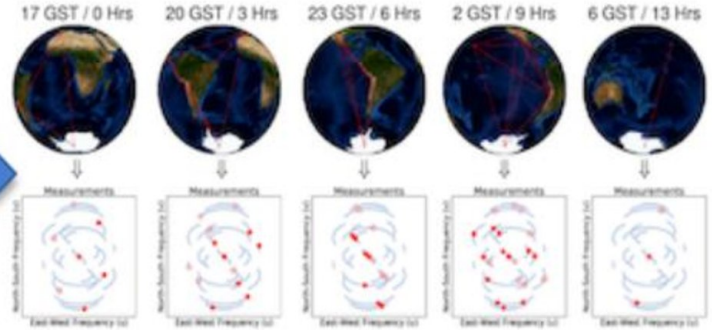
Matplotlib

EP performance crunch

- MATLAB-like interface
- Low level plotting backend for virtually all plotting tools in python
  - Seaborn, pandas plotting etc.
- De-facto standard for data visualizations in data science / machine learning communities
- Interactive (ipython) or in python scripts
- 2D, 3D
- Control over all visual aspects of a plot
- Output file formats
  - PNG, EPS, SVG, JPEG, …



Event Horizon Telescope



Petabytes of data (measurements)

Data correlation, calibration, reduction pipelines

Scientific, Astrophysics & Computing Algorithms

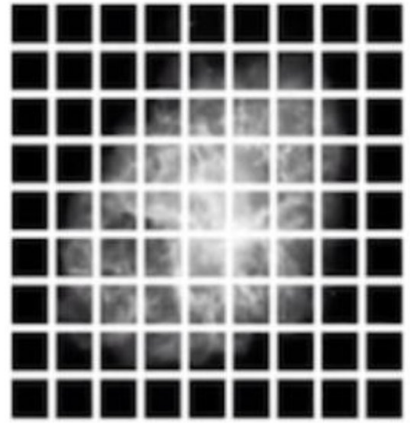


Image analysis

https://github.com/achael/eht-imaging/

Imaging, analysis and simulation software for radio interferometry

M87 – the first image of a black hole

Software: DiFX (Deller et al. 2011), CALC, PolConvert (Martí-Vidal et al. 2016), HOPS (Whitney et al. 2004), CASA (McMullin et al. 2007), AIPS (Greisen 2003), ParselTongue (Kettenis et al. 2006), GNU Parallel (Tange 2011), GILDAS, eht-imaging (Chael et al. 2016, 2018), Numpy (van der Walt et al. 2011), Scipy (Jones et al. 2001), Pandas (McKinney 2010), Astropy (The Astropy Collaboration et al. 2013, 2018), Jupyter (Kluyver et al. 2016), Matplotlib (Hunter 2007).

NumPy was one of the software used!

https://numpy.org/case-studies/blackhole-image/

matplotlib

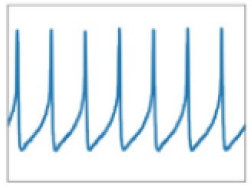
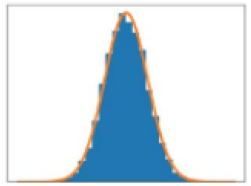
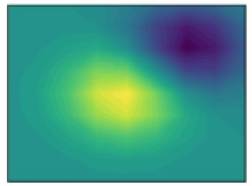
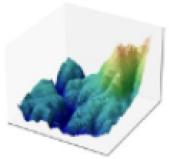
Version 3.3.2

Fork me on GitHub

Installation | Documentation | Examples | Tutorials | Contributing | Search

Matplotlib: Visualization with Python

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.

Matplotlib makes easy things easy and hard things possible.

Create

- Develop publication quality plots with just a few lines of code
- Use interactive figures that can zoom, pan, update...

Customize

- Take full control of line styles, font properties, axes properties...
- Export and embed to a number of file formats and interactive environments

Extend

- Explore tailored functionality provided by third party packages
- Learn more about Matplotlib through the many external learning resources

Latest release
3.3.2: docs | changelog

Last release for Python 2
2.2.5: docs | changelog

Development version
docs

Support Matplotlib

Documentation

SYSTEMS LABORATORY

Python compute ecosystem

EP performance crunch

SYSTEMS LABORATORY

Starting up

EP performance crunch

Install matplotlib - `$ pip install matplotlib`

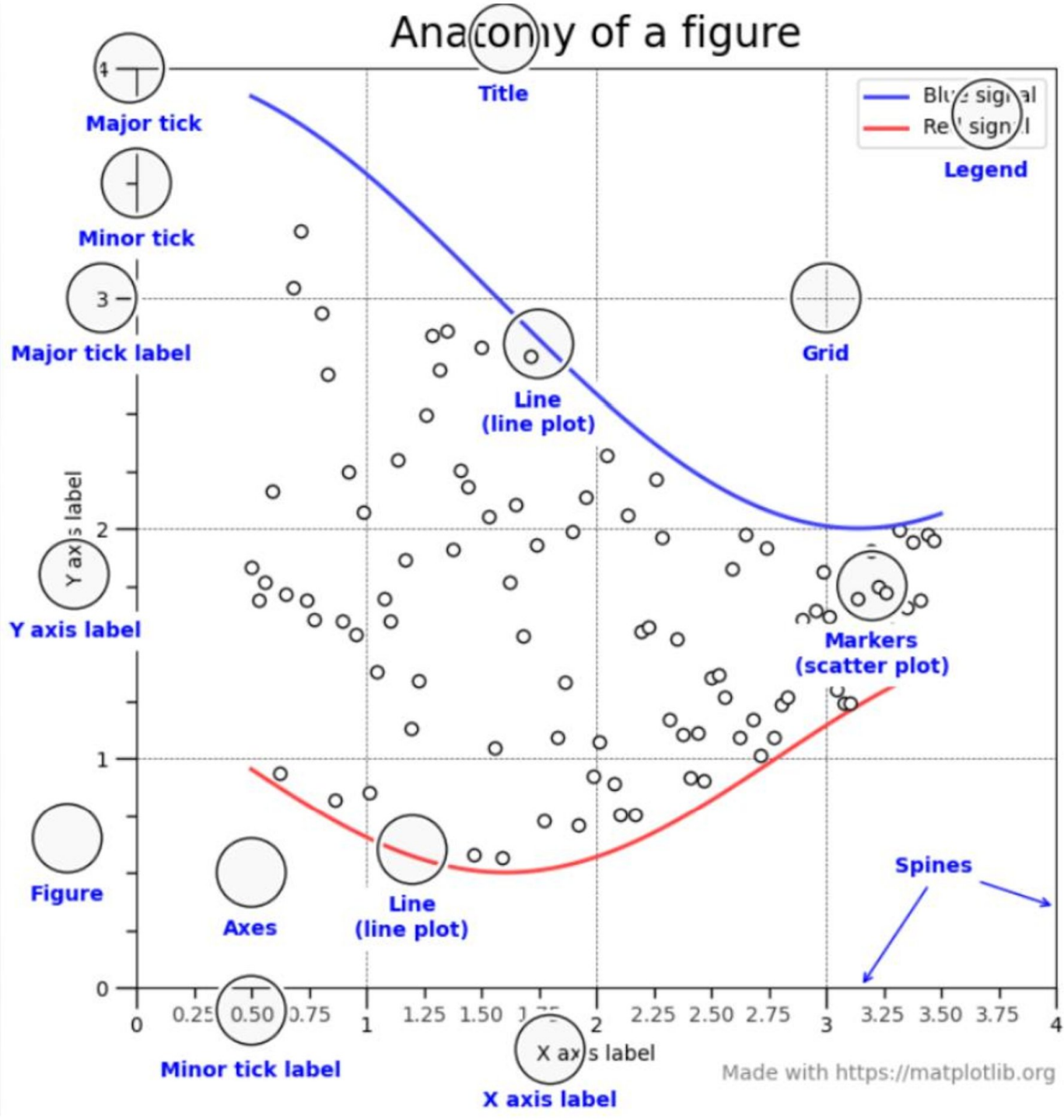
Import matplotlib - `>> import matplotlib.pyplot as plt`

```
(tfgpu) ~ $ ipython
Python 3.7.7 (default, May  7 2020, 21:25:33)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.11.1 -- An enhanced Interactive Python. Type '?' for help.

In [1]: import matplotlib.pyplot as plt

In [2]: plt.plot(...)
```

SYSTEMS LABORATORY

Anatomy of a figure

1. **All** aspects customizable
2. Axes
   a. Ranges, major / minor ticks, labels
3. Title
4. Markers
   a. Size, color, shape
5. Lines
   a. Thickness, colors, types
6. Grid
7. Legend



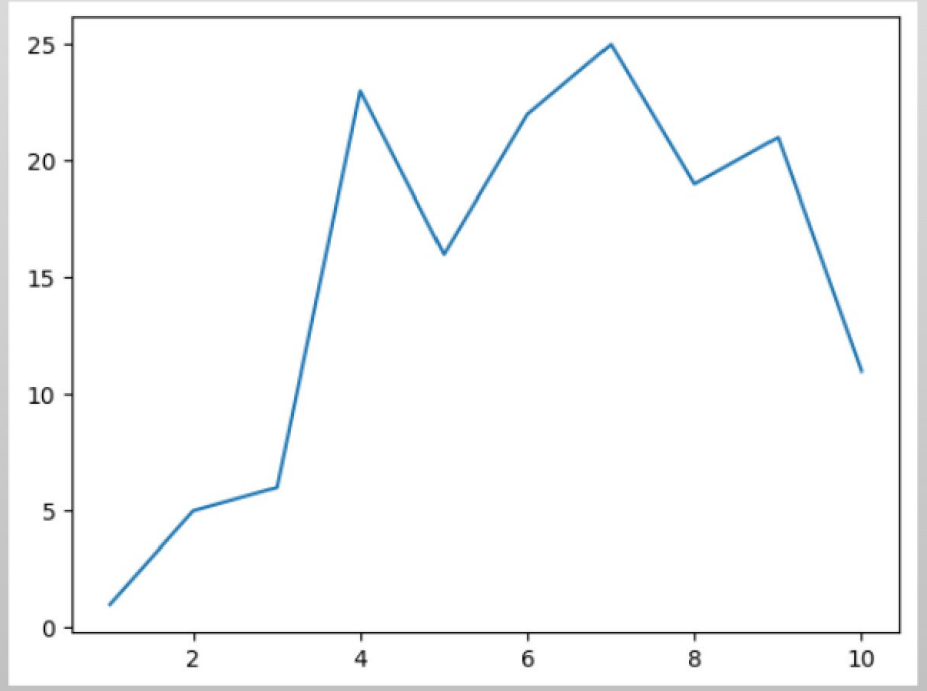
SYSTEMS LABORATORY

Plotting data from a file

EP performance crunch

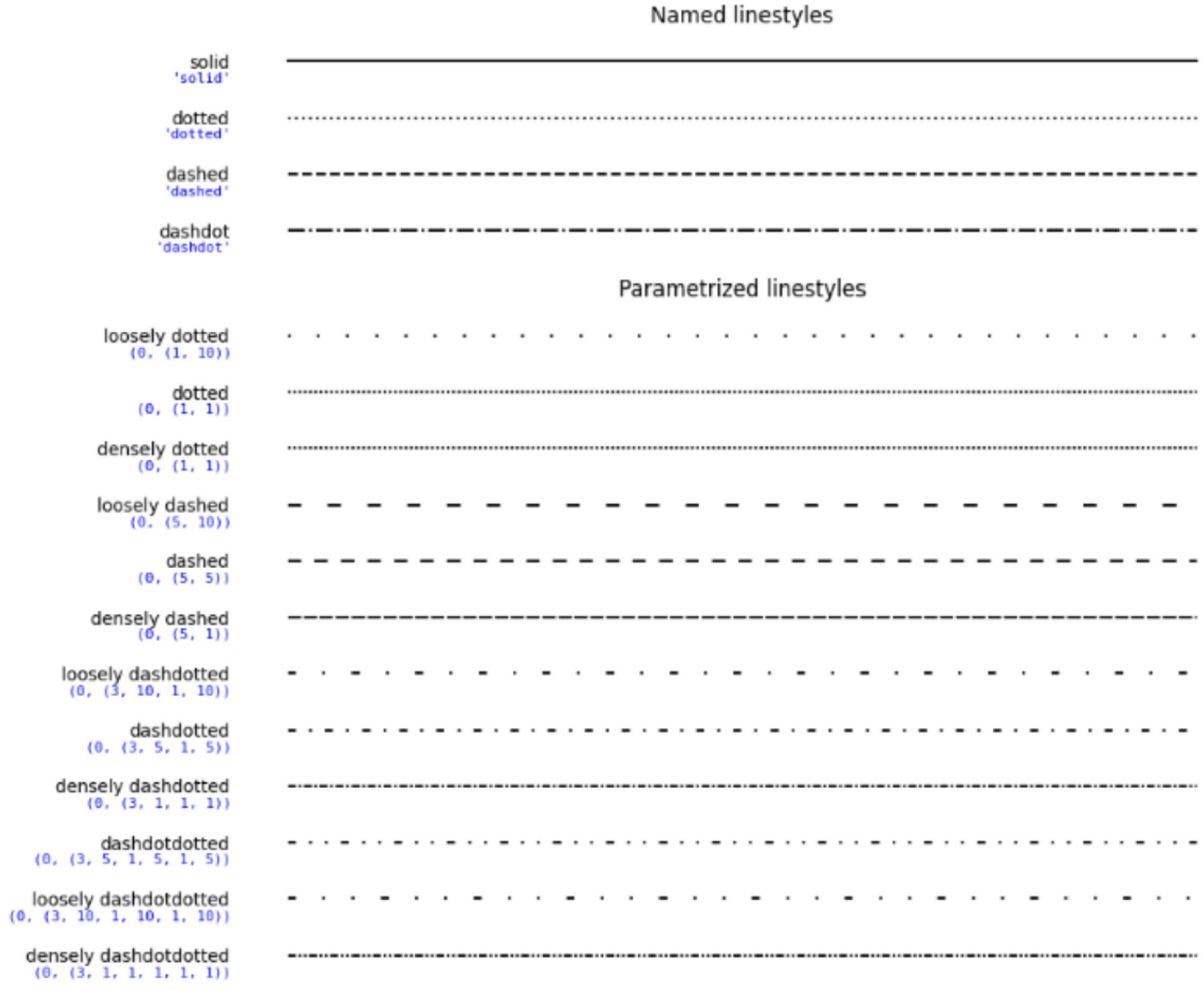
```
>> import matplotlib.pyplot as plt
>> import numpy as np
>> data = np.genfromtxt("file.csv", delimiter=",", names=["x", "y"])
>> plt.plot(data['x'], data['y'])
```

```
(tfgpu) ep $ cat file.csv
1,1
2,5
3,6
4,23
5,16
6,22
7,25
8,19
9,21
10,11
(tfgpu) ep $
```



SYSTEMS LABORATORY

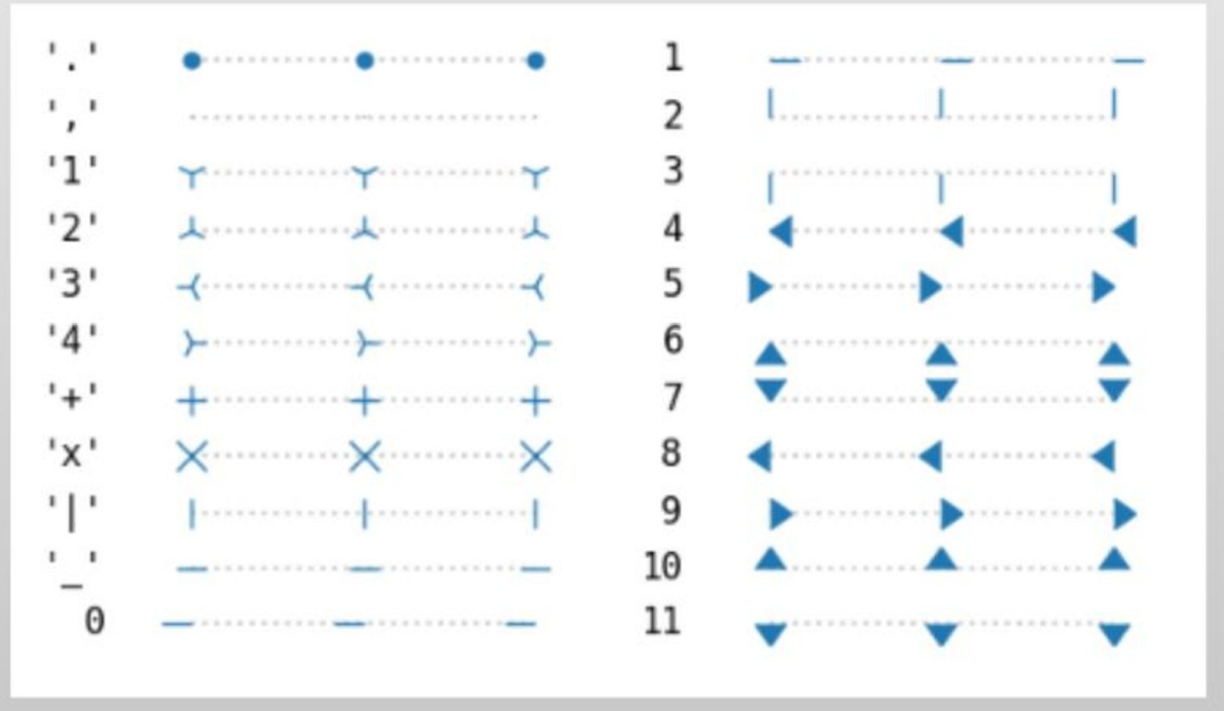
Linestyles

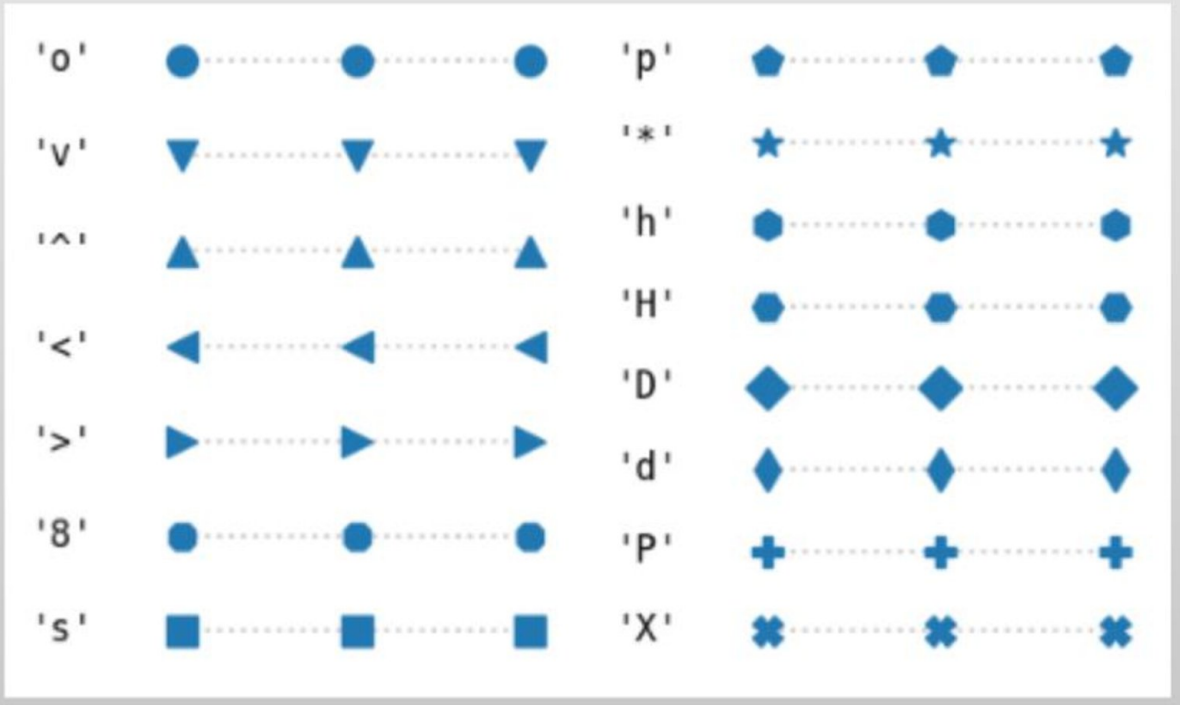


SYSTEMS LABORATORY

Markers

EP performance crunch



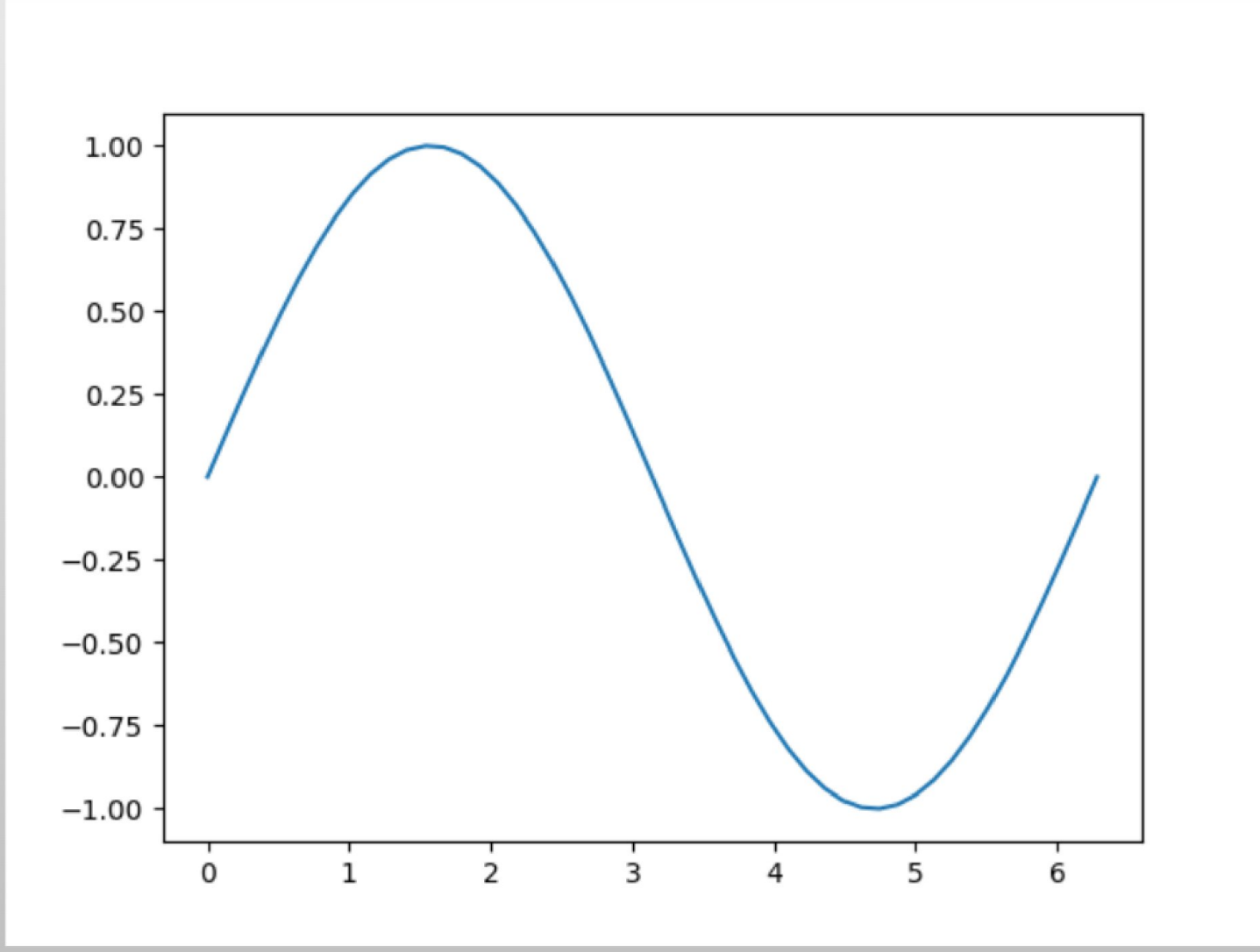


SYSTEMS LABORATORY

Matplotlib showcase

EP performance crunch

```python
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi)
y = np.sin(x)

plt.plot(x, y)
plt.show()
```

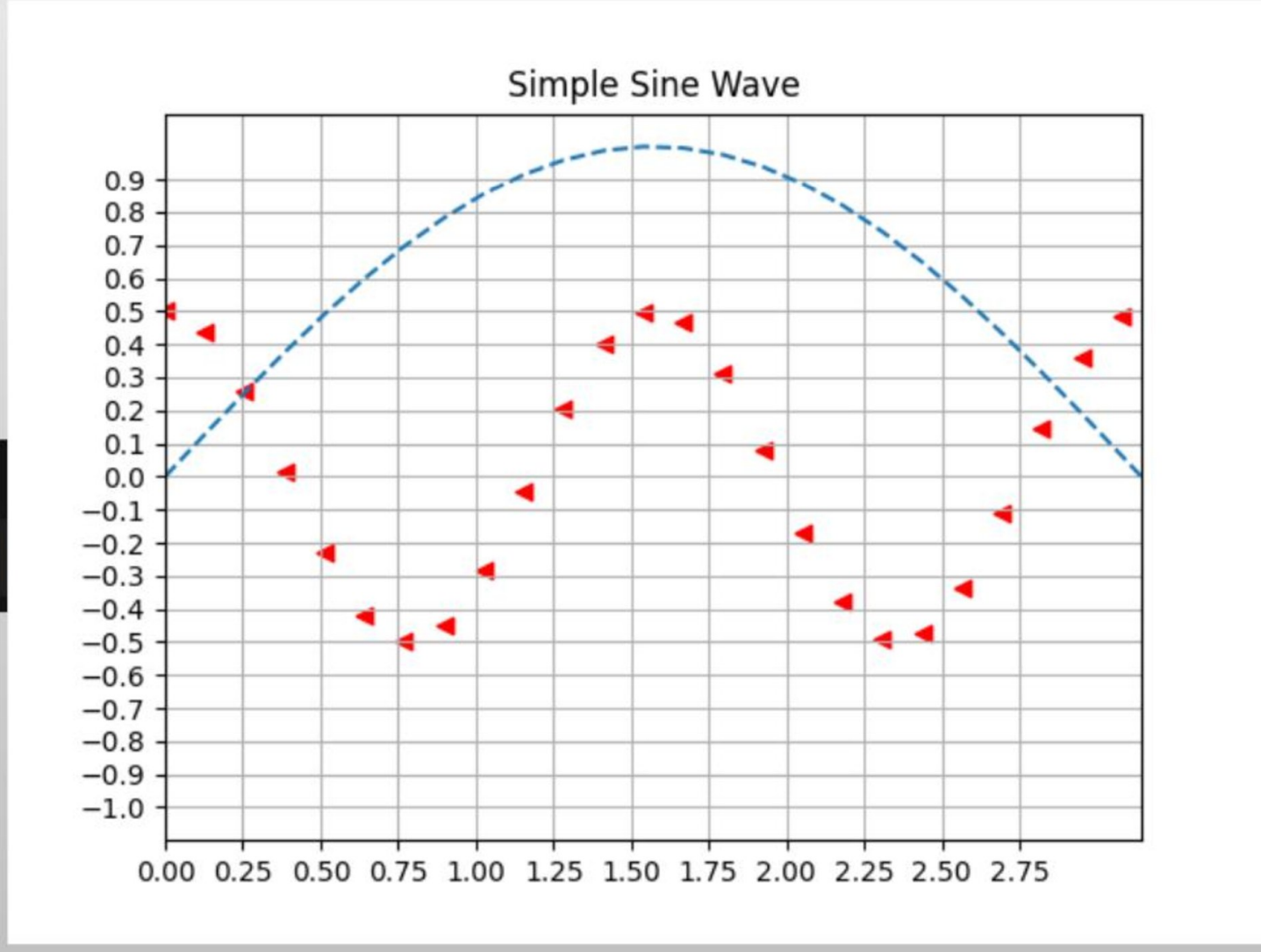


SYSTEMS LABORATORY

Matplotlib showcase

EP performance crunch

```
plt.plot(x, y, linestyle = '--')

plt.title('Simple Sine Wave')

plt.xlim(0, np.pi)

y2 = np.cos(2*x) ** 2 - 0.5
plt.scatter(x, y2, marker='<', c = 'r')

plt.xticks(np.arange(0, 3, 0.25))
plt.yticks(np.arange(-1, 1, 0.1))

plt.grid()
```



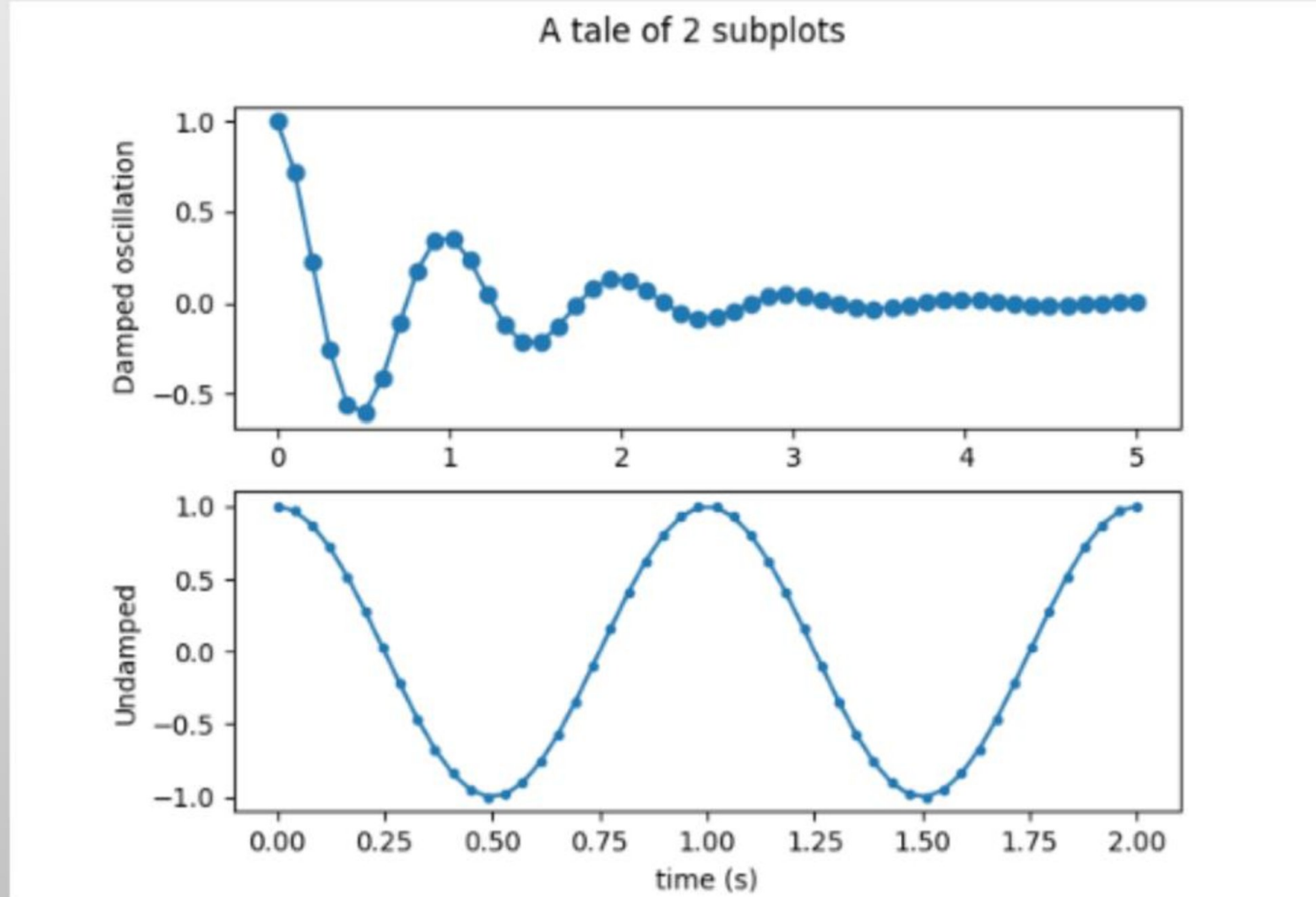
SYSTEMS LABORATORY

Subplots

EP performance crunch

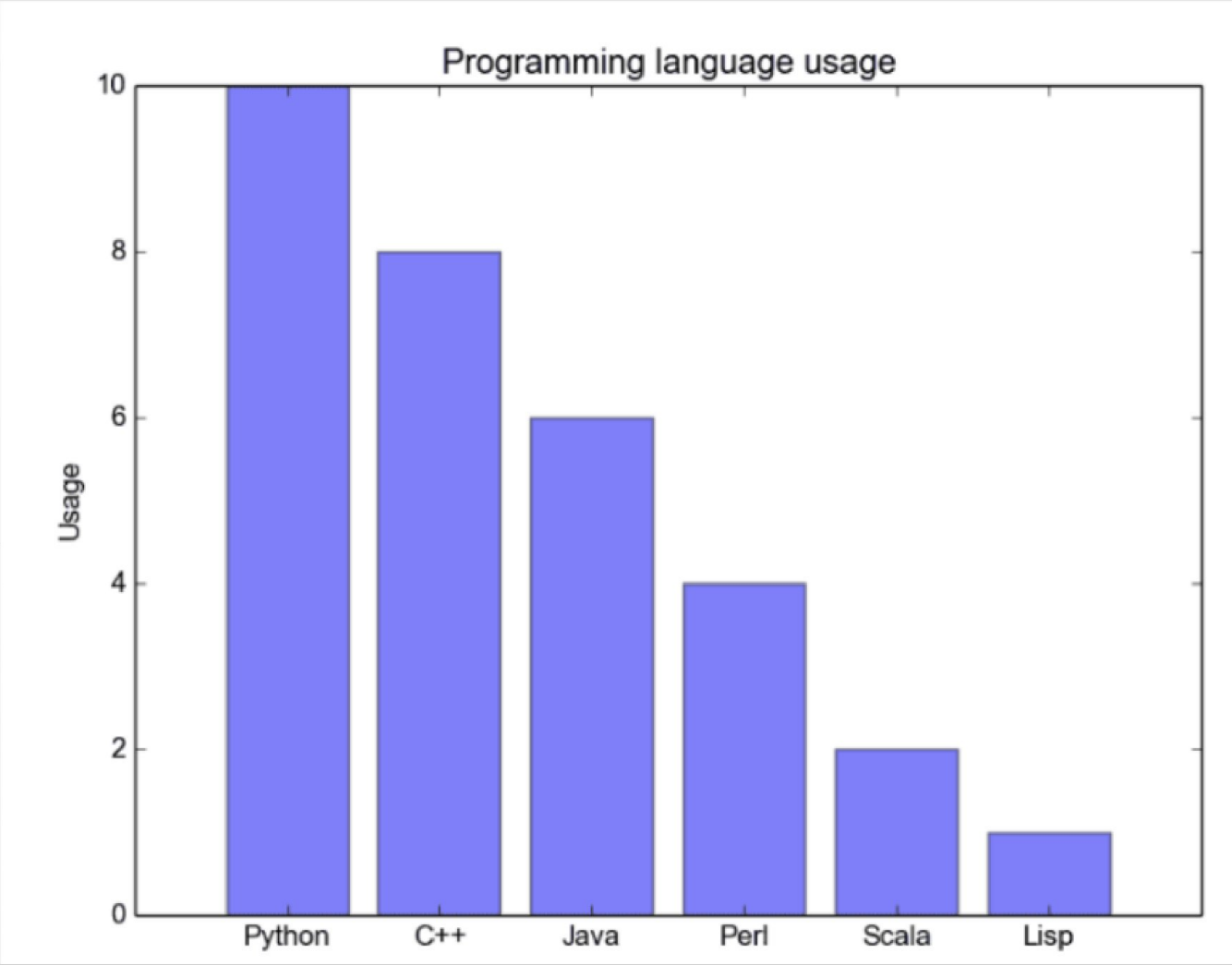
```
import numpy as np
import matplotlib.pyplot as plt
```

```
x1 = np.linspace(0.0, 5.0)
x2 = np.linspace(0.0, 2.0)

y1 = np.cos(2 * np.pi * x1) * np.exp(-x1)
y2 = np.cos(2 * np.pi * x2)

fig, (ax1, ax2) = plt.subplots(2, 1)
fig.suptitle('A tale of 2 subplots')

ax1.plot(x1, y1, 'o-')
ax1.set_ylabel('Damped oscillation')

ax2.plot(x2, y2, '.-')
ax2.set_xlabel('time (s)')
ax2.set_ylabel('Undamped')

plt.show()
```



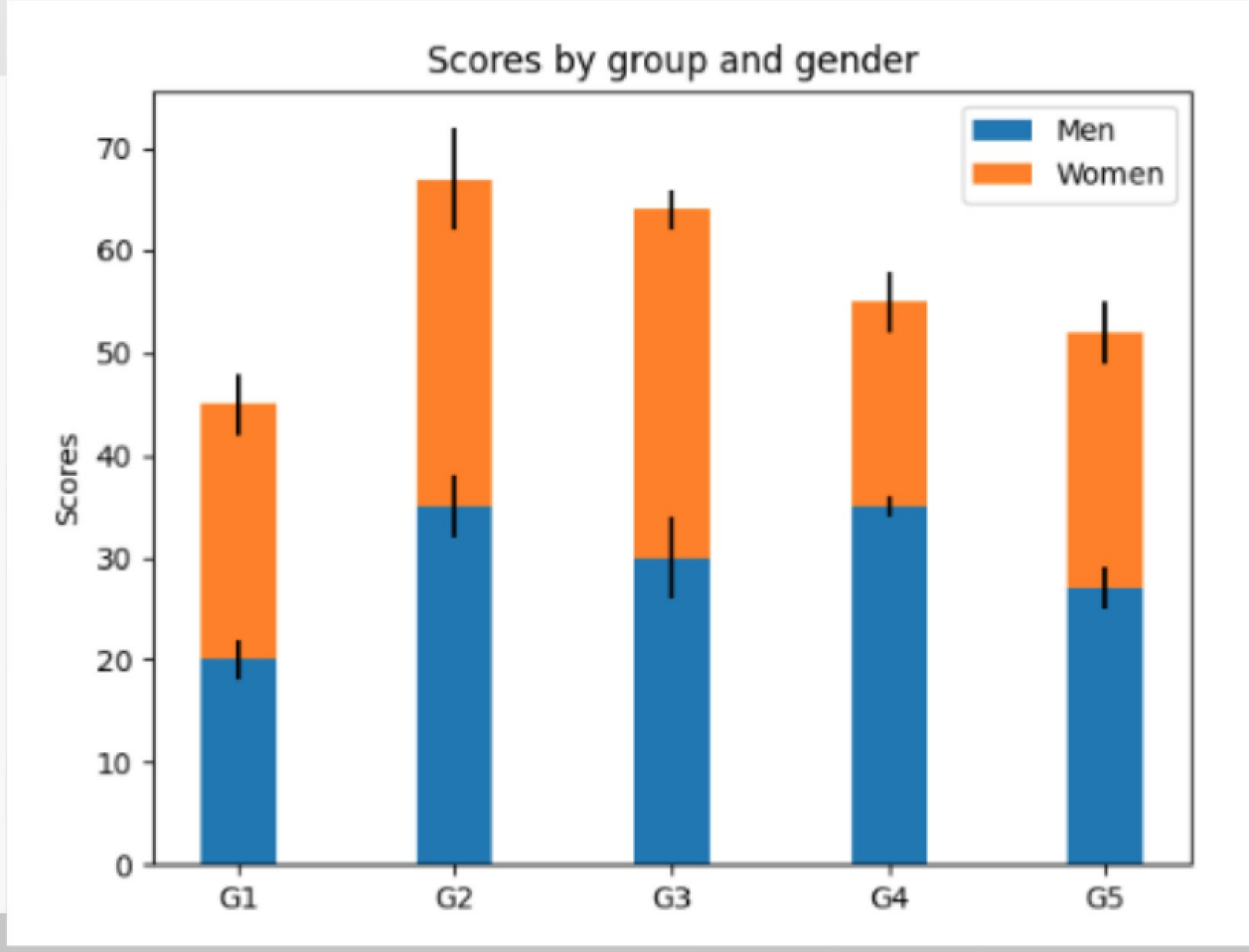
Bar chart


```
import matplotlib.pyplot as plt; plt.rcdefaults()
import numpy as np
import matplotlib.pyplot as plt

objects = ('Python', 'C++', 'Java', 'Perl', 'Scala', 'Lisp')
y_pos = np.arange(len(objects))
performance = [10,8,6,4,2,1]

plt.bar(y_pos, performance, align='center', alpha=0.5)
plt.xticks(y_pos, objects)
plt.ylabel('Usage')
plt.title('Programming language usage')

plt.show()
```



Stacked bar chart

```python
import matplotlib.pyplot as plt


labels = ['G1', 'G2', 'G3', 'G4', 'G5']
men_means = [20, 35, 30, 35, 27]
women_means = [25, 32, 34, 20, 25]
men_std = [2, 3, 4, 1, 2]
women_std = [3, 5, 2, 3, 3]
width = 0.35       # the width of the bars: can also be len(x) sequence

fig, ax = plt.subplots()

ax.bar(labels, men_means, width, yerr=men_std, label='Men')
ax.bar(labels, women_means, width, yerr=women_std, bottom=men_means,
       label='Women')

ax.set_ylabel('Scores')
ax.set_title('Scores by group and gender')
ax.legend()

plt.show()
```



SYSTEMS LABORATORY

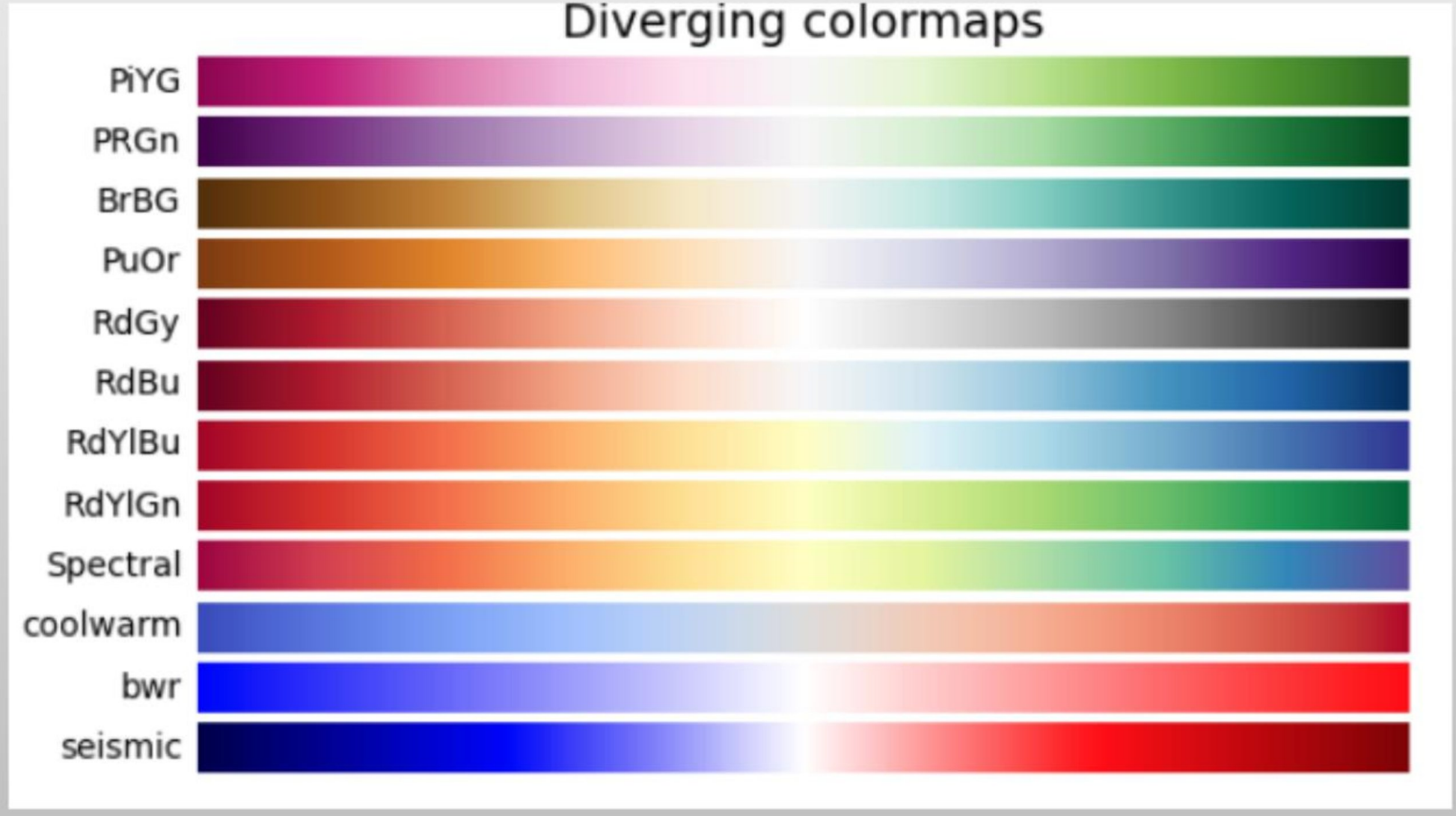
Colormaps

EP performance crunch

SYSTEMS LABORATORY

Colormaps

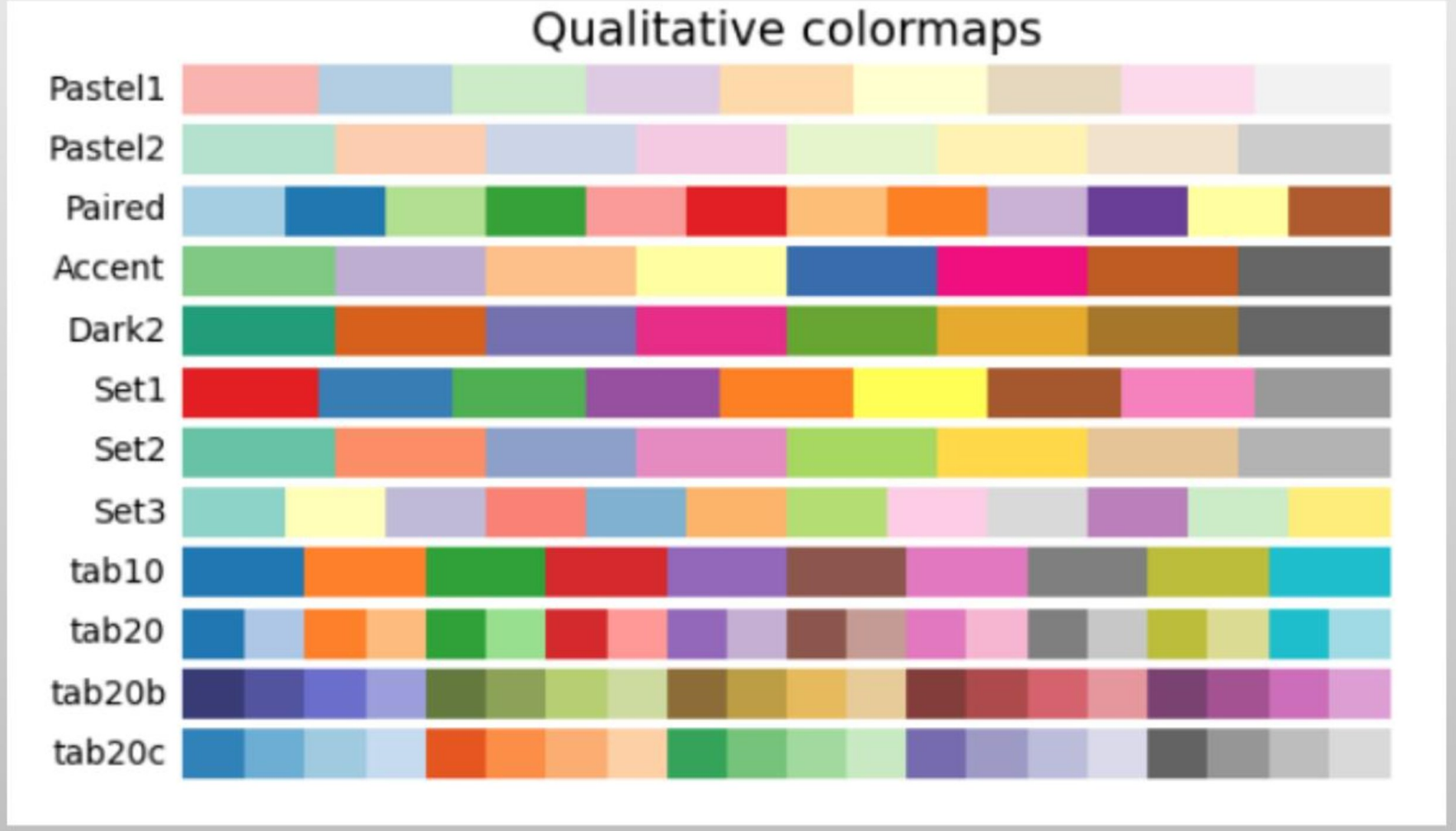
EP performance crunch



SYSTEMS LABORATORY

Colormaps

EP performance crunch

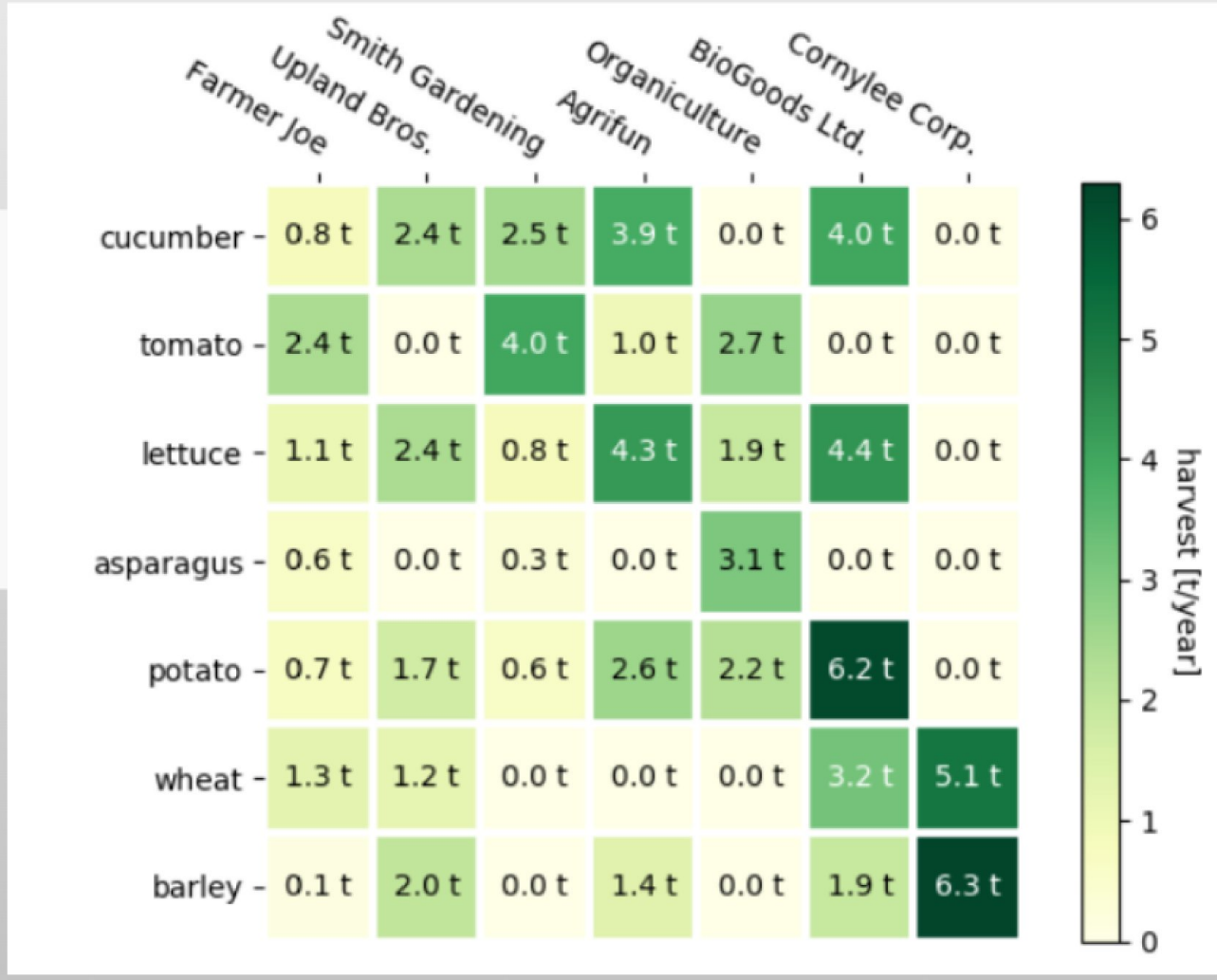


SYSTEMS LABORATORY

Colormaps

EP performance crunch

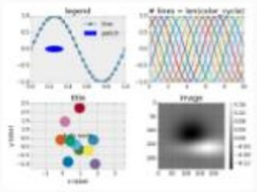
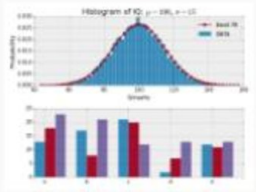
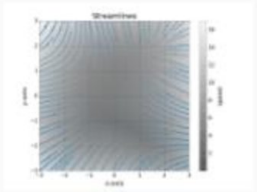
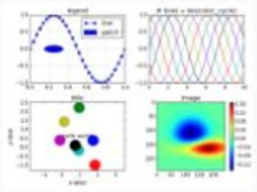
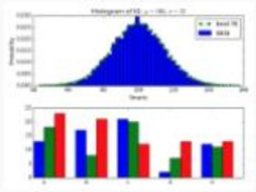
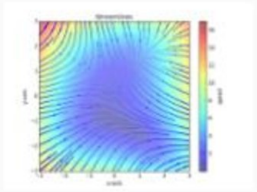
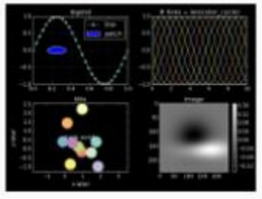
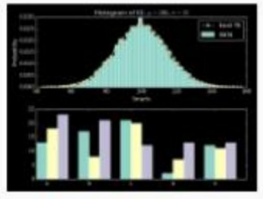
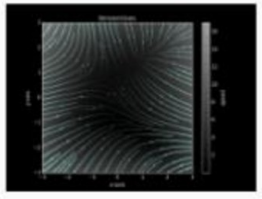
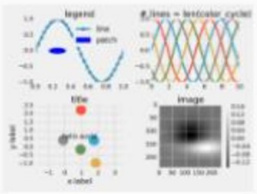
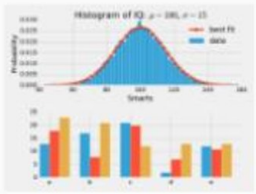
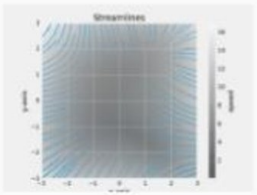
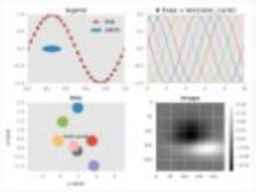
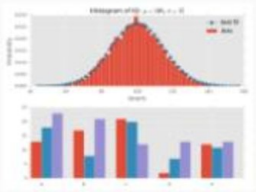
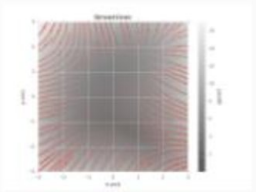
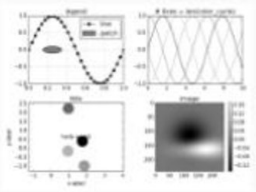
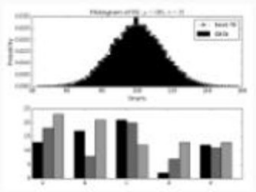
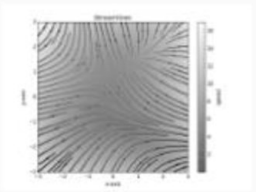
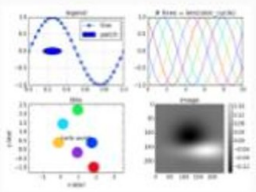
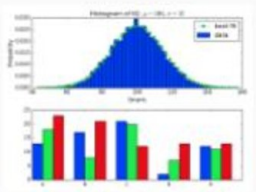
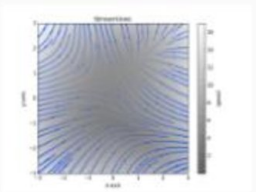
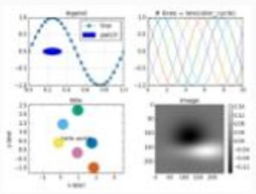
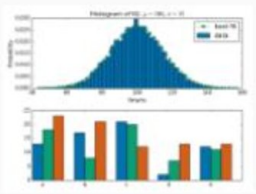
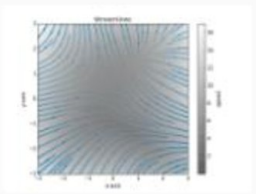
```
fig, ax = plt.subplots()

im, cbar = heatmap(harvest, vegetables, farmers, ax=ax,
                   cmap="YlGn", cbarlabel="harvest [t/year]")
texts = annotate_heatmap(im, valfmt="{x:.1f} t")

fig.tight_layout()
plt.show()
```



SYSTEMS LABORATORY

Styles

EP performance crunch

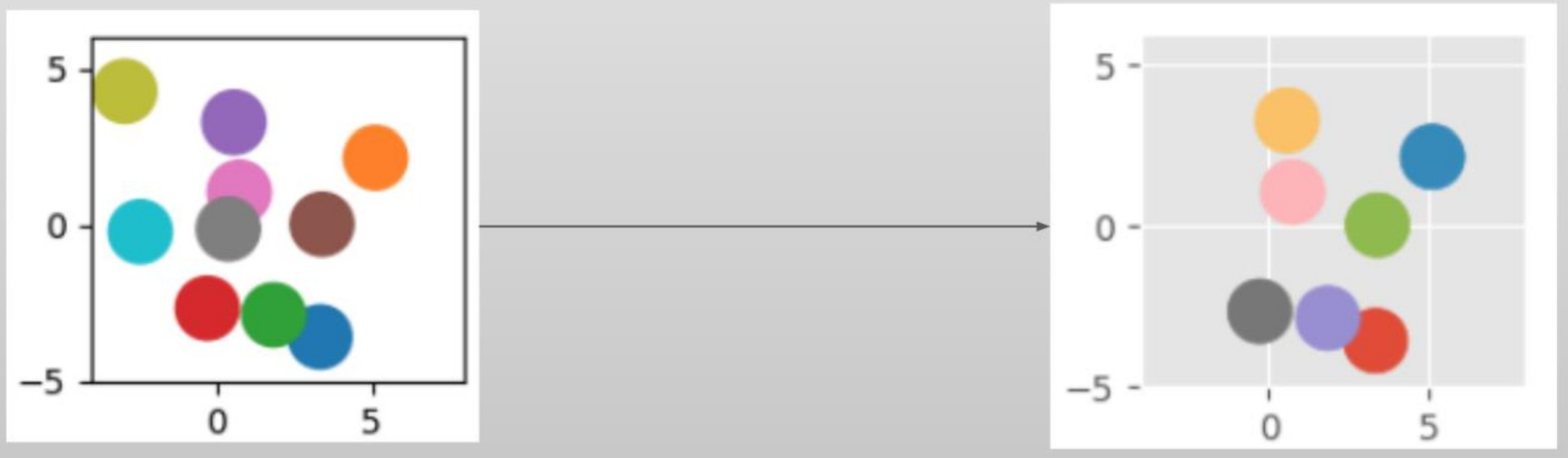
bmh			
classic			
dark_background			
fivethirtyeight			
ggplot			
grayscale			
seaborn-bright			
seaborn-colorblind			

SYSTEMS LABORATORY

Styles

EP performance crunch

```
>> plt.style.use('ggplot')
```

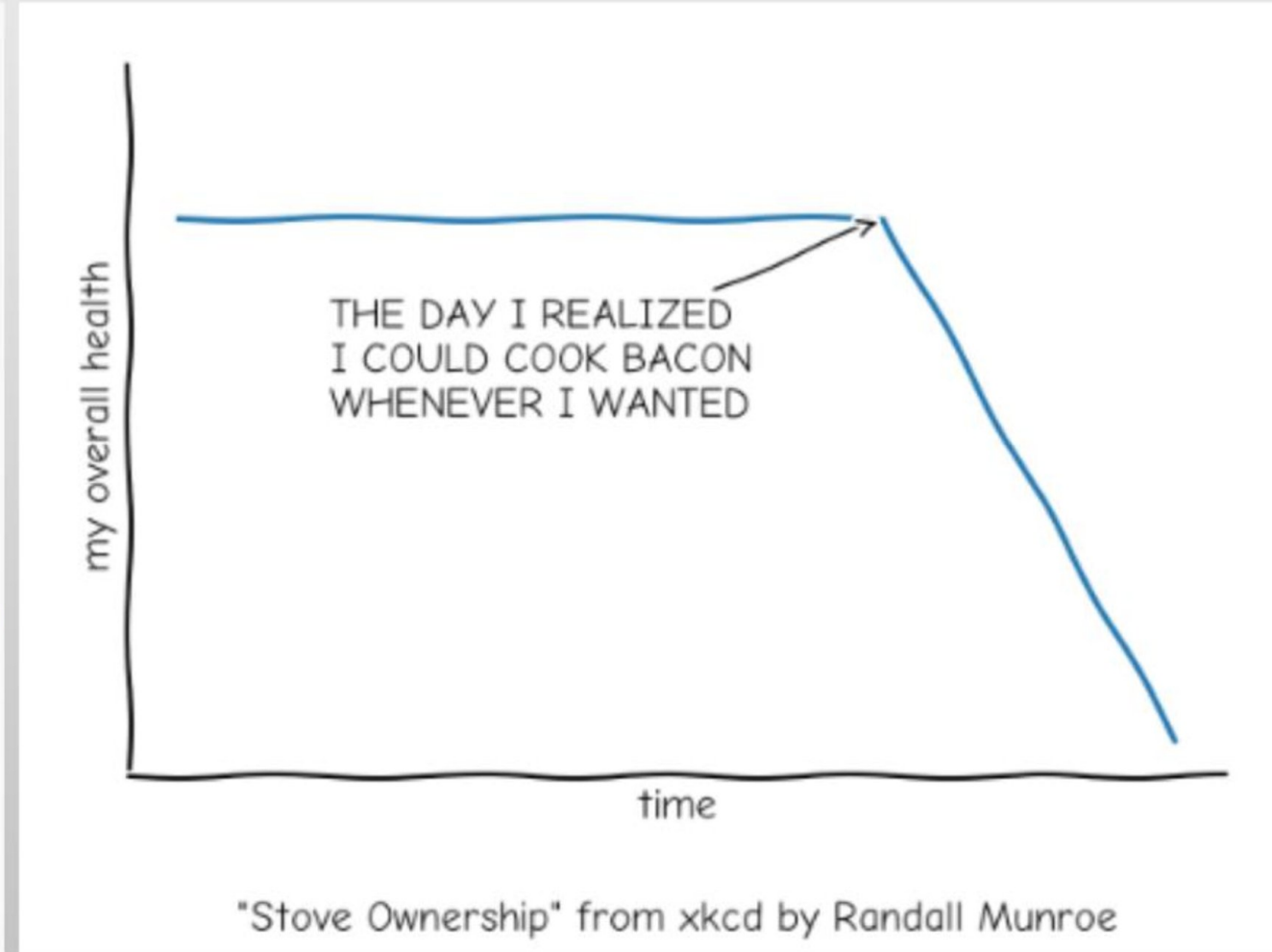


SYSTEMS LABORATORY

Xkcd style

EP performance crunch

```
with plt.xkcd():
    # Based on "Stove Ownership" from XKCD by Randall Munroe
    # https://xkcd.com/418/

    fig = plt.figure()
    ax = fig.add_axes((0.1, 0.2, 0.8, 0.7))
    ax.spines['right'].set_color('none')
    ax.spines['top'].set_color('none')
    ax.set_xticks([])
    ax.set_yticks([])
    ax.set_ylim([-30, 10])

    data = np.ones(100)
    data[70:] -= np.arange(30)

    ax.annotate(
        'THE DAY I REALIZED\nI COULD COOK BACON\nWHENEVER I WANTED',
        xy=(70, 1), arrowprops=dict(arrowstyle='->'), xytext=(15, -10))

    ax.plot(data)

    ax.set_xlabel('time')
    ax.set_ylabel('my overall health')
    fig.text(
        0.5, 0.05,
        '"Stove Ownership" from xkcd by Randall Munroe',
        ha='center')
```



SYSTEMS LABORATORY

EP performance crunch

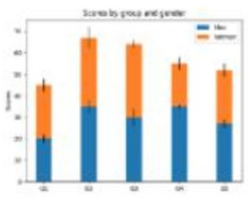
Some cool plots

Gallery

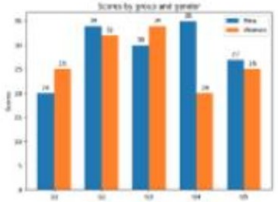
This gallery contains examples of the many things you can do with Matplotlib. Click on any image to see the full image and source code.

For longer tutorials, see our tutorials page. You can also find external resources and a FAQ in our user guide.

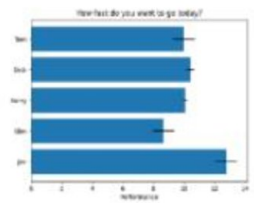
Lines, bars and markers



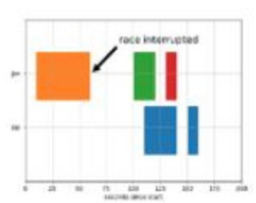
Stacked bar chart



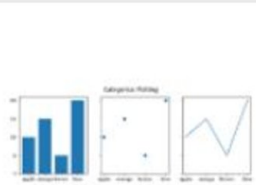
Grouped bar chart with labels

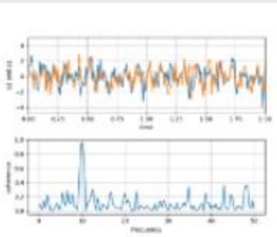


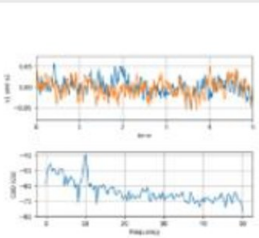
Horizontal bar chart



Broken Barh







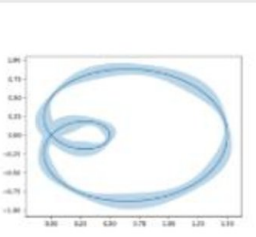


Table of Contents

Gallery

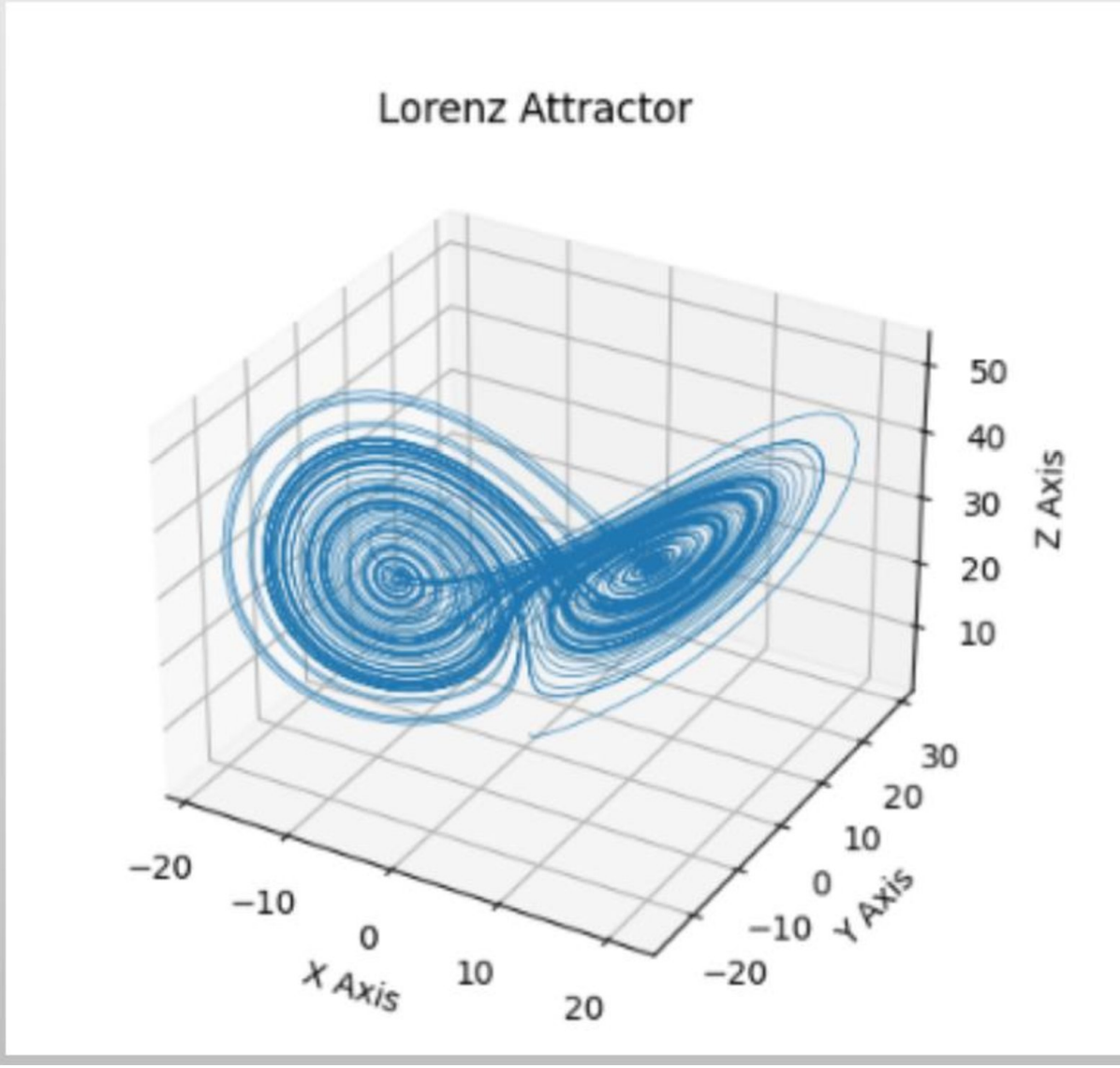
- Lines, bars and markers
- Images, contours and fields
- Subplots, axes and figures
- Statistics
- Pie and polar charts
- Text, labels and annotations
- Pyplot
- Color
- Shapes and collections
- Style sheets
- Axes Grid
- Axis Artist
- Showcase
- Animation
- Event handling
- Front Page
- Miscellaneous
- 3D plotting
- Our Favorite Recipes
- Scales

SYSTEMS LABORATORY

Lorenz Attractor

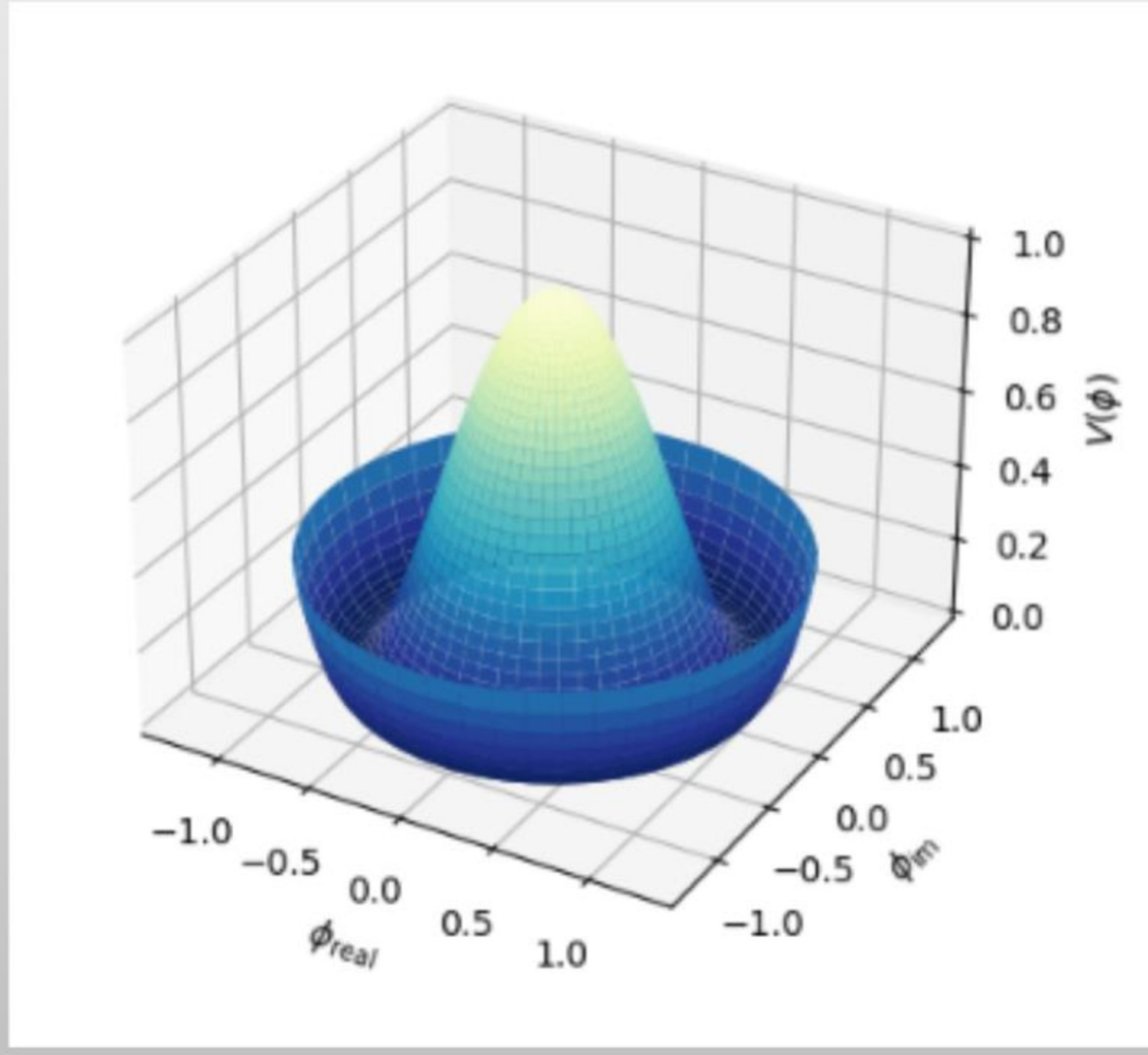
EP performance crunch

```python
def lorenz(x, y, z, s=10, r=28, b=2.667):
    """
    Given:
       x, y, z: a point of interest in three dimensional space
       s, r, b: parameters defining the lorenz attractor
    Returns:
       x_dot, y_dot, z_dot: values of the lorenz attractor's partial
           derivatives at the point x, y, z
    """
    x_dot = s*(y - x)
    y_dot = r*x - y - x*z
    z_dot = x*y - b*z
    return x_dot, y_dot, z_dot


dt = 0.01
num_steps = 10000

# Need one more for the initial values
xs = np.empty(num_steps + 1)
ys = np.empty(num_steps + 1)
zs = np.empty(num_steps + 1)

# Set initial values
xs[0], ys[0], zs[0] = (0., 1., 1.05)

# Step through "time", calculating the partial derivatives at the current point
# and using them to estimate the next point
for i in range(num_steps):
    x_dot, y_dot, z_dot = lorenz(xs[i], ys[i], zs[i])
    xs[i + 1] = xs[i] + (x_dot * dt)
    ys[i + 1] = ys[i] + (y_dot * dt)
    zs[i + 1] = zs[i] + (z_dot * dt)


# Plot
fig = plt.figure()
ax = fig.gca(projection='3d')

ax.plot(xs, ys, zs, lw=0.5)
ax.set_xlabel("X Axis")
ax.set_ylabel("Y Axis")
ax.set_zlabel("Z Axis")
ax.set_title("Lorenz Attractor")

plt.show()
```



3D surface with polar coordinates


```python
import matplotlib.pyplot as plt
import numpy as np


fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# Create the mesh in polar coordinates and compute corresponding Z.
r = np.linspace(0, 1.25, 50)
p = np.linspace(0, 2*np.pi, 50)
R, P = np.meshgrid(r, p)
Z = ((R**2 - 1)**2)

# Express the mesh in the cartesian system.
X, Y = R*np.cos(P), R*np.sin(P)

# Plot the surface.
ax.plot_surface(X, Y, Z, cmap=plt.cm.YlGnBu_r)

# Tweak the limits and add latex math labels.
ax.set_zlim(0, 1)
ax.set_xlabel(r'$\phi_\mathrm{real}$')
ax.set_ylabel(r'$\phi_\mathrm{im}$')
ax.set_zlabel(r'$V(\phi)$')

plt.show()
```

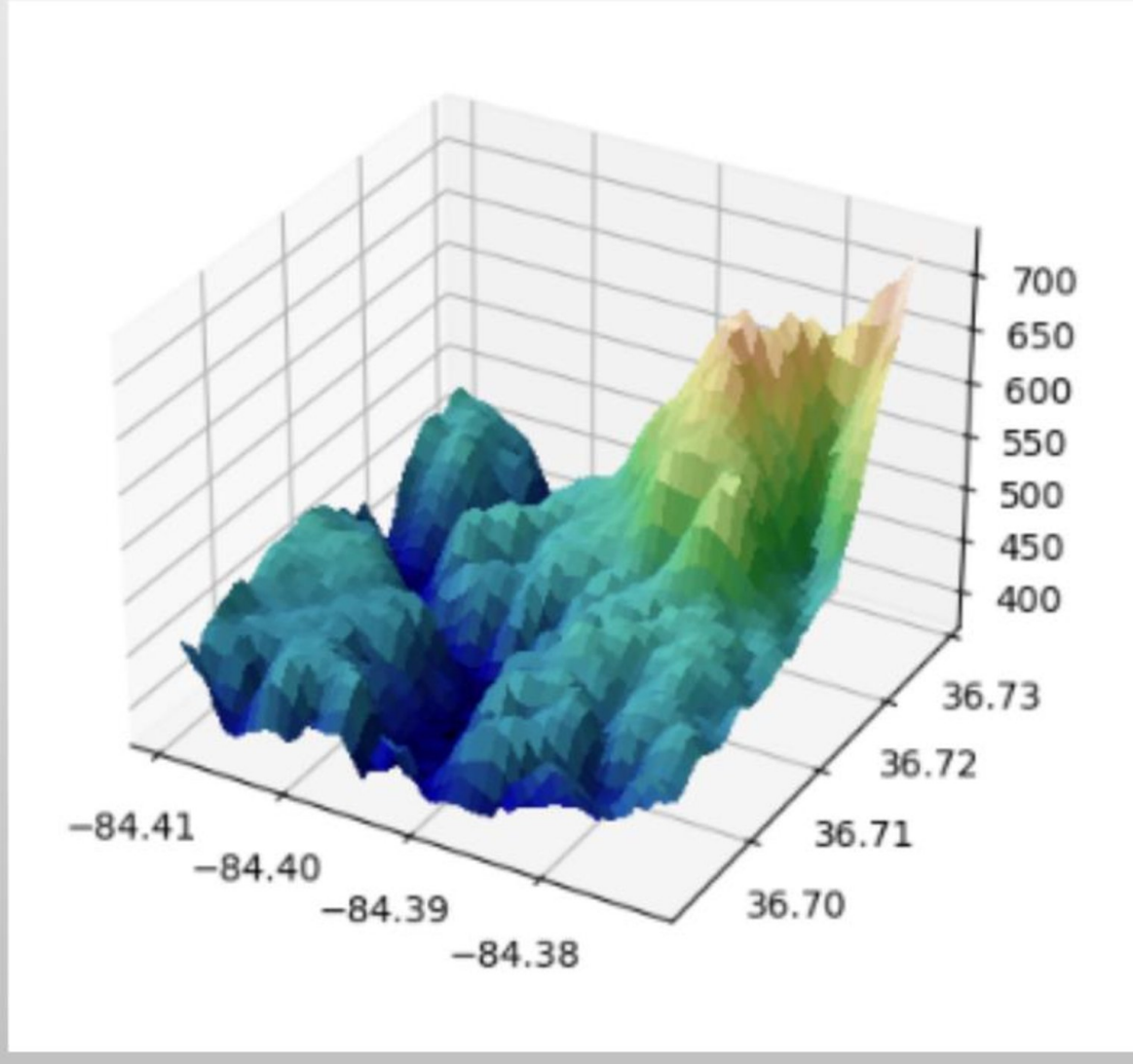


SYSTEMS LABORATORY

Custom hillshading in a 3D surface plot

EP performance crunch

```
from matplotlib import cbook
from matplotlib import cm
from matplotlib.colors import LightSource
import matplotlib.pyplot as plt
import numpy as np

# Load and format data
dem = cbook.get_sample_data('jacksboro_fault_dem.npz', np_load=True)
z = dem['elevation']
nrows, ncols = z.shape
x = np.linspace(dem['xmin'], dem['xmax'], ncols)
y = np.linspace(dem['ymin'], dem['ymax'], nrows)
x, y = np.meshgrid(x, y)

region = np.s_[5:50, 5:50]
x, y, z = x[region], y[region], z[region]

# Set up plot
fig, ax = plt.subplots(subplot_kw=dict(projection='3d'))

ls = LightSource(270, 45)
# To use a custom hillshading mode, override the built-in shading and pass
# in the rgb colors of the shaded surface calculated from "shade".
rgb = ls.shade(z, cmap=cm.gist_earth, vert_exag=0.1, blend_mode='soft')
surf = ax.plot_surface(x, y, z, rstride=1, cstride=1, facecolors=rgb,
                       linewidth=0, antialiased=False, shade=False)

plt.show()
```

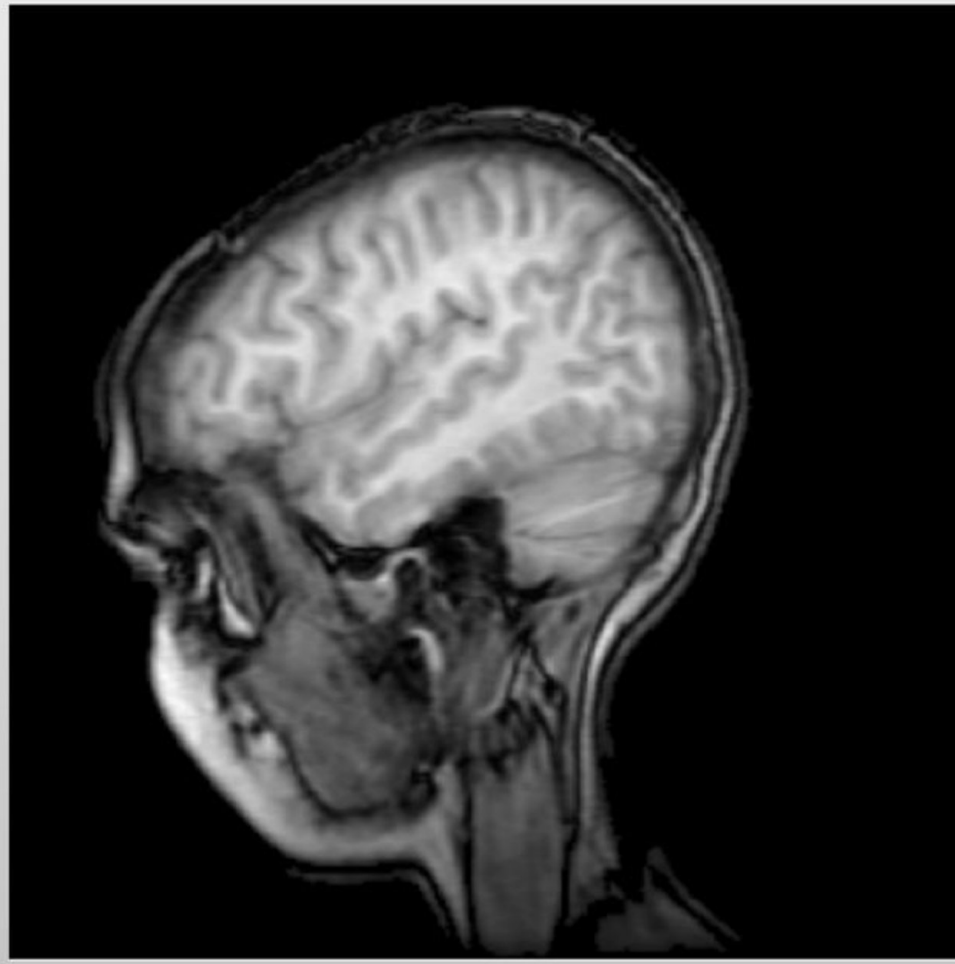


SYSTEMS LABORATORY

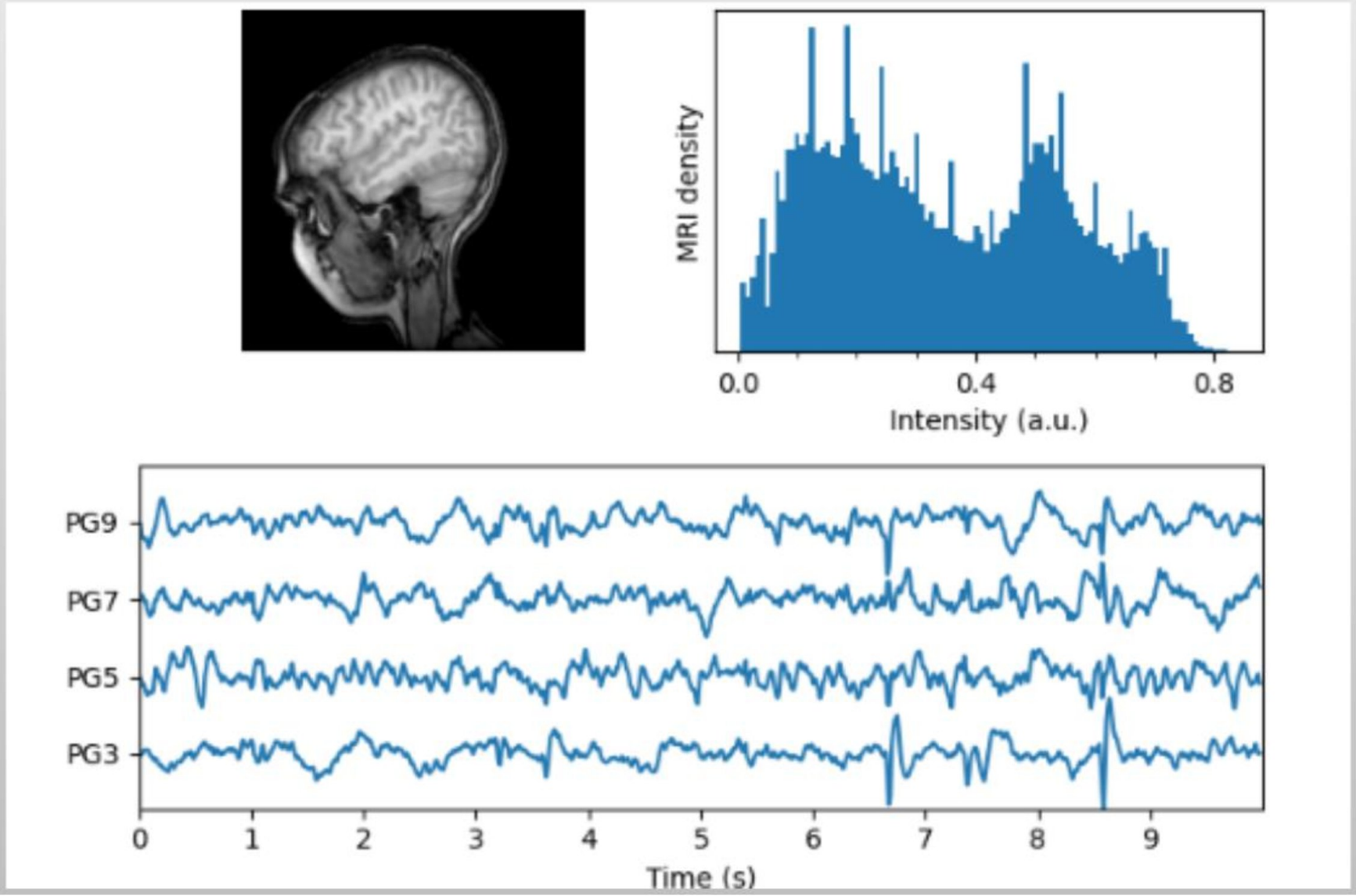
MRI visualization

EP performance crunch

```
import matplotlib.pyplot as plt
import matplotlib.cbook as cbook
import numpy as np


# Data are 256x256 16 bit integers.
with cbook.get_sample_data('s1045.ima.gz') as dfile:
    im = np.frombuffer(dfile.read(), np.uint16).reshape((256, 256))

fig, ax = plt.subplots(num="MRI_demo")
ax.imshow(im, cmap="gray")
ax.axis('off')

plt.show()
```



MRI visualization with EEG

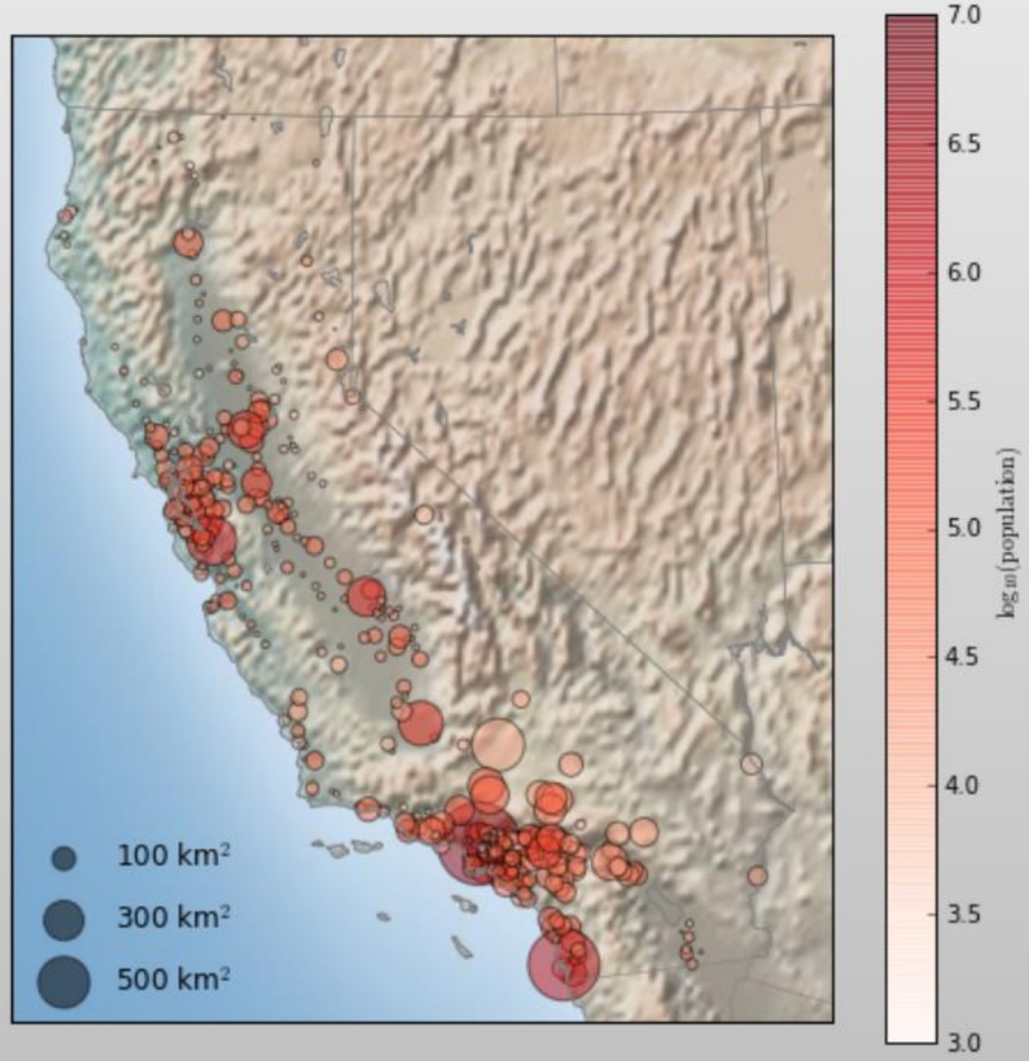


SYSTEMS LABORATORY

Plotting on a map with Basemap

EP performance crunch

```
# 1. Draw the map background
fig = plt.figure(figsize=(8, 8))
m = Basemap(projection='lcc', resolution='h',
            lat_0=37.5, lon_0=-119,
            width=1E6, height=1.2E6)
m.shadedrelief()
m.drawcoastlines(color='gray')
m.drawcountries(color='gray')
m.drawstates(color='gray')

# 2. scatter city data, with color reflecting population
# and size reflecting area
m.scatter(lon, lat, latlon=True,
          c=np.log10(population), s=area,
          cmap='Reds', alpha=0.5)

# 3. create colorbar and legend
plt.colorbar(label=r'$\log_{10}({\rm population})$')
plt.clim(3, 7)

# make legend with dummy points
for a in [100, 300, 500]:
    plt.scatter([], [], c='k', alpha=0.5, s=a,
                label=str(a) + ' km$^2$')
plt.legend(scatterpoints=1, frameon=False,
           labelspacing=1, loc='lower left');
```



SYSTEMS LABORATORY

Conclusion

EP performance crunch

- Always consider the **guidelines for good graphs** (most importantly - easy to read / understand, self-contained).
- The lecture presented some of the basic utility of matplotlib, but this is just scratching the surface.
- Any 2D or 3D plots you have seen can be rendered in very high quality using matplotlib.

SYSTEMS LABORATORY

References

EP performance crunch

1. Raj K. Jain, "*Art of Computer Systems Performance Analysis: Techniques for Experimental Design Measurements, Simulation and Modeling*", John Wiley, 2015.
2. Official Matplotlib documentation https://matplotlib.org/contents.html

SYSTEMS LABORATORY

Lecture 7
Summarising Measured Data

EP
performance crunch

(1232)

SYSTEMS LABORATORY

Airdocs + AR

EP performance crunch

Airdocs is an ongoing project at UPB that aims to associate documents with indoor places without using a positioning system, that usually involves either extensive training or instrumentation of the building. Airdocs uses radio fingerprints (WiFi, BLE) to create unique contexts based on which documents can be placed and retrieved using a mobile phone, without any previous setup.

We have a working Android application[0] that can place and retrieve documents, and wish to enrich it using the AR (augmented reality) in newer Android phones. AR Core provides justaline[1] [2] as an example to employ accelerometers and gyroscopes to enable AR.

The goal of this diploma project is to integrate Android code AR into Airdocs to create documents that are truly in the air.

[0] https://github.com/documenteinaer
[1] https://www.youtube.com/watch?v=IOKwGCQJVCw
[2] https://github.com/googlecreativelab/justaline-android

DI (1232)

SYSTEMS LABORATORY

EP performance crunch

Next week's lecture
Performance evaluation on Windows

SYSTEMS LABORATORY

Overview

EP performance crunch

- Ratio games
- Summarising measured data

SYSTEMS LABORATORY

EP performance crunch

Ratio games

SYSTEMS LABORATORY

Example

EP performance crunch

- Mice trials of 2 types of cancer medicine: **M1** and **M2**
  - Stats for the last 1000 mice

	M1	M2
1000	**900**	800
Advanced stage	100 / 30	400 / 210
	30%	**52.5%**
Early stage	900 / 870	600 / 590
	96.6%	**98.3%**

SYSTEMS LABORATORY

Example

EP performance crunch

Bench-mark	System	
	6502	8080
Block	41.16	51.50
Sieve	63.17	48.08
Sum	104.33	99.58
Avg	52.17	49.79

System	
6502	8080
1.00	1.25
1.00	0.76
2.00	2.01
1.00	1.01

System	
6502	8080
0.80	1.00
1.31	1.00
2.11	2.00
1.06	1.00

- Ratio of totals
  - 6502 is worse - takes 4.7% more time
- 6502 as base
  - 6502 is better - takes 1% less time
- 8080 as base
  - 6502 is worse - takes 6% more time

System	Throughput	Response	Power
A	10	2	5
B	4	1	4

- Throughput: A is better
- Response time: A is worse
- Power: A is better

Alternative	Without	With	Ratio
A on X	2	4	2.00
B on Y	3	5	1.66

- Incomparable bases
  - Need to try both on the same machine

SYSTEMS LABORATORY

EP performance crunch

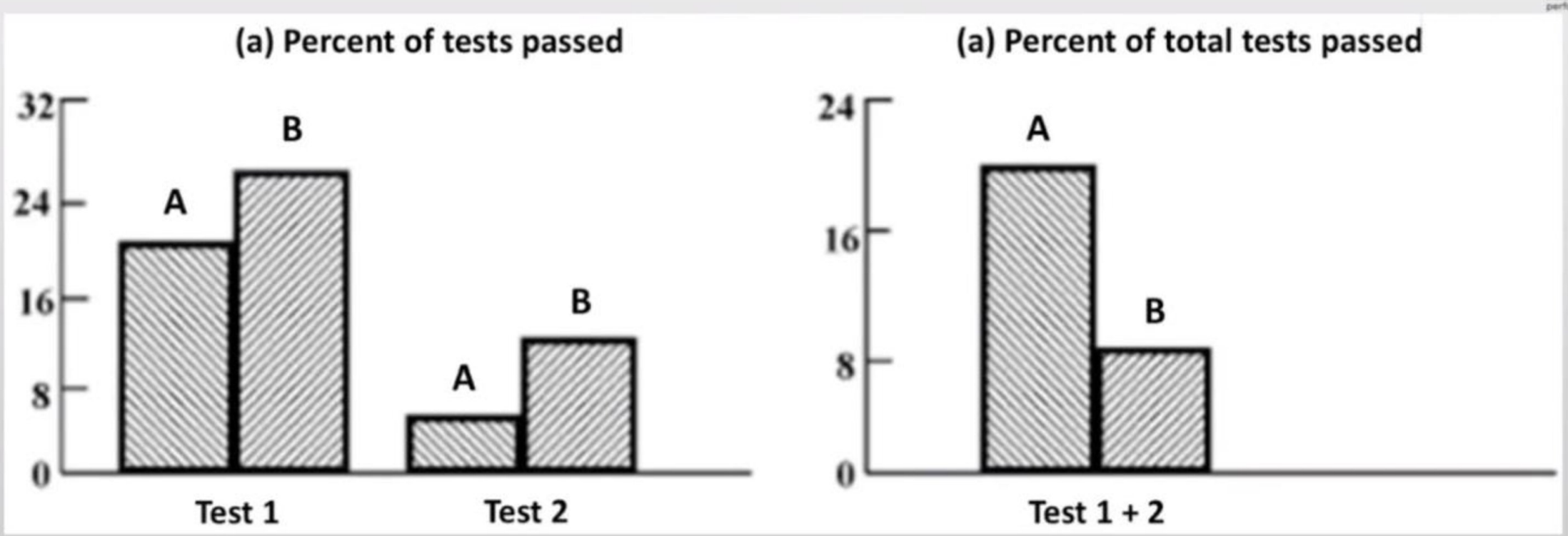
System A:

Test	Total	Pass	% Pass
1	300	60	20%
2	50	2	4%
Total	350	62	20.6%

System B:

Test	Total	Pass	% Pass
1	32	8	25%
2	500	40	8%
Total	532	48	9%

- System B is better on both tests
- System A is better overall



- Incomparable base
  - Can make one look better by repeating the favourable experiment

Other misuses of percentages

- 1000% sounds more impressive than 11-time
- Small sample sizes disguised in percentages
- Base = Initial
  - 400% reduction in prices ⇒ Base = Final

SYSTEMS LABORATORY

Guidelines

EP performance crunch

1. System better on all benchmarks
   - contradicting conclusions cannot be drawn by any ratio game technique
2. System better on all benchmarks
   - a better relative performance can be shown by selecting appropriate base
3. System better on some benchmarks
   - contradicting conclusions can be drawn in some cases (not in all cases)
4. Performance metric is an LB metric
   - better to use your system as the base
5. Performance metric is an HB meric
   - better to use your opponent as the base
6. Benchmarks that perform better on your system should be elongated and those that perform worse should be shortened

SYSTEMS LABORATORY

Takeaway

EP performance crunch

- Ratio games arise from the use of incomparable bases

SYSTEMS LABORATORY

EP performance crunch

Summarising measured data

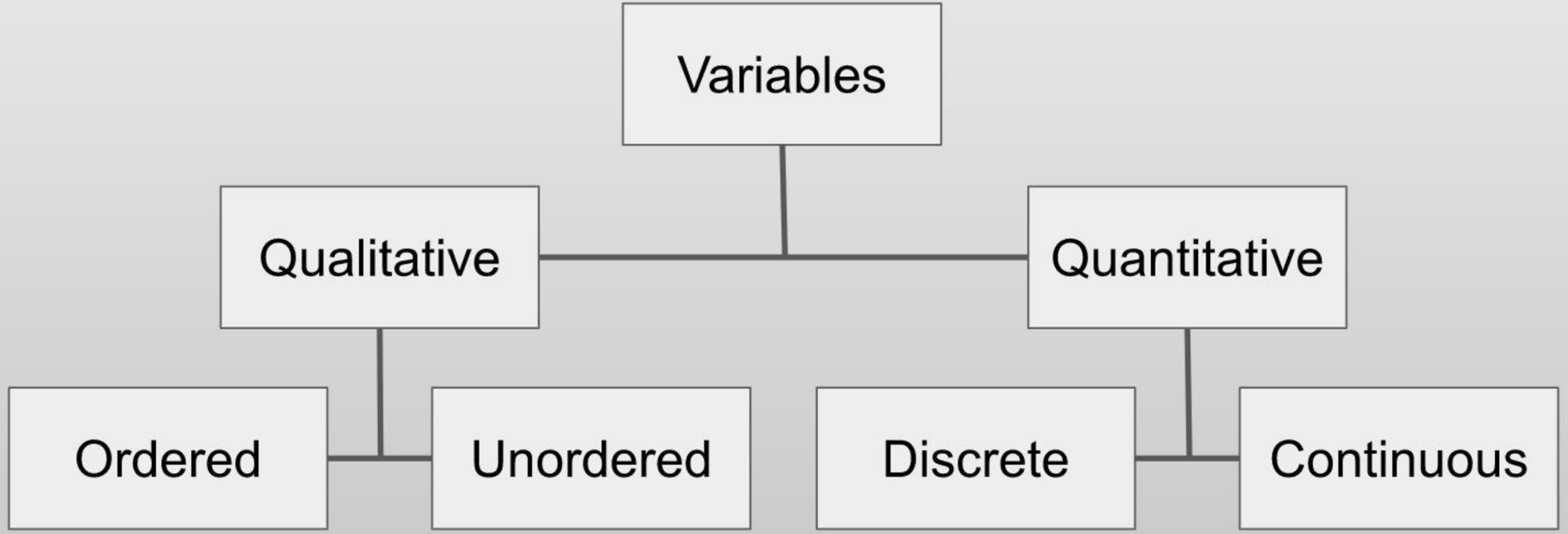
SYSTEMS LABORATORY

Overview

EP performance crunch

- Basic probability and statistics concepts: CDF, PDF, PMF, Mean, Variance, CoV, Normal distribution
- Summarising data by a single number: mean, median, mode, arithmetic/geometric/harmonic means
- Summarising variability: range, variance, percentiles, quartiles

Recap - types of variables



SYSTEMS LABORATORY

EP performance crunch

Basic concepts

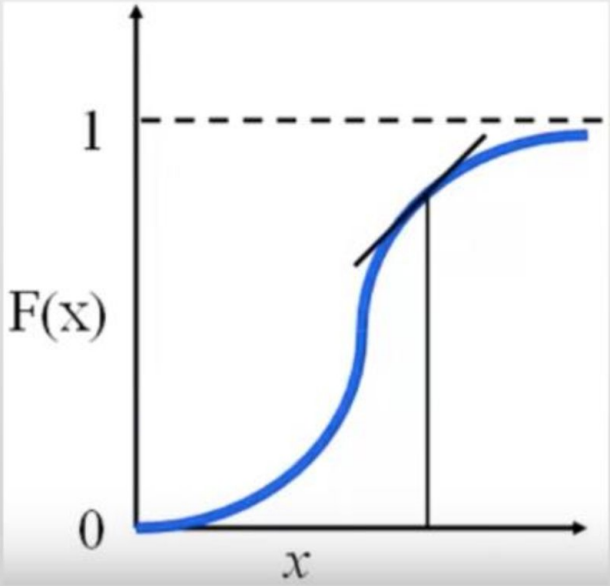
- Independent events
  - The occurence of one event does not affect the probability of the other event.
  - Examples
    - Successive throws of a coin - independent
    - Price of a stock on successive days - dependent
- Random variable
  - Takes one of a specified set of values with a specified probability.
  - Examples
    - Temperature
    - Execution time

CDF, PDF, PMF

- **Cumulative Distribution Function**
  - Starts at 0, ends at 1.

$$F(x) = P(X \leq x)$$

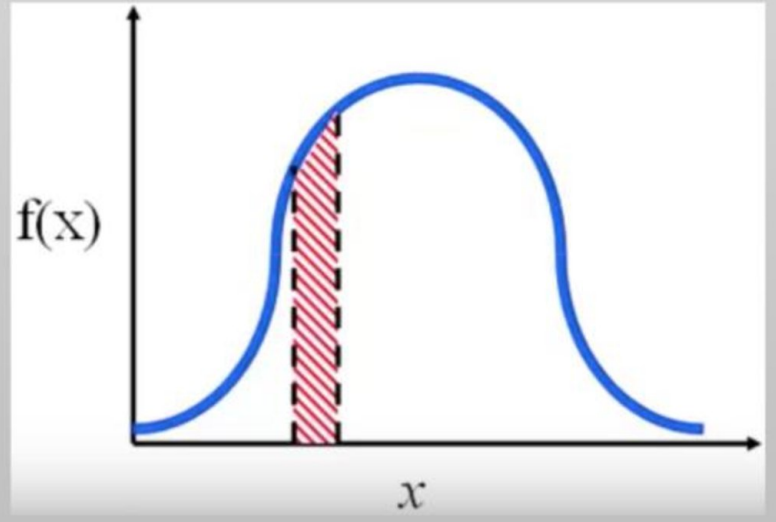
$$F_x(a) = P(x \leq a)$$



- **Probability Density Function**
  - Starts at 0, ends at 0.

$$f(x) = \frac{dF(x)}{dx}$$

$$P(a < x < b) = F(b) - F(a) = \int_a^b f(x)dx$$

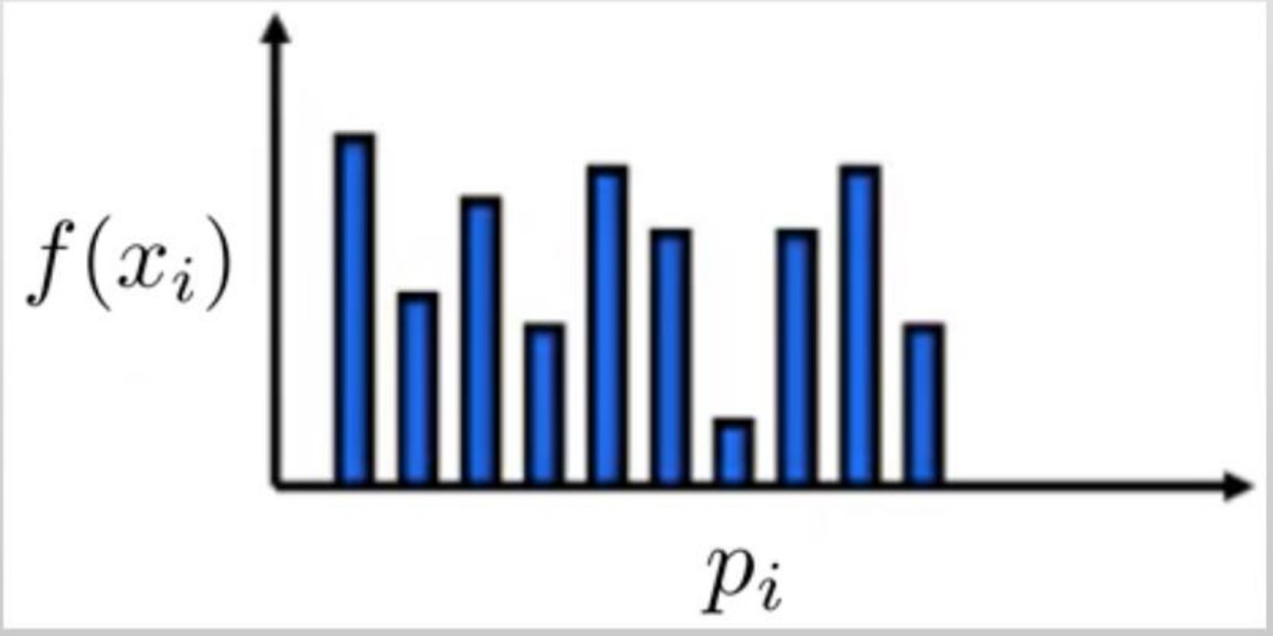


CDF, PDF, PMF

- **Probability Mass Function**
  - For discrete random variables

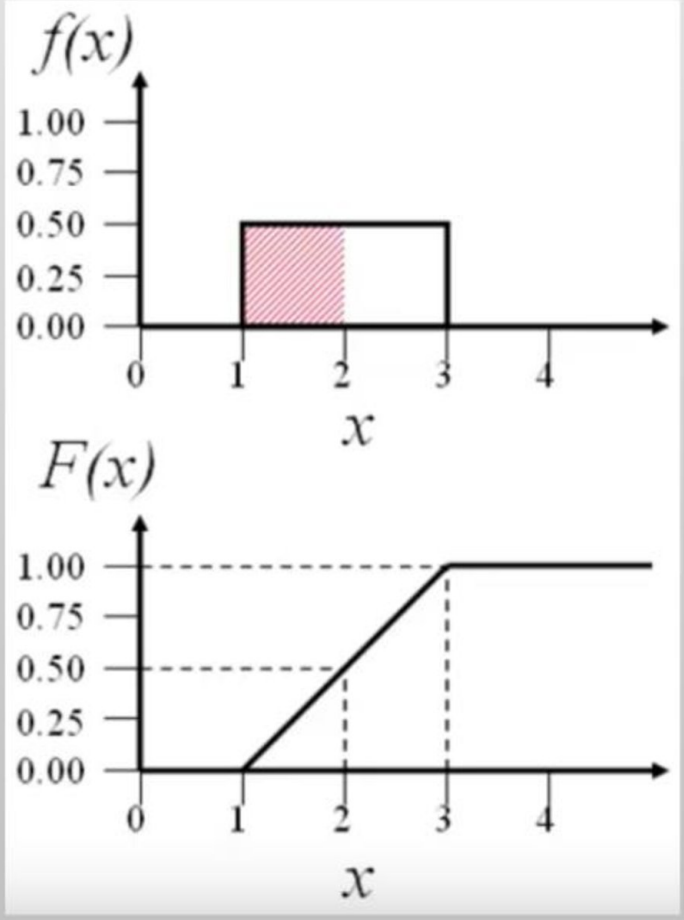
$$f(x) = P(X = x)$$

$$f(x_i) = p_i$$



Example

- Response time - uniformly distributed between 1 and 3 seconds

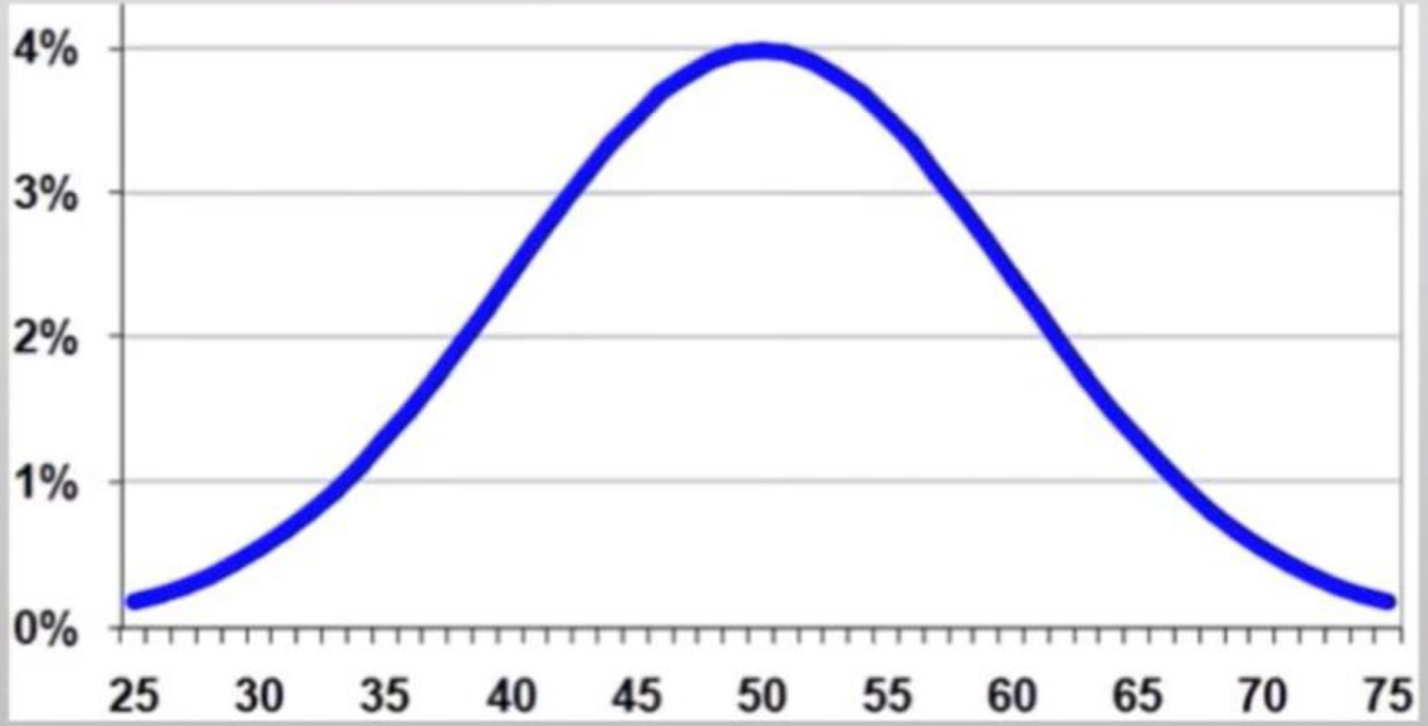


$$f(x) = \frac{1}{2}, \quad 1 \leq x \leq 3$$

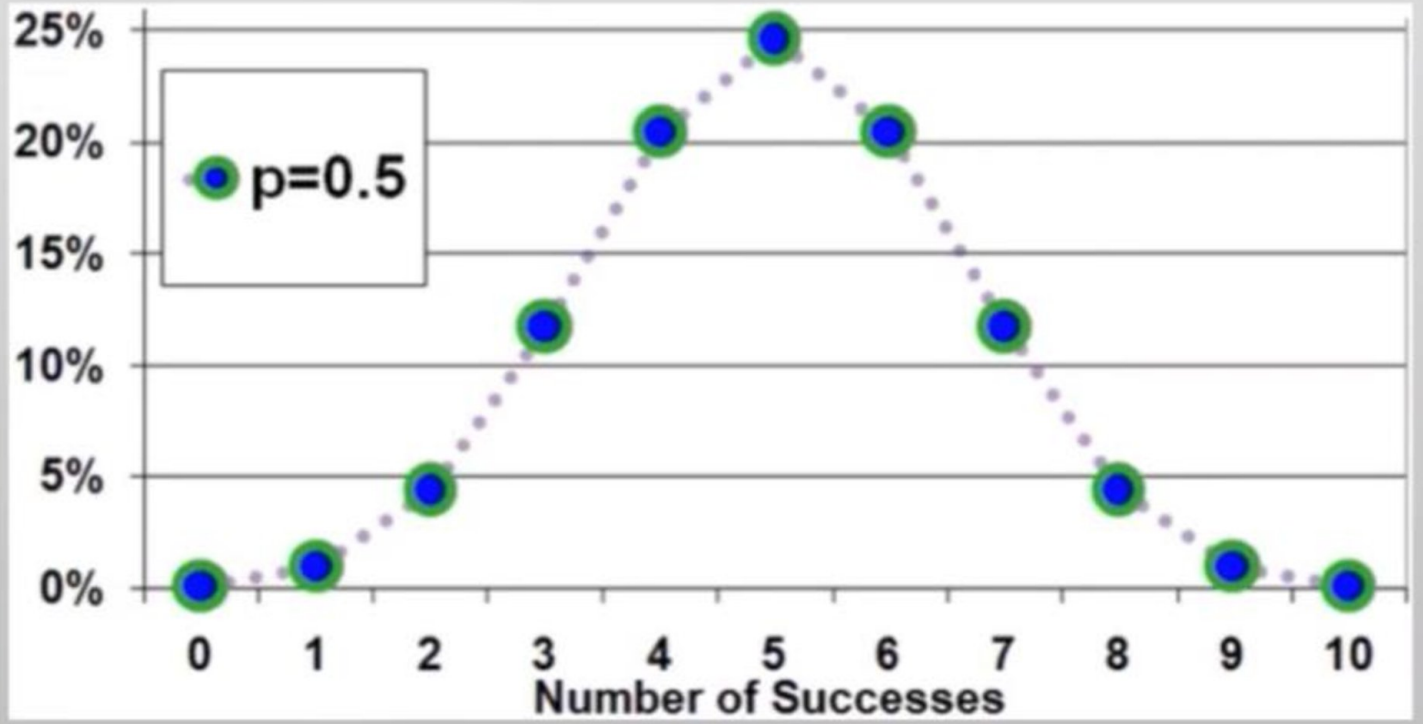
$$P(x \leq 2) = 0.5$$

$$F(x) = \int_{-\infty}^{x} f(x)dx$$

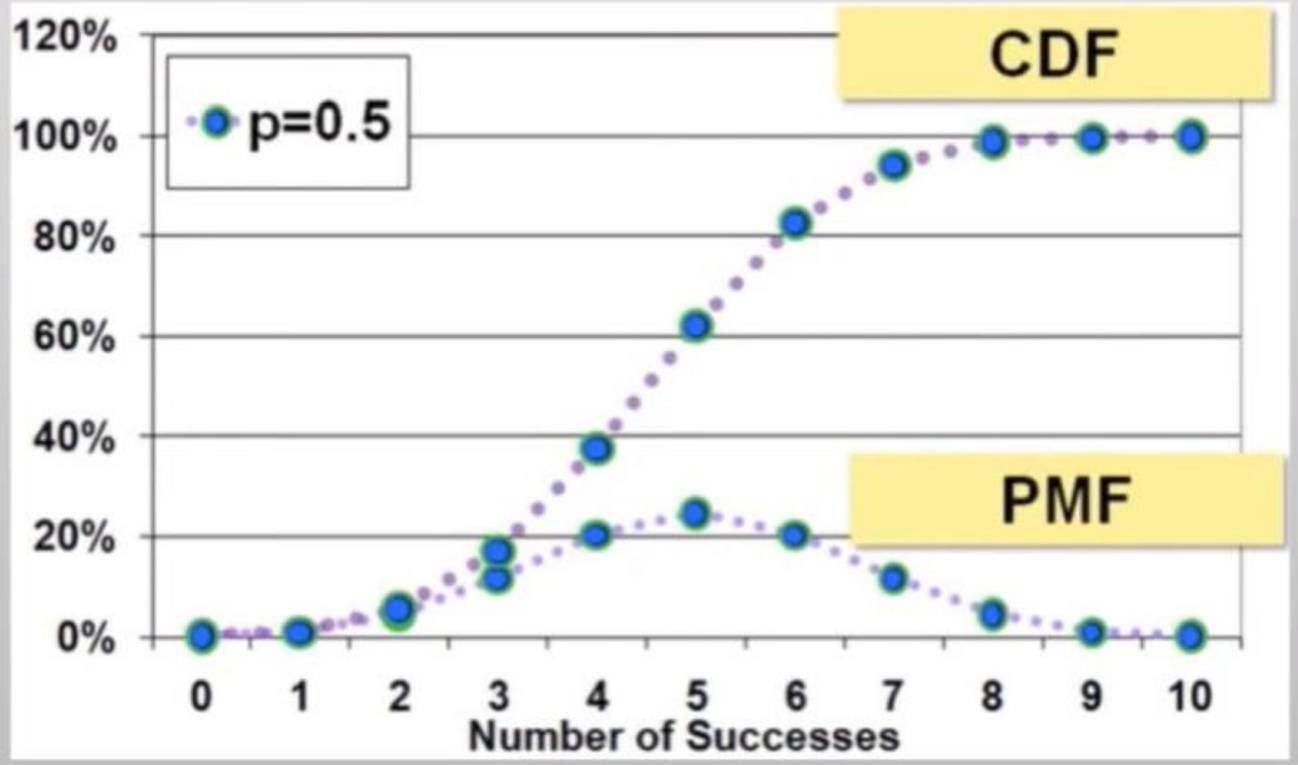
	Density	Cumulative
Discrete	PMF P(X=x)	CDF P(X≤x)
Continuous	PDF P(x1≤X≤x2)	



	Density	Cumulative
Discrete	PMF P(X=x)	CDF P(X≤x)
Continuous	PDF P(x1≤X≤x2)	



	Density	Cumulative
Discrete	PMF P(X=x)	CDF P(X≤x)
Continuous	PDF P(x1≤X≤x2)	



Mean, Variance, CoV

- **Mean**

$$\mu = E(x) = \sum_{i=1}^{n} p_i x_i = \int_{-\infty}^{+\infty} x f(x) dx$$

- **Variance**
  - Expected value of the square of the distance between x and its mean

$$Var(x) = \sigma^2 = E[(x-\mu)^2] = \sum_{i=1}^{n} p_i (x_i - \mu)^2 = \int_{-\infty}^{+\infty} (x_i - \mu)^2 f(x) dx$$

- **Coefficient of Variation**

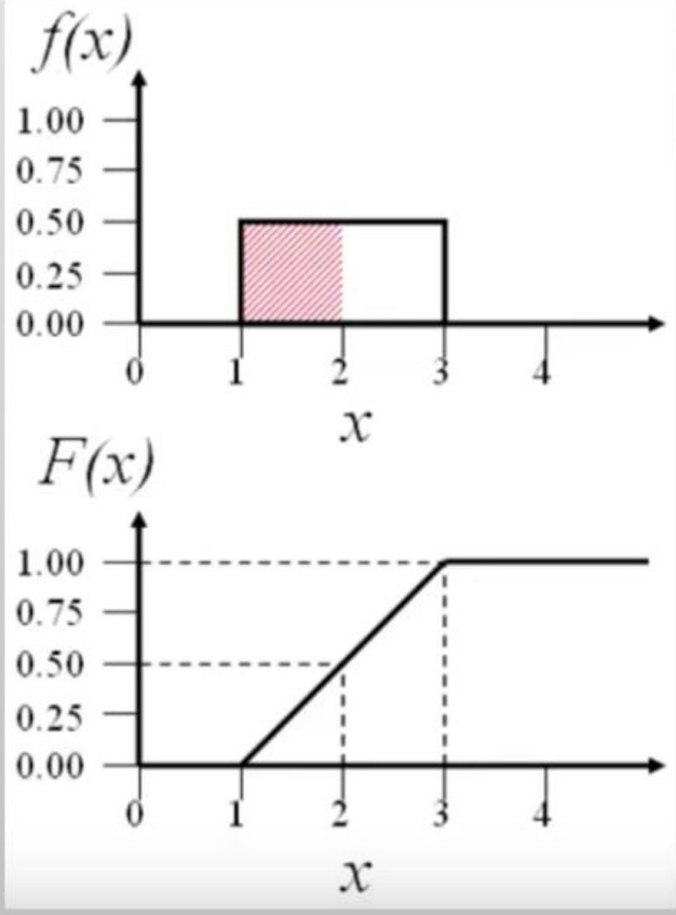
$$C.O.V. = \frac{Standard Deviation}{Mean} = \frac{\sigma}{\mu}$$

Standard deviation

- How much does the data vary from the average?
- Standard deviation is **a measure of spread**
  - **Low** - data is closely clustered around the mean (=average)
  - **High** - data is dispersed over a wider range of values
- It is used when the **distribution is “approximately normal”**, resembling a bell curve

Example

$$f(x) = \frac{1}{2}, \quad 1 \le x \le 3$$



- **Mean**

$$\mu = E(x) = \int_{-\infty}^{+\infty} x f(x) dx = \int_{1}^{3} x \frac{1}{2} dx = \frac{x^2}{4} = \frac{9}{4} - \frac{1}{4} = 2$$

- **Variance**

$$Var(x) = \int_{-\infty}^{+\infty} (x-\mu)^2 f(x) dx = \int_{1}^{3} (x-2)^2 \frac{1}{2} dx$$

$$= \frac{(x-2)^3}{6} = \frac{1}{6} + \frac{1}{6} = \frac{1}{3}$$

- **Standard Deviation** $\sigma = \frac{1}{\sqrt{3}}$

- **Coefficient of Variation**

$$C.O.V. = \frac{Standard Deviation}{Mean} = \frac{\sigma}{\mu} = \frac{1}{2\sqrt{3}}$$

SYSTEMS LABORATORY

Covariance and Correlation

EP performance crunch

Covariance

$$Cov(x) = \sigma_{xy} = E[(x - \mu_x)(y - \mu_y)] = E(xy) - E(x)E(y)$$

- Independent variables ⇒ covariance is zero

$$E(xy) = E(x)E(y)$$

- Reverse not true.

Correlation

- Normalised value of covariance
- Always between -1 and +1

$$Correlation(x, y) = \rho_{xy} = \frac{\sigma_{xy}^2}{\sigma_x \sigma_y}$$

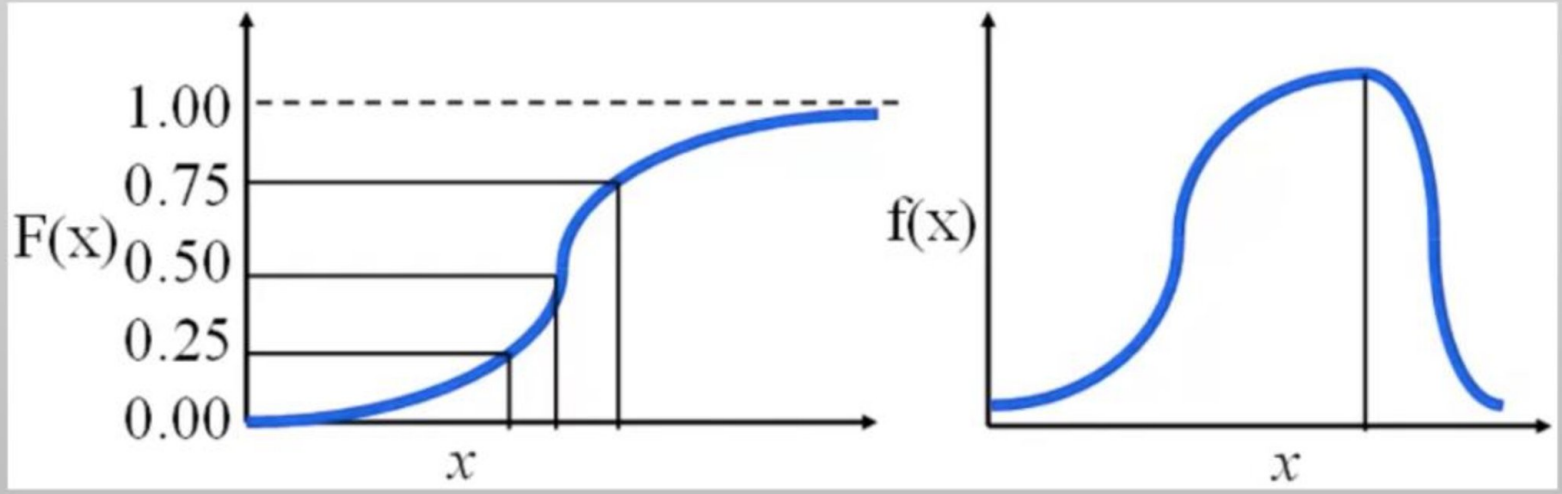
- Dependence implies correlation
- Reverse NOT true

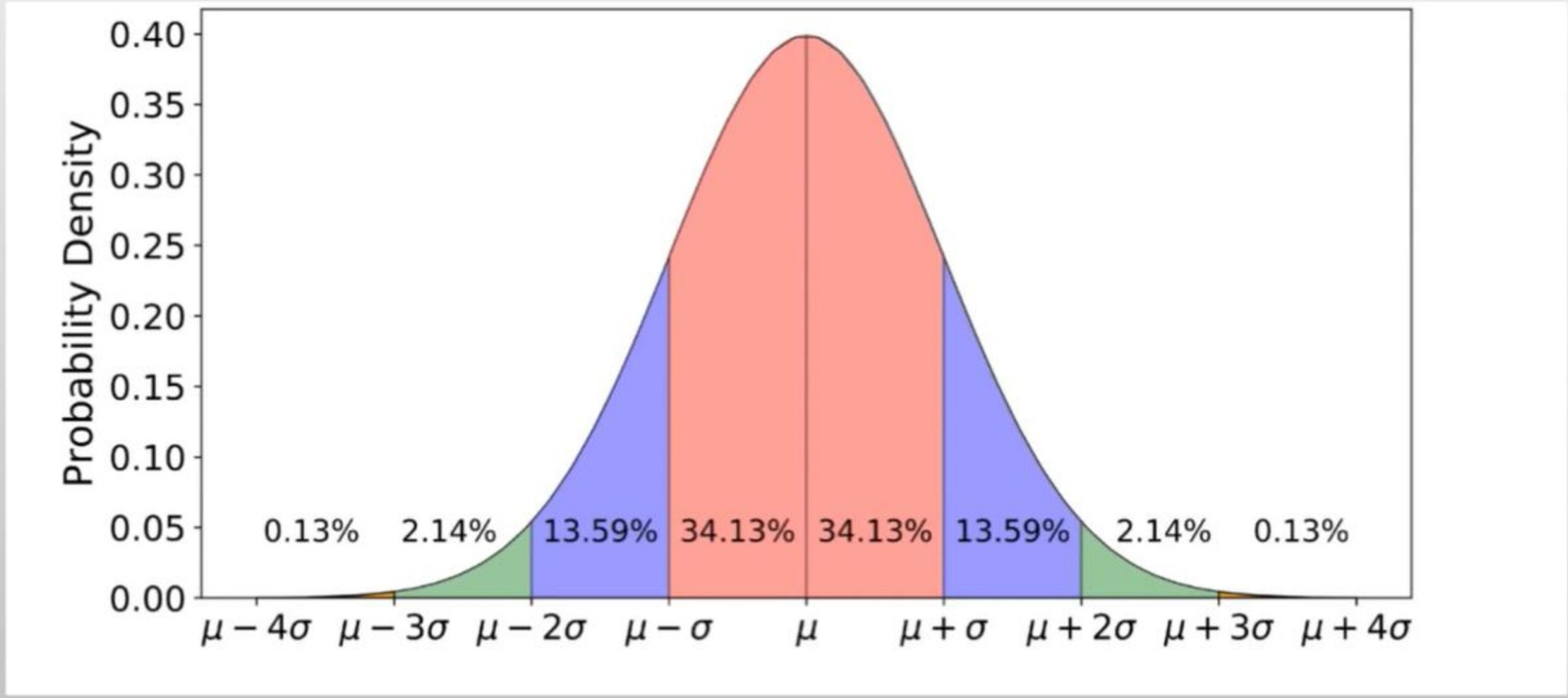
Quantiles, Median, Mode

- **Quantile**
  - The x value at which the CDF takes a value α is called the α-quantile or 100α-percentile.

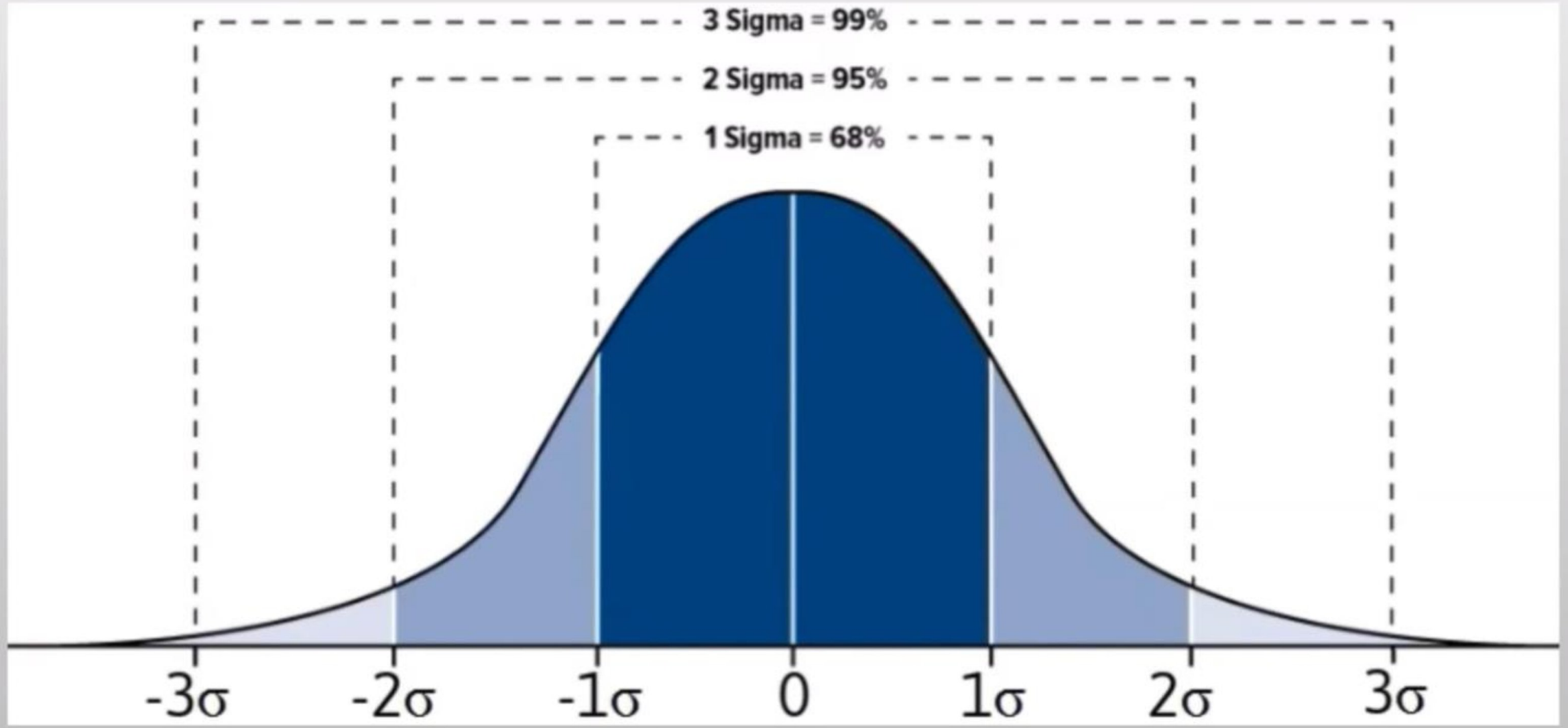
$$P(x \leq x_\alpha) = F(x_\alpha) = \alpha$$

- **Median**
  - 50-percentile or 0.5-quantile of a random variable
- **Mode**
  - The most likely value (x that has the highest probability; x at which PDF is maximum)




Probability Density
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0.13%
2.14%
13.59%
34.13%
34.13%
13.59%
2.14%
0.13%
$\mu-4\sigma$
$\mu-3\sigma$
$\mu-2\sigma$
$\mu-\sigma$
μ
$\mu+\sigma$
$\mu+2\sigma$
$\mu+3\sigma$
$\mu+4\sigma$

68-95-99.7 rule



SYSTEMS LABORATORY

Normal distribution

EP performance crunch

- The sum of a large number of independent observations from any distribution has a normal distribution

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}, \quad -\infty \leq x \leq +\infty$$

- Normal variate $N(\mu, \sigma)$
- Standard normal distribution $N(0, 1)$

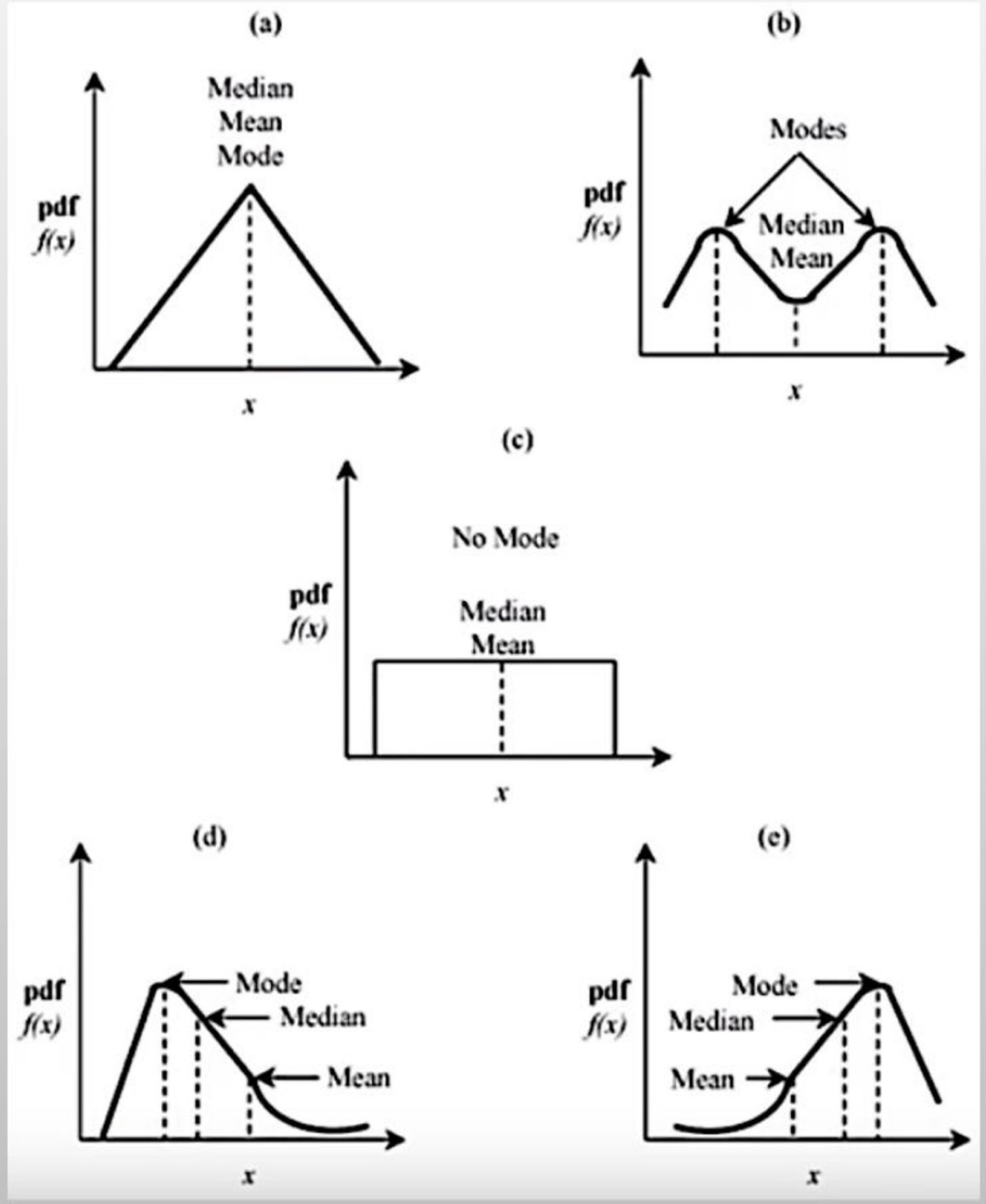
Summarising data by a single number

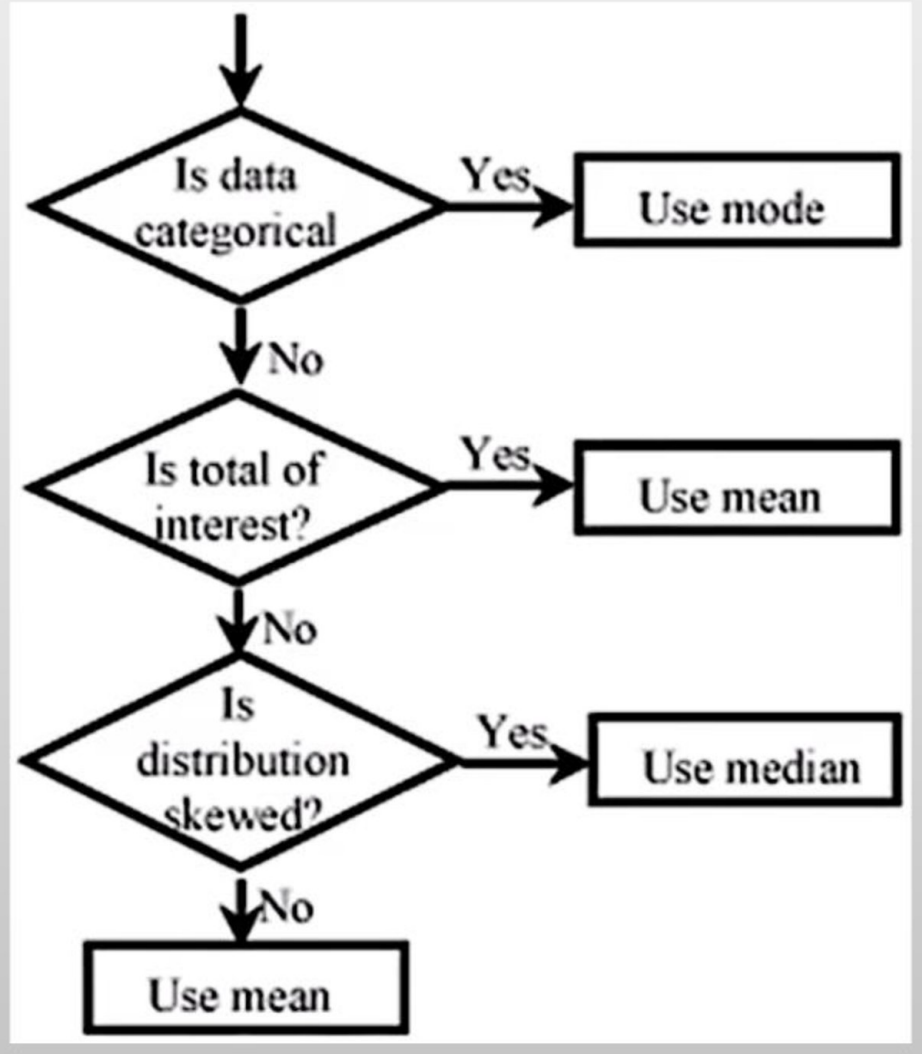
Indices of central tendencies

- Mean
- Median
- Mode

Population vs Sample

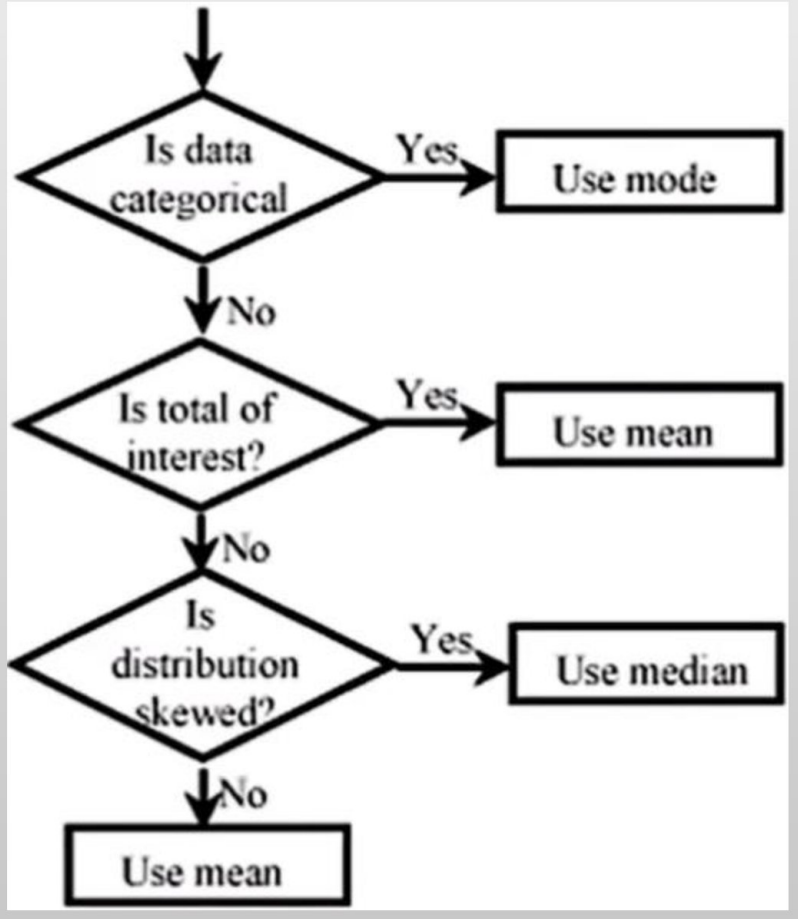
- Sample Mean
  - Sum of all observations divided by number of observations
- Sample Median
  - Sort obs in increasing order. Middle of the series. Even number ⇒ mean of middle 2 values.
- Sample Mode
  - Histogram ⇒ midpoint where the histogram peaks (most frequent category)

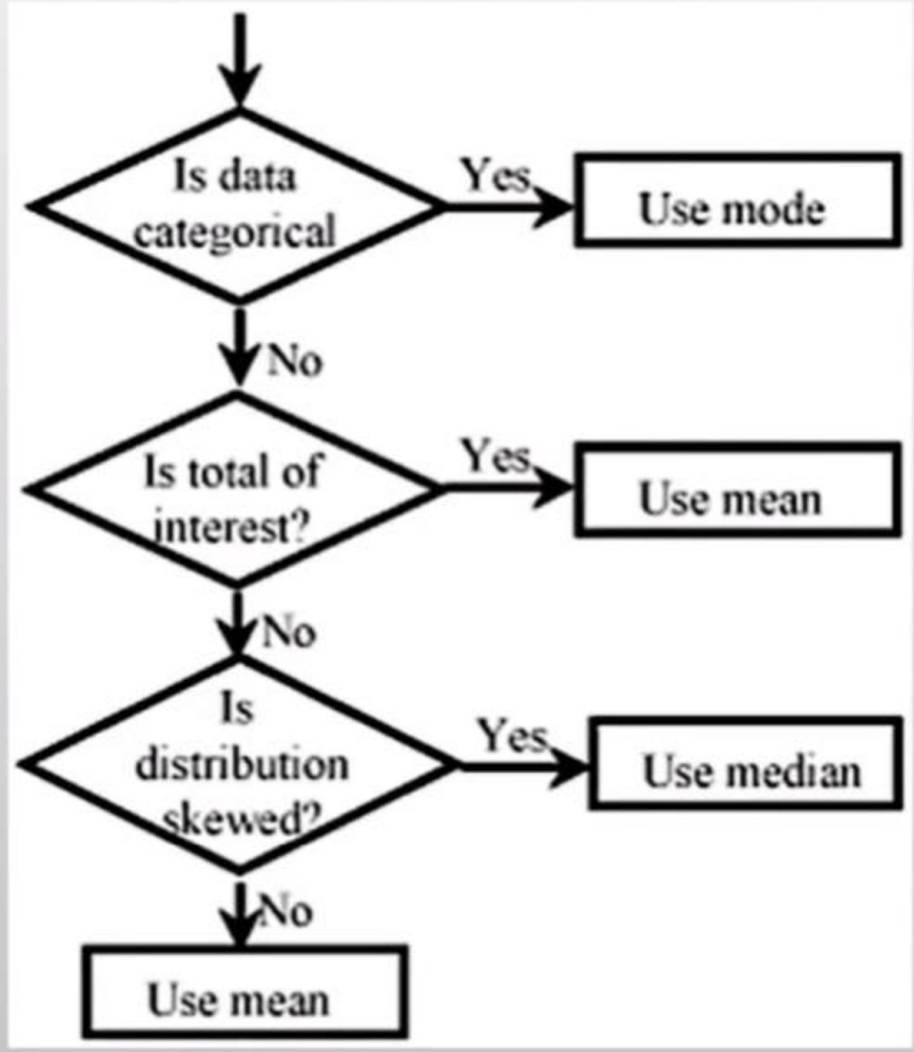

(a)
Median
Mean
Mode
pdf
f(x)
x
(b)
Modes
Median
Mean
pdf
f(x)
x
(c)
No Mode
Median
Mean
pdf
f(x)
x
(d)
Mode
Median
Mean
pdf
f(x)
x
(e)
Mode
Median
Mean
pdf
f(x)
x


Is data categorical
Yes
Use mode
No
Is total of interest?
Yes
Use mean
No
Is distribution skewed?
Yes
Use median
No
Use mean

Examples

- Most used resource in a system
- Interarrival time
- Load on a computer
- Average configuration
  - Number of devices
  - Memory size
  - Number of processors





If distribution not skewed?

SYSTEMS LABORATORY

Misuses of means

EP performance crunch

1. Using mean of significantly different values
   (10 + 1000)/2 = 505
2. Using mean without regard to the skewness of distribution

System A	System B
10	5
9	5
11	5
10	4
10	31
Sum=50	Sum=50
Mean=10	Mean=10
Typical=10	Typical=5

Misuses of means

3. Multiplying means to get the mean of a product

$$E(xy) \neq E(x)E(y)$$

Ex. Time sharing system

- Avg no of users - 23
- Avg no of processes per user - 2
- What is the avg no of processes?
  - Is it 46?
  - No!
  - The no of processes a user spawns depende on the load on the system

4. Taking a mean of a ratio with different bases

Geometric mean

$$\dot{x} = \left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}}$$

- Used if product of observations is of interest.

Example

- Performance improvements in 7 layers

Protocol Layer	Performance Improvement
7	18%
6	13%
5	11%
4	8%
3	10%
2	28%
1	5%

- Average improvement per layer

$$\{(1.18)(1.13)(1.11)(1.08)(1.10)(1.28)(1.05)\}^{\frac{1}{7}} - 1 = 0.13$$

SYSTEMS LABORATORY

Examples of multiplicative metrics

EP performance crunch

- Cache hit ratios over several levels of cache
- Cache miss ratios
- Percentage performance improvement between successive versions
- Avg error rate per hop (on a multi-hop path in a network)

SYSTEMS LABORATORY

EP performance crunch

Harmonic mean

$$\ddot{x} = \frac{n}{\frac{1}{x_1} + \frac{1}{x_2} + \ldots + \frac{1}{x_n}}$$

- Used when an arithmetic mean can be justified for $1/x_i$

Recap

- 3 ways to get a single number out of a sample
  - Mean
  - Mode
  - Median
- Flow chart - guideline to select between them
- Mean - 3 types
  - Arithmetic
  - Geometric
  - Harmonic

SYSTEMS LABORATORY

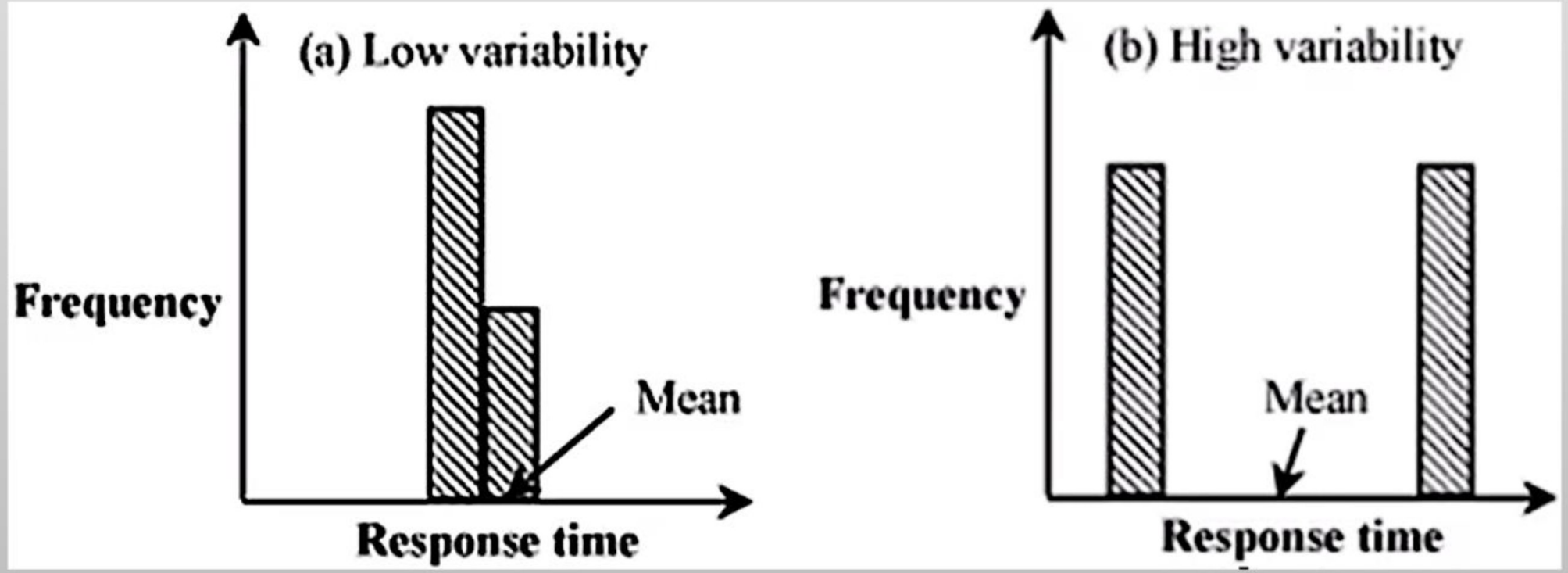
EP performance crunch

Summarising variability

Summarising variability

- “Then there is the man who drowned crossing a stream with an average depth of six inches”

  - W. I. E. Gates



Indices of dispersion

- Range
  - Minimum and maximum
- Variance or standard deviation
- 10- and 90- percentiles
- Semi inter-quantile range
- Mean absolute deviation

Range

- Range = Max - Min
- Larger ⇒ higher variability
- In most cases, not very useful
  - Min often comes out to zero
  - Max often comes out to an outlier
- Unless the variable is bounded, the Max increases with the no of observations, Min decreases with the no of obs. No stable point that gives good indication of actual range.
- **Useful if there is a reason to believe that the variable is bounded**

Variance

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2, \quad where \quad \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

- The divisor for s2 is n-1 and not n
  (because n-1 of the n differences are independent, the nth one can be computed as the sum of all n differences must be zero)
- The number of independent terms in a sum is also called **degrees of freedom**
- Expressed in units (square of units of observations)
- What is preferred to variance?
  - Variance < Std Dev < Coefficient of Variance

Percentiles

- 5-percentile and 95-percentile of a variable
  - Similar impact as Min and Max
- Can be done for any variable (even variables without bounds)
- Better than range?
- Quantile
  - Fraction between 0 and 1
  - 0.9 quantile = 90-percentile
- Decile
  - Percentiles at multiples of 10%
- Quartile
  - Percentiles at multiples of 25%
- Semi Inter-Quartile Range

$$SIQR = \frac{Q_3 - Q_1}{2} = \frac{x_{0.75} - x_{0.25}}{2}$$

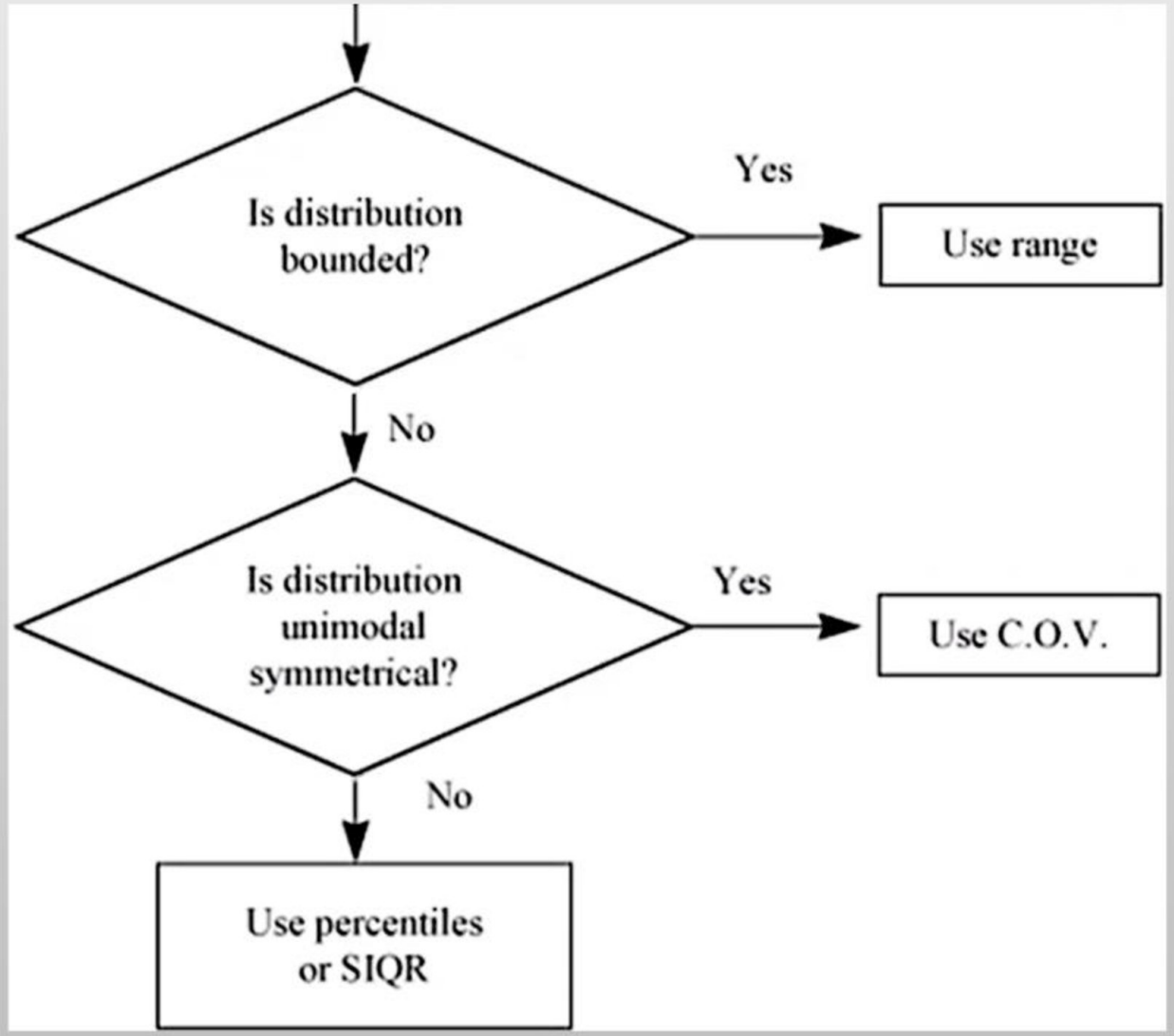
Comparison of variation measures

Range Sample variance SIQR

Affected by outliers Resistant to outliers

- Skewed distribution -> outliers highly likely -> use SIQR

Selecting index of dispersion



- Just guidelines
- E.g.
  - Networks
    - Upper range - worse case
    - Designed to carry 95 to 99-percentile of the observed load levels
  - Computing power
  - Power supplies - sustain peak demand

SYSTEMS LABORATORY

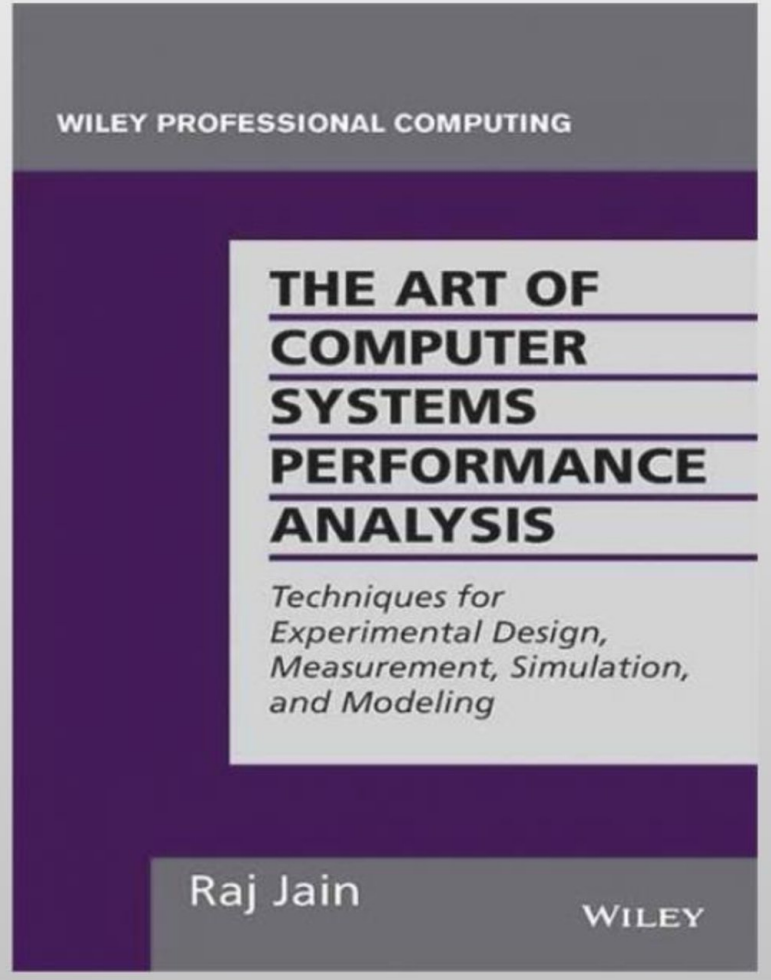
Summary

EP performance crunch

- Indices of central tendencies
  - Mode
  - Median
  - Mean
    - Arithmetic, Geometric, Harmonic means
- Indices of dispersion
  - Range
  - Variance
  - Percentiles
  - Quartiles
  - SIQR

References

1. Raj K. Jain, "*Art of Computer Systems Performance Analysis: Techniques for Experimental Design Measurements, Simulation and Modeling*", John Wiley, 2015.
2. Raj Jain, CSE 567M Computer Systems Analysis, Washington University in St. Louis, 2013.



SYSTEMS LABORATORY

Lecture 6 Comparing Systems

EP

performance crunch

DI (1232)

SYSTEMS LABORATORY

EP performance crunch

- CDF, PDF, PMF
- Mean, Variance, CoV
- Covariance, Correlation

DI (1232)

Mean, Variance, CoV

- **Mean**

$$\mu = E(x) = \sum_{i=1}^{n} p_i x_i = \int_{-\infty}^{+\infty} x f(x) dx$$

- **Variance**
  - Expected value of the square of the distance between x and its mean

$$Var(x) = \sigma^2 = E[(x-\mu)^2] = \sum_{i=1}^{n} p_i (x_i - \mu)^2 = \int_{-\infty}^{+\infty} (x_i - \mu)^2 f(x) dx$$

- **Coefficient of Variation**

$$C.O.V. = \frac{Standard Deviation}{Mean} = \frac{\sigma}{\mu}$$

Summarising data by a single number

- 3 ways to get a single number out of a sample
  - Mean
  - Mode
  - Median
- Flow chart - guideline to select between them
- Mean - 3 types
  - Arithmetic
  - Geometric
  - Harmonic

Indices of dispersion

- Range
  - Minimum and maximum
- Variance or standard deviation
- 10- and 90- percentiles
- Semi inter-quantile range

SYSTEMS LABORATORY

EP performance crunch

Comparing computer systems using sample data

SYSTEMS LABORATORY

Overview

EP performance crunch

- Sample vs Population
- Confidence interval for the mean
- Approximate visual test
- One sided confidence intervals
- Sample size for determining mean and proportions

l (1232)

Sample vs Population

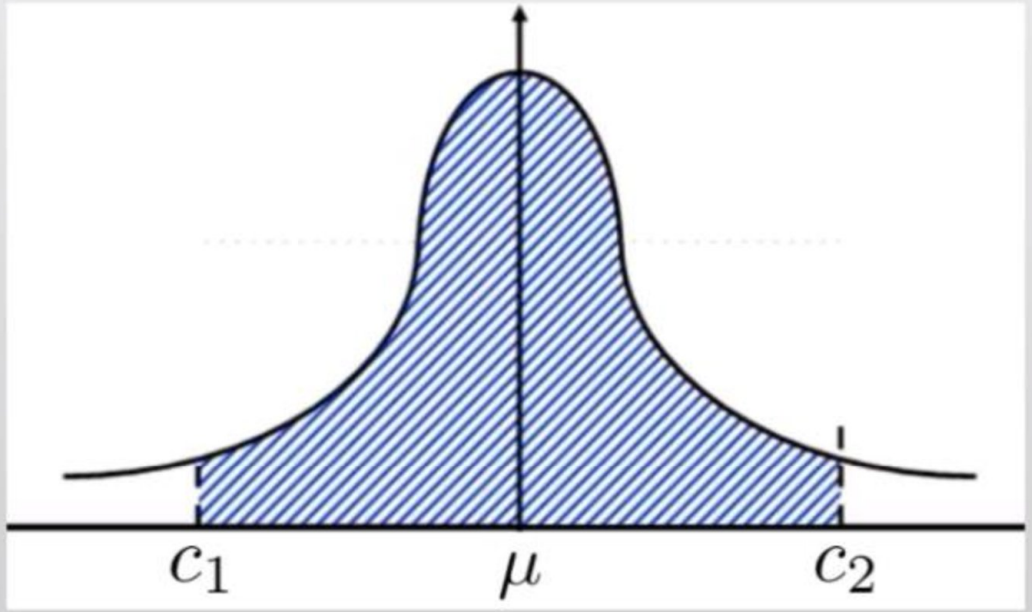
- Generate several million random numbers with mean **μ** and standard deviation **σ**
  Select a sample of **n** observations
  $\overline{x} \neq \mu$

- Sample mean ≠ Population mean
- Population (unknown) - greek letters
  - Population characteristics → parameters
- Sample - english letters
  - Sample estimates → statistics

Confidence interval for the mean

- k samples ⇒ k sample means
  - Cannot get a single estimate of μ
  - Use bounds c1 and c2

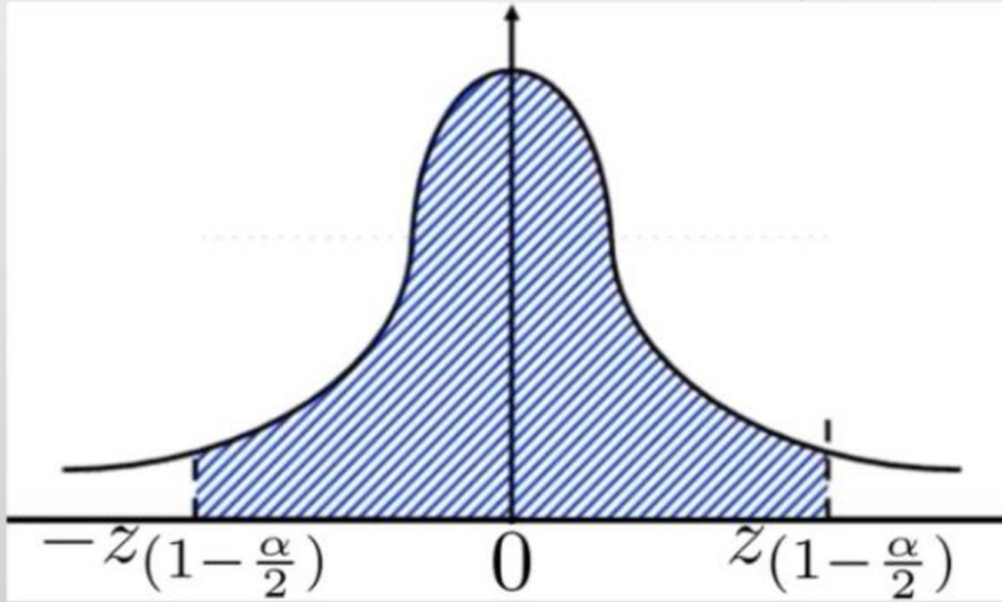
$$Probability\{c_1 \leq \mu \leq c_2\} = 1 - \alpha$$



- Confidence interval: (c_1, c_2)
- Significance level: α
- Confidence level: $100(1-\alpha)$
- Confidence coefficient: 1

Confidence interval

- 90% confidence interval
  - 5-percentile and 95-percentile of the sample means
- Central limit theorem
  - Sample mean is normally distributed for large sample sizes, regardless of the distribution from which we are sampling.



$$\overline{x} \sim N(\mu, \sigma/\sqrt{n})$$

Where μ = population mean and σ = population standard deviation

- Standard error: Standard deviation of the sample mean $s = \sigma/\sqrt{n}$
- 100(1-α)% confidence interval for μ:

$$(\overline{x} - z_{1-\frac{\alpha}{2}}\frac{s}{\sqrt{n}}, \overline{x} + z_{1-\frac{\alpha}{2}}\frac{s}{\sqrt{n}})$$

$$z_{1-\frac{\alpha}{2}} = (1-\frac{\alpha}{2}) - quantile\ \ of\ \ N(0,1)$$

Example

$\overline{x} = 3.90, \quad s = 0.95, \quad n = 32$

- 90% confidence interval for the mean $(\overline{x} - z_{1-\frac{\alpha}{2}} \frac{s}{\sqrt{n}}, \overline{x} + z_{1-\frac{\alpha}{2}} \frac{s}{\sqrt{n}})$

  $3.90 \mp$

SYSTEMS LABORATORY

EP performance crunch

TABLE A.2 Quantiles of the Unit Normal Distribution

p	0.00	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09
0.5	0.000	0.025	0.050	0.075	0.100	0.126	0.151	0.176	0.202	0.228
0.6	0.253	0.279	0.305	0.332	0.358	0.385	0.412	0.440	0.468	0.496
0.7	0.524	0.553	0.583	0.613	0.643	0.674	0.706	0.739	0.772	0.806
0.8	0.842	0.878	0.915	0.954	0.994	1.036	1.080	1.126	1.175	1.227

p	0.000	0.001	0.002	0.003	0.004	0.005	0.006	0.007	0.008	0.009
0.90	1.282	1.287	1.293	1.299	1.305	1.311	1.317	1.323	1.329	1.335
0.91	1.341	1.347	1.353	1.359	1.366	1.372	1.379	1.385	1.392	1.398
0.92	1.405	1.412	1.419	1.426	1.433	1.440	1.447	1.454	1.461	1.468
0.93	1.476	1.483	1.491	1.499	1.506	1.514	1.522	1.530	1.538	1.546
0.94	1.555	1.563	1.572	1.580	1.589	1.598	1.607	1.616	1.626	1.635
0.95	1.645	1.655	1.665	1.675	1.685	1.695	1.706	1.717	1.728	1.739
0.96	1.751	1.762	1.774	1.787	1.799	1.812	1.825	1.838	1.852	1.866
0.97	1.881	1.896	1.911	1.927	1.943	1.960	1.977	1.995	2.014	2.034
0.98	2.054	2.075	2.097	2.120	2.144	2.170	2.197	2.226	2.257	2.290

p	0.0000	0.0001	0.0002	0.0003	0.0004	0.0005	0.0006	0.0007	0.0008	0.0009
0.990	2.326	2.330	2.334	2.338	2.342	2.346	2.349	2.353	2.357	2.362
0.991	2.366	2.370	2.374	2.378	2.382	2.387	2.391	2.395	2.400	2.404
0.992	2.409	2.414	2.418	2.423	2.428	2.432	2.437	2.442	2.447	2.452
0.993	2.457	2.462	2.468	2.473	2.478	2.484	2.489	2.495	2.501	2.506
0.994	2.512	2.518	2.524	2.530	2.536	2.543	2.549	2.556	2.562	2.569
0.995	2.576	2.583	2.590	2.597	2.605	2.612	2.620	2.628	2.636	2.644
0.996	2.652	2.661	2.669	2.678	2.687	2.697	2.706	2.716	2.727	2.737
0.997	2.748	2.759	2.770	2.782	2.794	2.807	2.820	2.834	2.848	2.863
0.998	2.878	2.894	2.911	2.929	2.948	2.968	2.989	3.011	3.036	3.062
0.999	3.090	3.121	3.156	3.195	3.239	3.291	3.353	3.432	3.540	3.719

Example

$\overline{x} = 3.90, \quad s = 0.95, \quad n = 32$

- 90% confidence interval for the mean $\quad (\overline{x} - z_{1-\frac{\alpha}{2}} \frac{s}{\sqrt{n}}, \overline{x} + z_{1-\frac{\alpha}{2}} \frac{s}{\sqrt{n}})$

  $3.90 \mp (1.645)(0.95)/\sqrt{32} = (3.62, 4.17)$

- 95% confidence interval for the mean

  $3.90 \mp$

SYSTEMS LABORATORY

EP performance crunch

TABLE A.2 Quantiles of the Unit Normal Distribution

p	0.00	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09
0.5	0.000	0.025	0.050	0.075	0.100	0.126	0.151	0.176	0.202	0.228
0.6	0.253	0.279	0.305	0.332	0.358	0.385	0.412	0.440	0.468	0.496
0.7	0.524	0.553	0.583	0.613	0.643	0.674	0.706	0.739	0.772	0.806
0.8	0.842	0.878	0.915	0.954	0.994	1.036	1.080	1.126	1.175	1.227

p	0.000	0.001	0.002	0.003	0.004	0.005	0.006	0.007	0.008	0.009
0.90	1.282	1.287	1.293	1.299	1.305	1.311	1.317	1.323	1.329	1.335
0.91	1.341	1.347	1.353	1.359	1.366	1.372	1.379	1.385	1.392	1.398
0.92	1.405	1.412	1.419	1.426	1.433	1.440	1.447	1.454	1.461	1.468
0.93	1.476	1.483	1.491	1.499	1.506	1.514	1.522	1.530	1.538	1.546
0.94	1.555	1.563	1.572	1.580	1.589	1.598	1.607	1.616	1.626	1.635
0.95	1.645	1.655	1.665	1.675	1.685	1.695	1.706	1.717	1.728	1.739
0.96	1.751	1.762	1.774	1.787	1.799	1.812	1.825	1.838	1.852	1.866
0.97	1.881	1.896	1.911	1.927	1.943	1.960	1.977	1.995	2.014	2.034
0.98	2.054	2.075	2.097	2.120	2.144	2.170	2.197	2.226	2.257	2.290

p	0.0000	0.0001	0.0002	0.0003	0.0004	0.0005	0.0006	0.0007	0.0008	0.0009
0.990	2.326	2.330	2.334	2.338	2.342	2.346	2.349	2.353	2.357	2.362
0.991	2.366	2.370	2.374	2.378	2.382	2.387	2.391	2.395	2.400	2.404
0.992	2.409	2.414	2.418	2.423	2.428	2.432	2.437	2.442	2.447	2.452
0.993	2.457	2.462	2.468	2.473	2.478	2.484	2.489	2.495	2.501	2.506
0.994	2.512	2.518	2.524	2.530	2.536	2.543	2.549	2.556	2.562	2.569
0.995	2.576	2.583	2.590	2.597	2.605	2.612	2.620	2.628	2.636	2.644
0.996	2.652	2.661	2.669	2.678	2.687	2.697	2.706	2.716	2.727	2.737
0.997	2.748	2.759	2.770	2.782	2.794	2.807	2.820	2.834	2.848	2.863
0.998	2.878	2.894	2.911	2.929	2.948	2.968	2.989	3.011	3.036	3.062
0.999	3.090	3.121	3.156	3.195	3.239	3.291	3.353	3.432	3.540	3.719

Example

$\overline{x} = 3.90, \quad s = 0.95, \quad n = 32$

- 90% confidence interval for the mean $\left(\overline{x} - z_{1-\frac{\alpha}{2}} \frac{s}{\sqrt{n}}, \overline{x} + z_{1-\frac{\alpha}{2}} \frac{s}{\sqrt{n}}\right)$

  $3.90 \mp (1.645)(0.95)/\sqrt{32} = (3.62, 4.17)$

- 95% confidence interval for the mean

  $3.90 \mp (1.960)(0.95)/\sqrt{32} = (3.57, 4.23)$

- 99% confidence interval for the mean

  $3.90 \mp$

SYSTEMS LABORATORY

EP performance crunch

TABLE A.2 Quantiles of the Unit Normal Distribution

p	0.00	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09
0.5	0.000	0.025	0.050	0.075	0.100	0.126	0.151	0.176	0.202	0.228
0.6	0.253	0.279	0.305	0.332	0.358	0.385	0.412	0.440	0.468	0.496
0.7	0.524	0.553	0.583	0.613	0.643	0.674	0.706	0.739	0.772	0.806
0.8	0.842	0.878	0.915	0.954	0.994	1.036	1.080	1.126	1.175	1.227

p	0.000	0.001	0.002	0.003	0.004	0.005	0.006	0.007	0.008	0.009
0.90	1.282	1.287	1.293	1.299	1.305	1.311	1.317	1.323	1.329	1.335
0.91	1.341	1.347	1.353	1.359	1.366	1.372	1.379	1.385	1.392	1.398
0.92	1.405	1.412	1.419	1.426	1.433	1.440	1.447	1.454	1.461	1.468
0.93	1.476	1.483	1.491	1.499	1.506	1.514	1.522	1.530	1.538	1.546
0.94	1.555	1.563	1.572	1.580	1.589	1.598	1.607	1.616	1.626	1.635
0.95	1.645	1.655	1.665	1.675	1.685	1.695	1.706	1.717	1.728	1.739
0.96	1.751	1.762	1.774	1.787	1.799	1.812	1.825	1.838	1.852	1.866
0.97	1.881	1.896	1.911	1.927	1.943	1.960	1.977	1.995	2.014	2.034
0.98	2.054	2.075	2.097	2.120	2.144	2.170	2.197	2.226	2.257	2.290

p	0.0000	0.0001	0.0002	0.0003	0.0004	0.0005	0.0006	0.0007	0.0008	0.0009
0.990	2.326	2.330	2.334	2.338	2.342	2.346	2.349	2.353	2.357	2.362
0.991	2.366	2.370	2.374	2.378	2.382	2.387	2.391	2.395	2.400	2.404
0.992	2.409	2.414	2.418	2.423	2.428	2.432	2.437	2.442	2.447	2.452
0.993	2.457	2.462	2.468	2.473	2.478	2.484	2.489	2.495	2.501	2.506
0.994	2.512	2.518	2.524	2.530	2.536	2.543	2.549	2.556	2.562	2.569
0.995	2.576	2.583	2.590	2.597	2.605	2.612	2.620	2.628	2.636	2.644
0.996	2.652	2.661	2.669	2.678	2.687	2.697	2.706	2.716	2.727	2.737
0.997	2.748	2.759	2.770	2.782	2.794	2.807	2.820	2.834	2.848	2.863
0.998	2.878	2.894	2.911	2.929	2.948	2.968	2.989	3.011	3.036	3.062
0.999	3.090	3.121	3.156	3.195	3.239	3.291	3.353	3.432	3.540	3.719

Example

$\overline{x} = 3.90, \quad s = 0.95, \quad n = 32$

- 90% confidence interval for the mean $\left(\overline{x} - z_{1-\frac{\alpha}{2}} \frac{s}{\sqrt{n}}, \overline{x} + z_{1-\frac{\alpha}{2}} \frac{s}{\sqrt{n}}\right)$

  $3.90 \mp (1.645)(0.95)/\sqrt{32} = (3.62, 4.17)$

- 95% confidence interval for the mean

  $3.90 \mp (1.960)(0.95)/\sqrt{32} = (3.57, 4.23)$

- 99% confidence interval for the mean

  $3.90 \mp (2.576)(0.95)/\sqrt{32} = (3.46, 4.33)$

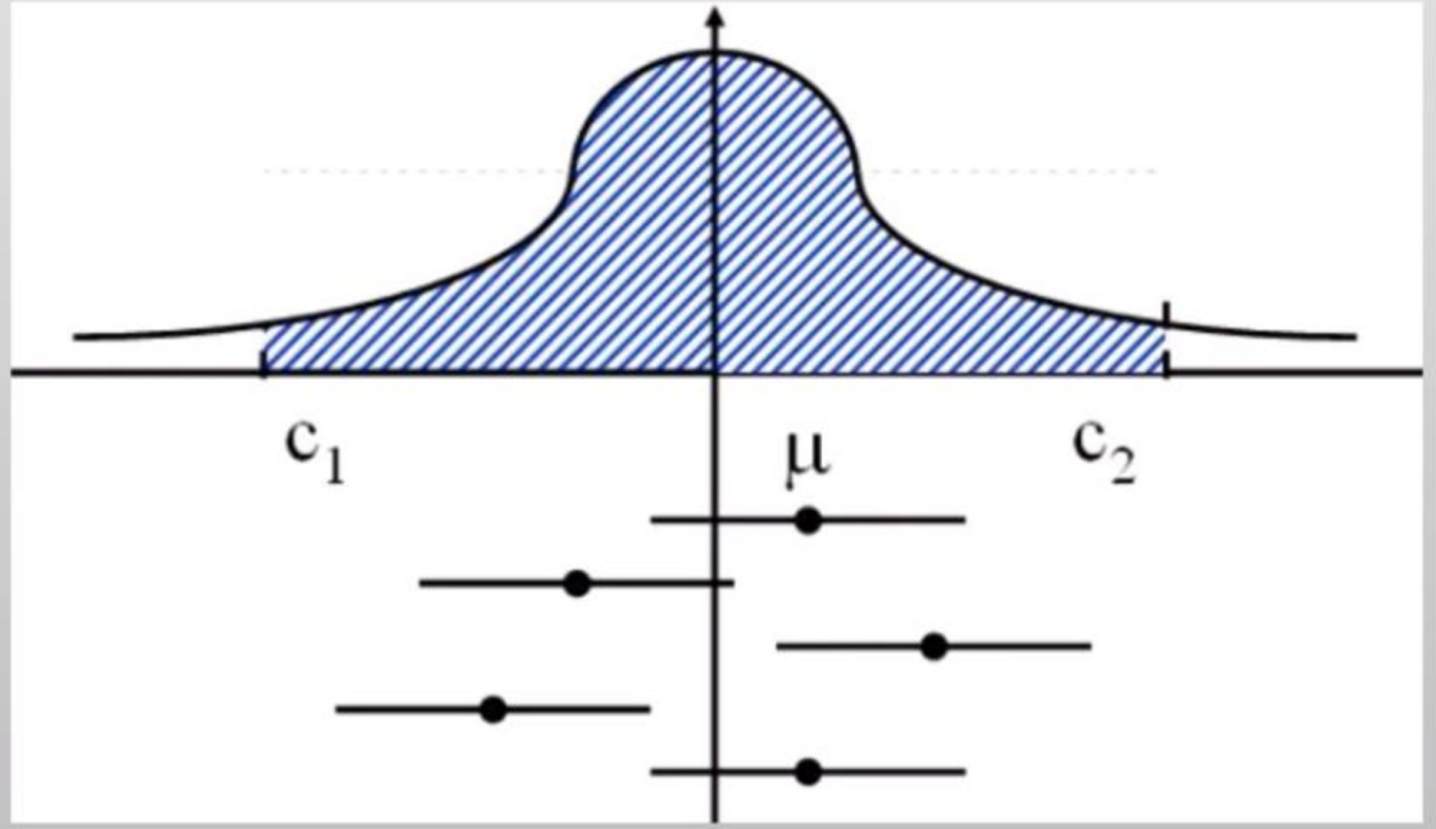
Example (continued)

Which confidence interval is wider?

- 90%
- 95%
- 99%

Confidence interval

- 90% confidence: take 100 samples and construct confidence interval for each sample, the interval would include the population mean in 90 cases.



Confidence interval for small samples

- How CI works - assume large number of observations
  ⇒ mean is normally distributed
- Large sample = 30
  - Not absolute bound
  - Exceptions exist (30+ observations and nowhere close to a normal distribution)
- What can you do if you do not have 30 observations?

Confidence interval for small samples

- 100(1-α)% confidence interval for n < 30

$$(\overline{x} - t_{[1-\frac{\alpha}{2};n-1]}\frac{s}{\sqrt{n}}, \overline{x} + t_{[1-\frac{\alpha}{2};n-1]}\frac{s}{\sqrt{n}})$$

- $t_{[1-\frac{\alpha}{2};n-1]} = (1 - \frac{\alpha}{2})$ - quantile of a t-variate with n-1 degrees of freedom

Example

- Sample: -0.04, -0.19, 0.14, -0.09, -0.14, 0.19, 0.04, 0.09
- Mean = 0
- Sample standard deviation = 0.138
- For 90% interval: $t_{[1-\frac{\alpha}{2};n-1]} = t_{[0.95;7]} =$

$$(\overline{x} - t_{[1-\frac{\alpha}{2};n-1]}\frac{s}{\sqrt{n}}, \overline{x} + t_{[1-\frac{\alpha}{2};n-1]}\frac{s}{\sqrt{n}})$$

SYSTEMS LABORATORY

EP performance crunch

TABLE A.4 Quantiles of the *t* Distribution

					p			
n	**0.6000**	**0.7000**	**0.8000**	**0.9000**	**0.9500**	**0.9750**	**0.9950**	**0.9995**
1	0.325	0.727	1.377	3.078	6.314	12.706	63.657	636.619
2	0.289	0.617	1.061	1.886	2.920	4.303	9.925	31.599
3	0.277	0.584	0.978	1.638	2.353	3.182	5.841	12.924
4	0.271	0.569	0.941	1.533	2.132	2.776	4.604	8.610
5	0.267	0.559	0.920	1.476	2.015	2.571	4.032	6.869
6	0.265	0.553	0.906	1.440	1.943	2.447	3.707	5.959
7	0.263	0.549	0.896	1.415	1.895	2.365	3.499	5.408
8	0.262	0.546	0.889	1.397	1.860	2.306	3.355	5.041
9	0.261	0.543	0.883	1.383	1.833	2.262	3.250	4.781
10	0.260	0.542	0.879	1.372	1.812	2.228	3.169	4.587
11	0.260	0.540	0.876	1.363	1.796	2.201	3.106	4.437
12	0.259	0.539	0.873	1.356	1.782	2.179	3.055	4.318
13	0.259	0.538	0.870	1.350	1.771	2.160	3.012	4.221
14	0.258	0.537	0.868	1.345	1.761	2.145	2.977	4.140
15	0.258	0.536	0.866	1.341	1.753	2.131	2.947	4.073
16	0.258	0.535	0.865	1.337	1.746	2.120	2.921	4.015
17	0.257	0.534	0.863	1.333	1.740	2.110	2.898	3.965
18	0.257	0.534	0.862	1.330	1.734	2.101	2.878	3.922
19	0.257	0.533	0.861	1.328	1.729	2.093	2.861	3.883
20	0.257	0.533	0.860	1.325	1.725	2.086	2.845	3.850
21	0.257	0.532	0.859	1.323	1.721	2.080	2.831	3.819
22	0.256	0.532	0.858	1.321	1.717	2.074	2.819	3.792
23	0.256	0.532	0.858	1.319	1.714	2.069	2.807	3.768
24	0.256	0.531	0.857	1.318	1.711	2.064	2.797	3.745
25	0.256	0.531	0.856	1.316	1.708	2.060	2.787	3.725
26	0.256	0.531	0.856	1.315	1.706	2.056	2.779	3.707
27	0.256	0.531	0.855	1.314	1.703	2.052	2.771	3.690
28	0.256	0.530	0.855	1.313	1.701	2.048	2.763	3.674
29	0.256	0.530	0.854	1.311	1.699	2.045	2.756	3.659
30	0.256	0.530	0.854	1.310	1.697	2.042	2.750	3.646
60	0.254	0.527	0.848	1.296	1.671	2.000	2.660	3.460
90	0.254	0.526	0.846	1.291	1.662	1.987	2.632	3.402
120	0.254	0.526	0.845	1.289	1.658	1.980	2.617	3.373

Example

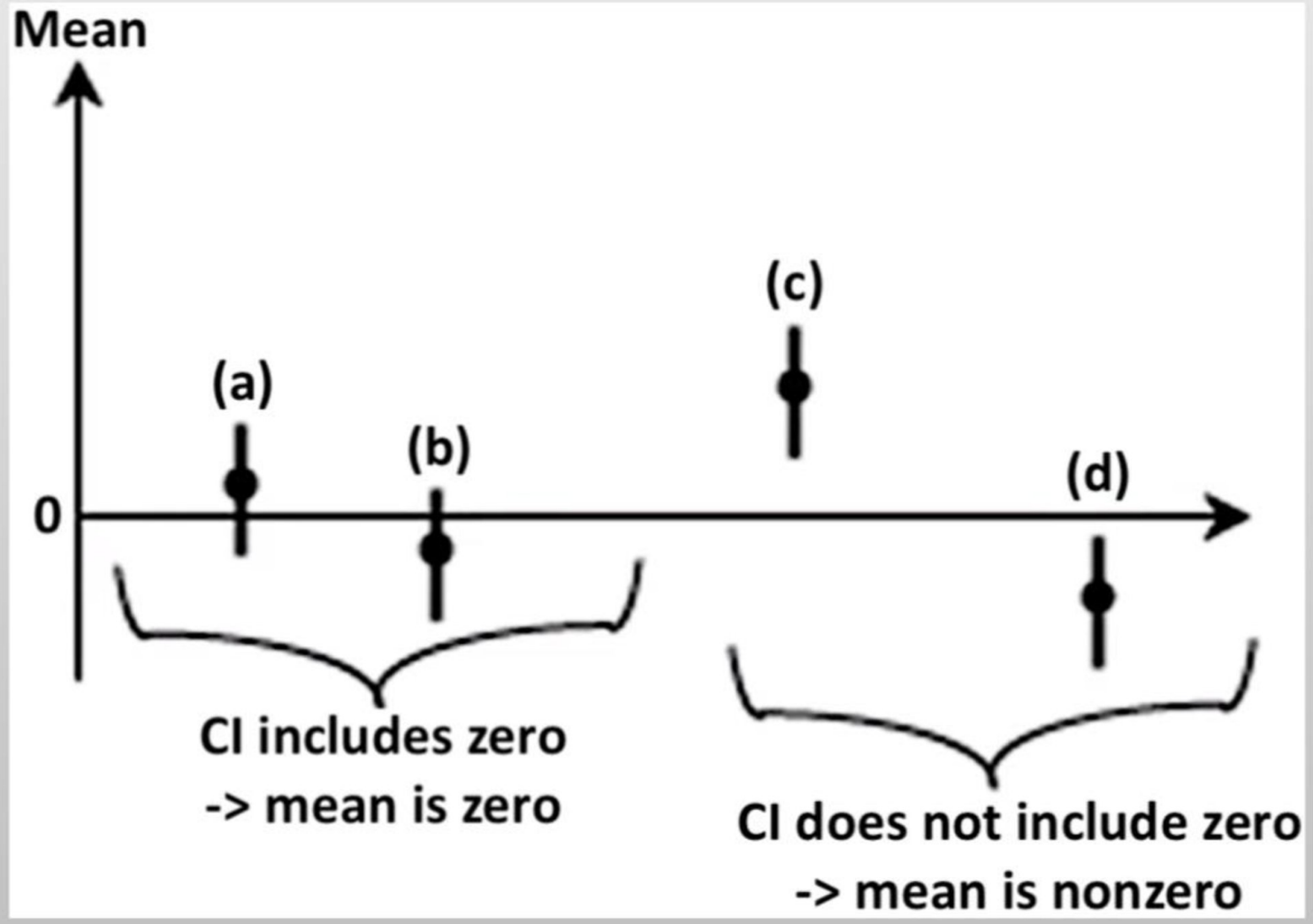
- Sample: -0.04, -0.19, 0.14, -0.09, -0.14, 0.19, 0.04, 0.09
- Mean = 0
- Sample standard deviation = 0.138
- For 90% interval: $t_{[1-\frac{\alpha}{2};n-1]} = t_{[0.95;7]} = 1.895$

$$(\overline{x} - t_{[1-\frac{\alpha}{2};n-1]}\frac{s}{\sqrt{n}}, \overline{x} + t_{[1-\frac{\alpha}{2};n-1]}\frac{s}{\sqrt{n}})$$

- Confidence interval for the mean

$$0 \mp 1.895 \times 0.138/\sqrt{8} = 0 \mp 0.092 = (-0.092, 0.092)$$

Testing for a zero mean



Example

- Difference in processor times: {1.5, 2.6, -1.8, 1.3, -0.5, 1.7, 2.4}
- Can we say with 99% confidence that one is superior to the other?

  Mean = 7.2 / 7 = 1.03
  Sample size = n = 7
  Sample variance = (22.84 - 7.2 * 7.2 / 7) / 6 = 2.57
  Sample standard deviation = sqr(2.57) = 1.6

- Confidence interval

$$1.03 \mp t * 1.6/\sqrt{7} = 1.03 \mp 0.6t$$

$$100(1-\alpha) = 99, \;\; \alpha = 0.01, \;\; 1-\alpha/2 = 0.995, \;\; t_{[0.995,6]} =$$

SYSTEMS LABORATORY

TABLE A.4 Quantiles of the *t* Distribution

	p							
n	**0.6000**	**0.7000**	**0.8000**	**0.9000**	**0.9500**	**0.9750**	**0.9950**	**0.9995**
1	0.325	0.727	1.377	3.078	6.314	12.706	63.657	636.619
2	0.289	0.617	1.061	1.886	2.920	4.303	9.925	31.599
3	0.277	0.584	0.978	1.638	2.353	3.182	5.841	12.924
4	0.271	0.569	0.941	1.533	2.132	2.776	4.604	8.610
5	0.267	0.559	0.920	1.476	2.015	2.571	4.032	6.869
6	0.265	0.553	0.906	1.440	1.943	2.447	3.707	5.959
7	0.263	0.549	0.896	1.415	1.895	2.365	3.499	5.408
8	0.262	0.546	0.889	1.397	1.860	2.306	3.355	5.041
9	0.261	0.543	0.883	1.383	1.833	2.262	3.250	4.781
10	0.260	0.542	0.879	1.372	1.812	2.228	3.169	4.587
11	0.260	0.540	0.876	1.363	1.796	2.201	3.106	4.437
12	0.259	0.539	0.873	1.356	1.782	2.179	3.055	4.318
13	0.259	0.538	0.870	1.350	1.771	2.160	3.012	4.221
14	0.258	0.537	0.868	1.345	1.761	2.145	2.977	4.140
15	0.258	0.536	0.866	1.341	1.753	2.131	2.947	4.073
16	0.258	0.535	0.865	1.337	1.746	2.120	2.921	4.015
17	0.257	0.534	0.863	1.333	1.740	2.110	2.898	3.965
18	0.257	0.534	0.862	1.330	1.734	2.101	2.878	3.922
19	0.257	0.533	0.861	1.328	1.729	2.093	2.861	3.883
20	0.257	0.533	0.860	1.325	1.725	2.086	2.845	3.850
21	0.257	0.532	0.859	1.323	1.721	2.080	2.831	3.819
22	0.256	0.532	0.858	1.321	1.717	2.074	2.819	3.792
23	0.256	0.532	0.858	1.319	1.714	2.069	2.807	3.768
24	0.256	0.531	0.857	1.318	1.711	2.064	2.797	3.745
25	0.256	0.531	0.856	1.316	1.708	2.060	2.787	3.725
26	0.256	0.531	0.856	1.315	1.706	2.056	2.779	3.707
27	0.256	0.531	0.855	1.314	1.703	2.052	2.771	3.690
28	0.256	0.530	0.855	1.313	1.701	2.048	2.763	3.674
29	0.256	0.530	0.854	1.311	1.699	2.045	2.756	3.659
30	0.256	0.530	0.854	1.310	1.697	2.042	2.750	3.646
60	0.254	0.527	0.848	1.296	1.671	2.000	2.660	3.460
90	0.254	0.526	0.846	1.291	1.662	1.987	2.632	3.402
120	0.254	0.526	0.845	1.289	1.658	1.980	2.617	3.373

EP performance crunch

Example

- Difference in processor times: {1.5, 2.6, -1.8, 1.3, -0.5, 1.7, 2.4}
- Can we say with 99% confidence that one is superior to the other?

  Mean = 7.2 / 7 = 1.03
  Sample size = n = 7
  Sample variance = (22.84 - 7.2 * 7.2 / 7) / 6 = 2.57
  Sample standard deviation = sqr(2.57) = 1.6

- Confidence interval

  $$1.03 \mp t * 1.6/\sqrt{7} = 1.03 \mp 0.6t$$

  $$100(1-\alpha) = 99,\ \ \alpha = 0.01,\ \ 1-\alpha/2 = 0.995,\ \ t_{[0.995,6]} = 3.707$$

- 99% confidence interval = (-1.21, 3.27)

Example (continued)

- Opposite signs ⇒ we cannot say with 99% confidence that the mean difference is significantly different from zero.
- Answer: They are the same.
- Answer: The difference is zero.

Example (continued)

- Difference in processor times: {1.5, 2.6, -1.8, 1.3, -0.5, 1.7, 2.4}
- 99% confidence interval = (-1.21, 3.27)

- Is the difference 1?

- Answer: Yes, the difference is 1.

SYSTEMS LABORATORY

Paired vs unpaired comparisons

EP performance crunch

- Paired
  - One-to-one correspondence between the ith test on system A and ith test on system B
  - Example: performance on i-th workload
  - Use confidence interval of the difference
- Unpaired
  - No correspondence
  - Number of observations may differ
  - Example
    - n people on system A, n on system B
    - Need more sophisticated method

Example

- Performance: {(5.4, 19.1), (16.6, 3.5), (0.6, 3.4), (1.4, 2.5), (0.6, 3.6) (7.3, 1.7)}
  Is one system better?
- Differences: {-13.7, 13.1, -2.8, -1.1, -3.0, 5.6}
  Sample mean = -0.32
  Sample variance = 81.62
  Sample standard deviation = 9.03
  Confidence interval for the mean

$$-0.32 \mp t\sqrt{81.62/6} = -0.32 \mp t(3.69), \;\; t_{[0.95,5]} = 2.015$$

  90% confidence interval: $-0.32 \mp (2.015)(3.69) = (-7.75, 7.11)$
- Answer: No, they are not different.

Unpaired observations

- Compute sample means

$$\overline{x}_a = \frac{1}{n_a} \sum_{i=1}^{n_a} x_{ia}$$

$$\overline{x}_b = \frac{1}{n_b} \sum_{i=1}^{n_b} x_{ib}$$

- Compute the sample standard deviations

$$s_a = \left\{ \frac{(\sum_{i=1}^{n_a} x_{ia}^2) - n_a x_a^{-2}}{n_a - 1} \right\}^{\frac{1}{2}}$$

$$s_b = \left\{ \frac{(\sum_{i=1}^{n_b} x_{ib}^2) - n_b x_b^{-2}}{n_b - 1} \right\}^{\frac{1}{2}}$$

Unpaired observations (continued)

- Compute mean difference $(\overline{x}_a - \overline{x}_b)$
- Compute standard deviation of the mean difference

$$s = \sqrt{\left(\frac{s_a^2}{n_a} + \frac{s_b^2}{n_b}\right)}$$

- Compute the effective number of degrees of freedom

$$\nu = \frac{\left(\frac{s_a^2}{n_a} + \frac{s_b^2}{n_b}\right)^2}{\frac{1}{n_a+1}\left(\frac{s_a^2}{n_a}\right)^2 + \frac{1}{n_b+1}\left(\frac{s_b^2}{n_b}\right)^2} - 2$$

- Compute the confidence interval for the mean difference

$$(\overline{x}_a - \overline{x}_b) \mp t_{[1-\alpha/2;\nu]} s$$

Example

- Times on system A: {5.36, 16.57, 0.62, 1.41, 0.64, 7.26}
  Times on system B: {19.12, 3.52, 3.38, 2.50, 3.60, 1.74}
- Are the two systems significantly different?

- System A
  Mean $\overline{x}_a = 5.31$
  Variance $s_a^2 = 37.92$
  $n_a = 6$

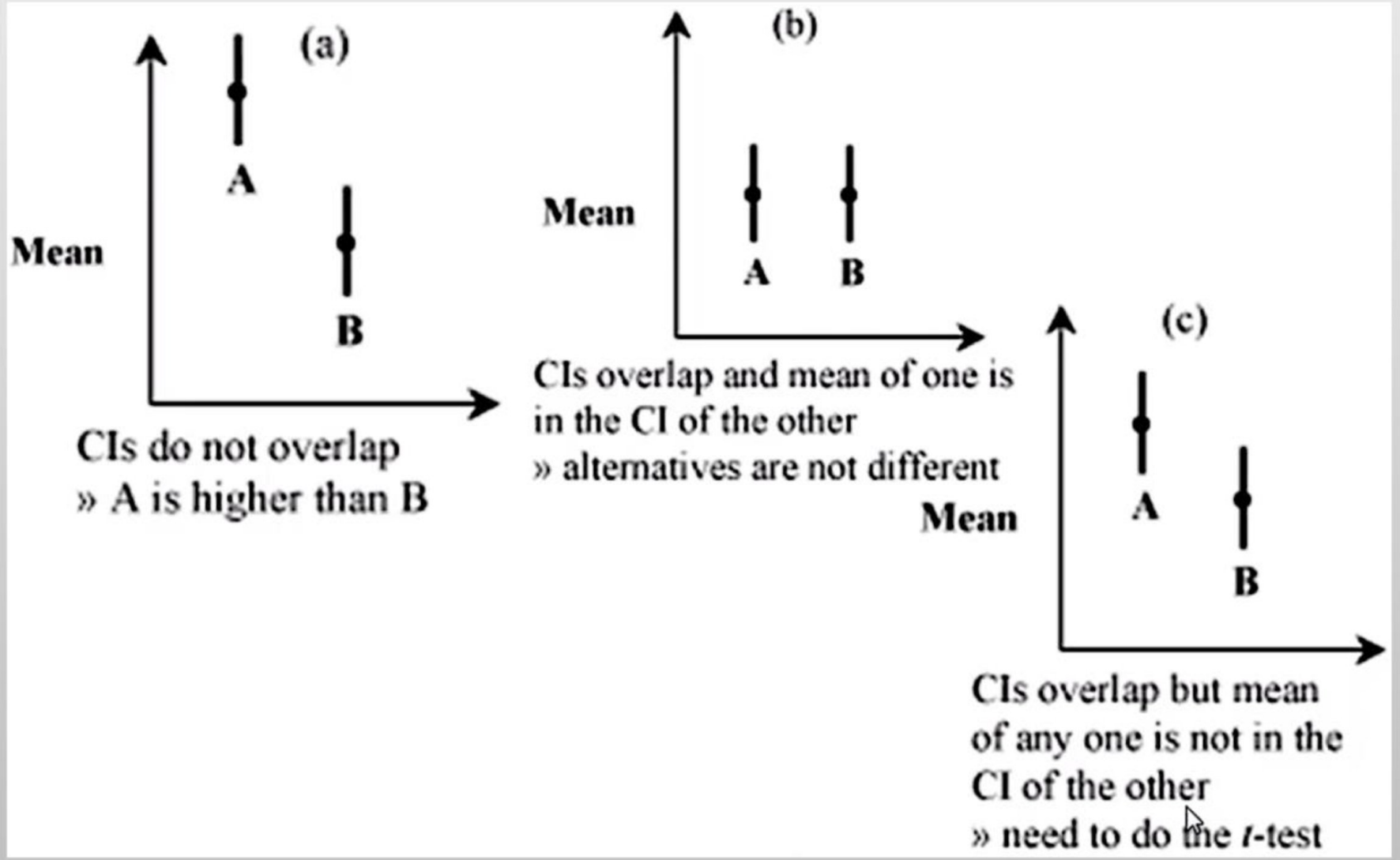
- System B
  Mean $\overline{x}_b = 5.64$
  Variance $s_b^2 = 44.11$
  $n_b = 6$

Example (continued)

- Mean difference $\overline{x}_a - \overline{x}_b$ = -0.33
- Standard deviation of the mean difference = 3.698
- Effective number of degrees of freedom f = 11.921
- 0.95-quantile of a t-variate with 12 degrees of freedom = 1.71
- 90% confidence interval for the difference = (-6.92, 6.26)

- The confidence interval includes zero
  ⇒ the two systems are not different

Approximate visual test



Example

- Times on system A: {5.36, 16.57, 0.62, 1.41, 0.64, 7.26}
  Times on system B: {19.12, 3.52, 3.38, 2.50, 3.60, 1.74}
  $t_{[0.95,5]} = 2.015$
- 90% confidence interval for the mean of A
  $5.31 \mp (2.015)\sqrt{37.92/6} = (0.24, 10.38)$
- 90% confidence interval for the mean of B
  $5.64 \mp (2.015)\sqrt{44.11/6} = (0.18, 11.10)$
- Confidence intervals overlap and the mean of one falls in the confidence interval of the other
  ⇒ The two systems are not different at this level of confidence

What level of confidence to use?

- 90%, 95%, 99% ?
- Base on the loss that you will sustain if the parameter is outside the range and the gain you would have if the parameter is inside the range.
- Low loss → low confidence is fine
  - Example: Lottery of $5million with probability 10^(-7)
    - 90% confidence → buy 9 million tickets
    - 0.01% confidence level if fine
  - 50% confidence may or may not be too low
  - 99% confidence may or may not be too high

One-sided confidence intervals

- Two side intervals - 90% confidence
  - P(difference > upper limit) = 5%
  - P(difference < lower limit) = 5%
- One-sided confidence interval
  - One sided question: Is the mean greater than 0?
- One-sided lower confidence interval for μ

$$\left(\overline{x} - t_{[1-\alpha;n-1]}\frac{s}{\sqrt{n}}, \overline{x}\right)$$

  Note t at 1-α (not at 1-α/2)
- One-sided upper confidence interval for μ

$$\left(\overline{x}, \overline{x} + t_{[1-\alpha;n-1]}\frac{s}{\sqrt{n}}\right)$$

- For large samples - use z instead of t

Example

- Time between crashes

System	Number	Mean	Stdv
A	972	124.10	198.20
B	153	141.47	226.11

- Assume unpaired observations
- Mean difference

$$\overline{x}_A - \overline{x}_B = 124.10 - 141.47 = -17.37$$

- Standard deviation of the difference

$$s = \sqrt{\left(\frac{s_a^2}{n_a} + \frac{s_b^2}{n_b}\right)} = \sqrt{\left(\frac{(198.20)^2}{972} + \frac{(226.11)^2}{153}\right)} = 19.35$$

Example (continued)

- Effective number of degrees of freedom

$$\nu = \frac{\left(\frac{s_a^2}{n_a} + \frac{s_b^2}{n_b}\right)^2}{\frac{1}{n_a+1}\left(\frac{s_a^2}{n_a}\right)^2 + \frac{1}{n_b+1}\left(\frac{s_b^2}{n_b}\right)^2} - 2 = \ldots = 191.05$$

- v > 30 ⇒ use z instead of t
- One-sided test ⇒ Use $z_{0.90} = 1.28$ for 90% confidence
- 90% confidence interval: (-17.37, -17.37+1.28*19.35) = (-17.37, 7.402)
- CI includes zero => system A is not more susceptible to crashes than system B.

Sample size for determining mean

- Large sample ⇒ narrower confidence interval - Higher confidence
- Question
  - How many observations n to get an accuracy of +/- r% and a confidence level of 100(1-α)%?

$$\overline{x} \mp z\frac{s}{\sqrt{n}}$$

- r% accuracy

$$CI = (\overline{x}(1 - r/100), \overline{x}(1 + r/100))$$

$$\overline{x} \mp z\frac{s}{\sqrt{n}} = \overline{x}\left(1 \mp \frac{r}{100}\right) \Rightarrow z\frac{s}{\sqrt{n}} = \overline{x}\frac{r}{100} \Rightarrow n = \left(\frac{100zs^2}{r\overline{x}}\right)$$

Example

- Sample mean of the response time = 20 sec
- Sample standard deviation = 5
- Question: How many repetitions are needed to get the response time accurate within 1 second at 95% confidence?
- Required accuracy is 1 in 20 = 5%

$$\overline{x} = 20, \quad s = 5, \quad z = 1.960, \quad r = 5$$

$$n = \left(\frac{100 \times 1.960 \times 5}{5 \times 20} \right)^2 = 9.8^2 = 96.04$$

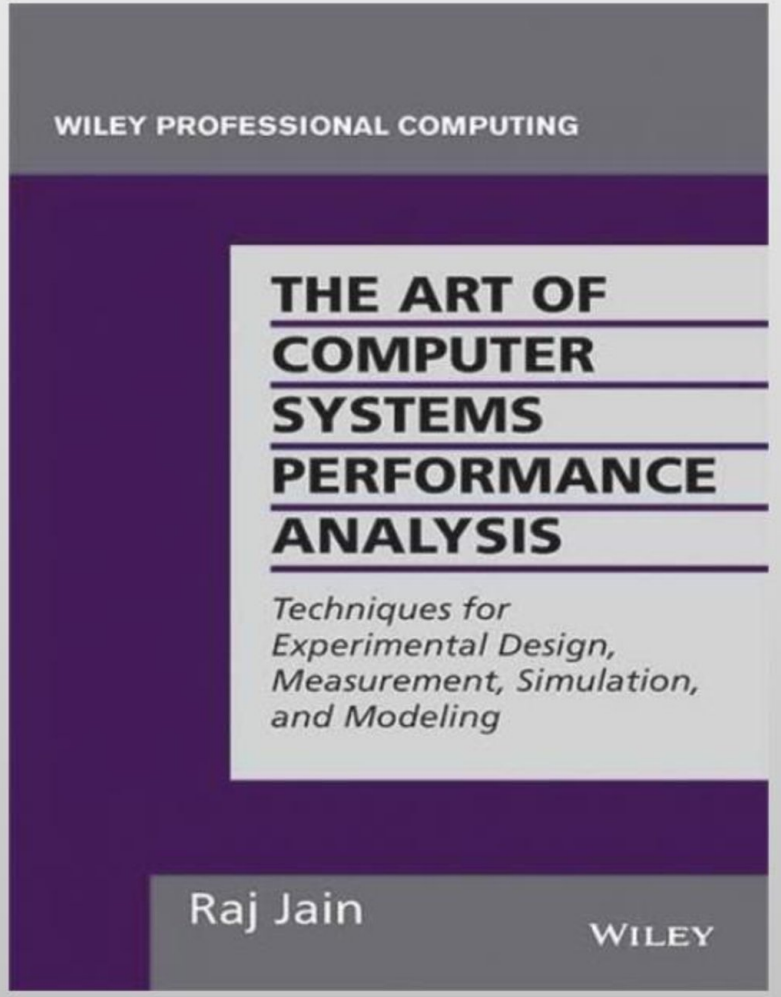
- Answer: 97 observations are needed

Takeaways

- All statistics based on a sample are random and should be specified with a confidence interval
- If the confidence interval includes zero, the hypothesis that the population mean is zero cannot be rejected
- Paired observations ⇒ test the difference for zero mean
- Unpaired observations ⇒ more sophisticated

References

1. Raj K. Jain, "*Art of Computer Systems Performance Analysis: Techniques for Experimental Design Measurements, Simulation and Modeling*", John Wiley, 2015.
2. Raj Jain, CSE 567M Computer Systems Analysis, Washington University in St. Louis, 2013.



QA TECHNIQUES

Murgan Mihai – mmurgan@bitdefender.com
George TRIF – gtrif@bitdefender.com
Adrian MOISE – amoise@bitdefender.com

SOFTWARE

Software consists of carefully-organized instructions and code written by programmers in any of various special computer languages which are not mathematically proven that they do what they should do.

History of the first bug.

Why do we need QA.

SOFTWARE TEST TYPES

Software Test types are introduced as a means of clearly defining the objective of a certain level for a program or project. A test type is focused on a particular test objective, which could be the testing of the function to be performed by the component or system.

The test objective could be to test non-functional quality characteristics, such as reliability or usability; the structure or architecture of the component or system; or related to changes, i.e confirming that defects have been fixed (confirmation testing or retesting) and looking for unintended changes (regression testing).

Depending on its objectives, testing will be organized differently. Hence there are four software test types:

- Functional testing
- Non-functional testing
- Structural testing
- Change related testing

TEST TYPES: FUNCTIONAL TESTING

Target: Test the functionalities (features) of a product

- Specification based:
  - uses Test Cases, derived from the specifications (Use Cases)
  - business process based, using business scenarios
- Focused on checking the system against the specifications
- Can be performed at all test levels
- Considers the external behavior of the system
- Black box design techniques will be used
- Security testing is part of functional testing, related to the detection of threats

TEST TYPES: NON-FUNCTIONAL TESTING

Non-functional testing = testing the attributes of a component or system that do not relate to functionality: the way a system operates, rather than specific behaviors of that system. Is also performed at all levels like functional testing.

Non-functional testing includes:

- Performance testing
- Documentation testing
- Compliance testing
- Load testing (how much load can be handled by the system?)
- Stress testing (evaluate system behavior at limits and out of limits)
- Usability testing
- Reliability testing
- Portability testing
- Maintainability testing
- …

NON-FUNCTIONAL TESTING TECHNIQUES

Usability testing = used to determine the extent to which the software product is understood, easy to learn, easy to operate and attractive to the users under specified conditions.

Usability testing includes the following five components:

- **Learnability**: How easy is it for users to accomplish basic tasks the first time they encounter the design?
- **Efficiency**: How fast can experienced users accomplish tasks?
- **Memorability**: When users return to the design after a period of not using it, does the user remember enough to use it effectively the next time, or does the user have to start over again learning everything?
- **Errors**: How many errors do users make, how severe are these errors and how easily can they recover from the errors?
- **Satisfaction**: How much does the user like using the system?

NON-FUNCTIONAL TESTING TECHNIQUES

Instalability testing = The process of testing the instalability of a software product.

- Does the installation work?
- How easy is to install the system?
- Does installation affect other software?
- Does the environment affect the product?
- Does it uninstall correctly?

Reliability testing = Is about exercising an application so that failures are discovered and removed before the system is deployed. The purpose of reliability testing is to determine product reliability, and to determine whether the software meets the customer's reliability requirements.

NON-FUNCTIONAL TESTING TECHNIQUES

Maintainability testing: It basically defines that how easy it is to maintain the system. This means that how easy it is to analyze, change and test the application or product.

Portability testing: It refers to the process of testing the ease with which a computer software component or application can be moved from one environment to another, e.g. moving of any application from Windows Vista to Windows 10. This is usually measured in terms of the maximum amount of effort permitted. Results are measured in terms of the time required to move the software and complete the and documentation updates.

Bitdefender

NON-FUNCTIONAL TESTING TECHNIQUES

Load Test = A test type concerned with measuring the behavior of a component or system with increasing load, e.g. number of parallel users and/or numbers of transactions to determine what load can be handled by the component or system

Stress Test = Testing conducted to evaluate a system or component at or beyond the limits of its specified requirements.

Performance Test = The process of testing to determine the performance of a software product. Performance can be measured watching:

- Response time
- Throughput
- Resources utilization

NON-FUNCTIONAL TESTING TECHNIQUES

Spike Test = Keeping the system, periodically, for short amounts of time, beyond its specified limits

Endurance Test = a Load Test performed for a long time interval (week(s))

Volume Test = Testing where the system is subjected to large volumes of data

TEST TYPES: STRUCTURAL TESTING

Targeted to test:

- internal structure (component)
- architecture (system)

Uses only white box design techniques:

- Can be performed at all test levels
- Used also to help measure the coverage (% of items being covered by tests)
- Tool support is critical

TEST TYPES: CHANGE RELATED TESTING

Change related testing = CONFIRMATION & REGRESSION TESTING

Confirmation testing = Re-testing of a module or product, to confirm that the previously detected defect was fixed

- Implies the use of a bug tracking tool
- Confirmation testing is not the same as the “debugging”(debugging is a development activity, not a testing activity)

Regression testing = Re-testing of a previously tested program following modification to ensure that defects have not been introduced or uncovered as a result of the changes made. It is performed when the software or its environment is changed

- Can be performed at all test levels
- Can be automated (because of cost and schedule reasons)

AUTOMATED TESTING

- Executes tests fast and reliable
- Expensive is short-term, money saver in the long-term
- Interesting and captivating
- Cannot replace human observation (user friendliness, customer experience)
- Smoke / Sanity / Other scripts

CLICK TO ADD SUBTITLE

Windows Internals

Process Monitor

Registry

Registry operations, such as creating, enumerating, querying, and deleting keys and values.

File System

Operations on local storage and remote file systems, including file systems or devices added while Procmon was running.

Network

UDP and TCP- network activity, including source and destination addresses (but not the actual data that was transmitted or received).

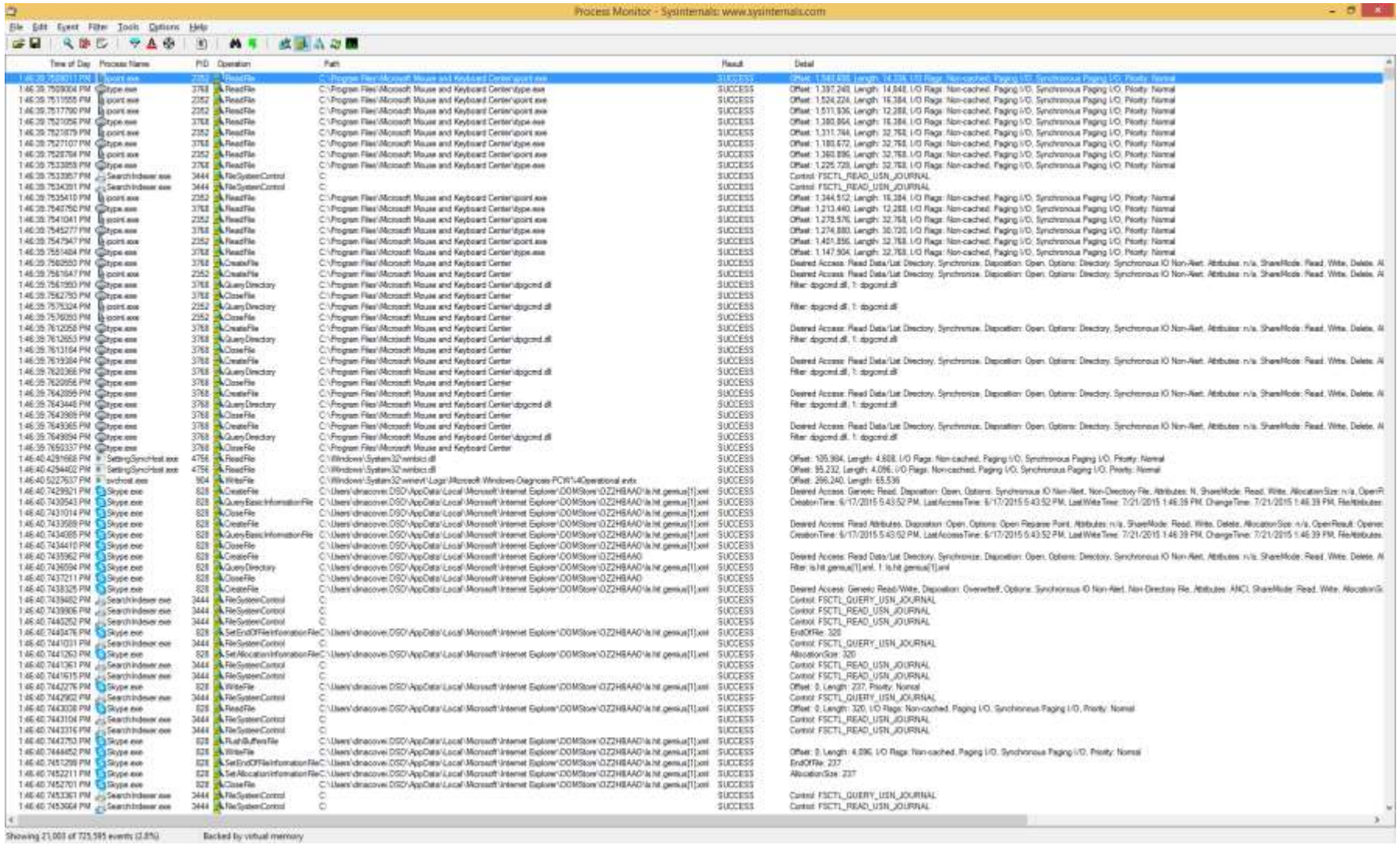
Process

Process and thread events such as process creation by a parent process, process start, thread create, thread exit, process exit

Profiling

Generates and logs an event for every process and thread on the system

Gui


Process Monitor - Sysinternals: www.sysinternals.com
File Edit Event Filter Tools Options Help
Time of Day Process Name PID Operation Path Result Detail

Column Display Defaults

- Time of Day
- Process Name
- PID
- Operation
- Path
- Result
- Detail

Results examples

- SUCCESS
- ACCESS DENIED
- SHARING VIOLATION
- NAME COLLISION
- NAME NOT FOUND ,PATH NOT FOUND ,NO SUCH FILE
- NAME INVALID

Results examples

- NO MORE ENTRIES NO MORE FILES
- END OF FILE
- BUFFER TOO SMALL
- FILE LOCKED WITH ONLY READERS
- FILE LOCKED WITH WRITERS
- IS DIRECTORY

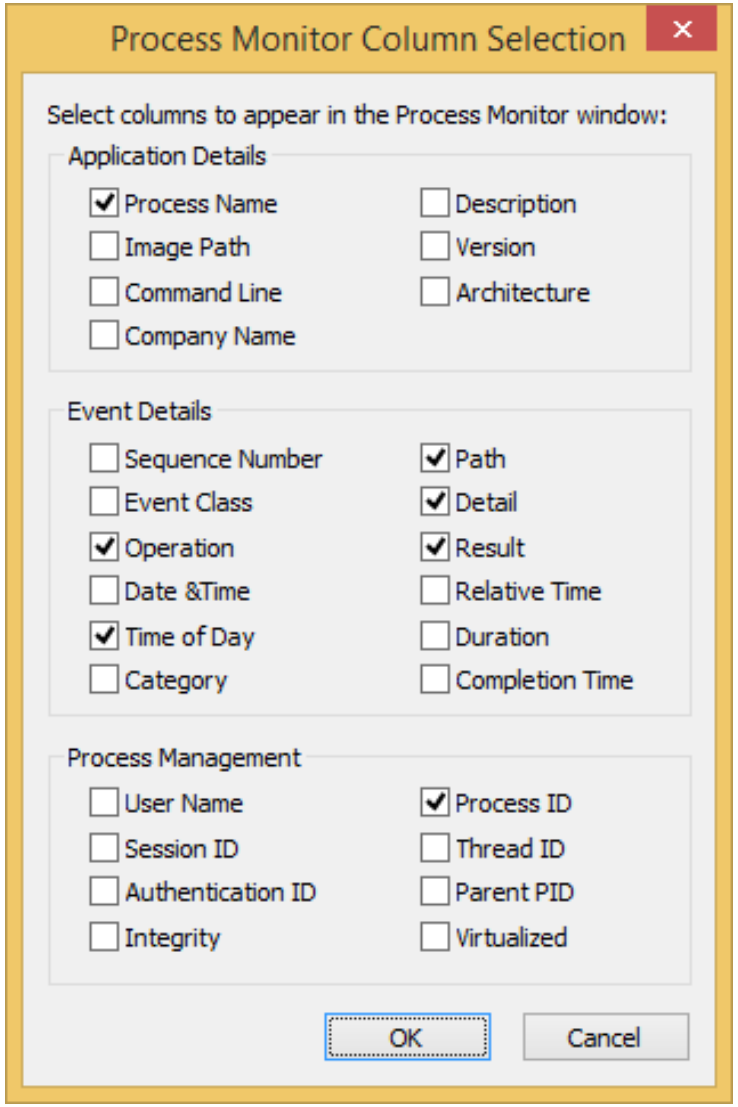
Results examples

- INVALID DEVICE REQUEST
- INVALID PARAMETER
- NOT GRANTED
- CANCELLED
- BAD NETWORK PATH
- BAD NETWORK NAME

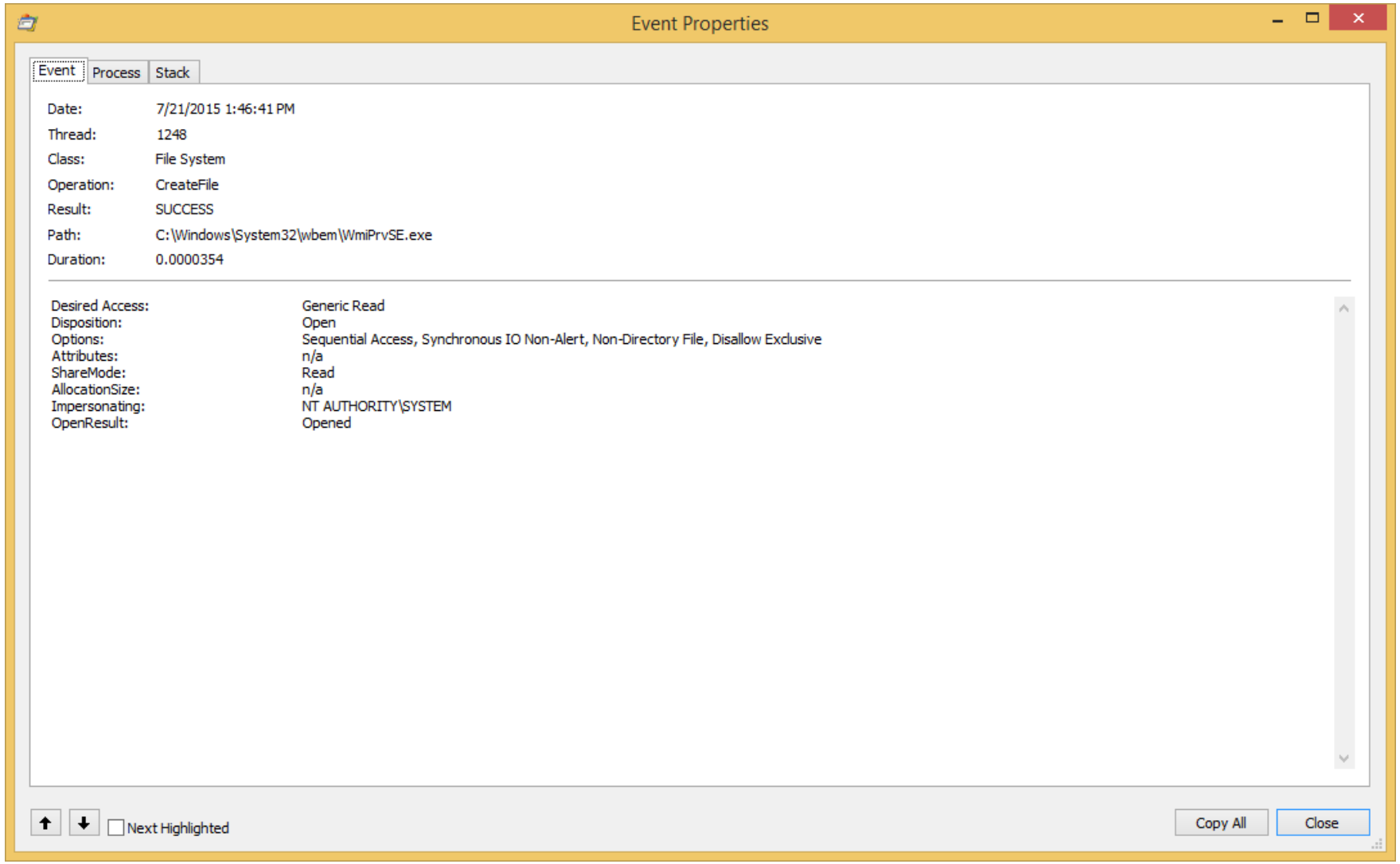
Results examples

- MEDIA WRITE PROTECTED
- KEY DELETED
- NOT IMPLEMENTED

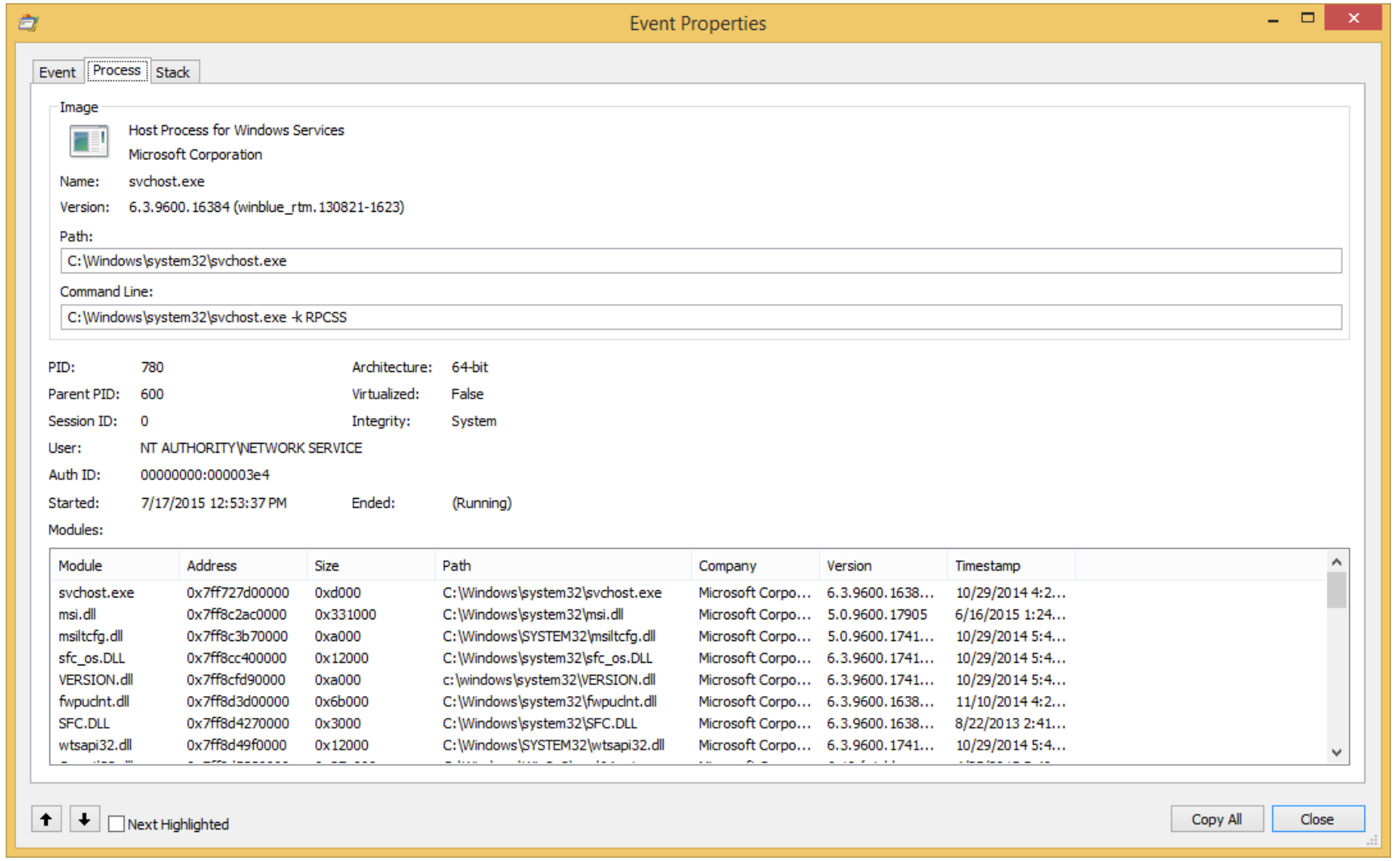
Customize Column View



Event properties

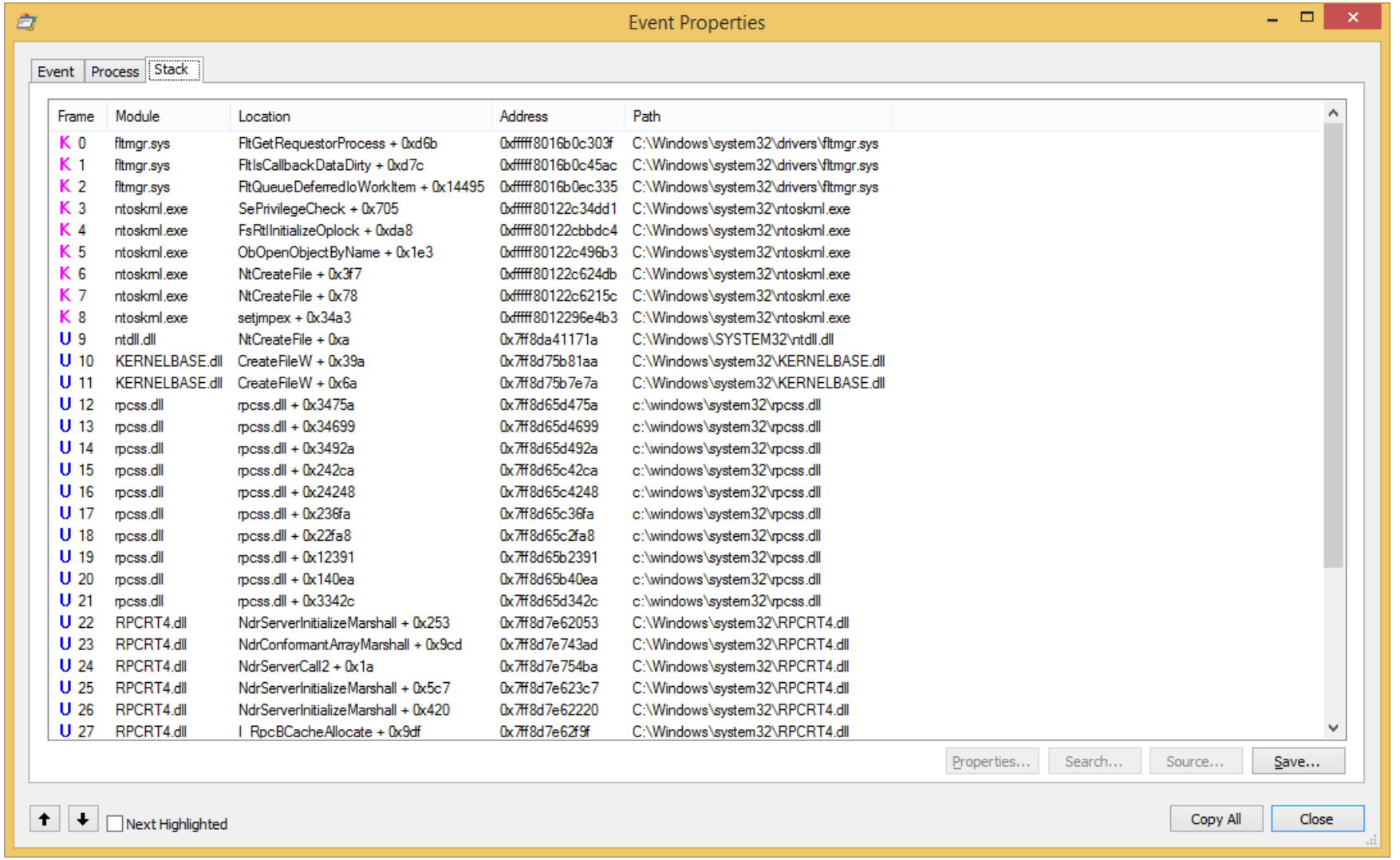


Event properties



Bitdefender

Event properties

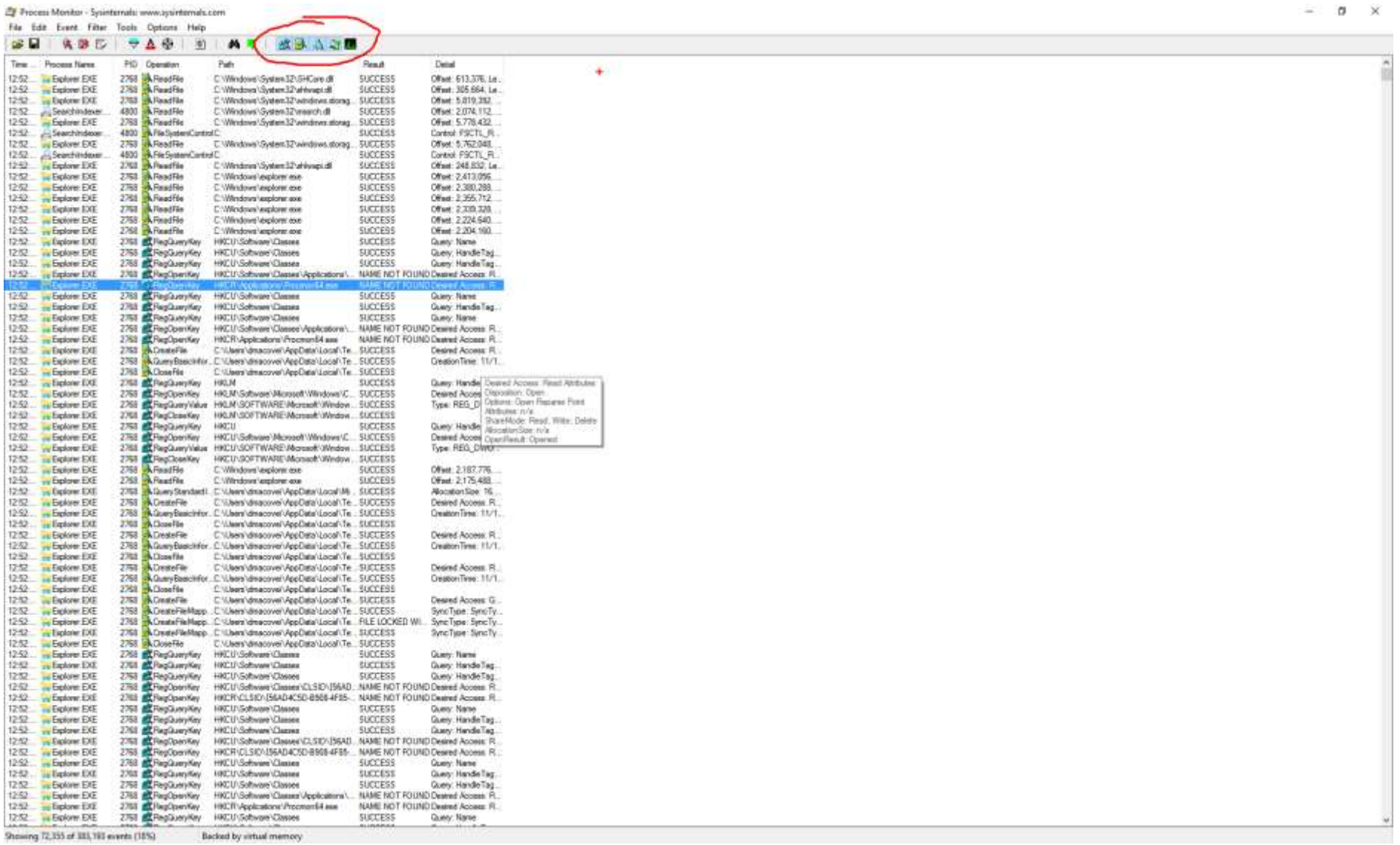


Bitdefender

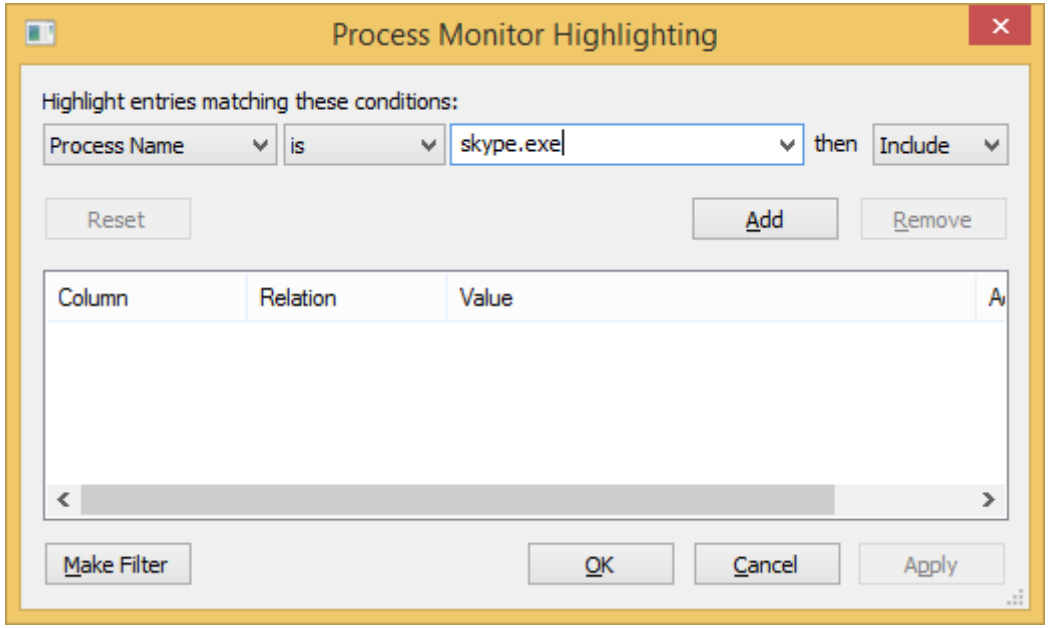
Operations

- Finding an Event
- Copying Event Data
- Jumping to a Registry or File Location
- Searching Online

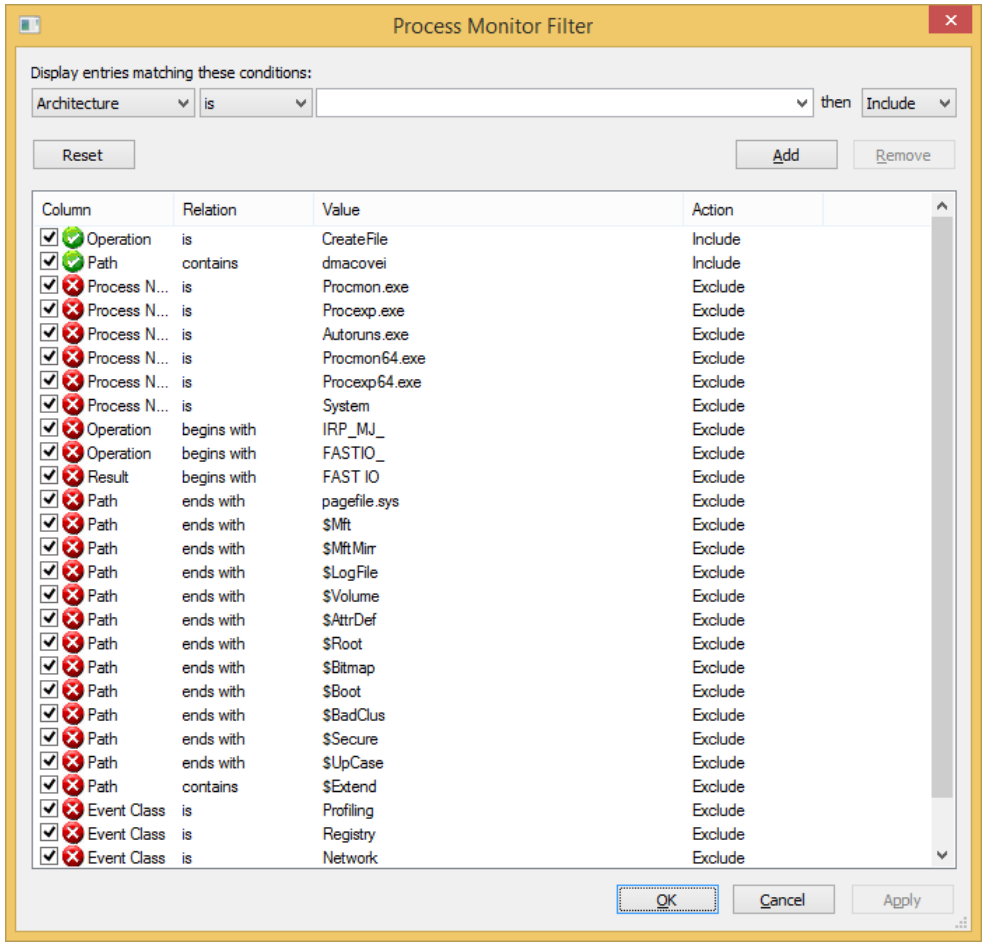
Filtering and Highlighting



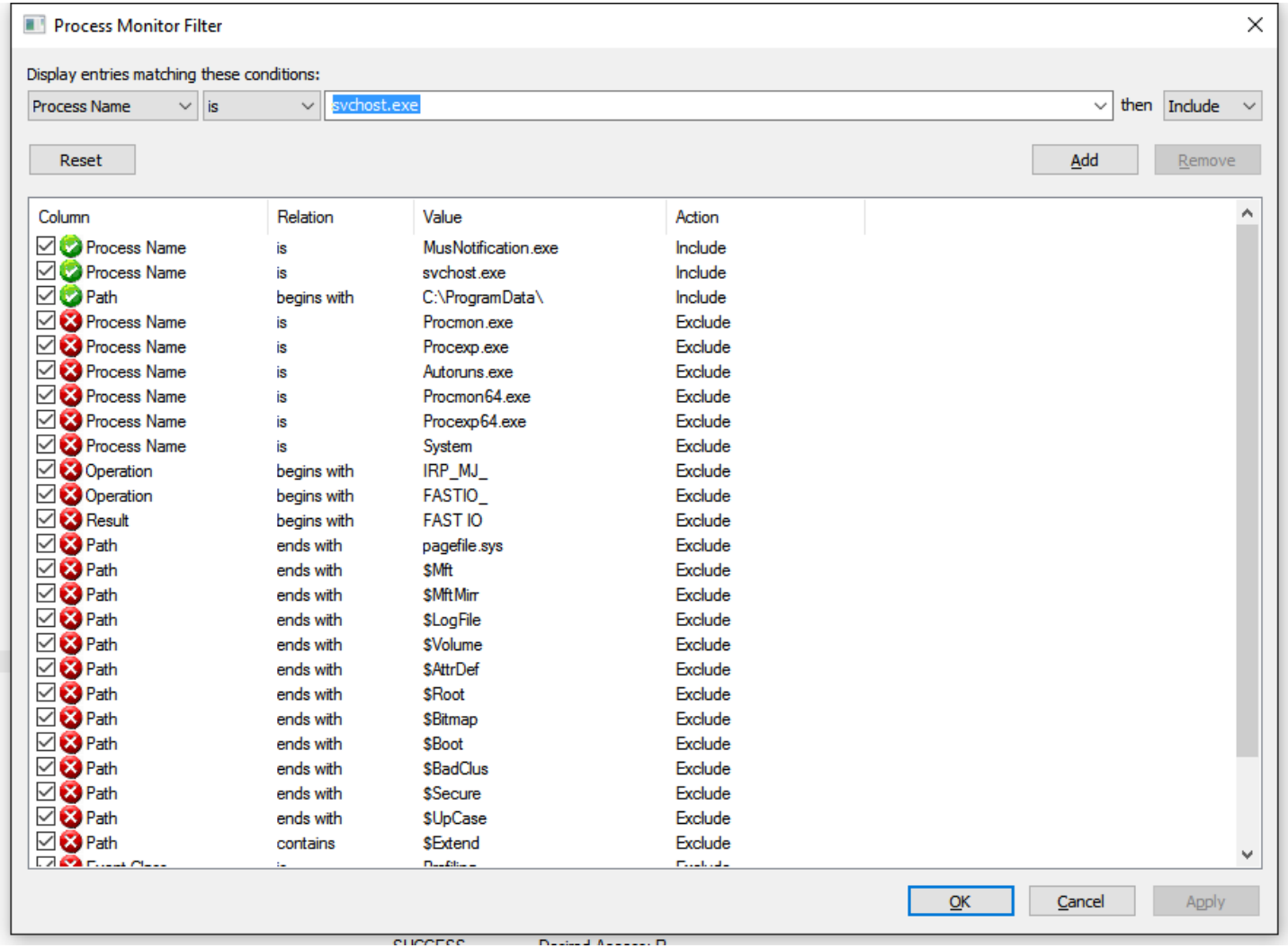
Filtering and Highlighting



Filtering and Highlighting

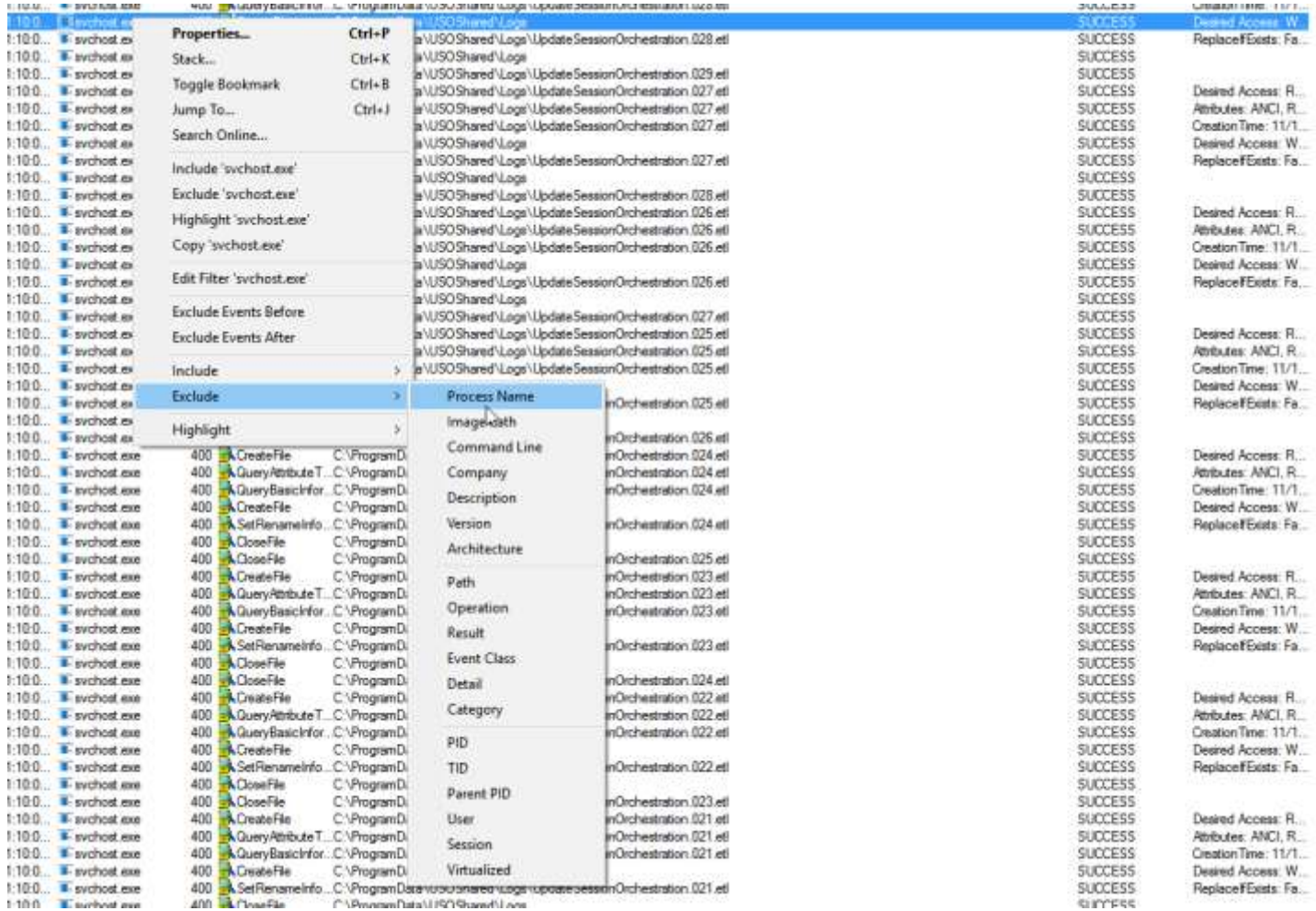


Filtering and Highlighting



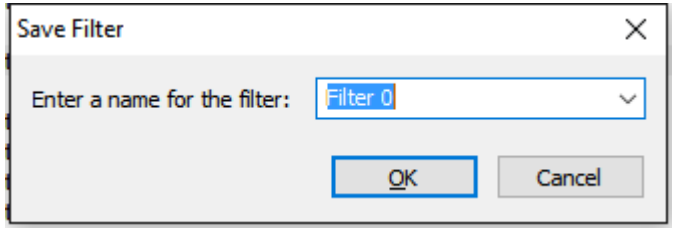
Bitdefender

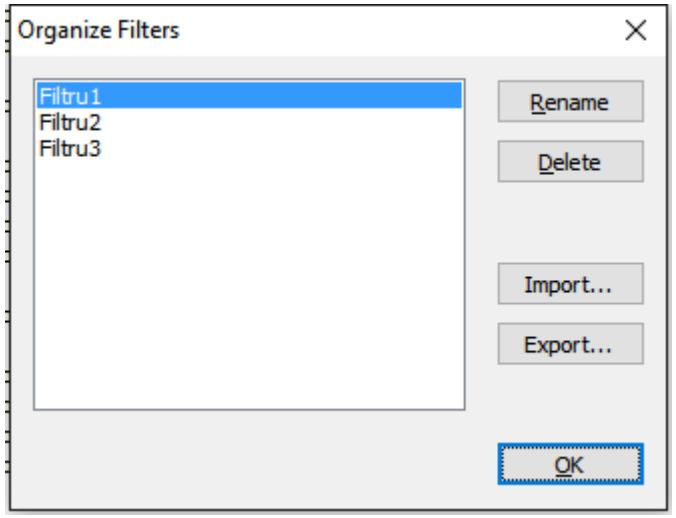
Filtering and Highlighting



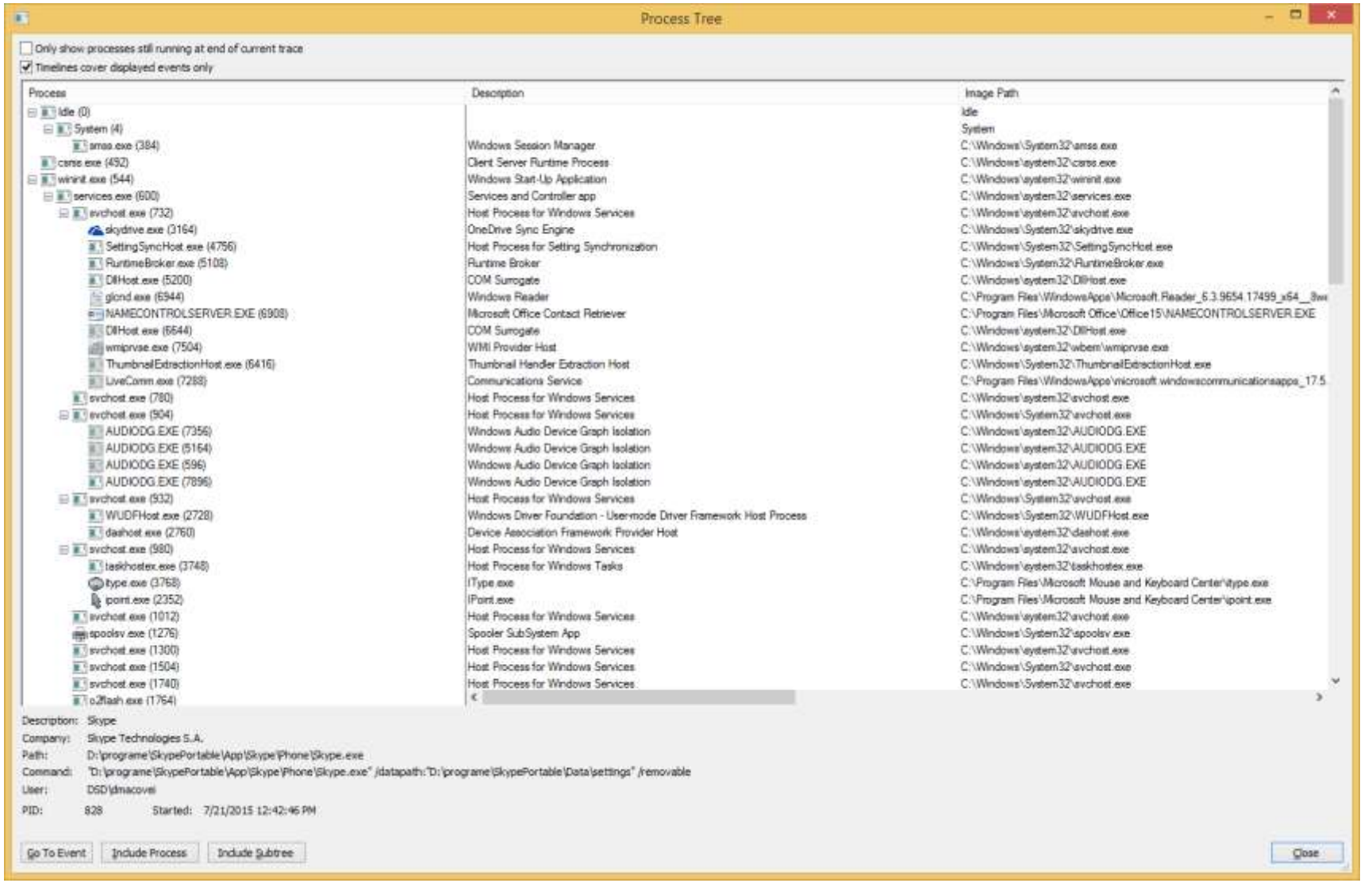
Bitdefender

Filtering and Highlighting



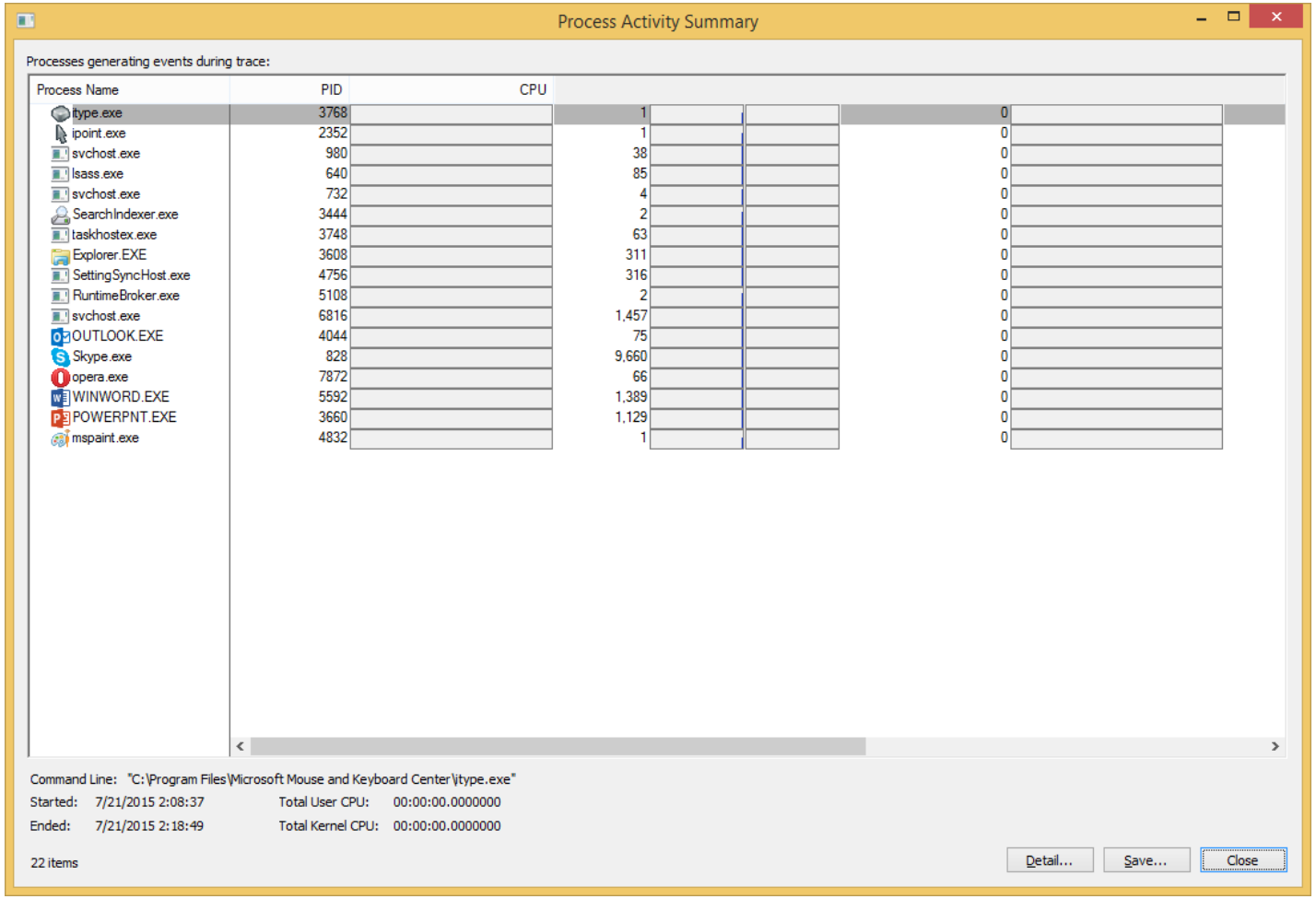


Process Tree



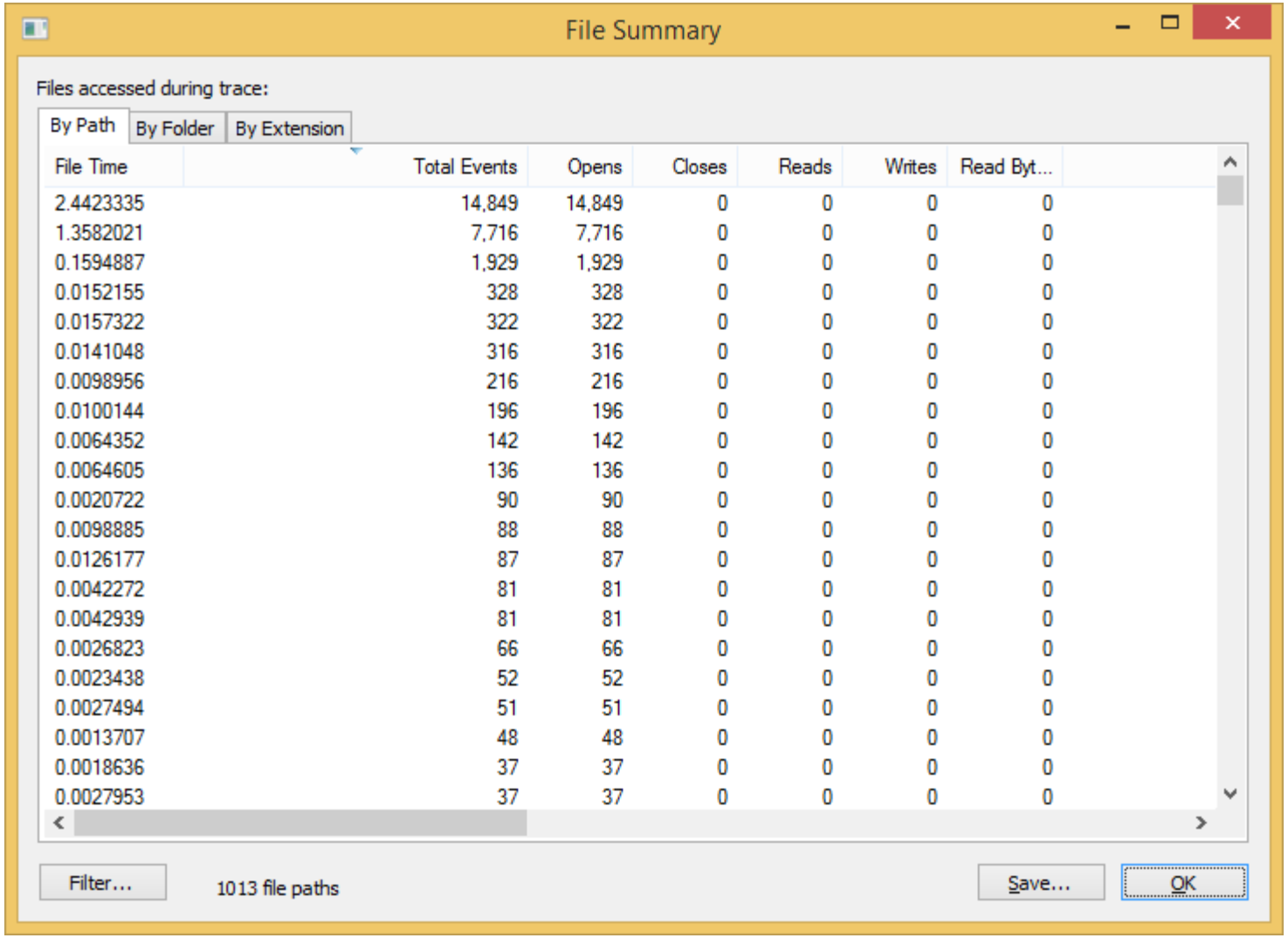
Bitdefender

Process activity summary



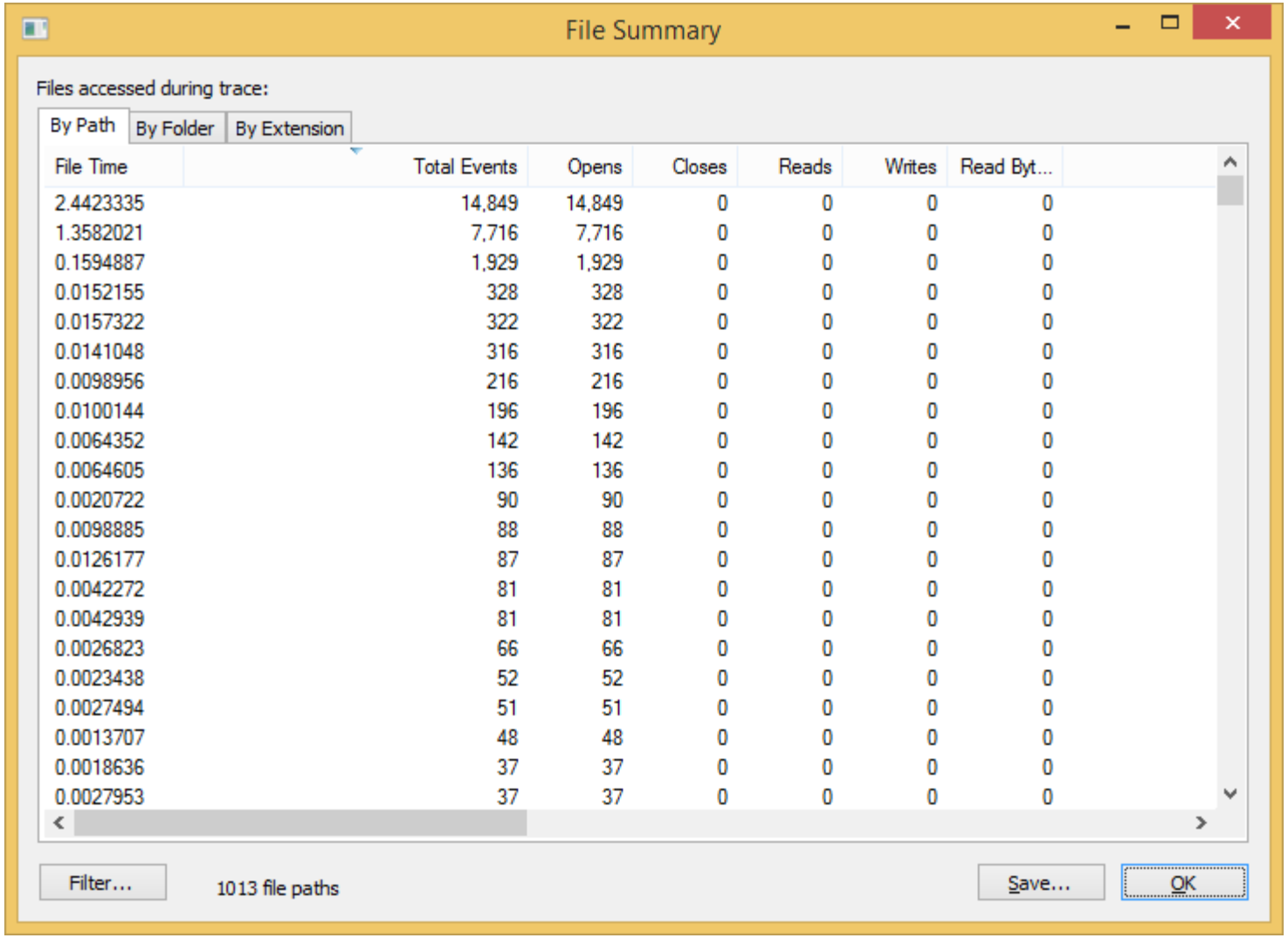
Bitdefender

File summary



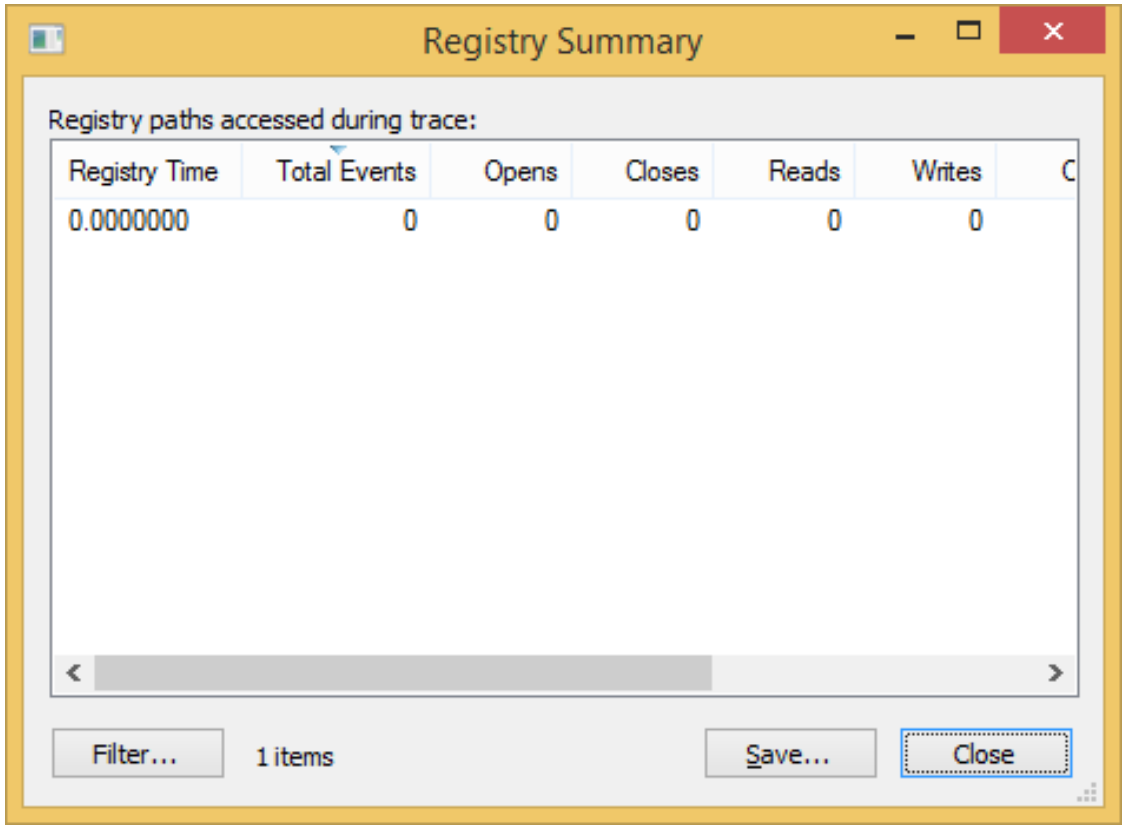
Bitdefender

File summary

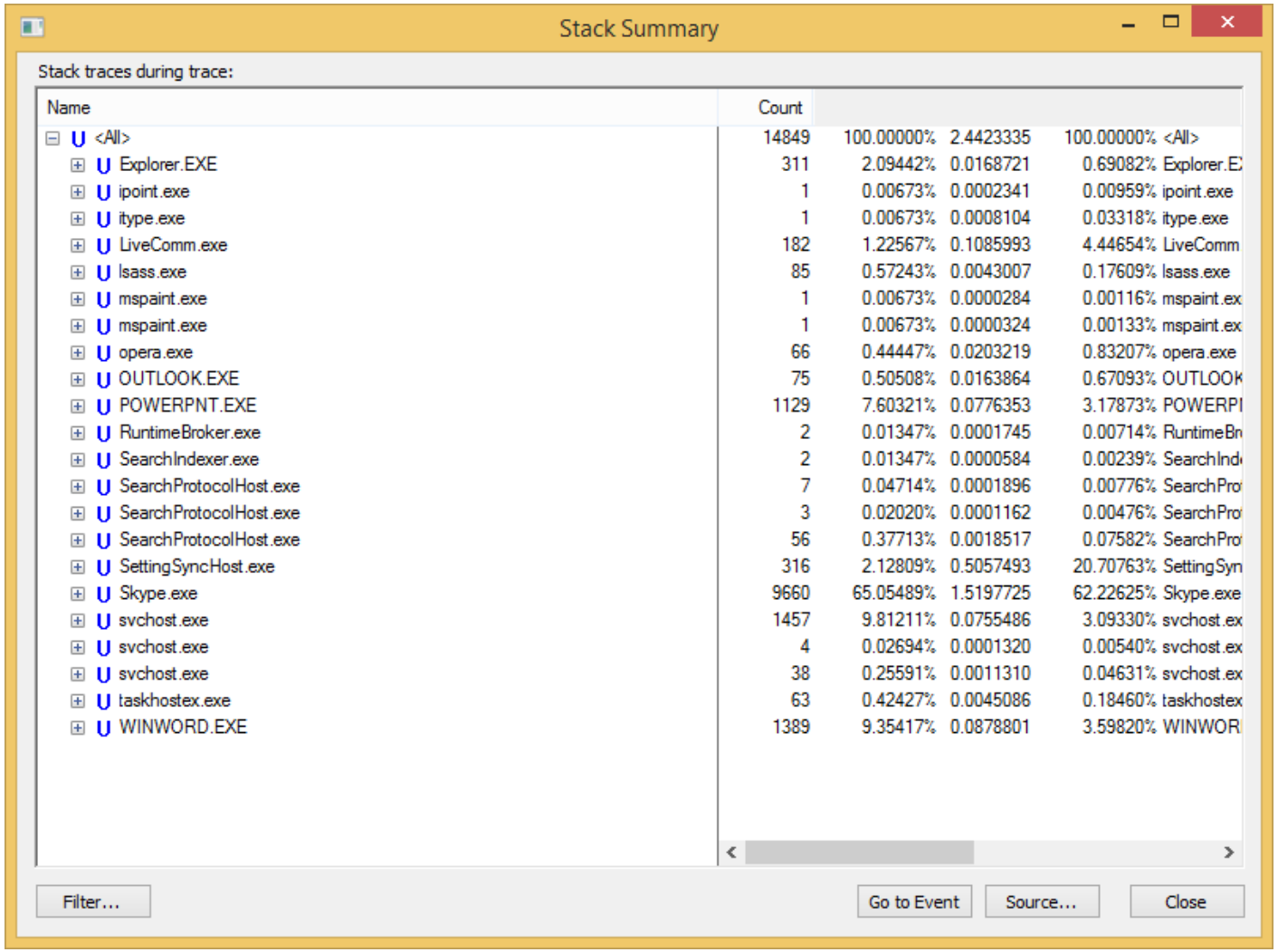


Bitdefender

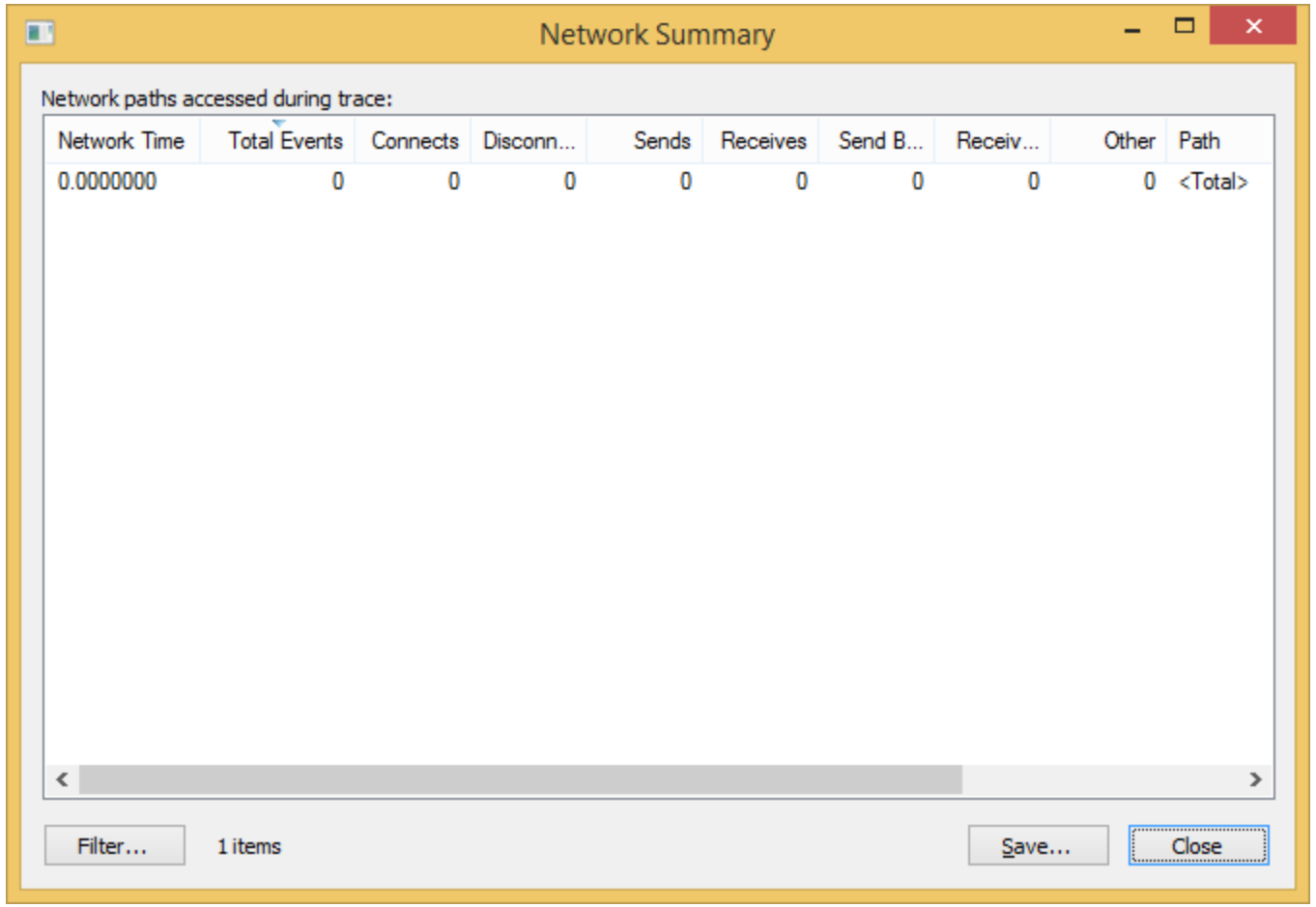
Registry summary



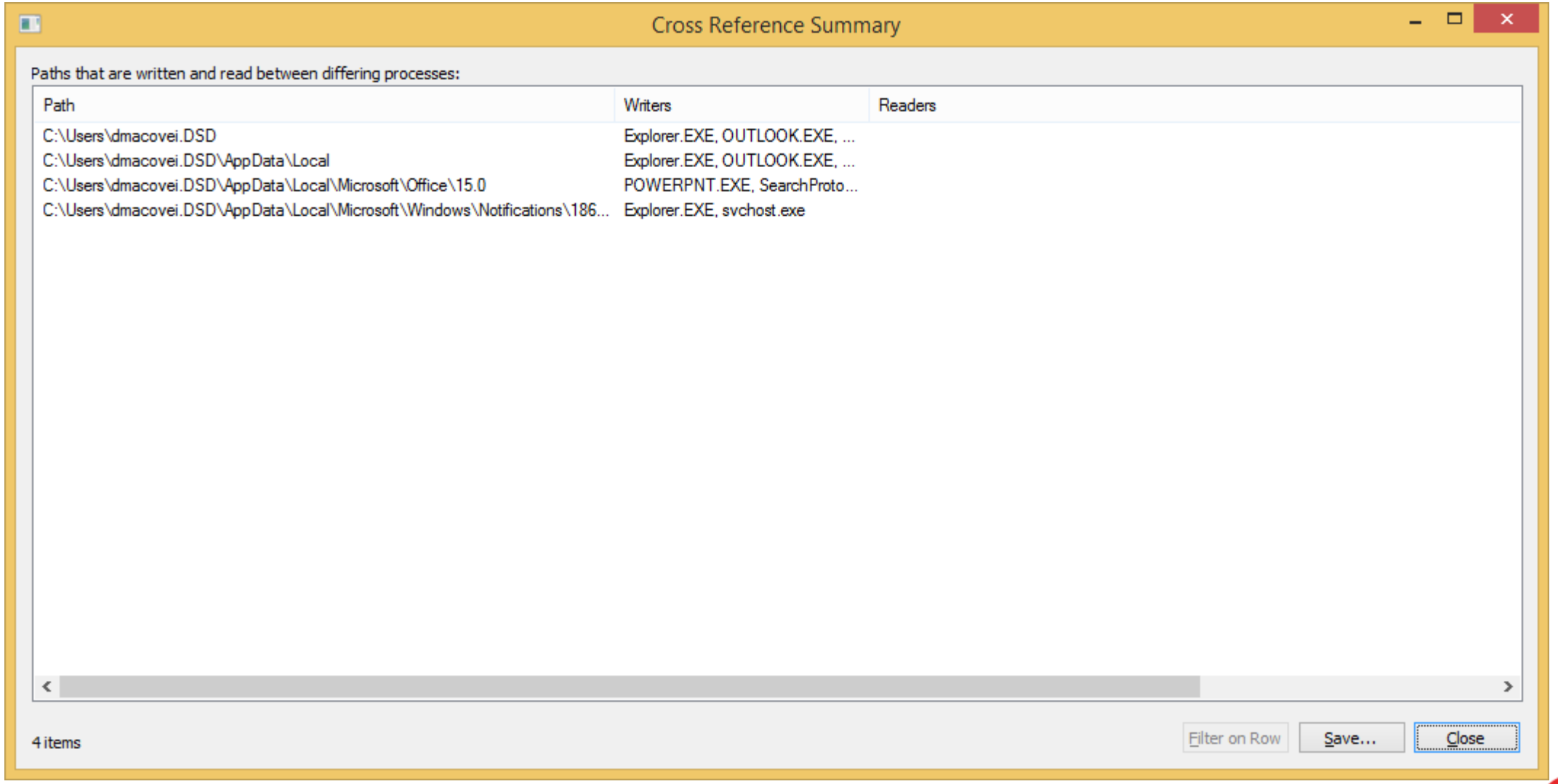
Stack summary



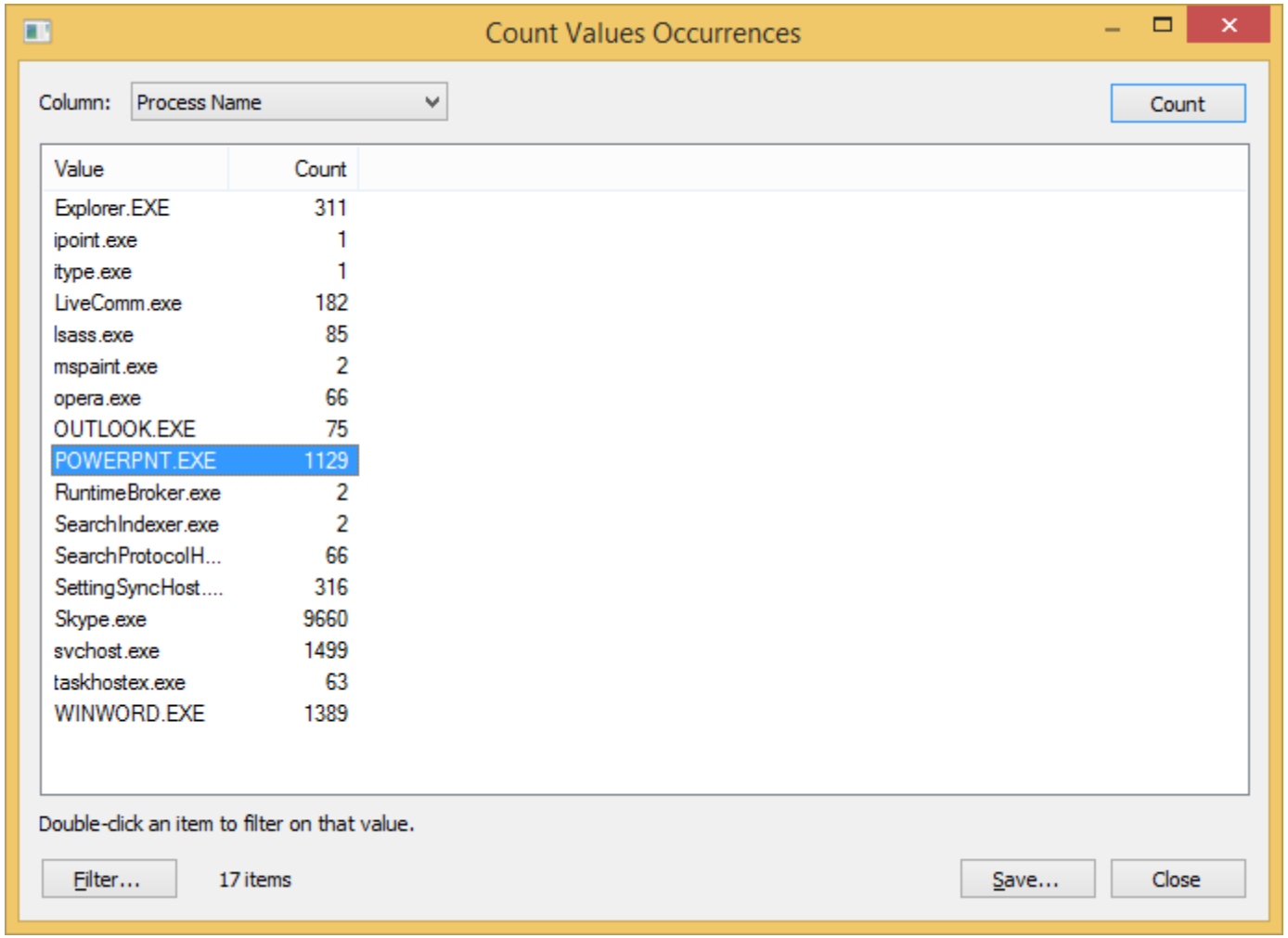
Network summary



Cross reference summary

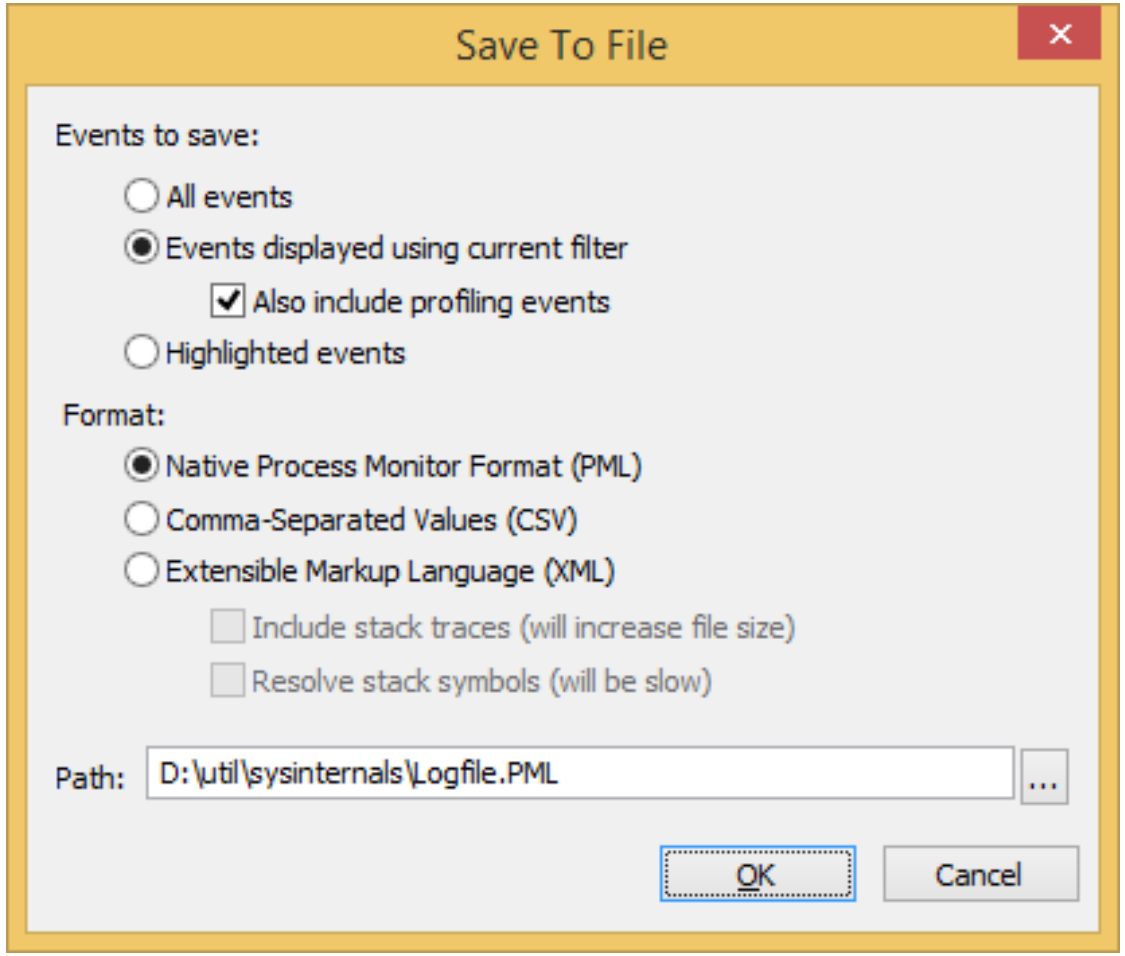


Count Occurencies

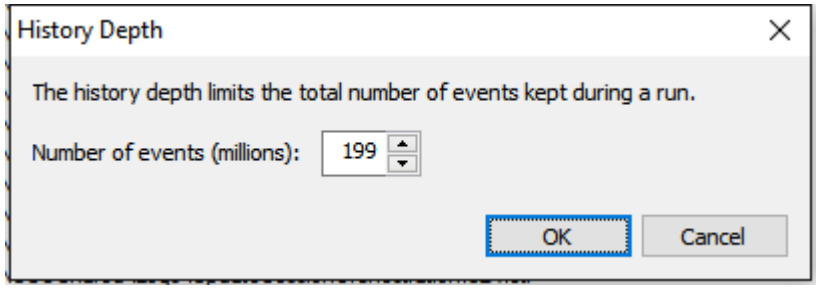


Bitdefender

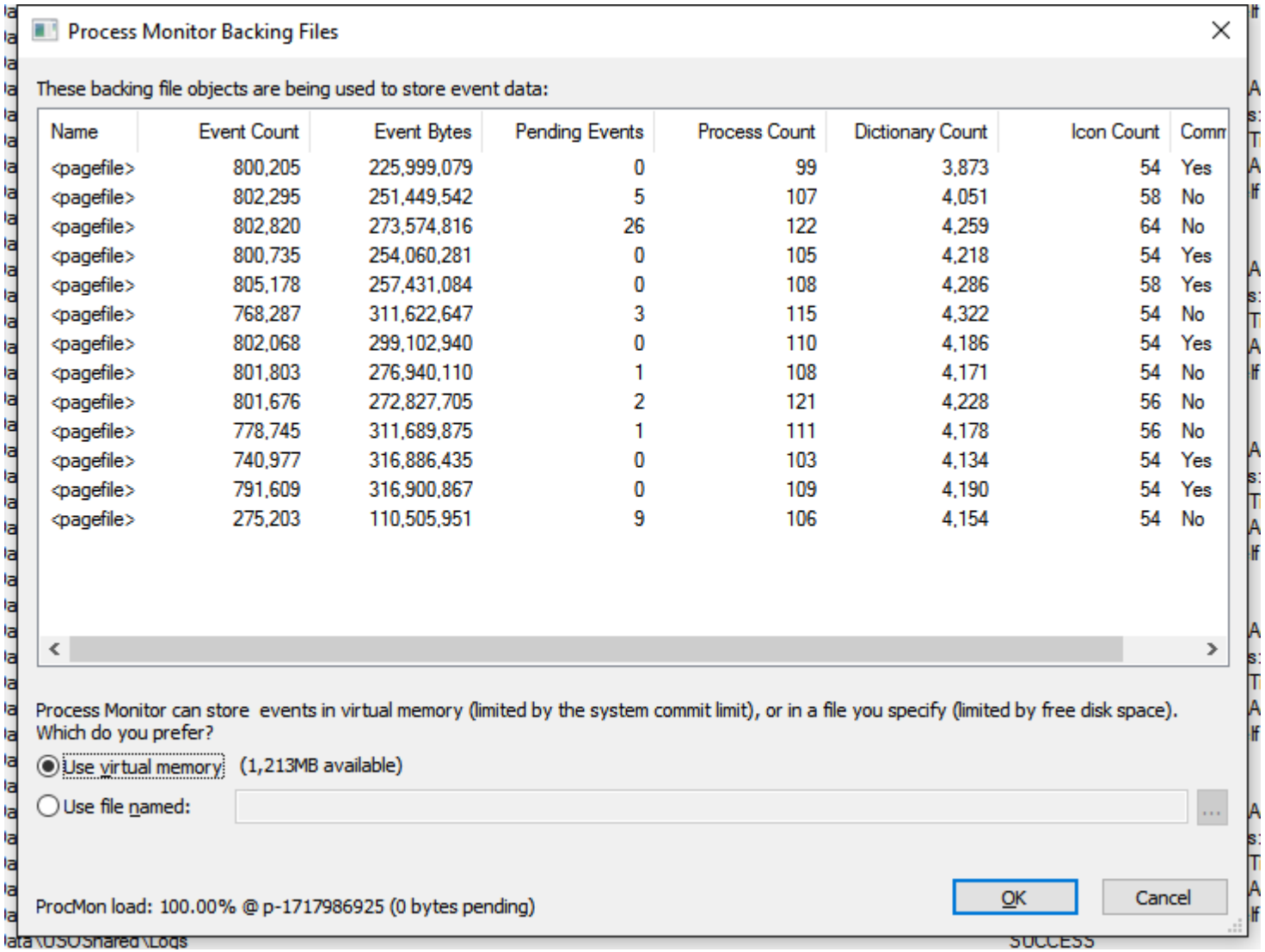
Save a trace


Save To File
Events to save:
All events
Events displayed using current filter
Also include profiling events
Highlighted events
Format:
Native Process Monitor Format (PML)
Comma-Separated Values (CSV)
Extensible Markup Language (XML)
Include stack traces (will increase file size)
Resolve stack symbols (will be slow)
Path:
D:\util\sysinternals\Logfile.PML
...
OK
Cancel

Long Running Traces & Log Size



Long Running Traces & Log Size



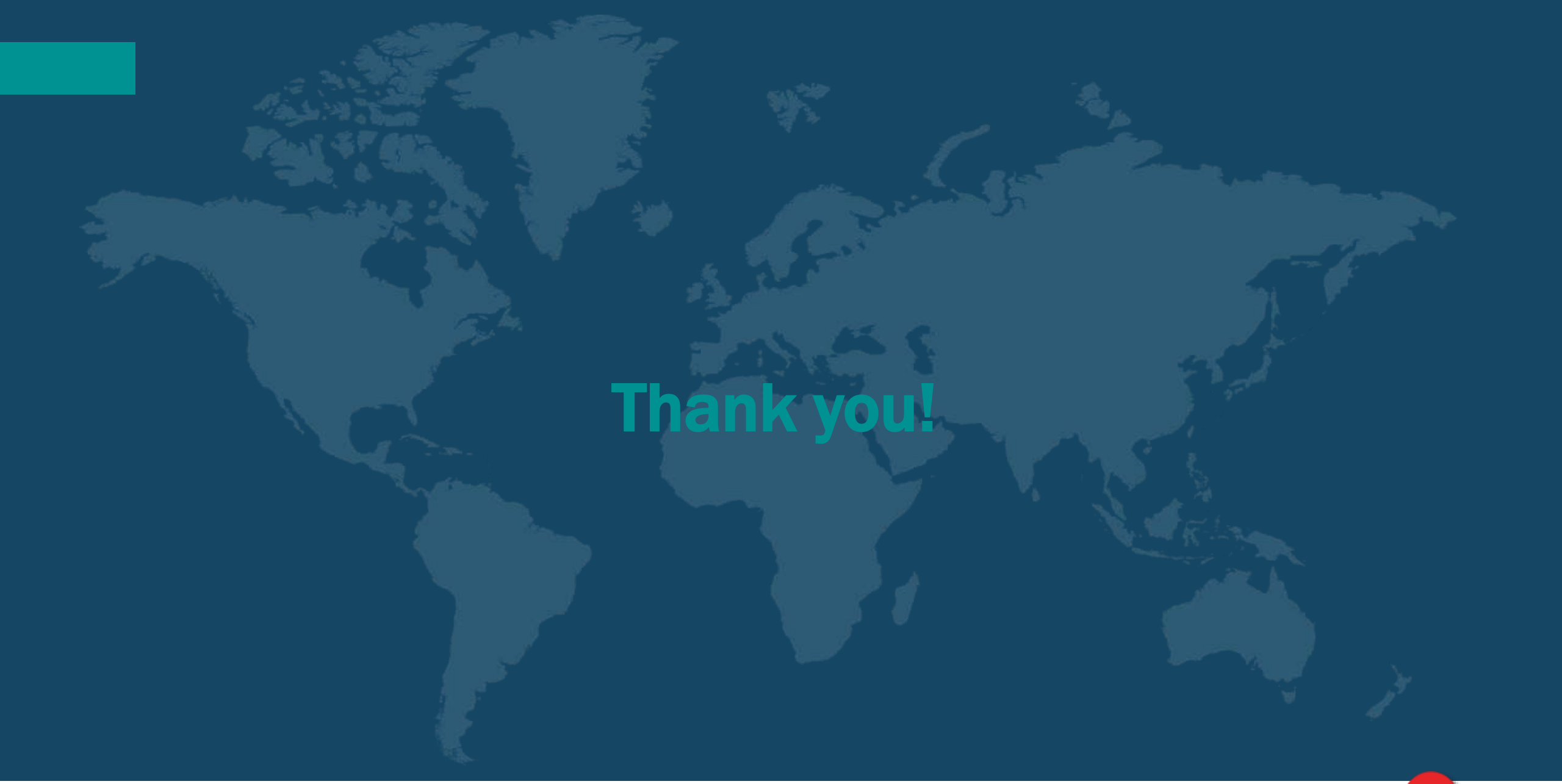
Bitdefender

Q&A

Feedback & Questions

Murgan Mihai
Dan Macovei
Raphael Stroe

mmurgan@bitdefender.com
dmacovei@bitdefender.com
rstroe@bitdefender.com


Thank you!

SYSTEMS LABORATORY

Lecture 10 Regression Models

EP

performance crunch

OI (1232)

SYSTEMS LABORATORY

Simple Linear Regression Models

OI (1232)

SYSTEMS LABORATORY

Overview

EP performance crunch

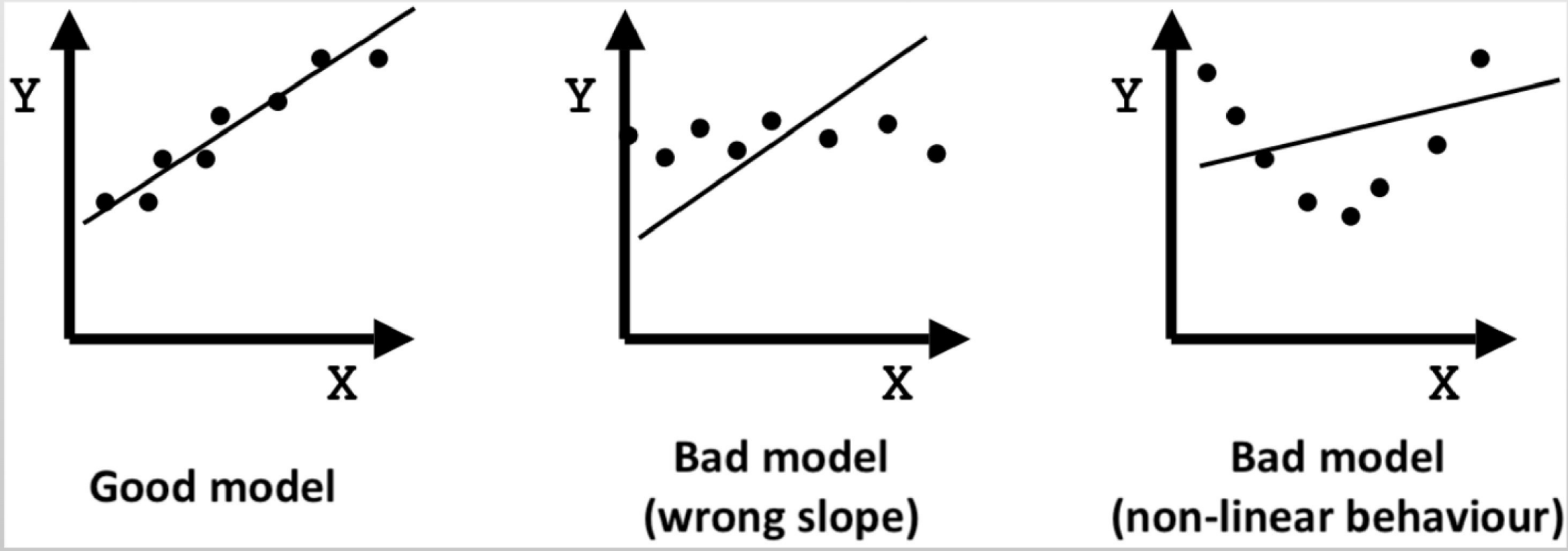
1. Definition of a good model
2. Estimation of model parameters
3. Allocation of variation
4. Standard deviation of errors
5. Confidence intervals for regression parameters
6. Confidence intervals for predictions
7. Visual tests for verifying regression assumption

OI (1232)

Simple linear regression models

- Regression model
  - Predict a response for a given set of predictor variables
  - Response variable
    - Estimated variable
  - Predictor variables
    - Variables used to predict the response predictors or factors
- Linear regression models
  - Response is a linear function of predictors
- Simple linear regression models
  - Only one predictor

Good model


Y
X
Good model
Y
X
Bad model
(wrong slope)
Y
X
Bad model
(non-linear behaviour)

Good model (continued)

- Regression models attempt to minimize the distance measured vertically between the observation point and the model line
- The length of the line segment is called residual / modelling error / error
- The negative and positive errors should cancel out
  - ⇒ zero overall error
  - Many lines satisfy this criterion

Good model (continued)

- Choose the line that minimizes the sum of squares of the errors

$$\widehat{y} = b_0 + b_1 x$$

  - $\widehat{y}$ is the predicted response when the predictor variable is x
  - b_0 and b_1 are fixed regression parameters to be determined from the data

- Given *n* observation pairs $\{(x_1, y_1), ..., (x_n, y_n)\}$, the estimated response $\widehat{y_i}$ for the ith observation is

$$\widehat{y_i} = b_0 + b_1 x_i$$

- The error is

$$e_i = y_i - \widehat{y_i}$$

Good model (continued)

- The best linear model minimizes the sum of squared errors (SSE)

$$\sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} \left(y_i - b_0 - b_1 x_i\right)^2$$

- Subject to the constraint that the mean error is zero

$$\sum_{i=1}^{n} e_i = \sum_{i=1}^{n} \left(y_i - b_0 - b_1 x_i\right) = 0$$

- This is equivalent to minimizing the variance of errors

Estimation of mode parameters

- Regression parameters that give minimum error variance

$$b_1 = \frac{\sum xy - n\overline{x}\,\overline{y}}{\sum x^2 - n\overline{x}^2} \quad \text{and} \quad b_0 = \overline{y} - b_1\overline{x}$$

- where

$$\overline{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \overline{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

$$\sum xy = \sum_{i=1}^{n} x_i y_i \qquad \sum x^2 = \sum_{i=1}^{n} x_i^2$$

Example

- The number of disk I/O's and processor times of seven programs were measured as {(14, 2), (16, 5), (27, 7 (42, 9), (39, 10), (50, 13), (83, 20)}.
- For this data

$$n = 7,\ \sum xy = 3375,\ \sum x = 271,\ \sum x^2 = 13.855$$
$$\sum y = 66,\ \sum y^2 = 828,\ \overline{x} = 38.71,\ \overline{y} = 9.43$$

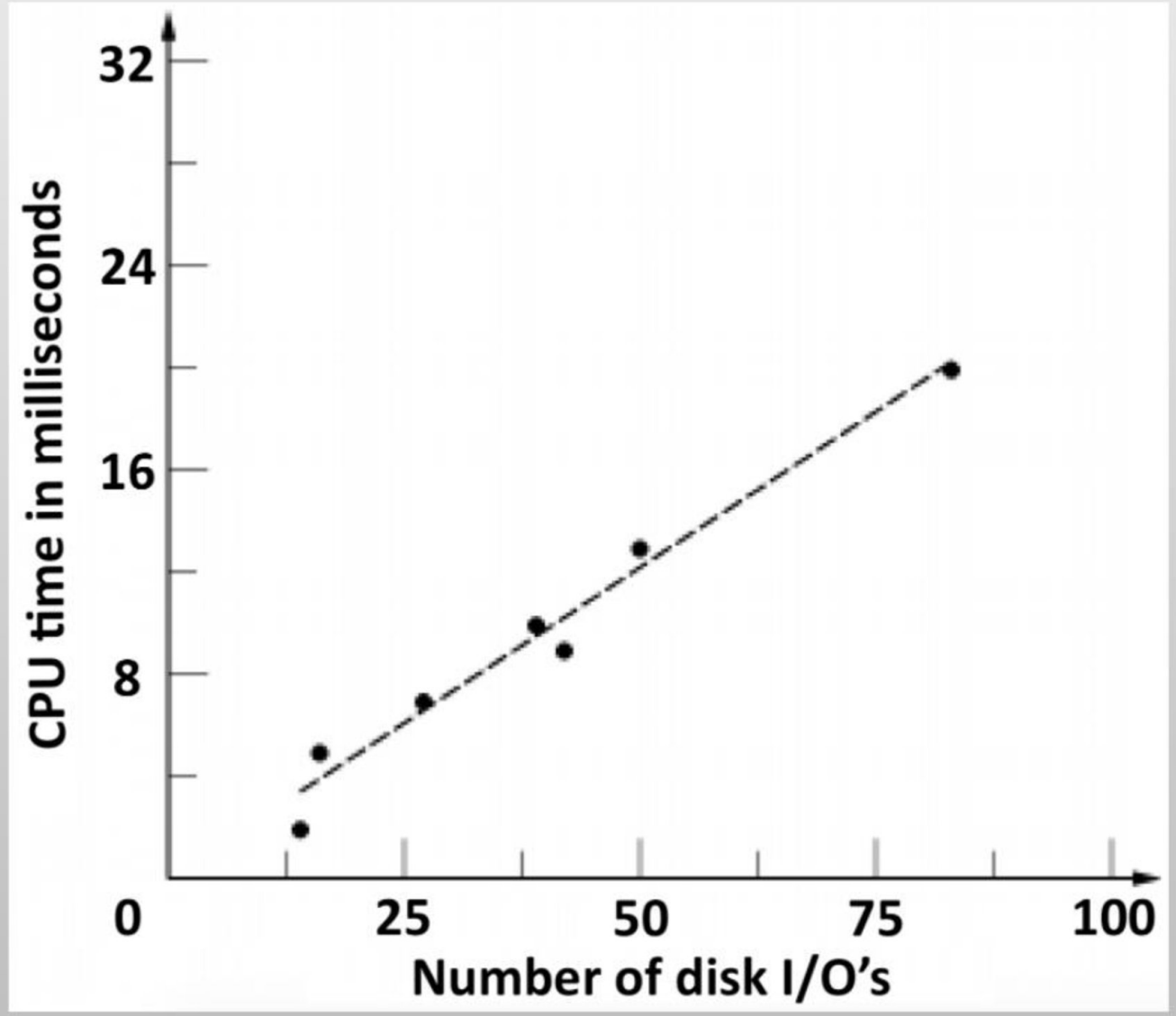
$$b_1 = \frac{\sum xy - n\overline{x}\,\overline{y}}{\sum x^2 - n(\overline{x})^2} = \frac{3375 - 7 \times 38.71 \times 9.43}{13.855 - 7 \times (38.71)^2} = 0.2438$$

$$b_0 = \overline{y} - b_1\overline{x} = 9.43 - 0.2438 \times 38.71 = -0.0083$$

- The desired linear model is:

$$CPU\ time = -0.0083 + 0.2438(Number\ of\ disk\ I/O's)$$

Example (continued)



Example (continued)

	Disk I/O's	CPU Time	Estimate	Error	Error^2
	x_i	y_i	$\widehat{y}_i = b_0 + b_1 x_i$	$e_i = y_i - \widehat{y}_i$	e_i^2
	14	2	3.4043	-1.4043	1.9721
	16	5	3.8918	1.1082	1.2281
	27	7	6.5731	0.4269	0.1822
	42	9	10.2295	-1.2295	1.5116
	39	10	9.4982	0.5018	0.2518
	50	13	12.1795	0.8205	0.6732
	83	20	20.2235	-0.2235	0.0500
$\sum$	272	66	66.0000	0.00	5.8690

Derivation of regression parameters

- The error of the ith observation

$$e_i = y_i - \widehat{y}_i = y_i - (b_0 + b_1 x_i)$$

- The mean error for a sample of n observations

$$\overline{e} = \frac{1}{n}\sum_{i=1}^{n} e_i = \frac{1}{n}\sum_{i=1}^{n} \{y_i - (b_0 + b_1 x_i)\} = \overline{y} - b_0 - b_1\overline{x}$$

- Setting mean error to zero

$$b_0 = \overline{y} - b_1\overline{x}$$

- Substitute b0 in the error expression

$$e_i = y_i - \overline{y} + b_1\overline{x} - b_1 x_i = (y_i - \overline{y}) - b_1(x_i - \overline{x})$$

Derivation (continued)

$$SSE = \sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} \{(y_i - \overline{y})^2 + 2b_1(y_i - \overline{y})(x_i - \overline{x}) + b_1^2(x_i - \overline{x})^2\}$$

$$\frac{SSE}{n-1} = \frac{1}{n-1}\sum_{i=1}^{n}(y_i - \overline{y})^2 - 2b_1\frac{1}{n-1}\sum_{i=1}^{n}(y_i - \overline{y})(x_i - \overline{x}) + b_1^2\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2$$

$$= s_y^2 - 2b_1 s_{xy}^2 + b_1^2 s_x^2$$

- Divided by n-1 to facilitate the last notation (n-1 is not related to the degrees of freedom)

Derivation (continued)

$$\frac{1}{n-1}\frac{d(SSE)}{db_1} = -2s_{xy}^2 + 2b_1 s_x^2 = 0$$

$$b_1 = \frac{s_{xy}^2}{s_x^2} = \frac{\sum xy - n\overline{x}\,\overline{y}}{\sum x^2 - n(\overline{x})^2}$$

Allocation of variation

- Error variance without regression = variance of the response

$$Error = e_i = Observed\,response - Predicted\,response = y_i - \overline{y}$$

- Variance of errors without regression =

$$\frac{1}{n-1}\sum_{i=1}^{n} e_i^2 = \frac{1}{n-1}\sum_{i=1}^{n}(y_i - \overline{y})^2 = Variance\,of\,y$$

Allocation of variation (continued)

- The sum of squared errors without regression

$$\sum_{i=1}^{n}(y_i - \overline{y})^2$$

- SST = total sum of squares
  - It is a measure of *y*'s variability and is called **variation of y**

$$SST = \sum_{i=1}^{n}(y_i - \overline{y})^2 = \left(\sum_{i=1}^{n} y_i^2\right) - n\overline{y}^2 = SSY - SS0$$

- SSY is the sum of squares of *y* (or Σy^2)
- SS0 is the sum of squares of $\overline{y}$ and is equal to $n\overline{y}^2$

Allocation of variation (continued)

- The difference between SST and SSE is the sum of squares explained by the regression (SSR)

$$SSR = SST - SSE$$

$$SST = SSR + SSE$$

- Coefficient of determination (the fraction of the variation that is explained determines the goodness of the regression)

$$R^2 = \frac{SSR}{SST} = \frac{SST - SSE}{SST}$$

Allocation of variation (continued)

- The higher the value of R^2 the better the regression
  - $R^2 = 1 \Rightarrow$ perfect fit
  - $R^2 = 0 \Rightarrow$ no fit

$$Sample\,correlation(x,y) = R_{xy} = \frac{s_{xy}^2}{s_x s_y}$$

$$Coefficient\ of\ determination = \{Correlation\ coefficient(x,y)\}^2$$

- Shortcut formula for SSE

$$SSE = \sum y^2 - b_0 \sum y - b_1 \sum xy$$

Example

- Consider the disk I/O - CPU time data from the previous example

$$SSE = \sum y^2 - b_0 \sum y - b_1 \sum xy$$
$$= 828 + 0.0083 \times 66 - 0.2438 \times 3375 = 5.87$$

$$SST = SSY - SS0 = \sum y^2 - n(\overline{y})^2$$
$$= 828 - 7 \times (9.43)^2 = 205.71$$

$$SSR = SST - SSE = 205.71 - 5.87 = 199.84$$

$$R^2 = \frac{SSR}{SST} = \frac{199.84}{205.71} = 0.9715$$

- The regression explains 97% of the CPU time's variation

Standard deviation of errors

- Variance of errors
  - Divide sum of squared errors by its degrees of freedom
  - Errors are obtained after calculating 2 regression parameters ⇒ n-2 degrees of freedom

$$s_e = \sqrt{\frac{SSE}{n-2}}$$

- MSE (mean squared errors) = SSE/(n-2)
- s_e (standard deviation of errors) = $\sqrt{MSE}$
- SSY - n degrees of freedom
  - Obtained from n independent observations without estimating any parameters
- SS0 - 1 degree of freedom
  - Computer simply from $\overline{y}$
- SST - n-1 degrees of freedom
  - Only requires one parameter to be calculated from the data $\overline{y}$

Standard deviation of errors (continued)

- SSR (= SST - SSE) has the remaining 1 degree of freedom

$$SST = SSY - SS0 = SSR + SSE$$
$$(n-1) = n - 1 = 1 + (n-2)$$

- Degrees of freedom add up (similar to sums of squares)

Example

- Consider the disk I/O - CPU time data example
- Degrees of freedom of the sums

$$\begin{array}{lrcrcrcr} SS: & SST & = & SSY & - & SS0 & = & SSR & + SSE \\ & 205.71 & = & 828 & - & 622.29 & = & 199.84 & + 5.87 \\ DF: & 6 & = & 7 & - & 1 & = & 1 & + 5 \end{array}$$

- Mean squared error

$$MSE = \frac{SSE}{DF\ for\ errors} = \frac{5.87}{5} = 1.17$$

- Standard deviation of errors

$$s_e = \sqrt{MSE} = \sqrt{1.17} = 1.08$$

CI for regression parameters

- Regression coefficients b0 and b1 are estimates from a single sample of size n
  - ⇒ random
  - ⇒ using another sample, the estimates may be different
- Consider β0 and β1 the true parameters of the population

$$y = \beta_0 + \beta_1 x$$

- Computed coefficients b0 and b1 are estimates of β0 and β1

$$s_{b_0} = s_e \left[\frac{1}{n} + \frac{\overline{x}^2}{\sum x^2 - n\overline{x}^2} \right]^{\frac{1}{2}}$$

$$s_{b_1} = \frac{s_e}{\left[\sum x^2 - n\overline{x}^2 \right]^{\frac{1}{2}}}$$

CI for regression parameters (continued)

- The 100(1-α)% confidence intervals for b0 and b1 can be computed using the 1-α/2 quantile of a t variate with n-2 degrees of freedom
  $t_{[1-\alpha/2;\ n-2]}$
- The confidence intervals
  $$b_0 \mp t s_{b_0}$$
  $$b_1 \mp t s_{b_1}$$
- If a confidence interval includes zero ⇒ the regression parameter cannot be considered different from zero at the 100(1-α)% CI

Example

- Consider the disk I/O - CPU time data example

$$n = 7,\ \overline{x} = 38.71,\ \sum x^2 = 13.855,\ s_e = 1.0834$$

$$s_{b_0} = s_e\left(\left[\frac{1}{n} + \frac{\overline{x}^2}{\sum x^2 - n\overline{x}^2}\right]^{\frac{1}{2}}\right.$$

$$= 1.0834\left[\frac{1}{7} + \frac{(38.71)^2}{13.855 - 7 \times 38.71 \times 38.71}\right]^{\frac{1}{2}} = 0.8311$$

$$s_{b_1} = \frac{s_e}{\left[\sum x^2 - n\overline{x}^2\right]^{\frac{1}{2}}}$$

$$= \frac{1.0834}{[13.855 - 7 \times 38.71 \times 38.71]^{\frac{1}{2}}} = 0.0187$$

Example (continued)

$$b_0 \mp ts_{b_0} \qquad b_1 \mp ts_{b_1} \qquad t_{[1-\alpha/2;\ n-2]} = t_{[0.95;\ 5]}$$

- 90% confidence interval for b0

$$-0.0083 \mp (2.015)(0.8311) = (-1.6830, 1.6663)$$

- CI includes zero - the hypothesis that this parameter is zero cannot be rejected at 0.10 significance level ⇒ b0 is essentially zero

- 90% confidence interval for b1

$$0.2438 \mp (2.015)(0.0187) = (0.2061, 0.2814)$$

- CI does not include zero - the slope b1 is significantly different from zero at this confidence level

Example

- Best linear models
  Time on UNIX = 0.017 (Data size in bytes) + 24
  Time on ARGUS = 0.034 (Data size in bytes) +30

- The regression explains 81% and 75% of the variation, respectively.

Does ARGUS take longer time per byte than UNIX?
Does ARGUS have a longer setup time than UNIX?

Example (continued)

UNIX			
Param-eter	**Mean**	**Std. Dev.**	**Confidence Interval**
b0	26.898	2.005	(23.2968, 30.4988)
b1	0.017	0.003	(0.0128, 0.0219)
ARGUS			
Param-eter	**Mean**	**Std. Dev.**	**Confidence Interval**
b0	31.068	4.711	(22.6076, 39.5278)
b1	0.034	0.006	(0.0231, 0.0443)

- Intervals for intercepts overlap, while those for slopes do not
- Setup times are not significantly different in the 2 systems, while the per byte times (slopes) are different

CI for predictions

- Mean value of the predicted response

$$\widehat{y}_p = b_0 + b_1 x_p$$

- Standard deviation of the mean of a future sample of m observations

$$s_{\widehat{y}_{mp}} = s_e \left[\frac{1}{m} + \frac{1}{n} + \frac{(x_p - \overline{x})^2}{\sum x^2 - n\overline{x}^2} \right]^{\frac{1}{2}}$$

CI for predictions (continued)

- m = 1 ⇒ standard deviation of a single future observation

$$s_{\widehat{y}_{1p}} = s_e\left[1 + \frac{1}{n} + \frac{(x_p - \overline{x})^2}{\sum x^2 - n\overline{x}^2}\right]^{\frac{1}{2}}$$

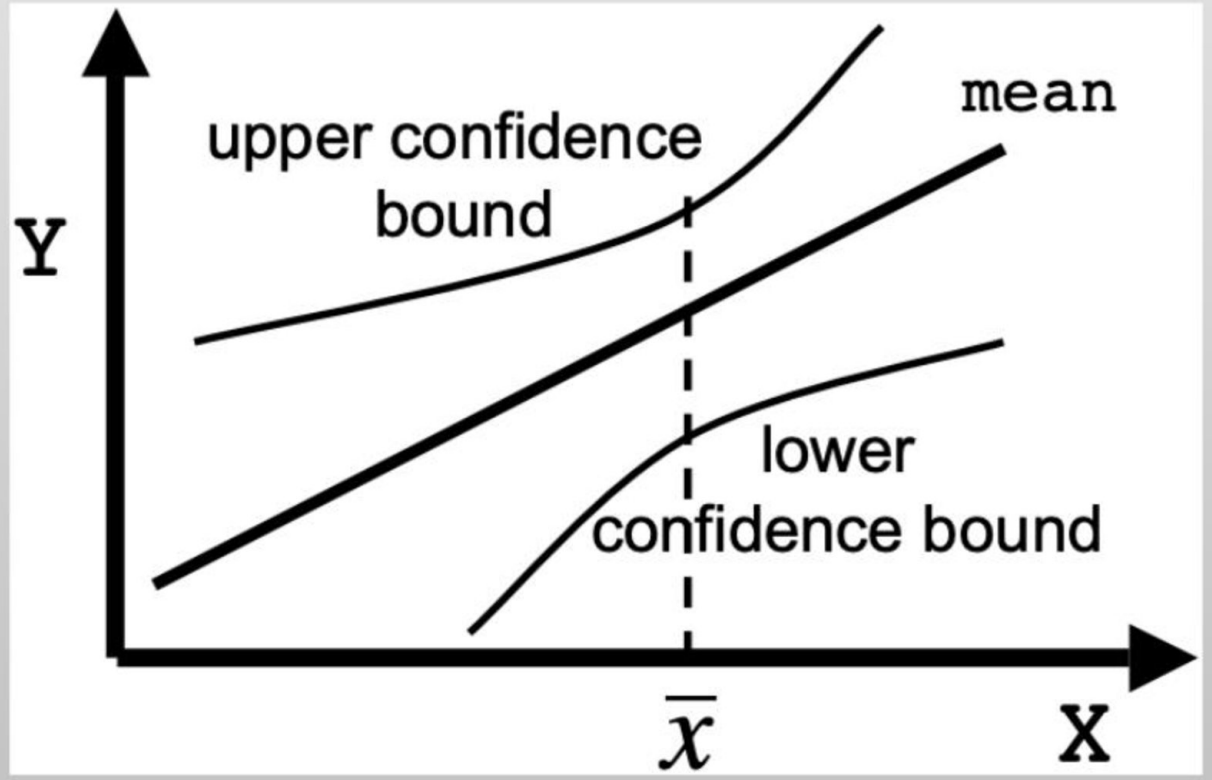
- m = ∞ ⇒ standard deviation of the mean of a large number of future observations at xp

$$s_{\widehat{y}_p} = s_e\left[\frac{1}{n} + \frac{(x_p - \overline{x})^2}{\sum x^2 - n\overline{x}^2}\right]^{\frac{1}{2}}$$

- 100(1-α)% confidence interval for the mean can be constructed using a t quantile read at n-2 degrees of freedom

CI for predictions (continued)

- Goodness of the prediction decreases as we move away from the center



Example

- Consider the disk I/O - CPU time data example

  CPU time = -0.0083 + 0.2438(Number of disk I/O's)

- What is the mean CPU time for a program with 100 disk I/O's?

  CPU time = -0.0083 + 0.2438(100) = 24.3674

- Standard deviation of errors $s_e = 1.0834$

Example (continued)

- The standard deviation of the predicted mean of a large number of observations

$$s_{\widehat{y}_p} = s_e\left[\frac{1}{n} + \frac{(x_p - \overline{x})^2}{\sum x^2 - n\overline{x}^2}\right]^{\frac{1}{2}}$$

$$s_{\widehat{y}_p} = 1.0834\left[\frac{1}{7} + \frac{(100 - 38.71)^2}{13855 - 7(38.71)^2}\right]^{\frac{1}{2}} = 1.2159$$

- 90% CI for the predicted mean

$$24.3574 \mp (2.015)(1.2159) = (21.9174, 26.8174)$$

Example (continued)

- CPU time of a single future program with 100 disk I/O's

$$s_{\widehat{y}_{1p}} = s_e\left[1 + \frac{1}{n} + \frac{(x_p - \overline{x})^2}{\sum x^2 - n\overline{x}^2}\right]^{\frac{1}{2}}$$

$$s_{\widehat{y}_p} = 1.0834\left[1 + \frac{1}{7} + \frac{(100 - 38.71)^2}{13855 - 7(38.71)^2}\right]^{\frac{1}{2}} = 1.6286$$

- 90% CI for a single prediction

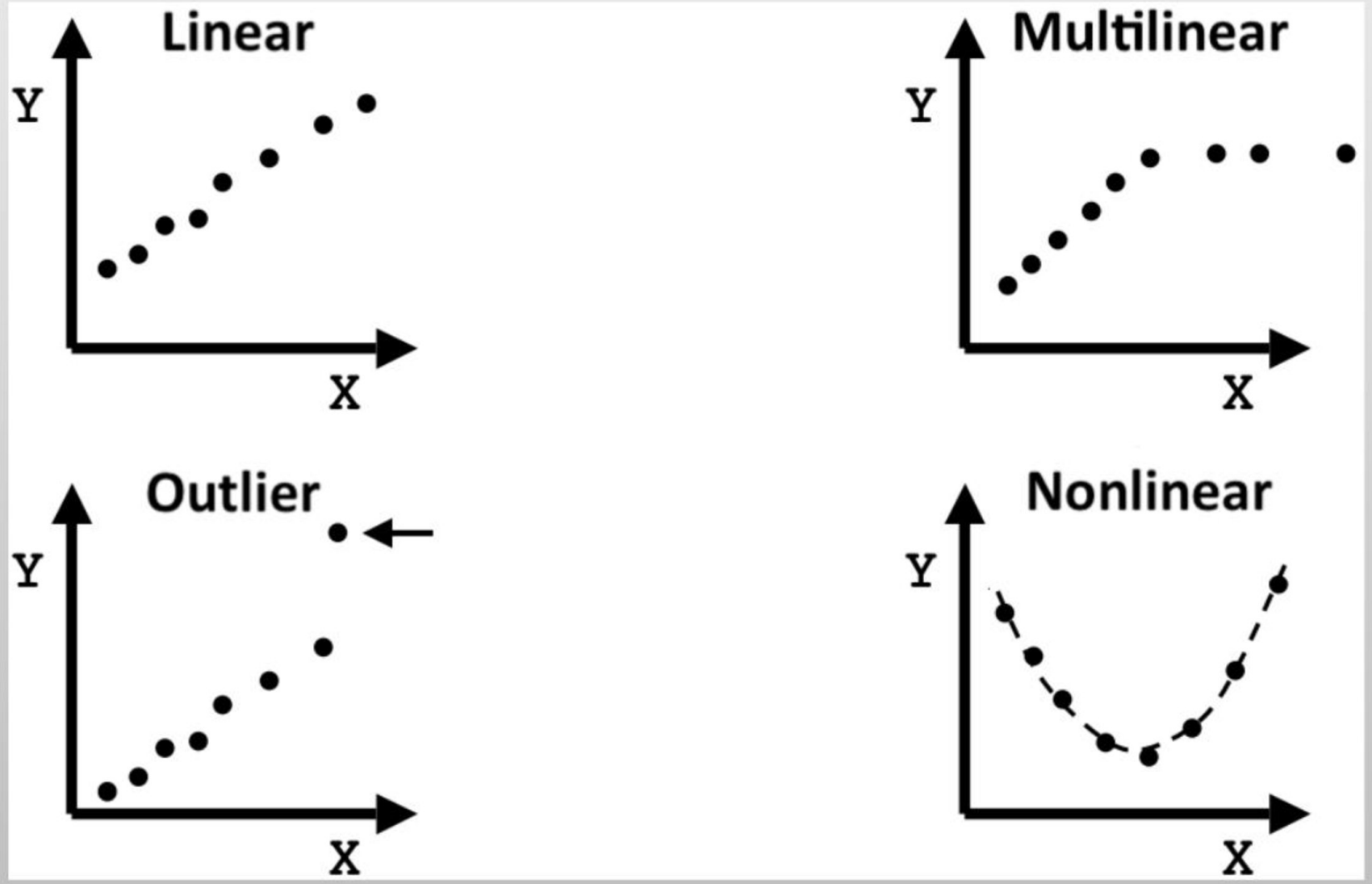
$$24.3574 \mp (2.015)(1.6286) = (21.0858, 27.6489)$$

Visual tests for regression assumptions

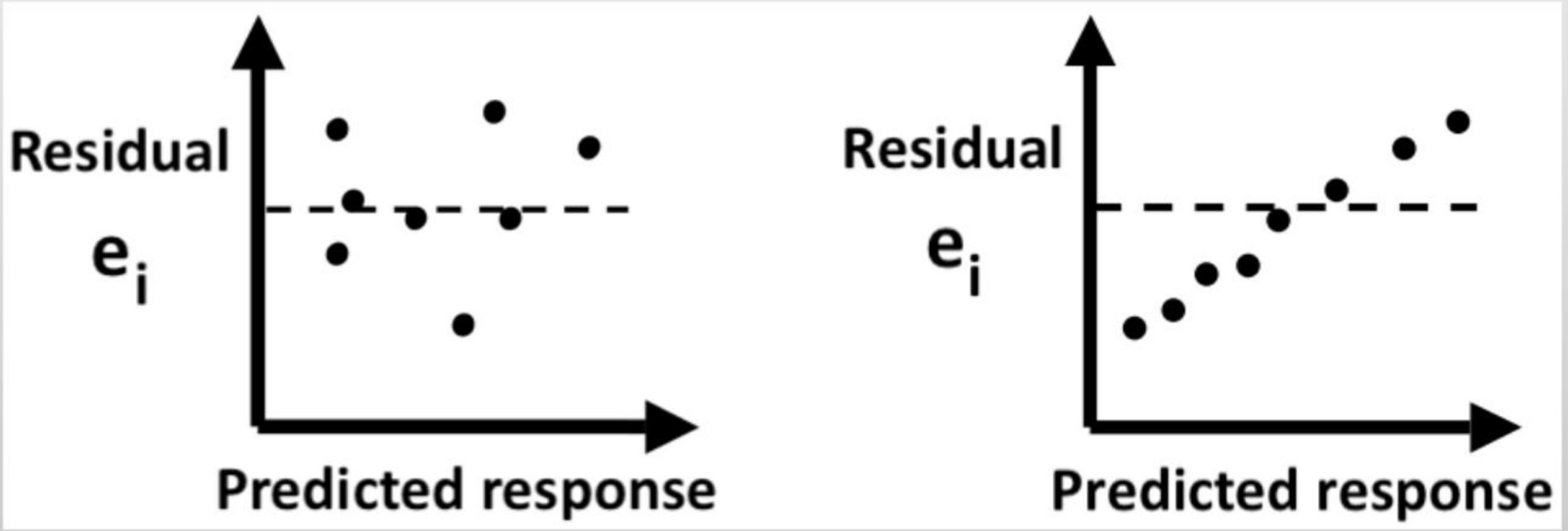
Regression assumptions

1. The true relationship between the response variable y and the predictor variable x is linear
2. The predictor variable x is non-stochastic and it is measured without any error
3. The model errors are statistically independent
4. The errors are normally distributed with zero mean and a constant standard deviation

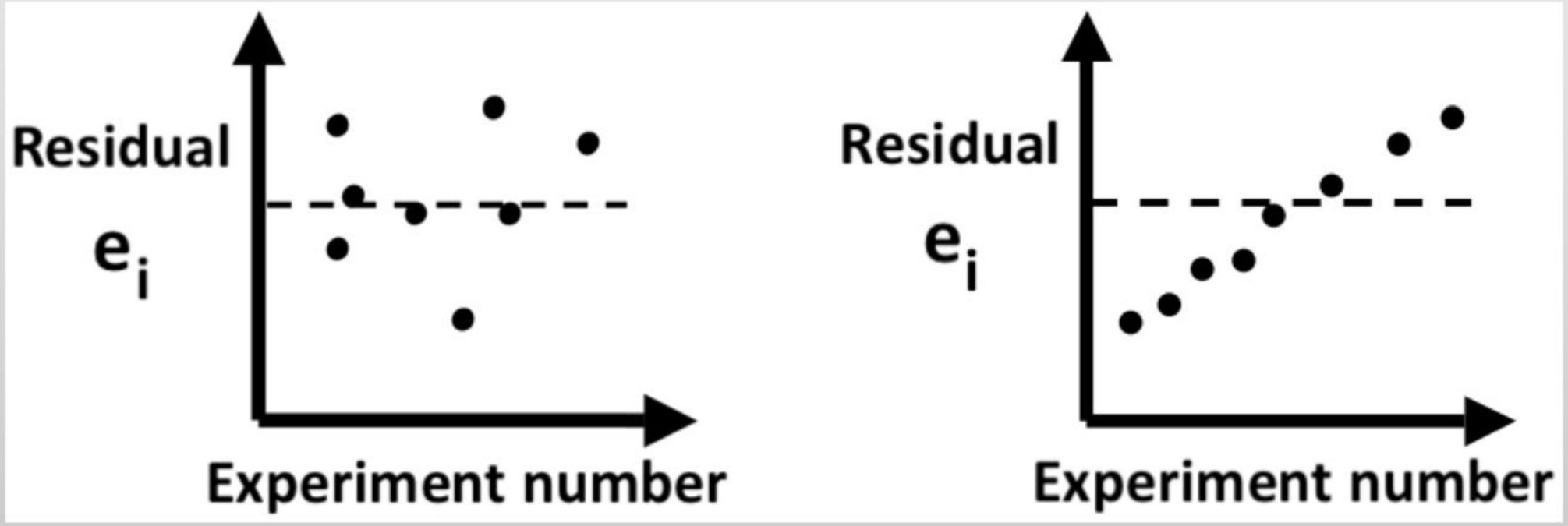
1. Linear relationship



2. Independent errors

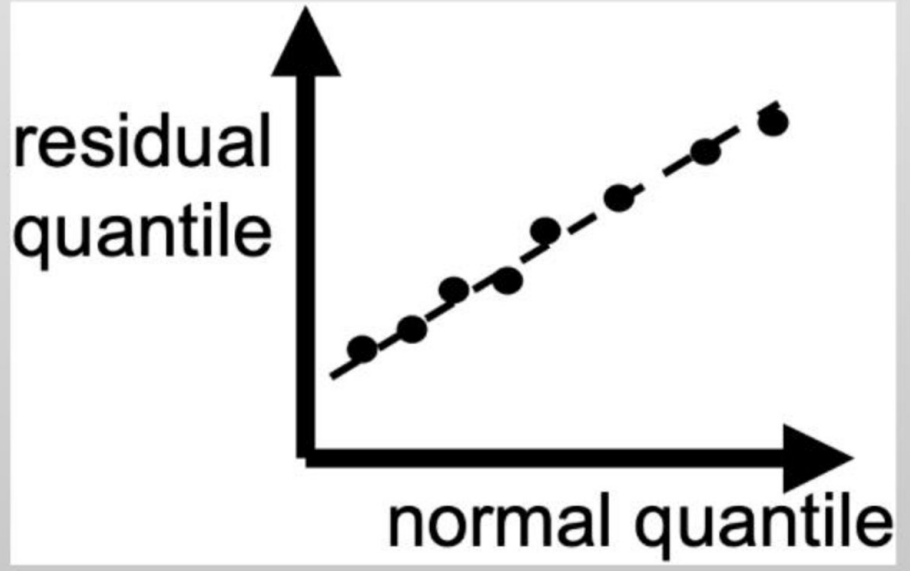


2. Independent errors (continued)

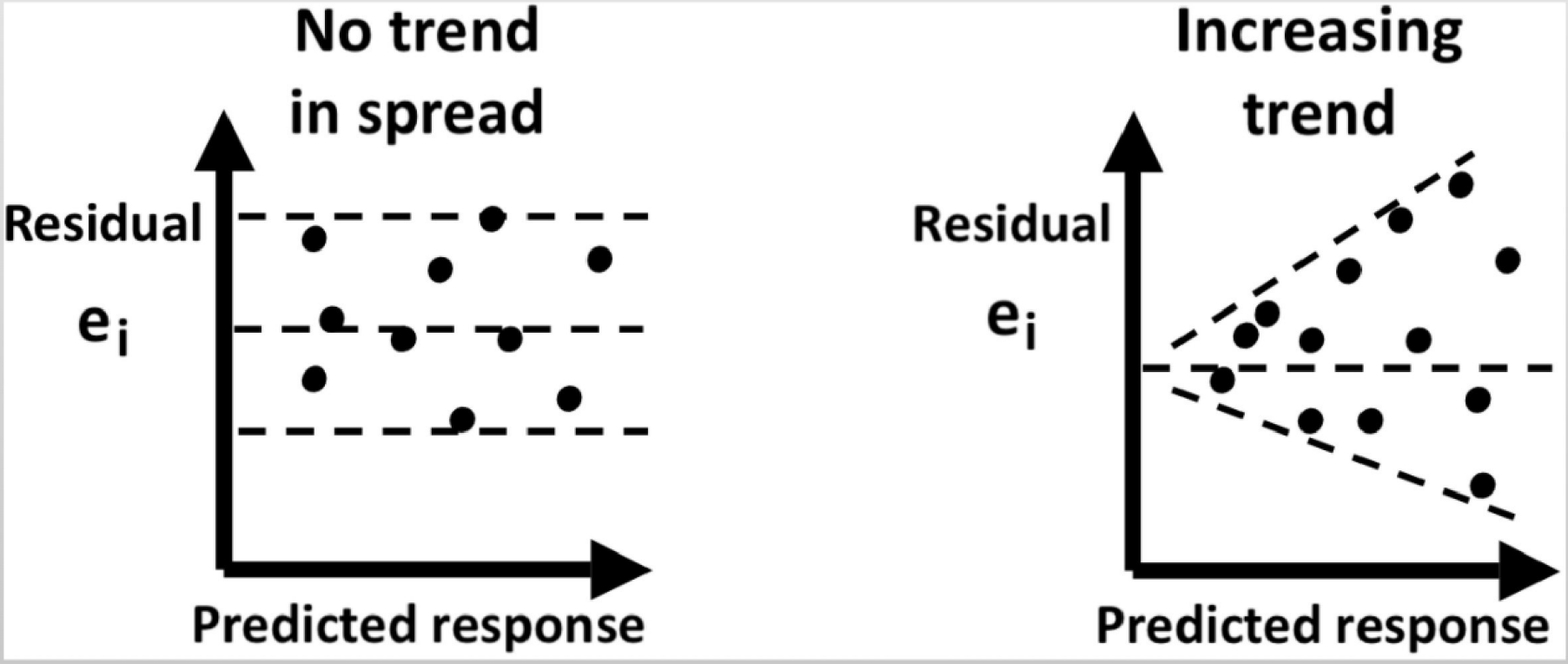


3. Normally distributed errors

- Use a quantile-quantile plot of e_i versus N(0,1)
  - Linear ⇒ the assumption is satisfied

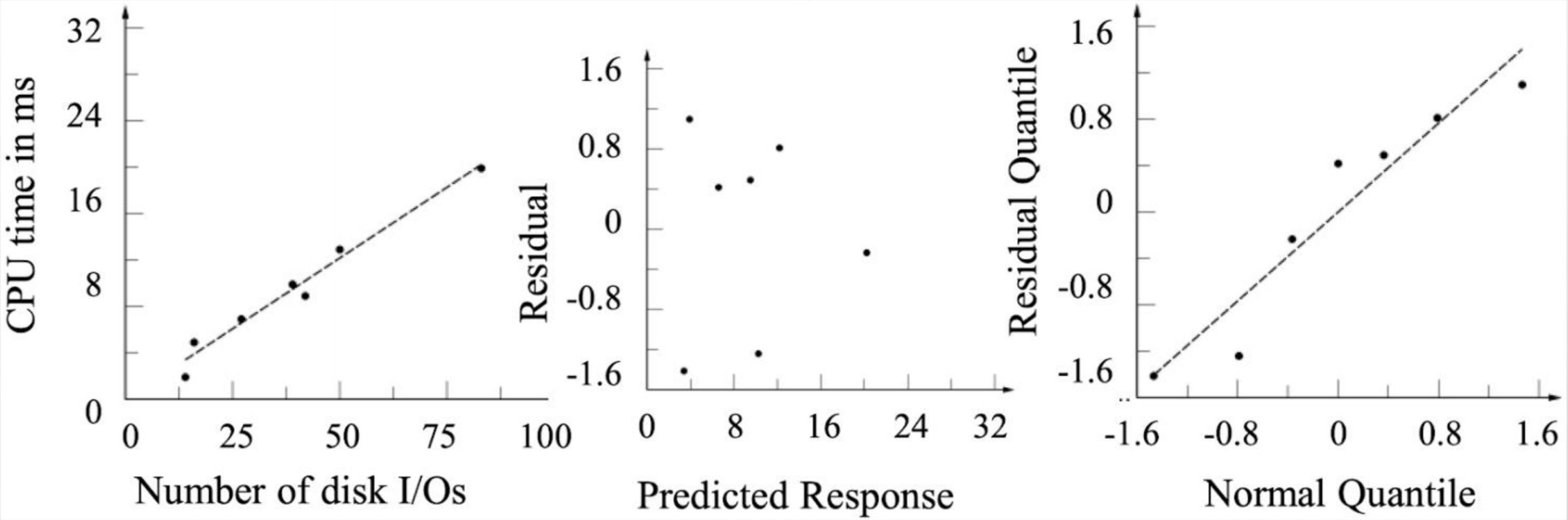


4. Constant standard deviation of errors



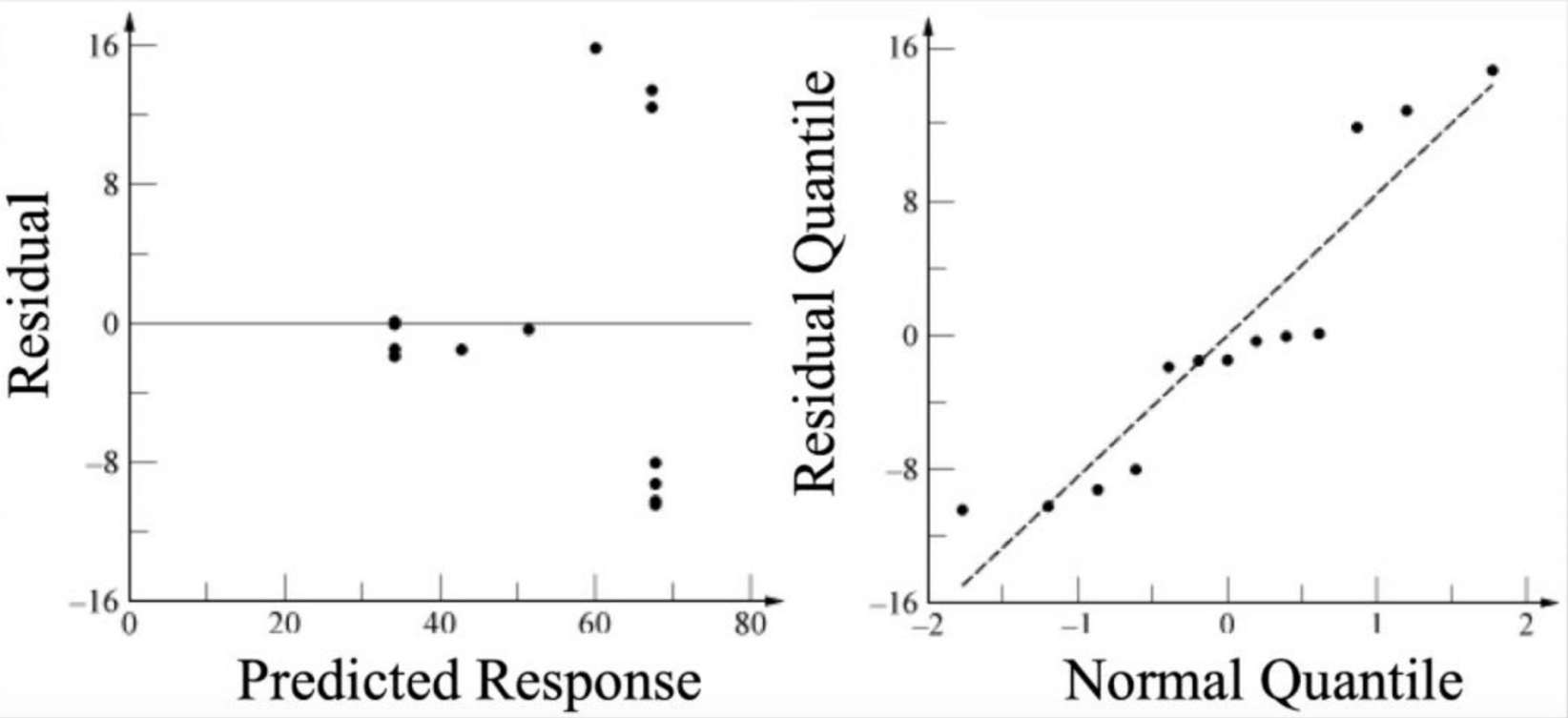
Example

- Consider the disk I/O - CPU time data example



1. Relationship is linear
2. No trend in residuals ⇒ seem independent
3. Linear normal quantile-quantile plot
    - Larger deviations at lower values but all values are small

Example



1. Larger errors at larger responses
2. Normality of errors is questionable

Summary

- Terminology
  - Simple Linear Regression model
  - Sums of Squares, Mean Squares
  - Degrees of freedom
  - Percent of variation explained
  - Coefficient of determination
  - Correlation coefficient

SYSTEMS LABORATORY

Summary

EP performance crunch

- Simplest model
- All estimates from a sample = random / guess
  - Need CI
  - Even if parameters of the model (regression parameters, responses), or a means
- Use formulas
- Aware of the many assumptions made
  - Same apply for any model, even more complex ones
  - Need to verify
- How to verify vizually
  - Assumptions of linearity, error independence, error normality
  - For taking practical decisions quickly

- Multiple Linear Regression
  - More than one predictor variables
- Regression with Categorical Predictors
  - Predictor variables are categories such as CPU type, disk type, etc.
- Curvilinear Regression
  - Relationship is nonlinear
- Transformations
  - Errors are not normally distributed or the variance is not homogeneous

References

1. Raj K. Jain, "*Art of Computer Systems Performance Analysis: Techniques for Experimental Design Measurements, Simulation and Modeling*", John Wiley, 2015.
2. Raj Jain, CSE 567M Computer Systems Analysis, Washington University in St. Louis, 2013.